

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 15.09.2026, 21:37:23
Файлов исходного кода: 111
Суммарный объём кода: 1.98 MB
Глав/билетов в пособии: 36

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: ...]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref]
6. 6. Графы. Представление графов, DFS, BFS [id: ...]
7. 7. Графы. Планарные. Покраска [id: graph-plana]
8. 7. Графы. Планарные. Покраска [id: planarity-e]
9. 8. Графы. Компоненты связности [id: graph-comp]
10. 9. Графы. Топологическая сортировка [id: top-s]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articul]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cyc]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-a]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: ...]
22. 18. Графы. Остовное дерево. Краскал [id: mst-k]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Б...
24. 19. Графы. Остовное дерево. Прима [id: mst-pri]
25. 20. Графы. Остовное дерево. Борувка [id: mst-b]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends \
            # На Ubuntu политика ImageMagick иногда запрещает
            # разрешаем то, что нужно пайплайну (text -> pdf)
            for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml
            if [ -f "$policy" ]; then
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискреткам
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
     <title> Дискретка для тупых – Интерактивный гид по дискреткам
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub"
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useState<
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefined;
 const next = index < chapters.length - 1 ? chapters[index + 1] : undefined;

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters.length]);

 return (
 <div className="min-h-screen bg-slate-950 text-slate-50" >

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
```tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler') => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab }) => {
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };

  return (
```

```
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 w-6 h-6" />
          <Sparkles className="w-6 h-6" />
        </div>
        <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```



```
export function PythonCompiler() {
  const [code, setCode] = useState(STARTER_CODE);
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Нажмите «Запустить»");
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState("");

  const runCode = useCallback(async () => {
    if (running) return;

    setRunning(true);
    setOutput("");
    setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

    const stdout: string[] = [];
    const stderr: string[] = [];
    const inputLines = stdin.split(/\r?\n/);

    try {
      const runtime = await getPythonRuntime();
      setRuntimeReady(true);
      setRuntimeMessage("Python готов — код выполняется");

      runtime.setStdout({ batched: (message) => stdout.push(message) });
      runtime.setStderr({ batched: (message) => stderr.push(message) });
      runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
      await runtime.runPythonAsync(code);

      const text = [...stdout, ...stderr].join("");
      setOutput(text || "Готово: программа ничего не выполнила");
    } catch (error) {
      const captured = [...stdout, ...stderr].join("");
      setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
      setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
    } finally {
      setRunning(false);
    }
  }, [code, stdin, runtimeReady]);
```

```

        <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
  <button
    type="button"
    onClick={reset}
    className="inline-flex items-center gap-2 text-slate-800 transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" />
  </button>
  <button
    type="button"
    onClick={() => void runCode()}
    disabled={running}
    className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
  >
    {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запускаю..."}
    {running ? "Запускаю..." : "Запустить"}
  </button>
</div>
</div>

<textarea
  value={code}
  onChange={(event) => setCode(event.target.value)}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) && event.key === "Enter") {
      event.preventDefault();
      void runCode();
    }
  }}
  spellCheck={false}
  aria-label="Код Python"
  className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
  placeholder="Напишите Python-код..."

```

```
-----
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-indigo-600" >
        <span>Ctrl или Cmd + Enter запускает код. Введите код в поле ввода.
      </span>
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-amber-600" >
        <span>Первый запуск скачивает примерно несколько килобайт кода.
      </span>
    </div>
    <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
      <Info className="w-4 h-4 shrink-0 text-emerald-600" >
        <span>Код выполняется в браузере и не отправляется на сервер.
      </span>
    </div>
  </div>
</div>
</main>
);
}
....
```

Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+def main():
+ print("Привет, мир!")
+ return "Ваш код выполнен успешно!"
+
+main()
+
+`;
```



```

+
+ const runCode = useCallback(async () => {
+ if (running) return;
+
+ setRunning(true);
+ setOutput("");
+ setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+ const stdout: string[] = [];
+ const stderr: string[] = [];
+ const inputLines = stdin.split(/\r?\n/);
+
+ try {
+ const runtime = await getPythonRuntime();
+ setRuntimeReady(true);
+ setRuntimeMessage("Python готов – код выполняется");
+
+ runtime.setStdout({ batched: (message) => stdout
+ runtime.setStderr({ batched: (message) => stderr
+ runtime.setStdin({ stdin: () => (inputLines.length
+ await runtime.runPythonAsync(code);
+
+ const text = [...stdout, ...stderr].join("");
+ setOutput(text || "Готово: программа ничего не выв
+ } catch (error) {
+ const captured = [...stdout, ...stderr].join("")
+ setOutput(`${captured}${captured && !captured.en
+ setRuntimeMessage("Не удалось выполнить код – пр
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };

```

```

late-800 transition-colors"
+
+ >
+ <RotateCcw className="w-3.5 h-3.5" />
+ </button>
+ <button
+ type="button"
+ onClick={() => void runCode()}
+ disabled={running}
+ className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+ >
+ {running ? <Loader2 className="w-3.5
+ {running ? "Запускаю..." : "Запустить"}
+ </button>
+ </div>
+ </div>
+
+ <textarea
+ value={code}
+ onChange={(event) => setCode(event.target
+ onKeyDown={(event) => {
+ if ((event.ctrlKey || event.metaKey) &
+ event.preventDefault();
+ void runCode();
+ }
+ }}
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea

```

```

+ </div>
+ <div className="flex gap-2 rounded-xl border
+ <Info className="w-4 h-4 shrink-0 text-eme
+ Код выполняется в браузере и не отпра
+ </div>
+ </div>
+ </div>
+ </main>
+);
+}
+...

```

## `vite.config.ts`

### Полное содержимое после изменений

```

````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локальн
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
},

```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7

AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

****CRITICAL DIRECTIVE****:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas). При запросе пользователя создать или сгенерировать новую страницу ****строго**** придерживаться готовых шаблонов разметки, стилей и компонентов.

Правила для малых страниц (Билетов / Глав)

- **Единый стиль:**** Всегда используй Tailwind CSS и HTML-шаблоны.
- **Структура JSON/TS:**** Каждая новая малая страница должна быть описана в файле `src/data/content.ts` с использованием `export const content = { ... }` для вставки в `src/data/content.ts`.
- **Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
 - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``).
 - Обязательная сетка аналогий (2 колонки: неформальный язык / строгий язык).

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

```
{
  "name": "algv0-exam-guide",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "audit:content": "node scripts/audit-coverage.mjs",
    "audit:terms": "node scripts/audit-terms.mjs",
    "export:txt": "node scripts/export/collect-code.mjs",
    "export:png": "node scripts/export/render-pages.mjs",
    "export:pdf": "node scripts/export/build-pdf.mjs",
    "export": "npm run export:txt && npm run export:png",
    "export:guide": "node scripts/export/build-guide-pdf.mjs",
    "export:shots": "node scripts/export/capture-viz-shots.mjs",
    "verify:unit": "node scripts/verify/live-matrix.test.mjs",
    "verify:code": "node scripts/verify/check-reference.mjs",
    "verify:links": "node scripts/verify/check-links.mjs",
    "verify:dom": "node scripts/verify/run-browser-check.mjs",
    "verify:probe": "node scripts/verify/run-probe.mjs",
    "verify": "npm run verify:unit && npm run verify:links",
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
  }
}
```

Универсальное пособие • полный исходный код

```
-----  
npm run dev          # предпросмотр на http://0.0.0.0:5  
npm run build        # сборка в dist/ (single-file)  
````
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в о

| Команда              | Шаг       | Результат                               |
|----------------------|-----------|-----------------------------------------|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/`                     |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export`     | всё сразу | txt + png + pdf                         |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core  
````
```

Настройки через переменные окружения:

- \* `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- \* `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

### Автоматизация (CI)

`.github/workflows/rebuild-pdf.yml` запускается **\*\*на каж**  
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док  
и коммитит обновлённые `public/export/full\_code\_all\_pages`  
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

## Аудит контента

```
```bash
```

Заголовки

- * Заголовки секций: Используют стандартный тег `<h2>` и
- * Подзаголовки: Используют тег `<h3>`.

Текст и параграфы

- * Обычный текст содержится в тегах `<p>`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы.
- * Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- * Они помещены внутрь оберток с классом `<div class="img">`
- * Все отрисовки находятся внутри векторного тега `<svg>`
- * Подпись к картинке, если она есть, идет сразу после `<img>`

Списки

- * Стандартные списки оформлены через `<ul class="list-n">`

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```
-----
* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**
---
```

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни кода
8. Графы. Компоненты связности	`graph-components`	Нет кода
24. Классы сложности, сведение задач	`complexity-classes`	Нет кода

Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---

ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",

    /* Path mapping */
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    },

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "vite.config.ts"]
}
```

```
    },  
  });  
}
```

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";  
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";  
import { chapters } from "../data/content";  
import { Navbar } from "../components/Navbar";  
import { Sidebar } from "../components/Sidebar";  
import { MobileToc } from "../components/MobileToc";  
import { ChapterView } from "../components/ChapterView";  
import { ChapterNav } from "../components/ChapterNav";  
import { SimulatorModal } from "../components/hints/SimulatorModal";  
import { LiteGuide } from "../components/LiteGuide";  
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {  
  if (typeof window === "undefined") return { lite: false };  
  const q = new URLSearchParams(window.location.search);  
  const lite = q.has("lite") || q.get("view") === "lite";  
  return { lite, topic: q.get("topic") ?? undefined };  
}
```

```
export default function App() {  
  const liteMode = useMemo(readLiteMode, []);
```

```

    setSidebarMode((m) => {
      const next = m === "wide" ? "compact" : m === "co
      window.localStorage.setItem("sidebarMode", next);
      return next;
    });
  }, []));
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.in
  };
  apply();
  window.addEventListener("resize", apply);
  return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
  0,
  chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[i

const goTo = useCallback((id: string) => {
  setActiveChapterId(id);
  window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
  setCompilerOpen(true);
  requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
  {/* Сайдбар только на десктопе — на мобильном
  <div
    id="sidebar-root"
    className={`hidden lg:block shrink-0 border
      -all duration-200 print:hidden ${sidebarCol
    " : "lg:w-56"}` }
  >
    {!sidebarCollapsed && (
      <Sidebar selectedChapterId={activeChapterId}
    )}
  </div>
  {/* Вкладка-переключатель содержания у левого
  <button
    type="button"
    onClick={toggleSidebar}
    className="hidden lg:flex items-center gap-1
    border-slate-700 bg-slate-900/90 px-1 py-3
    aria-label={sidebarMode === "wide" ? "Сжать
    ание"}
    title={sidebarMode === "wide" ? "Сжать содер
    }
  >
    {sidebarCollapsed ? <PanelLeftOpen className="
    <span className="text-[10px] font-bold [wri
  </button>

  <main id="main-content" className="flex-1 min
    {/* Шапка главы с быстрыми переходами */}
    <div className="flex items-center justify-b
      <div className="min-w-0">
        <div className="text-[10px] uppercase t
          {activeChapter.category || "Универсальн
        </div>
        <h2 className="text-base sm:text-xl font
          {activeChapter.title}
        </h2>
      </div>
    </div>

```

```
        onHeightChange={setCompilerHeight}
        onClose={() => setCompilerOpen(false)}
      />
    ))
  </div>
);
}
```

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

/** Числовые константы TreeWalker: глобального `NodeFile
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
        const surface = m[1].toLowerCase();
        const id = labelToId.get(surface);
        if (!id) continue;

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
        perTerm.set(id, usedTerm + 1);
        perSurface.set(surface, usedSurface + 1);

        if (m.index > last) frag.appendChild(document.createElement("span"));
        const span = document.createElement("span");
        span.className = "term-hint";
        span.dataset.termId = id;
        span.tabIndex = 0;
        span.setAttribute("role", "button");
        span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
        span.textContent = m[1];
        frag.appendChild(span);
        last = m.index + m[1].length;
    }

    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createElement("span"));
    textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
  box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
  box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
  box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
  box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}
```

```
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
    color: #a5b4fc;
    border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
    text-decoration: none;
    border-radius: 3px;
    padding: 0 1px;
    cursor: pointer;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
    background: rgba(99, 102, 241, 0.25);
    color: #fff;
    outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}
```



```
    line-height: 1.3 !important;
  }
  .chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
  }
  .chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
  }
  .chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
  }
  .chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
  }
  .chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
  }
}
```

```
/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}
```

```
/* Индикатор длительности кадра в пошаговых визуализаци  
@keyframes demoProgress {
```

```
main {
  padding: 0 !important;
  max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
  color: #111827 !important;
  background-color: transparent !important;
  background-image: none !important;
  box-shadow: none !important;
  text-shadow: none !important;
  border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
  background-color: #f3f4f6 !important;
  color: #111827 !important;
  font-size: 9pt;
}

article.chapter-body .prose table {
  border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
  border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
  page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {
  id: string;
  title: string;
  type: "html" | "markdown";
  content: string;
  description?: string;
  category?: string;
}
```

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
      <p class="text-sm font-mono text-slate-300">
    ] [k] </p>
  </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины <span class="font-mono text-emerald-300">
">i</span>. Любой отрезок накрывается <b>двумя
перация <b>идемпотентна</b>: <span class="font-mono text-emerald-300">min</span>, <span class="font-mono text-emerald-300">
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-indigo-300 italic">
  <p class="text-xl font-mono text-white">
  <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрыва
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N</span>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-slate-700">
    <p class="text-[10px] uppercase tracking-wider">
```

```

        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Аналогия 2: Торт-пирамидка
        </div>
        <p class="text-slate-300 text-sm mb-4">
            ия, зато красивый, как пирамидка: каждый из
            сех блоков  $2^{kx} \times 2^{ky}$ ), и ответ на запрос
            ой операции нет: крем обратно в торт не запы
             $n(x, x) = x$ ), и цена  $O(1)$  – сто коржей вмес
            <div class="rounded-lg bg-slate-950">
                <div><span class="text-slate-500">
                     $n \ O(1)$ </span></div>
                <div><span class="text-slate-500">
                    = 100 копий матрицы</span></div>
            </div>
            <p class="text-[12px] text-slate-400">
                шает). Менять элементы тоже нельзя. Сумму ч
            </div>
        </div>
        <p class="text-slate-400 text-sm mt-4">Поря
        ерению) → <b>2D: построение</b> (уровни <span
        ос</b> (прямоугольник раскладывается на чет
        </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">
    <p class="text-slate-300 text-sm mb-4">Эти
        за <span class="font-mono text-emerald-300">
        они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
    <p class="text-sm font-bold text-white">
    <p class="text-slate-300 text-sm mb-3">
        цы, у разреженной таблицы – размер, умножен
        <table class="w-full text-xs text-left">
            <thead>
                <tr class="text-slate-500 border">
                    <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-1" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; vertical-align: middle;">P[i][j]</p>
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-1">ум на его части не вычислишь. Приходится хранить $\log N \times \log M$ на ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: **лицу** – перекрывающиеся блоки посчитают **лицу** – у минимума нет обратной операции. Стр

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-amber-400">

<div class="bg-rose-950/30 p-5 rounded-lg border">

<ul class="list-disc pl-5 space-y-2 text">

Сумма через разреженную таблицу префиксные (билет 5).

Память. $N = M = 1000$ даём N и M , считайте ячейки, потом пишите код.

Один log вместо двух. В $\log N$ и $\log M$ st[0][0] st[0][1] st[1][0] st[1][1] st[2][0] st[2][1] st[3][0] st[3][1] st[4][0] st[4][1] st[5][0] st[5][1] st[6][0] st[6][1] st[7][0] st[7][1] st[8][0] st[8][1] st[9][0] st[9][1] st[10][0] st[10][1] st[11][0] st[11][1] st[12][0] st[12][1] st[13][0] st[13][1] st[14][0] st[14][1] st[15][0] st[15][1] st[16][0] st[16][1] st[17][0] st[17][1] st[18][0] st[18][1] st[19][0] st[19][1] st[20][0] st[20][1] st[21][0] st[21][1] st[22][0] st[22][1] st[23][0] st[23][1] st[24][0] st[24][1] st[25][0] st[25][1] st[26][0] st[26][1] st[27][0] st[27][1] st[28][0] st[28][1] st[29][0] st[29][1] st[30][0] st[30][1] st[31][0] st[31][1] st[32][0] st[32][1] st[33][0] st[33][1] st[34][0] st[34][1] st[35][0] st[35][1] st[36][0] st[36][1] st[37][0] st[37][1] st[38][0] st[38][1] st[39][0] st[39][1] st[40][0] st[40][1] st[41][0] st[41][1] st[42][0] st[42][1] st[43][0] st[43][1] st[44][0] st[44][1] st[45][0] st[45][1] st[46][0] st[46][1] st[47][0] st[47][1] st[48][0] st[48][1] st[49][0] st[49][1] st[50][0] st[50][1] st[51][0] st[51][1] st[52][0] st[52][1] st[53][0] st[53][1] st[54][0] st[54][1] st[55][0] st[55][1] st[56][0] st[56][1] st[57][0] st[57][1] st[58][0] st[58][1] st[59][0] st[59][1] st[60][0] st[60][1] st[61][0] st[61][1] st[62][0] st[62][1] st[63][0] st[63][1] st[64][0] st[64][1] st[65][0] st[65][1] st[66][0] st[66][1] st[67][0] st[67][1] st[68][0] st[68][1] st[69][0] st[69][1] st[70][0] st[70][1] st[71][0] st[71][1] st[72][0] st[72][1] st[73][0] st[73][1] st[74][0] st[74][1] st[75][0] st[75][1] st[76][0] st[76][1] st[77][0] st[77][1] st[78][0] st[78][1] st[79][0] st[79][1] st[80][0] st[80][1] st[81][0] st[81][1] st[82][0] st[82][1] st[83][0] st[83][1] st[84][0] st[84][1] st[85][0] st[85][1] st[86][0] st[86][1] st[87][0] st[87][1] st[88][0] st[88][1] st[89][0] st[89][1] st[90][0] st[90][1] st[91][0] st[91][1] st[92][0] st[92][1] st[93][0] st[93][1] st[94][0] st[94][1] st[95][0] st[95][1] st[96][0] st[96][1] st[97][0] st[97][1] st[98][0] st[98][1] st[99][0] st[99][1] st[100][0] st[100][1] st[101][0] st[101][1] st[102][0] st[102][1] st[103][0] st[103][1] st[104][0] st[104][1] st[105][0] st[105][1] st[106][0] st[106][1] st[107][0] st[107][1] st[108][0] st[108][1] st[109][0] st[109][1] st[110][0] st[110][1] st[111][0] st[111][1] st[112][0] st[112][1] st[113][0] st[113][1] st[114][0] st[114][1] st[115][0] st[115][1] st[116][0] st[116][1] st[117][0] st[117][1] st[118][0] st[118][1] st[119][0] st[119][1] st[120][0] st[120][1] st[121][0] st[121][1] st[122][0] st[122][1] st[123][0] st[123][1] st[124][0] st[124][1] st[125][0] st[125][1] st[126][0] st[126][1] st[127][0] st[127][1] st[128][0] st[128][1] st[129][0] st[129][1] st[130][0] st[130][1] st[131][0] st[131][1] st[132][0] st[132][1] st[133][0] st[133][1] st[134][0] st[134][1] st[135][0] st[135][1] st[136][0] st[136][1] st[137][0] st[137][1] st[138][0] st[138][1] st[139][0] st[139][1] st[140][0] st[140][1] st[141][0] st[141][1] st[142][0] st[142][1] st[143][0] st[143][1] st[144][0] st[144][1] st[145][0] st[145][1] st[146][0] st[146][1] st[147][0] st[147][1] st[148][0] st[148][1] st[149][0] st[149][1] st[150][0] st[150][1] st[151][0] st[151][1] st[152][0] st[152][1] st[153][0] st[153][1] st[154][0] st[154][1] st[155][0] st[155][1] st[156][0] st[156][1] st[157][0] st[157][1] st[158][0] st[158][1] st[159][0] st[159][1] st[160][0] st[160][1] st[161][0] st[161][1] st[162][0] st[162][1] st[163][0] st[163][1] st[164][0] st[164][1] st[165][0] st[165][1] st[166][0] st[166][1] st[167][0] st[167][1] st[168][0] st[168][1] st[169][0] st[169][1] st[170][0] st[170][1] st[171][0] st[171][1] st[172][0] st[172][1] st[173][0] st[173][1] st[174][0] st[174][1] st[175][0] st[175][1] st[176][0] st[176][1] st[177][0] st[177][1] st[178][0] st[178][1] st[179][0] st[179][1] st[180][0] st[180][1] st[181][0] st[181][1] st[182][0] st[182][1] st[183][0] st[183][1] st[184][0] st[184][1] st[185][0] st[185][1] st[186][0] st[186][1] st[187][0] st[187][1] st[188][0] st[188][1] st[189][0] st[189][1] st[190][0] st[190][1] st[191][0] st[191][1] st[192][0] st[192][1] st[193][0] st[193][1] st[194][0] st[194][1] st[195][0] st[195][1] st[196][0] st[196][1] st[197][0] st[197][1] st[198][0] st[198][1] st[199][0] st[199][1] st[200][0] st[200][1] st[201][0] st[201][1] st[202][0] st[202][1] st[203][0] st[203][1] st[204][0] st[204][1] st[205][0] st[205][1] st[206][0] st[206][1] st[207][0] st[207][1] st[208][0] st[208][1] st[209][0] st[209][1] st[210][0] st[210][1] st[211][0] st[211][1] st[212][0] st[212][1] st[213][0] st[213][1] st[214][0] st[214][1] st[215][0] st[215][1] st[216][0] st[216][1] st[217][0] st[217][1] st[218][0] st[218][1] st[219][0] st[219][1] st[220][0] st[220][1] st[221][0] st[221][1] st[222][0] st[222][1] st[223][0] st[223][1] st[224][0] st[224][1] st[225][0] st[225][1] st[226][0] st[226][1] st[227][0] st[227][1] st[

```

        else
            st[r][c][kx][ky] = min(
    
```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 &lt;&lt; ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 &lt;&lt; kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);
    
```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальный квадрат. Но.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 ≈ 100
- Префиксная сумма для той же матрицы

Почему? Потому что для каждого из нескольких сотен на сторону), а запросов очень много.

- [Префиксная сумма](#) – вторая половина этого сравнения:
- [Префиксная сумма](#)

```
<div class="text-center my-6">
  <p class="text-slate-400 text-xs uppercase">
    <p class="my-1 font-bold leading-none">
      <span class="text-2xl text-slate-300">Й</span>
    </p>
  <p class="text-slate-300 font-mono text-sm">
  </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Split – таможня
    </div>
    <p class="text-slate-300 text-sm">С
    х». Спуск от корня: узел, который вместе с
    скрытия</b> – досматривается только одна ветвь
  </div>
```

```
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
      Аналогия 2: Merge – стыковка
    </div>
    <p class="text-slate-300 text-sm">М
    аг – одно сравнение: чей у у корня больше,
    сохраняет порядок х). Итого  $O(h)$  сравнений
  </div>
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border">
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
    быстрее  $n \cdot \log n$ .</p>
  <p class="text-slate-400 text-sm mb-1">
  <p class="text-slate-400 text-sm">• «У6
```



```
-----
        <summary class="p-4 font-bold text-slate-700 border-slate-700/50">
            Код: вставка, merge, erase (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded">
def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key: # равные x — левый
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
    root = insert(root, key, pri)</pre>
        <pre class="bg-slate-950 p-4 rounded">
def merge(a, b): # все x в a < все x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else: root = merge(root.left, root.right)
    return root</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "splay-tree",
        title: "4. Splay-дерево",
    },
}
```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево  $\beta$  (между  $x$  и  $p$ ) – единств
 $x$ , и под  $p$ .</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

      /  \ \
     x    y
    /  \ \
   α    β
      /  \ \
     x    y
    /  \ \
   α    β

      x выше
      -----> α
      /  \ \
     p    r
    /  \ \
   β    γ

      p выше
      <-----
      /  \ \
     p    r
    /  \ \
   β    γ
  </pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
    амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x):    3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x):    3 × Zig-Zig    + Zig → высота 8
бамбук из 16 узлов:      наивно 16 → 16, Zig-Zig 16 → 10<
```

```
<p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем <b>три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути</b>. Отсюда амортизация: дорогой за
дом потенциала: потенциал  $\Phi$  = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
    <p class="text-slate-300 text-sm mb-2">
    <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
    <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
    <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

        rotate_up(x)
    else:                                     # ZIG-ZAG: НИЖНЯЯ
        rotate_up(x)
        rotate_up(x)
    return x

def find(root, key):                         # поиск: спуск + s
    cur, last = root, None
    while cur:
        last = cur
        if key == cur.key: break
        cur = cur.left if key < cur.key else cur.right
    return splay(last) if last else None</pre>
    <p class="text-xs text-slate-400">В
        Одна строка – и вся амортизация.</p>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сумма подматрицы за O(1): 1D и 2D, дв
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>

    <div class="space-y-8">

        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-emerald-4

```

```

        <td class="py-2 pr-3 font-mono">
      </td>
    </tr>
  </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
  ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-rose-300 t
    <p class="text-[11px] text-slate-500
  </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-blue-400 m
  <div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800
      <div class="absolute -top-3 left-4 l
      rder border-emerald-500 shadow-md">
        Аналогия 1: Сетка из вафельных
      </div>
      <p class="text-slate-300 text-sm mb
    о накопилось от угла (0,0) до меня». Хочешь
    у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
  Тонкая: одна матрица
</div>
  <p class="text-slate-300 text-sm mb
одной матрицы (плюс рамка из нулей). Никаки
прямоугольника от угла. Всё остальное полу
</div>
  <div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
  Не умеет: всё, что нельзя вычес
</div>
  <p class="text-slate-300 text-sm mb
максимума или gcd вычитания не существует:
е запросы – это <a class="ticket-link" data
платит она за это памятью <span class="fon
</div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
  <p class="text-sm font-bold text-white"
  <table class="w-full text-xs text-left"
    <thead>
      <tr class="text-slate-500 borde
        <th class="py-1.5 pr-2 font
        <th class="py-1.5 pr-2 font
        <th class="py-1.5 pr-2 font
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate
        <td class="py-2 pr-3 font-b
        <td class="py-2 pr-3 font-m
        <td class="py-2 pr-3 font-m
      </tr>
      <tr class="border-b border-slate

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код: 1D (тот же, что выполняет пане
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [3, 1, 4, 1, 5, 9, 2, 6]  # исходный ма
n = len(a)  # префиксов будет n + 1
P = [0]  # P[0] = 0 – пустой преф
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
    <p class="text-slate-300 text-sm">П
  - часть демонстрации: лента подсвечивает, из
    </div>
  </details>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код: 2D – построение и запрос прямо
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = []  # рамка из нуле
for rr in range(n + 1):
    S.append([0] + [0] * m)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2  # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

    },
    {
      id: "graph-planar-colors",
      title: "7. Графы. Планарные. Покраска",
      type: "html",
      description: "Формула Эйлера и теорема о 4 красках.",
      category: "Графы. Теория",
      content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Графы. Планарные. Покраска
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
        <p class="text-xl font-mono text-white">
          4. Если граф планарный, то его можно раскрасить 4 цветами.
        </p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">
            . Границы не пересекаются в одной точке по-
            ы не сливались цветами, Всегда можно всего 4
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "top-sort",

```



```

    </div>
</section>`,
  },
  {
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Граф называется сильно связным, если из любой вершины можно добраться до любой другой вершины по directed edge.
        <p class="text-xl font-mono text-white">Граф можно разбить на компоненты сильной связности (SCC), инвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением
          </div>
          <p class="text-slate-300 text-sm mb-4">Граф, состоящий из вершин, расположенных в 2D-плоскости, с directed edge в 8-и направлениях по односторонним улицам. Как это выглядит на карте.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },

```

```

        description: "Вершины, удаление которых убивает свя
        category: "Графы. Продвинутые",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
          теи > 1).</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия 1: Центральный роутер
          </div>
          <p class="text-slate-300 text-sm mb-4">
            правильный роутер - и два сегмента офиса б
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
d-длина.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия: Копаем туннель под .
        </div>
        <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-4

```

```

        которое торчит из посещенных в непосещенных
    </div>
    <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                ✦ Аналогия: Заражение вирусом
            </div>
            <p class="text-slate-300 text-sm mb-2">
                ли вирус). Мы стартуем из одного города, и
                Сеть растет только из одного связного куска
            </p>
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "mst-boruvka",
        title: "20. Графы. Остовное дерево. Борувка",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
                    <h3 class="text-xl font-bold text-emerald-400">
                        <div class="bg-slate-900 p-4 rounded-lg mb-4">
                            <p class="text-lg text-emerald-300 italic">
                                <p class="text-xl font-mono text-white">
                                    ается с соседом.</p>
                                </div>
                            <div class="grid grid-cols-1 gap-6">

```

```

        rder border-emerald-500 shadow-md">
            Аналогия 1: Бумеранг (Префикс
        </div>
        <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

    </div>
</section>`,
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description: "",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded">24</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Пример: сортировка чисел</p>
        <p class="text-base sm:text-lg font-mono">
          <b>P</b>: класс decision-задач, решаемый за полиномиальное время.
          <b>NP</b>: класс decision-задач с сертификатом.
        </p>
      </div>
      <p>
        <b>Сведение (Reduction) A ≤<sub>p</sub> B</b>: задача A сводится к задаче B.
      </p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          Аналогия 1: Судоку (P vs NP)
        </div>
        <p class="text-slate-300 text-sm mb-4">Пример сортировки чисел. Класс <b>NP</b> – задача нахождения решения (например, полную сетку (ответ)), он БЫСТРО (за класс P) решается.</p>
      </div>
    </div>
  </div>
</section>`
  }
]

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"

  <div class="bg-slate-900 p-4 rounded-lg mb-4"
    <p class="text-lg text-blue-300 italic"
    <div class="text-left font-mono text-sm"
      <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
      <p><strong class="text-white">Эйлер
      <div class="p-3 bg-slate-950 rounded"
        <p class="text-sm text-blue-300"
        <p><strong class="text-white">П
        <p><strong class="text-white">Ц
      </div>
    </div>
  </div>
</div>

<p class="text-slate-300 text-sm">
  <strong>Билет 13 (Различие пути и цикла
  <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
  <br>• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
  <div class="bg-slate-800 p-6 rounded-lg bor"
    <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
      Аналогия 1: Путь – из дома в бар,
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Ты вышел из дома <strong>(нечётная)
      талый пришёл в бар <strong>(вторая нечётная
      Домой вернуться не смог, потому что
    </p>

```

```

    </div>
</section>`,
  },
  {
    id: "bridges-code",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Мосты через DFS Тарьяна: tin/low, усл",
    category: "Графы. Продвинутые",
    content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-
  <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-400">Мосты

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
        <div class="text-left font-mono text-sm">
          <p><strong class="text-white">tin[v
          <p><strong class="text-white">low[v
          <div class="p-3 bg-slate-950 rounded">
            <span class="text-rose-400 font
            <p class="text-xl font-bold tex
            <p class="text-xs text-slate-400
          </div>
        </div>
      </div>
    </div>
  </div>

  <p class="text-slate-300 text-sm">
    Если из сына u и всех его потомков <strong>
    Единственная ниточка. Порвётся – граф р
  </p>
</div>

```



```
-----
<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n;                // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем
    for (int to : adj[v]) {
        if (to == p) continue;          // не идём назад
        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                   // рекурсивный в
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
}
```

```

        <p><span class="text-emerald-400 font-
        <p><span class="text-rose-400 font-l
    </div>
</div>

<p class="text-slate-300 text-sm">
    <strong>Следствие:</strong> Для планарн
ode>.
    Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bord
        <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
            K5: Полный граф на 5 вершинах
        </div>
        <p class="text-slate-300 text-sm mb-4">
            У K5: <code class="text-white">V=5,
-white">10 &gt; 9</code> – БАХ, непланарен!
            Все 5 вершин соединены со всеми. На
            Теорема Куратовского: K5 и K3,3 – д
            Если внутри графа спрятано <b>подра
            – плоской раскладки не будет.
        </p>
    </div>

<div class="bg-slate-900 p-6 rounded-lg bord
    <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
        K3,3: 3 дома и 3 колодца
    </div>
    <p class="text-slate-300 text-sm mb-4">
        Три дома хотят соединиться с тремя
        Нарисовать без пересечений невозможно
        У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
```

```
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left"
  <thead>
```

```
<tr class="text-slate-500 border"
  <th class="py-2 pr-3 font-b
  <th class="py-2 pr-3 font-b
  <th class="py-2 font-bold">
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ 
а свободен</td></tr>
```

```
<tr class="border-b border-slate-
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

        веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код: жадная раскраска и проверка
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше  $\Delta + 1$  цвета
color = [-1] * n                # -1 = ещё не покрашен
for v in order:                 # порядок влияет на чи
    used = set()
    for to in graph[v]:
        if color[to] != -1:
            used.add(color[to]) # цвета уже покрашены
    c = 0
    while c in used:             # минимальный свободны
        c += 1
    color[v] = c
chi = max(color) + 1            #  $\chi$ , который выдал жад

# Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
    if col[start] != -1:
        continue
    col[start] = 0
    dq = deque([start])
    while dq:
        v = dq.popleft()
        for to in graph[v]:
            if col[to] == -1:
                col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
```

```
  <strong>Важно:</strong> Это работает то-
  еплятся к точкам сочленения. То есть, <strong>
  ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
  <div class="bg-slate-800 p-6 rounded-lg border-
```

```
    <div class="absolute -top-3 left-4 bg-slate-800
    rder-rose-500 shadow-md">
```

```
      Аналогия: Главный перекресток
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb-4">
```

```
      Представь город, где два района соединены
      её перекроют (удалят вершину) — районы будут
      g>хрупкость</strong> системы.
```

```
    </p>
```

```
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
  <div class="absolute -top-3 left-4 bg-emerald-500
  er border-emerald-500 shadow-md">
```

```
    Телеком: Центральный свитч
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb-4">
```

```
    У тебя несколько компьютеров в одной комнате
    абелем (мостом). Сами свитчи — это точки со-
```

```
  </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
  <summary class="p-4 font-bold text-slate-400 border-
  order-slate-700/50">
```

```
    Душный мод: Код и корень DFS (Скрыто)
```

```
  </summary>
```

```

        <ul class="list-disc list-inside text-sm text-rose-400">
            <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
            <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
            <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
        </ul>
    </div>
</div>
</section>`,
    },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { ticket1Segtree } from "./tickets/ticket1_segtr
import { tickets18to20Mst } from "./tickets/ticket18_20
import { chapters as newChapters } from "./content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
  if (!toRemove.includes(c.id)) {
    dedup.set(c.id, c);
  }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
  newChapters.forEach((c) => {
    if (!toRemove.includes(c.id)) dedup.set(c.id, c);
  });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  // Главы без номера билета (введение) стоят ДО билетов
  const numOf = (t: string) => {
    const m = t.match(/(\d+)/);
    return m ? parseInt(m[0], 10) : -1;
  };
  return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
  "segment-trees": [1],
  "sparse-table": [2],
  treap: [3],
  "splay-tree": [4],
  "prefix-sums-2d": [5],
  "graph-dfs-bfs": [6],
  "planarity-euler-formula": [7],

```

```

-----
const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
  "euler-path-vs-cycle", "bridges-code", "planarity-euler
  "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
  "dijkstra", "bellman-ford", "floyd", "aho-corasick",
  "mst", "string-kmp", "string-z-func", "segment-trees", "s
  "dynamic-programming", "johnson-algo", "graph-articulat
  "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 212 | РАЗМ

```

-----
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
  /** Номер строки в коде пользователя (1-based). */
  line: number;
  /** Имя функции-фрейма ("<module>" – верхний уровень)
  func: string;
  /** Локальные переменные: имя → герг (усечённый), для
  locals: Record<string, string>;
  /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
    pass

def __safe(v):
    try:
        r = repr(v)
    except Exception:
        r = "<...>"
    return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
    """Небольшой JSON-снимок: UI использует его без парсера"""
    if v is None or isinstance(v, (bool, int, str)):
        return v
    if isinstance(v, float):
        return v if v == v and v not in (float("inf"), float("-inf")) else "inf/-inf"
    if depth >= 4:
        return __safe(v)
    limit = 80 if depth < 2 else 32
    if isinstance(v, (list, tuple)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, (set, frozenset)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, dict):
        # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
        # предел 80 незаметно обрезал уровни 4x4 и 8x8
        dict_limit = 192 if depth == 0 else limit
        return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
    # deque, heap-подобные и другие итерируемые учебные
    if type(v).__name__ == "deque":
        return [__snapshot(x, depth + 1) for x in list(v)]
    return __safe(v)

```

```

# PEP 709: с 3.12 компрекеншны встроены во фреймворк
# на своей строке каждую итерацию. Склеиваем по
if __steps and __steps[-1]["line"] == frame.f_lineno:
    __steps[-1]["locals"] = loc
    __steps[-1]["values"] = values
else:
    __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
if len(__steps) >= __CAP:
    # Нельзя просто отключить trace: бесконечный цикл
    # тогда продолжит работать вечно. Мягко прерываем
    __was_truncated = True
    raise __TraceLimit()
# Финальные локалы программы (после последней строки)
if event == "return" and frame.f_code.co_filename == __FILE__:
    loc, values = __capture(frame)
    __final.update(loc)
    __final_values.update(values)
return __tracer

__err = ""
try:
    __sys.settrace(__tracer)
    __init_globals = {
        "__name__": "__main__",
        "A": [5, 8, 3, 12, 7, 2, 0, 0],
        "a": [5, 8, 3, 12, 7, 2, 0, 0],
    }
    exec(compile(__src, "<user-code>", "exec"), __init_globals, __init_globals)
except __TraceLimit:
    pass
except Exception:
    import traceback as __tb
    __err = __tb.format_exc(limit=3)
finally:
    __sys.settrace(None)

__OUT = __json.dumps({
    "steps": __steps,

```

```
    const rest = Object.entries(cur)
      .filter(([k]) => !inSync.has(k))
      .sort(([a], [b]) => a.localeCompare(b));
    return [...syncEntries, ...rest];
  }
}
```

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 639 | РАЗМ

```
import type { DemoKind } from "../components/hints/MiniI

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мо
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
  id: string;
  /** Варианты написания в тексте (регистр не важен). */
  labels: string[];
  title: string;
  /** Объяснение «на пальцах» – 1–2 фразы. */
  short: string;
  /** Строгая формулировка / что важно не забыть. */
  formal?: string;
  complexity?: string;
  /** Какая мини-анимация показывается в карточке. */
  demo?: DemoKind;
  /** id полного симулятора (см. vizRegistry) – кнопка
  simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
```

```

{
  id: "heap",
  labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
  title: "Куча (priority queue)",
  short:
    "Не отсортированный список, а «мешок, из которого можно извлечь элемент с минимальным значением»",
  formal:
    "Полное бинарное дерево в массиве: дети узла  $i$  лежат в  $2i$  и  $2i+1$  местах",
  complexity: "вставка/извлечение  $O(\log n)$ , «поиск минимума»  $O(1)$ ",
  demo: "heap",
  simulator: "heap-beam-search",
},
{
  id: "stack",
  labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
  title: "Стек (LIFO)",
  short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
  formal: "push / pop / top за  $O(1)$ . На стеке живёт рекурсия",
  demo: "stack",
  simulator: "stack-dfs",
},
{
  id: "queue",
  labels: ["очередь", "очереди", "очередью", "FIFO"],
  title: "Очередь (FIFO)",
  short: "Очередь в столовой: кто первым встал, тот первым и ест",
  formal: "push_back / pop_front за  $O(1)$ . Основа BFS",
  demo: "queue",
  simulator: "queue-bfs",
},
{
  id: "dsu",
  labels: ["DSU", "СНМ", "система непересекающихся множеств"],
  title: "DSU — система непересекающихся множеств",
  short:
    "«Кто твой староста?» Каждый элемент знает своего старосту",
  formal:
    "find со сжатием путей + union по рангу/размеру. Массив parent",
  demo: "dsu",
  simulator: "dsu",
},

```

```

    },
    {
      id: "suffix-link",
      labels: ["суффиксная ссылка", "суффиксные ссылки",
        title: "Суффиксная ссылка",
        short:
          "«Куда откатиться, если следующая буква не подошла",
        строки.",
        formal: "link(v) = go(link(parent(v)), ch). Считает
        demo: "suffix-link",
        simulator: "aho-corasick",
    },
    {
      id: "toposort",
      labels: ["топологическая сортировка", "топсорт", "топ",
        title: "Топологическая сортировка",
        short:
          "Порядок «сначала штаны, потом ботинки»: каждая с",
        циклов.",
        formal: "DFS + вершины в обратном порядке выхода, л",
        complexity: "O(V + E)",
        demo: "toposort",
        simulator: "top-sort",
    },
    {
      id: "relaxation",
      labels: ["релаксация", "релаксацию", "релаксации",
        title: "Релаксация ребра",
        short:
          "«А через соседа не быстрее?» Если известный путь",
          ,
        formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
        ан и Флойд.",
        demo: "relax",
        simulator: "dijkstra",
    },
    {
      id: "prefix-sum",

```

```

    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается",
    formal: "Ребро  $(u,v)$  — мост  $\iff \text{low}[v] > \text{tin}[u]$ , где  
ребро в родителя — нет).",
    complexity: " $O(V + E)$  одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "то"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
    formal: "Для не-корня:  $v$  — точка сочленения  $\iff \exists$  ребро",
    demo: "articulation",
    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная",
    formal: " $V-1$  ребро, никаких циклов. Краскал (жадно  
но).",
    demo: "mst",
    simulator: "mst",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизир",
        "амортизированное", "амортизированного", "амортизир"
    ]
}
```

```

    formal: "Косарайю: DFS с сохранением порядка выхода",
    complexity: "O(V + E)",
    demo: "scc",
    simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функ",
        "префикс", "префикса", "суффиксом", "суффикс", "с",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция  $\pi$  (КМП)",
    short:
        "Для каждой позиции запоминаем: какой кусок начался",
        "от кусок.",
    formal:
        " $\pi[i]$  — длина наибольшего собственного префикса  $s$ ",
        "возвращая указатель по тексту.",
    complexity: "O(n + m)",
    demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "z-function",
    labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функция"],
    title: "Z-функция",
    short:
        "Для каждой позиции: сколько символов подряд, начинающихся",
        "с каждой буквы.",
    formal:
        " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i..n]$ ",
    complexity: "O(n)",
    demo: "prefix-function",
    simulator: "string-z-func",
},
{
    id: "dp",

```

```

    labels: [
        "жадный алгоритм", "жадный", "жадно", "жадная",
        "самое дешевое ребро", "самое дешёвое ребро", "са
    ],
    title: "Жадный алгоритм",
    short:
        "На каждом шаге хватаем то, что лучше прямо сейчас",
    formal:
        "Корректность обычно доказывается «леммой о разре
    demo: "mst",
    simulator: "mst",
},
{
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова
    title: "Эйлеров путь и цикл",
    short:
        "Пройти по каждому ребру ровно один раз, не отрыв
    formal:
        "Связный граф: эйлеров цикл    все степени чётные;
    simulator: "euler-path-vs-cycle",
},
{
    id: "planarity",
    labels: ["планарный", "планарность", "планарного",
    title: "Планарность",
    short:
        "Граф планарен, если его можно нарисовать на листе
    ,
    formal:
        " $V - E + F = 2$  для связного планарного графа. Кур
    ,
    simulator: "planarity-euler-formula",
},
{
    id: "splay",
    labels: [
        "splay", "splay-дерево", "AVL", "AVL-дерево", "вс

```



```

    title: "Алгоритм Форда–Беллмана",
    short:
        "Тупой, но честный:  $V-1$  раз проходим по ВСЕМ рёбрам",
    formal:
        "После  $k$ -й итерации гарантированно верны все кратчайшие пути. Если найдётся отрицательный цикл, то не существует кратчайшего пути.",
    complexity: " $O(V \cdot E)$ ",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
        "промежуточная вершина", "промежуточную вершину",
    ],
    title: "Флойд–Уоршелл: разрешаем вершину  $k$ ",
    short:
        "Внешний цикл – это не «шаг», а «какие пересадки сделать, чтобы сделать пересадку в  $k$ ?",
    formal:
        " $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$ . Порядок вершин не важен.",
    complexity: " $O(V^3)$  времени,  $O(V^2)$  памяти",
    demo: "floyd",
    simulator: "floyd",
},
{
    id: "prim",
    labels: ["Прима", "Прим", "Prim"],
    title: "Алгоритм Прима",
    short:
        "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
    formal:
        "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
    complexity: " $O(E \log V)$ ",

```

```

    "Считаются одним проходом: пока есть непосещённая
complexity: "O(V + E)",
demo: "components",
simulator: "graph-dfs-bfs",
},
{
  id: "graph",
  labels: [
    "граф", "графа", "графе", "графы", "графов", "графа",
    "вершины", "вершина", "вершин", "ребра с весом",
  ],
  title: "Граф",
  short:
    "Точки (вершины) и связи между ними (рёбра). Города,
  formal:
    "G = (V, E). Ребро бывает ориентированным (улицы с односторонним движением),
  demo: "graph",
  simulator: "graph-dfs-bfs",
},
{
  id: "coord-compress",
  labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
  title: "Сжатие координат",
  short:
    "Координаты бывают гигантские и дробные, а нам нужно хранить только
    порядковым номером.",
  formal:
    "sort + unique + binary search: значение → индекс",
  complexity: "O(n log n)",
  demo: "coord-compress",
},
{
  id: "treap",
  labels: ["Treap", "декартово дерево", "декартова деревья"],
  title: "Treap = Tree + Heap",
  short:
    "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету."

```

```

    short:
        "«Я не умею решать A, зато умею B». Если A можно
formal:
        "A  $\leq$  B: есть полиномиальная функция, переводящая
        NP-полную задачу к новой.",
demo: "np",
},
{
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
    title: "Запрос на отрезке",
    short:
        "Спрашиваем не про один элемент, а про кусок массива
        предподсчёты.",
    formal:
        "Суммы — префиксные суммы  $O(1)$ . Минимум без измене
demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "с
    title: "Строка как массив символов",
    short:
        "Строковые алгоритмы держатся на трёх вопросах: г
        (Z-функция) и где встречаются слова словаря (бо
    formal:
        "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функци
demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворо
    title: "Поворот (rotation) в дереве",
    short:
        "Локальная перевеска двух узлов: ребёнок встаёт на
        дившемся месту. Порядок ключей при этом не лом

```

```
      demo: "zig-zag",
      simulator: "splay-tree",
    },
  ];

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  e.labels.map((label) => ({ label, id: e.id }))
).sort((a, b) => b.label.length - a.label.length);
```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 520 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
  {
    id: "intro",
    title: "Введение. Базовая база: куча, очередь, бинарное дерево",
    type: "html",
    category: "Введение",
    content: `<section id="intro-content" class="mb-12">
  <div class="flex items-center mb-6 flex-wrap gap-2">
    <span class="bg-indigo-600 text-white px-4 py-1">Введение</span>
    <h2 class="text-3xl font-bold text-white">Базовая база</h2>
  </div>

  <div class="bg-slate-900 border border-indigo-500/40 p-6">
    <p class="text-slate-200 text-sm leading-relaxed">
      <b class="text-indigo-300">Базовая база</b> – это структура данных, которая позволяет хранить и
      управлять данными. В этой главе мы рассмотрим, как реализовать
      базовую базу с помощью массива, кучи, очереди и стека, бинарного
      дерева как массив, куча, очередь и стек, бинарное дерево.
      Кодим по минимуму: там, где за нас уже написали.</p>
    </div>
  </section>`
  }
];
```

```

        <td class="p-3 font-bold text-white">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 font-mono text-emerald-500">
    </tr>
    <tr class="hover:bg-slate-800/50 bg-slate-800/50">
        <td class="p-3 font-bold text-white">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 text-slate-500 italic">
    </tr>
    <tr class="hover:bg-slate-800/50">
        <td class="p-3 font-bold text-white">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 font-mono text-emerald-500">
    </tr>
    <tr class="hover:bg-slate-800/50 bg-slate-800/50">
        <td class="p-3 font-bold text-white">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 font-mono text-emerald-500">
    </tr>
    <tr class="hover:bg-slate-800/50">
        <td class="p-3 font-bold text-white">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 border-r border-slate-800">
        <td class="p-3 text-slate-500 italic">

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-indigo-400 mb
  <p class="text-slate-300 text-sm mb-4">Дерево н
    единицы, то родственников выдаёт простая ари
    чейках <span class="font-mono text-emerald-300
    <span class="font-mono text-emerald-300">n /
    па.</p>
    <pre class="bg-slate-950 border border-slate-700
tree = [None, 40, 20, 60, 10, 30, 50, 70]    # tree[0] н
#      корень n=1
#      дети 40:  2*1=2 → 20    и    2*1+1=3 → 60
#      дети 20:  2*2=4 → 10    и    2*2+1=5 → 30
#      родитель 30: 5 // 2 = 2 → 20</pre>
  <p class="text-slate-300 text-sm mb-4">Цена: пу
    > деревья – кучу (ниже) и дерево отрезков из
    emerald-300">2n+1</span> и <span class="font-m
    ">(n-1)//2</span>.</p>
  <details class="bg-slate-900 border border-slate
    <summary class="cursor-pointer px-4 py-2 text
    <div class="px-4 pb-4 text-slate-300 text-sm
      На уровне <span class="font-mono text-emera
      /span> вершин, а перед ним суммарно <span c
      class="font-mono text-emerald-300">k</span>
      lass="font-mono text-emerald-300">n</span>
      0">2n</span> и <span class="font-mono text-
      де, включая heapq.
    </div>
  </details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-emerald-400 mb
  <p class="text-slate-300 text-sm mb-4"><b>Куча
    ель ≤ своих детей. Отсюда магия: минимум всег
    минимума стоят <span class="font-mono text-e
    елей/детей. Полностью сортировать кучу не нуж
    <pre class="bg-slate-950 border border-slate-700

```

```

<p class="text-slate-300 text-sm mb-4">Писать с
o text-emerald-300">O(n log n)</span> в худшем
анных данных. Внутри он ищет уже упорядоченные
вставками на короткихранах. Сортировка <b>ус
<pre class="bg-slate-950 border border-slate-700
sorted([3, 1, 2])          # [1, 2, 3] – новый с
sorted([(1, "6"), (1, "a")], key=lambda t: t[0])  # ра
<p class="text-slate-300 text-sm">Что лежит вну
отвечают: слияние двух ранов длины k стоит O(
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-l
<h3 class="text-xl font-bold text-purple-400 mb
<p class="text-slate-300 text-sm mb-4"><b>AVL-д
>каждой</b> вершины выполнено условие баланса
class="font-mono text-emerald-300">|h_L – h_R
зывают фактором баланса; из условия следует в
авка и удаление за O(log n) даже в худшем слу
<pre class="bg-slate-950 border border-slate-700
Малый поворот (цепочка вправо-вправо стрелочками):

```

```

      z           x
    L<gt; x      =>   L<gt; y
      L<gt; y      z   y

```

Зигзаг (вправо-влево): сначала малый поворот нижнего из
потом малый поворот верхнего – две перерисовки стрелок:

```

      z           z           x
    L<gt; x      =>   L<gt; x      =>   L<gt; y
      L<gt; y      L<gt; y      z   y

```

```

<p class="text-slate-300 text-sm mb-4">Поворот
родителя y среднего звена и перевесить одно
отов вдоль пути вставки, и условие <span clas
<details class="bg-slate-900 border border-slate
<summary class="cursor-pointer px-4 py-2 text
<div class="px-4 pb-4 text-slate-300 text-sm
x = right(z)<br/>

```

```

<div class="absolute -top-3 left-4 bg-slate-500 border-emerald-500 shadow-md">
  Аналогия 2: Турнирная сетка
</div>
<p class="text-slate-300 text-sm">Куча матчей, не нужно переигрывать все матчи. Снял показавшихся.</p>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-emerald-500">
  <h3 class="text-xl font-bold text-indigo-400 mb-2">База закрыта
  <p class="text-slate-300 text-sm leading-relaxed">База закрыта — теперь ею пользуются остальные. Кратчайшие пути живут в билетах 14–16 (<a data-goto="string-edges-code"
    class="text-indigo-300 hover:text-white underline font-semibold cursor-pointer">Дейкс</a>),
    остовные деревья — в билетах 18–20 (<a data-goto="string-edges-code"
    class="text-indigo-300 hover:text-white underline font-semibold cursor-pointer">Краскал, Прим, Бор</a>),
    обходы и связность — в билетах 6–13 (<a data-goto="string-edges-code"
    class="text-indigo-300 hover:text-white underline font-semibold cursor-pointer">КМП</a>),
    строки — в билетах 21–23 (<a data-goto="string-edges-code"
    class="text-indigo-300 hover:text-white underline font-semibold cursor-pointer">КМП</a>),
    Начинать удобно с <a data-goto="graph-dfs-bfs" class="text-indigo-300 hover:text-white underline font-semibold cursor-pointer font-bold">билета
  </p>
</div>

</div>
</section>
},
{

```



```

        охождение улицы), Дейкстра сломается, так как
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border-1
    <summary class="p-4 font-bold text-slate-300
    -b border-slate-700/50">
        Строгое доказательство (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <p>Алгоритм использует приоритетную очередь
    g V). Гарантирует корректность только для неориентирован
    </div>
</details>

    <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
    id: "bellman-ford",
    title: "Билет 15. Форд-Беллман",
    type: "html",
    category: "Графы",
    content: `<section id="bellman-ford-content" class="
    <div class="flex items-center mb-6">
        <span class="bg-blue-600 text-white px-4 py-1 rounded
        <h2 class="text-3xl font-bold text-white">Форд-Беллман
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-1
            <h3 class="text-xl font-bold text-rose-400">Описание
        </div>

        <div class="bg-slate-900 p-4 rounded-lg mb-6">
            <p class="text-lg text-rose-300 italic">Алгоритм
            <p class="text-xl font-mono text-white">Алгоритм

```

```
    <h2 class="text-3xl font-bold text-white">Флойд
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-indigo-400">Аналогия 1
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-indigo-300 italic">Аналогия 1: Автомагистрали
      <p class="text-xl font-mono text-white">
    </div>
```

```
  <div class="grid grid-cols-1 xl:grid-cols-2"
    <!-- Аналогия 1 -->
```

```
    <div class="bg-slate-800 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
        Аналогия 1: Автомагистрали
      </div>
```

```
      <p class="text-slate-300 text-sm mb-4">
        евле долететь из Москвы в Париж, если мы сд
      <div id="slot-chapter-image-floyd"
    </div>
```

```
    <!-- Аналогия 2 -->
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
```

```
        Время работы
      </div>
      <p class="text-slate-300 text-sm mb-4">
        три вложенных цикла. Сложность  $O(V^3)$ . Если
    </div>
```

```
</div>
```

```
  <div id="slot-floyd-viz" class="mt-8"></div>
```

```
</div>
</div>
```

```

<div class="bg-slate-900 p-6 rounded-lg border-2"
  <div class="absolute -top-3 left-4 bg-rose-900
    -500 shadow-md">
    Аналогия 2: Матёшка (Терминальные)
  </div>
  <p class="text-slate-300 text-sm mb-4">Представ
    нашли и "he", потому что оно спрятано внутри
    терминальные ссылки</b> (term_link) – прямые мос
  <p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
  <summary class="p-4 font-bold text-slate-400 outl
    pen:border-slate-700/50">
    Внутренняя структура автомата и код BFS (Скры
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <p>Каждая вершина имеет таблицу переходов <code>
      ссылку <code>term_link</code> (ближайшая мен
    <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
    map<char, int> go;          // Предвычисленные пер
    int link = 0;              // Суффиксная ссылка: куда
    int term_link = 0;         // Терминальная ссылка: матр
    bool is_terminal = false;
    vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {
        nodes[child].link = 0;
        q.push(child);
    }
}

```

```
      <div class="bg-slate-700/50 p-6 rounded-xl border" style="border-color: #4b4b9b; border-radius: 12px; border-width: 2px; margin: 10px 0 10px 10px;">
        <h3 class="text-xl font-bold text-purple-400">Вопрос
        <p class="text-slate-300 text-sm mb-4">Когда могут быть большими дробными числами (долгота вычисления) значения  $v_i$  для  $i$  от 0 до N, и уже по ним строим графы.</p>
      </div>
    </div>
  </section>
}
];
```

ФАЙЛ 23/111: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 200 | РАЗМЕР: 1.5 КБ

```
export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
  explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> = {
  "sparse-table": [
    {
      question: "Сколько памяти занимает двумерная разреженная матрица размера N x M?",
      options: [
        "N x M — столько же, сколько сама матрица",
        "N x M x log N x log M: на каждом уровне почти N x M элементов",
        "N x M x 4 — по одной ячейке на каждый угол каждого элемента",
      ],
      correctIndex: 1,
      explanation: "Уровней log N по строкам и log M по столбцам. В сумме примерно 10 x 10 = 100 копий — ровно такова сложность алгоритма.",
    },
  ],
};
```

```

        "N × M – как исходная матрица, плюс рамка из нулей",
        "N × M × log N × log M",
        "N + M: одна строка и один столбец"
    ],
    correctIndex: 0,
    explanation: "Сумма накапливается: каждая клетка в матрице  
здесь как раз разреженная таблица (билет 2)",
},
{
    question: "Запрос суммы прямоугольника (r1,c1)...(r2,c2)",
    options: [
        "S[r2][c2] - S[r1][c2] - S[r2][c1] + S[r1][c1]",
        "S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]",
        "S[r2+1][c2+1] - S[r1][c1]"
    ],
    correctIndex: 1,
    explanation: "С рамкой из нулей S[i][j] – это сумма элементов  
внутри квадрата (или прямоугольника) с вершинами (0,0) и (i,j).  
вариант без +1 даёт ответ со сдвигом на клетку – (i,j-1)",
},
{
    question: "Почему левый верхний угол в формуле присутствует?",
    options: [
        "Он входит и в левую, и в верхнюю полосу – его надо считать",
        "Он всегда равен нулю, поэтому прибавление безопасно",
        "Так компенсируется рамка из нулей"
    ],
    correctIndex: 0,
    explanation: "Отрезая полосу слева и полосу сверху, мы теряем  
баланс: это и есть формула включений-исключений.",
},
{
    question: "Можно ли тем же приёмом считать минимум?",
    options: [
        "Да, если заменить вычитание на min",
        "Нет: у минимума нет обратной операции, нужна разность",
        "Да, но только для неотрицательных чисел"
    ],
    correctIndex: 1,

```

```

    {
      question: "Куда ведёт эйлеров путь, если нечётных",
      options: [
        "Всё ещё можно пройти по всем рёбрам ровно раз",
        "Такого графа не существует",
        "Эйлерова пути нет — нужно удалять лишние рёбра",
      ],
      correctIndex: 2,
      explanation: "Если нечётных вершин больше 2, эйлеров",
        "ровно 2 нечётные или 0."
    }
  ],
  "bridges-code": [
    {
      question: "После DFS у нас  $low[u] = 4$ , а  $tin[v] =$ ",
      options: [
        "Да, так как  $low[u] (4) > tin[v] (2)$ ",
        "Нет,  $low[u]$  должно быть меньше или равно",
        "Только если граф связный и неориентированный"
      ],
      correctIndex: 0,
      explanation: "Условие моста:  $low[u] > tin[v]$ . В н",
    },
    {
      question: "Для чего нужен массив  $low[v]$ ?",
      options: [
        "Для того чтобы было больше переменных в коде",
        "Чтобы знать минимальный  $tin$  предка, до которого",
        "Чтобы хранить глубину рекурсии"
      ],
      correctIndex: 1,
      explanation: " $low[v]$  хранит минимальное время входа",
        "поиска мостов и точек сочленения."
    },
    {
      question: "Какова временная сложность алгоритма по",
      options: [
        " $O(V^2)$  — ищем мосты в квадрате",

```

```
    }  
  ]  
};
```

ФАЙЛ 24/111: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 292 | РАЗМ

```
import { createContext, useContext, useEffect, useState } from "react";  
import type { DebugValue } from "../debugRunner";
```

```
/**
```

```
 * Прямая связь «компилятор ↔ открытая визуализация».
```

```
 *
```

```
 * Компилятор передаёт не «магический номер строки эталонного
```

```
 * выполненного Python-кода. Визуализация читает захардкоженные
```

```
 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие
```

```
 * любой пользовательский код работает, если в нём использованы
```

```
 */
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-state";
```

```
export interface VizStateEventDetail {
```

```
  chapterId: string;
```

```
  /** Строка и функция реально выполненного пользователем кода */
```

```
  line?: number;
```

```
  func?: string;
```

```
  /** Машинно-читаемые globals + locals: числа остаются строками */
```

```
  variables?: Record<string, DebugValue>;
```

```
  /** Имена, изменённые выделенной строкой. */
```

```
  changed?: string[];
```

```
}
```

```
/** Последний снимок хранится, чтобы поздно смонтировать визуализацию
```

```

    }, [chapterId]));

    return runtime;
}

export type VizStepSelector = (variables: Record<string>

/**
 * Подписка старых покадровых демонстраций. Если переда
 * номер кадра выбирается по именам/значениям переменны
 */
export function useVizStepSync(
    currentStep: number,
    goToStep: (n: number) => void,
    maxStep: number,
    selectByVariables?: VizStepSelector
) {
    const chapterId = useContext(VizChapterContext);

    useEffect(() => {
        if (!chapterId) return;

        const apply = (detail?: VizStateEventDetail) => {
            if (!detail || detail.chapterId !== chapterId) re
            // Без явного селектора компонент либо рисует use
            // либо остаётся ручным. Номер Python-строки нико
            const requested =
                selectByVariables && detail.variables ? selectB
            // null означает: компонент рисует runtime-перемен
            // не увидел знакомых захардкоженных имён.
            if (requested === null || !Number.isFinite(request
            const target = Math.max(0, Math.min(Math.max(0, m
            if (target !== currentStep) goToStep(target);
        };

        apply(latestByChapter.get(chapterId));
        const handler = (e: Event) => apply((e as CustomEvent
        window.addEventListener(CHANNEL, handler);
    }
}

```



```

at: (i: number, j: number) => number | null;
/** Подпись размера для бейджа связи: «5x5». */
sizeLabel: string;
/** true, если хотя бы одна ячейка – число. */
hasNumbers: boolean;
}

const MAX_MATRIX_SIDE = 64;

const toCellNumber = (value: DebugValue | undefined): number | null {
  if (typeof value === "number" && Number.isFinite(value)) return value;
  // float("inf") снапшотится строкой "inf" – показываем, что это не число
  if (typeof value === "string") {
    const t = value.trim().toLowerCase();
    if (t === "inf" || t === "+inf" || t === "infinity" || t === "-inf" || t === "-infinity") return Number.POSITIVE_INFINITY;
    const n = Number(t);
    return Number.isFinite(n) ? n : null;
  }
  return null;
};

/** Ключ-пара вида "(0, 3)" или "0, 3" → [0, 3]. */
const parsePairKey = (key: string): [number, number] | null {
  const m = key.match(/^(\s*(-?\d+)\s*,\s*(-?\d+)\s*)$/);
  if (!m) return null;
  return [Number(m[1]), Number(m[2])];
};

export function liveMatrix(value: DebugValue | undefined): Matrix {
  if (value === undefined || value === null) return null;

  const cells = new Map<string, number>();
  let rows = 0;
  let cols = 0;
  const put = (i: number, j: number, v: DebugValue | undefined) => {
    if (!Number.isInteger(i) || !Number.isInteger(j) || i < 0 || j < 0 || i >= MAX_MATRIX_SIDE || j >= MAX_MATRIX_SIDE) return;
    cells.set(`${i},${j}`, toCellNumber(v));
  };
  const get = (i: number, j: number): number | null => {
    if (!Number.isInteger(i) || !Number.isInteger(j) || i < 0 || j < 0 || i >= MAX_MATRIX_SIDE || j >= MAX_MATRIX_SIDE) return null;
    return cells.get(`${i},${j}`);
  };
  return {
    rows,
    cols,
    put,
    get,
    hasNumbers: cells.size > 0,
    sizeLabel: `${rows}x${cols}`,
  };
}

```

```

    }
    const innerRecord = vizRecord(cell);
    if (innerRecord) {
      for (const [k2, v2] of Object.entries(innerRecord)) {
        const j = Number(k2);
        if (Number.isInteger(j)) put(i, j, v2);
      }
    }
  }
}

if (rows === 0 || cols === 0) return null;
return {
  rows,
  cols,
  at: (i, j) => (i >= 0 && j >= 0 && i < rows && j < cols),
  sizeLabel: `${rows}x${cols}`,
  hasNumbers: cells.size > 0,
};
}

/** Первое имя из списка, которое реально есть в снимке
export function pickLiveVar(
  vars: Record<string, DebugValue> | undefined,
  names: string[]
): { name: string; value: DebugValue } | null {
  if (!vars) return null;
  for (const name of names) {
    if (Object.prototype.hasOwnProperty.call(vars, name)) {
      return { name, value: vars[name] };
    }
  }
  return null;
}

/** Индекс, ограниченный реальной размерностью данных
export const clampIndex = (value: number | null, size: number): number {
  if (value === null || !Number.isFinite(value) || size <= 0) {
    return null;
  }
  return Math.max(0, Math.min(size - 1, Math.trunc(value)));
}

```

ФАЙЛ 25/111: src/data/vizSync.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 1566 | РАЗМЕР: 100 КБ

```
/**
 * Захардкоженная «карта синхронизации» компилятора с визуализацией
 *
 * Для каждой страницы (главы) здесь вручную перечислены переменные,
 * которые понимает её визуализация: i, j, k, v, dist, ...
 * Компилятор выполняет любой Python-код и передаёт результат
 * имён напрямую; привязки к номерам или тексту строк эмулируются
 */

export interface SyncVariable {
  /** Имя переменной, как в коде визуализации: i, j, k, ... */
  name: string;
  /** Роль в алгоритме (показывается во всплывающей подсказке) */
  role: string;
  /** Характерный диапазон/значение на демонстрации. */
  range?: string;
}

export interface PageSync {
  /** Заголовок демонстрации на странице (если интерактивная) */
  vizTitle?: string;
  /** Чему соответствует один шаг визуализации (для тултипа) */
  stepNote?: string;
  /** Переменные синхронизации. Пусто – на странице нет переменных */
  variables: SyncVariable[];
  /** Полный референсный Python-код, который кнопка-глава запускает */
  code?: string;
}

/** Справочные значения, указанные внутри самих симуляторов */
export const PAGE_SYNC: Record<string, PageSync> = {
  "segment-trees": {
    vizTitle: "Дерево отрезков",
    stepNote: "Шаг = спуск по вершине v: либо читаем tree[v], либо записываем",
  },
}
```

```

def __init__(s, x, y):
    s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
    """Декартово дерево из отсортированных по x точек:
    pts = sorted(pts, key=lambda p: p[0])
    stack = []
    for x, y in pts:
        cur = Node(x, y)
        last = None
        while stack and stack[-1].y < y:    # поднимаемс
            last = stack.pop()
        cur.l = last
        if stack:
            stack[-1].r = cur
        stack.append(cur)
    return stack[0] if stack else None

def split(t, m):
    """(L, R): в L всё с x <= m, в R — с x > m."""
    global step
    if t is None:
        return (None, None)
    step += 1
    if t.x <= m:
        a, b = split(t.r, m)
        t.r = a
        return (t, b)
    a, b = split(t.l, m)
    t.l = b
    return (a, t)

def merge(a, b):
    """Слияние: все x в a меньше всех x в b."""
    global step
    if a is None: return b
    if b is None: return a
    step += 1

```

```

    { name: "n", role: "число узлов = число пар", gap: 100,
      { name: "mid", role: "разрез для вкладки Split: в
      { name: "key", role: "какую точку удаляет вкладка
      { name: "step", role: "номер шага спуска/слияния
      { name: "height", role: "высота дерева: в среднем
      { name: "root", role: "корень – узел с максимальн
    ],
  },
  "splay-tree": {
    vizTitle: "Splay: Zig / Zig-Zig / Zig-Zag на узлах
    stepNote: "Шаг = одно применение случая: переменная
      rota.",
    code: `# Splay-дерево: три случая поворота на узлах
x, p, g = None, None, None      # узел, родитель, дед (о
xk, pk, gk = 0, 0, 0           # их ключи-числа: 20 / 4
case = "zig"                   # какой случай сработал:
step = 0                       # номер поворота
root = None                    # корень текущего дерева

# --- тело алгоритма ---
class N:
    def __init__(s, k):
        s.k = k; s.l = None; s.r = None; s.p = None

def rot(x):                    # один поворот: поднять
    p = x.p
    g = p.p if p else None
    if p and p.l is x:
        p.l = x.r
        if x.r: x.r.p = p
        x.r = p
    elif p:
        p.r = x.l
        if x.l: x.l.p = p
        x.l = p
    if p: p.p = x
    x.p = g
    if g:

```

```

splay(root.l.r)`,
    variables: [
        { name: "case", role: "какой случай сработал: \"Zig-Zag\"",
          a" },
        { name: "step", role: "номер применения случая — 1 или 2"},
        { name: "xk / x", role: "ключ узла, который поднимается"},
        { name: "pk / p", role: "ключ родителя x перед поворотом"},
        { name: "gk / g", role: "ключ деда — нужен в Zig-Zag"},
        { name: "root", role: "корень дерева после расшейвления"},
    ],
},
"sparse-table": {
    vizTitle: "Разреженная таблица (1D и 2D)",
    stepNote: "Шаг = одна ячейка st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])",
    code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
    st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])

for row in st:
    print(row) # готовая таблица: None там, где не надо

# запрос min на отрезке за O(1): отрезок покрывают ДВА блока
L, R = 1, 6
K = (R - L + 1).bit_length() - 1
print("min на [" + str(L) + ", " + str(R) + "] =",
      min(st[L][K], st[R - (1 << K) + 1][K]),
      "| блоки длины", 1 << K)`,
    variables: [
        { name: "a", role: "исходный массив, из которого строится таблица"},
        { name: "n", role: "длина массива a", range: "n = 1..100"},
        { name: "LOG", role: "число уровней таблицы: log2 n"},
    ],
}

```

```

    r1 = fib(k - 1)
    r2 = fib(k - 2)
    memo[k] = r1 + r2
    return memo[k]

print("итог:", fib(n))`,
    variables: [
        { name: "n", role: "целевой номер числа Фибоначчи"},
        { name: "k", role: "аргумент текущего рекурсивного"},
        { name: "memo", role: "кэш уже посчитанных значений"},
        { name: "r1, r2", role: "ответы двух рекурсивных"},
    ],
},
"heap-beam-search": {
    vizTitle: "Куча (priority queue)",
    stepNote: "Шаг = просеивание: вставленный элемент в
        .",
    code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
    variables: [
        { name: "n", role: "текущее число элементов в куче"},
        { name: "i", role: "индекс элемента, который сейчас"},
        { name: "(i-1)//2", role: "индекс родителя вершины"},
        { name: "2*i+1, 2*i+2", role: "индексы левого и правого"},
    ],
},
"stack-dfs": {
    vizTitle: "Стек и обход в глубину",
    stepNote: "Шаг = push новой вершины на стек или pop",
    code: `st = [] # стек = рекурсия
st.append("A") # push – зашли в вершину
st.append("B")
v = st[-1] # вершина на вершине стека:

```

```

        { name: "top / front", role: "верх стопки (стек)" },
        { name: "i", role: "индекс в массиве кучи; родите." },
    ],
},
"graph-dfs-bfs": {
    vizTitle: "DFS и BFS на графе",
    stepNote: "Шаг = переход из текущей вершины v к непосещенной",
    code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
visited, order = set(), []

stack = ["A"] # DFS: свой стек вместо рекурсии
while stack:
    v = stack.pop() # шаг: достали вершину с вершины
    if v in visited:
        continue
    visited.add(v); order.append(v)
    for u in adj[v]: # соседи v → на стек
        stack.append(u)
print("DFS:", order)

visited, order = set(), []
q = deque(["A"]) # BFS: очередь, обход по слоям
while q:
    v = q.popleft() # шаг: достали из начала очереди
    if v in visited:
        continue
    visited.add(v); order.append(v)
    for u in adj[v]: # соседи v → в конец очереди
        q.append(u)
print("BFS:", order)`
    variables: [
        { name: "v", role: "текущая вершина обхода", gang: 1 },
        { name: "to / u", role: "очередной сосед вершины v", gang: 1 },
        { name: "visited", role: "множество уже посещённых вершин", gang: 1 },
        { name: "order", role: "порядок, в котором обходился граф", gang: 1 },
    ],
},

```



```

        y", range: "-1 ... comps" },
        { name: "used", role: "булев вариант разметки — да/нет", range: "0 ... 1" },
        { name: "comps", role: "счётчик найденных компонент", range: "0 ... 1" },
        { name: "c", role: "номер компоненты, которую рассматриваем", range: "0 ... 1" },
        { name: "v", role: "вершина, где обход находится", range: "0 ... 1" },
        { name: "stack", role: "фронт обхода: стек (DFS)", range: "0 ... 1" },
        { name: "i / step", role: "номер шага — перематываем", range: "0 ... 1" }
    ],
},
"top-sort": {
    vizTitle: "Топологическая сортировка",
    stepNote: "Шаг = выход рекурсии из вершины v: она достигнута",
    code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2], 4: []}
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:
        # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)
    # выход из v!

for v in adj:
    if v not in used:
        dfs(v)

answer = order[::-1]
# разворот — топологический
for i, v in enumerate(answer):
    print(f"позиция {i}: вершина {v}")`,
    variables: [
        { name: "v", role: "текущая вершина DFS", range: "0 ... 1" },
        { name: "to", role: "вершина, куда ведёт ребро из v", range: "0 ... 1" },
        { name: "i", role: "перебор стартовых вершин во вложенном цикле", range: "0 ... 1" },
        { name: "order", role: "список выхода из рекурсии", range: "0 ... 1" }
    ],
},
"scc-kosaraju": {
    vizTitle: "Компоненты сильной связности (Косарайю)"

```

```

        continue
    stack = [v]
    comp[v] = c
    while stack:
        u = stack.pop()
        for to in adjT[u]:
            if to not in comp:
                comp[to] = c
                stack.append(to)
    print("SCC", c, "собрана из", v, ":", comp)
    c += 1

print("порядок выхода:", order, "| компонент:", c, "| р
variables: [
    { name: "adj / adjT", role: "граф и его транспози
    { name: "v", role: "текущая вершина DFS (в фазе 1
    { name: "to", role: "сосед по ребру: в фазе 1 пря
    { name: "used", role: "помеченные первым проходом
    { name: "order", role: "порядок ВЫХОДА из DFS пер
    { name: "i", role: "индекс прохода по order на вто
    { name: "stack", role: "стек DFS второго прохода:
    { name: "u", role: "вершина, снятая со стека во в
    { name: "comp", role: "номер компоненты сильной с
    { name: "c", role: "счётчик компонент: каждая нов
    { name: "n", role: "число вершин", range: "5" },
],
},
"graph-articulation": {
    vizTitle: "Точки сочленения",
    stepNote: "Шаг = обновление low[v] по ребру; вершин
    code: `tin, low, timer = {}, {}, 0

i = 0                # номер шага DFS
v, to = 0, 1         # текущее ребро DFS: v → to
tin[v] = tin.get(v, timer); low[v] = tin[v]

# шаг: возврат из to в v
i += 1

```

Универсальное пособие • полный исходный код

```
-----
deg = {v: len(adj[v]) for v in range(n)}    # степени вершин
odd = [v for v in range(n) if deg[v] % 2 == 1]    # нечётные
start = odd[0] if odd else 0                # путь начинаем с
used = [False] * len(edges)                # какие рёбра уже пройдены
path = [start]                              # маршрут: последовательность вершин
last = start                                # где мы сейчас
stack = [start]                             # стек для отката
v = start
step = 0
```

```
# --- тело алгоритма ---
```

```
while stack:
    v = stack[-1]
    moved = False
    for to, i in adj[v]:
        if not used[i]:
            used[i] = True
            stack.append(to)
            path.append(to)
            last = to
            step += 1
            moved = True
            break
    if not moved:
        stack.pop()                        # из v все рёбра пройдены
```

```
print("степени:", deg)
print("нечётных вершин:", len(odd), odd, "->", "цикл из 2 вершин")
print("маршрут:", path, "| рёбер пройдено:", sum(used), "из", len(edges))
```

```
variables: [
    { name: "path", role: "маршрут — последовательность вершин"},
    { name: "last / v", role: "вершина, где обход находится"},
    { name: "used", role: "какие рёбра уже пройдены (True/False)"},
    { name: "deg", role: "степени вершин: чётные всюду, нечётные в цикле"},
    { name: "odd", role: "список нечётных вершин — по ним начинаем"},
    { name: "stack", role: "стек отката алгоритма Иерархического"},
    { name: "step / i", role: "номер шага обхода", range(1, len(edges) + 1)}
],
```

```
},
```

```

        { name: "i / step", role: "номер шага – перематывае",
        { name: "V, E, F", role: "для вкладки «Планарности
          9" },
      ],
    },
    dijkstra: {
      vizTitle: "Дейкстра",
      stepNote: "Шаг = старт, извлечение ближайшей вершины",
      code: `import heapq

g = {"A": [( "B", 4), ( "C", 2)], "B": [( "G", 4), ( "D", 5)
    , "E": [( "F", 1)], "F": []}
dist = {"A": 0}
prev = {}
done = set()
pq = [(0, "A")]
while pq:
    du, u = heapq.heappop(pq)
    if u in done:
        continue
    done.add(u)
    print(f"обрабатываем {u} (dist={du})")
    for v, w in g[u]:
        if v in done:
            continue
        print(f"смотрим ребро {u}→{v} (w={w})")
        if du + w < dist.get(v, float("inf")):
            dist[v] = du + w
            prev[v] = u
            heapq.heappush(pq, (dist[v], v))
print("итог:", dist)`
      variables: [
        { name: "u", role: "вершина с минимальным dist ср",
        { name: "v", role: "сосед вершины u, до которого",
        { name: "w", role: "вес ребра (u, v)", range: "по",
        { name: "dist[v]", role: "лучшее известное рассто",
      ],
    },
  },

```

```

-----
    print("отрицательный цикл: улучшается ребро", cycle)
else:
    print("отрицательных циклов нет:", {k: dist[k] for
    variables: [
        { name: "n", role: "число вершин графа – ровно n–"},
        { name: "m", role: "число рёбер, по которым прохо"},
        { name: "i", role: "номер итерации (после i итера"},
        { name: "u, v, w", role: "текущее ребро: откуда,"},
        { name: "dist[v]", role: "расстояние; если обнов"},
    ],
},
floyd: {
    vizTitle: "Флойд–Уоршелл",
    stepNote: "Шаг = инициализация матрицы, смена проме",
    code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, IN
    NF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0
n = 7
for k in range(n):
    print(f"посредник k={k}")
    for i in range(n):
        for j in range(n):
            if i != j and i != k and j != k and d[i][k]
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны")`,
    variables: [
        { name: "n", role: "число вершин – размер квадрата"},
        { name: "k", role: "промежуточная вершина – ВНЕШНИ"},
        { name: "i", role: "начало пути – строка матрицы"},
        { name: "j", role: "конец пути – столбец матрицы"},
        { name: "dist[i][j]", role: "кратчайший путь i → j"},
        { name: "ни", role: "номер итерации"},
    ],
},
"johnson-algo": {
    vizTitle: "Алгоритм Джонсона (перевзвешивание)",

```

```

-----
for i, (w, a, b) in enumerate(edges):
    u, v = a, b
    ra, rb = find(a), find(b)
    if ra != rb:                                # разные компоненты
        parent[ra] = rb
        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
    variables: [
        { name: "n", role: "число вершин (A..I); в остов включены"},
        { name: "edges", role: "список рёбер (вес, u, v) -"},
        { name: "i", role: "номер текущего ребра в отсортированном"},
        { name: "u, v, w", role: "концы и вес ребра, который"},
        { name: "parent", role: "DSU: представитель множества"},
        { name: "rank", role: "высота дерева DSU – union by rank"},
        { name: "mst", role: "взятые рёбра остова – рисуем"},
        { name: "total", role: "вес остова; на этом графе"},
    ],
},
"string-kmp": {
    vizTitle: "Префикс-функция (КМП)",
    stepNote: "Шаг = обработка символа s[i]: откаты по префиксам",
    code: `s = "aabaabaaba"
n = len(s)
i, j = 1, 0
pi = [None] * n                # пустые клетки: значения префикс-функции
pi[0] = 0

for i in range(1, n):          # шаг: считаем pi[i]
    j = pi[i - 1]               # длина совпадения
    while j > 0 and s[i] != s[j]:
        j = pi[j - 1]          # откат назад
    if s[i] == s[j]:
        j += 1
    pi[i] = j

```

```

        овпадают с картинкой «водопада». ",
        code: `words = ["HERS", "HIS", "SHE", "HE"]    # обра
text = "USHERS"                                     # текст, в котор
n = len(text)                                       # длина текста
state = 0                                           # текущее состоян
i = 0                                               # индекс текущег
c = ""                                             # сам символ
nodes = 1                                           # сколько вершин
queue = []                                         # очередь BFS, к
fail = {}                                          # fail[v] = верш
matches = []                                       # найденные вхож

# --- тело алгоритма ---
nxt = {}                                           # nxt[(v, ch)] = u — переходы бора
term = {}                                          # term[v] = слово, которое заканчивае

def add_word(w):
    """Добавить образец в бор: недостающие вершины полу
    global nodes
    v = 0
    for ch in w:
        if (v, ch) not in nxt:
            nxt[(v, ch)] = nodes
            nodes += 1
        v = nxt[(v, ch)]
    term[v] = w

for w in words:
    add_word(w)

# BFS по уровням: fail-ссылка ребенка = куда приведёт f
fail[0] = 0
queue = sorted(v for (p, ch), v in nxt.items() if p == 0)
for v in queue:
    fail[v] = 0
qi = 0
while qi < len(queue):
    v = queue[qi]

```

```

    { name: "term", role: "в каких вершинах заканчивае",
    { name: "queue", role: "очередь BFS, которой нате",
    { name: "matches", role: "найденные вхождения: (и",
    { name: "nodes", role: "сколько вершин в боре (ко",
    ],
},
"complexity-classes": {
    vizTitle: "Классы сложности: рост по n и ops",
    stepNote: "Шаг = одна точка (n, ops): визуализация",
    code: `n = 1                                # размер входа
ops = 0                                         # сколько операций насчитал
cls = "P"                                     # класс, к которому относим
step = 0                                       # кадр схемы: 0 вход → 1 алг
k = 0                                         # длина сертификата: сколько
cert = []                                    # сам сертификат – предполага
ok = True                                    # проверился ли сертификат з

# --- тело алгоритма ---
# Полиномиальный алгоритм: перебор всех пар –  $\Theta(n^2)$ .
# Точки (n, ops) сами ложатся на график вкладки «Рост».
for n in range(2, 11):
    step = 1                                # запустили алгоритм на вход
    ops = 0
    for i in range(n):
        for j in range(i, n):
            ops += 1
    step = 2                                # ответ готов
    cls = "P"                               #  $n^2/2$  операций – полином, з
    print("n =", n, "| ops =", ops, "|", cls)

# Сертификат NP-задачи (3-SAT): решение угадано, провер
step = 3
cert = [1, 0, 1, 0]                        # значения переменных x1...x4
k = len(cert)                              # проверяем все k бит
ok = all(bit in (0, 1) for bit in cert)
cls = "NP"                                  # угадать + быстро проверить
print("сертификат:", cert, "| k =", k, "| принят:", ok,
    variables: [

```



```

        color[v] = c
        chi = max(chi, c)
        i += 1
        print("v =", v, "занято:", sorted(used), "-> цвет",
print("chi =", chi, "| delta =", delta, "| chi <= delta
        variables: [
            { name: "color", role: "цвета вершин (0 = не покр
            { name: "v", role: "текущая вершина", range: "0 ...
            { name: "delta / chi", role: "максимальная степен
            { name: "i / step", role: "номер кадра", range: "1
        ],
    },
    "planarity-euler-formula#bipartite": {
        vizTitle: "Двудольность: BFS в 2 цвета",
        stepNote: "Шаг = сосед u получает цвет 3 - color[v]
        code: `n = 8
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4],
color = [0] * n                # 0 = не покрашена, иначе 1
queue = []                     # очередь BFS
v = -1                          # вершина, из которой смотрим
u = -1                          # сосед, которого красим сей
ok = True                       # двудольный ли граф
i = 0

# --- тело алгоритма ---
for start in range(n):
    if color[start] != 0:
        continue
    color[start] = 1
    queue = [start]
    while queue and ok:
        v = queue.pop(0)
        for u in adj[v]:
            if color[u] == 0:
                color[u] = 3 - color[v]        # противополож
                queue.append(u)
                i += 1
            elif color[u] == color[v]:

```

```

variables: [
    { name: "ecolor", role: "цвета рёбер – рёбра на г",
    { name: "e", role: "номер текущего ребра", range:
    { name: "v, u", role: "концы текущего ребра – под
    { name: "inc", role: "для каждой вершины: какие ц
    { name: "chi_e / delta", role: "χ' – сколько цвет
],
},
"graph-components#dfs": {
    vizTitle: "Компоненты связности: DFS-покраска",
    stepNote: "Шаг = вершина v достаётся из стека; сосе
    code: `n = 10                                # вершин в графе
adj = [[1, 3], [0, 2, 3], [1, 3], [0, 1, 2], [5, 6], [4
comp = [-1] * n                                # -1 = вершина ещё не покраш
used = [False] * n                             # то же самое булевым массив
comps = 0                                       # счётчик найденных компонен
c = 0                                           # номер текущей компоненты
v = -1                                          # вершина, в которой обход н
stack = []                                     # фронт обхода (стек для DFS
i = 0                                           # номер шага – двигает покад

# --- тело алгоритма ---
for start in range(n):
    if comp[start] != -1:
        continue
    comps += 1                                # непокрашенная вершина => Н
    c = comps
    stack = [start]
    comp[start] = c
    used[start] = True
    v = start
    while stack:
        v = stack.pop()                       # для BFS: v = stack.pop(0)
        i += 1
        for to in adj[v]:
            if comp[to] == -1:
                comp[to] = c
                used[to] = True

```

```

        if rank[ra] == rank[rb]:
            rank[ra] += 1
        comps -= 1
        return True

for e, (a, b) in enumerate(edges):
    u, v = a, b
    step = e + 1
    union(a, b)
print("компонент:", comps)
print("корни:", [find(x) for x in range(n)])`,
    variables: [
        { name: "parent", role: "DSU-массив: parent[v] = 0 ... n-1" },
        { name: "rank", role: "высота дерева DSU – union" },
        { name: "e / step", role: "номер обрабатываемого ребра" },
        { name: "u, v", role: "концы текущего ребра – подмножества" },
        { name: "comps", role: "сколько множеств осталось" },
    ],
},
"complexity-classes#hierarchy": {
    vizTitle: "Иерархия классов  $P \subseteq NP \subseteq PSPACE \subseteq EXP$ ",
    stepNote: "Шаг не нужен: схема подсвечивает тот овал, который является подмножеством предыдущего",
    code: `cls = "NP" # имя класса: P, NP, PSPACE, EXP
n = 10 # размер входа
ops = 0 # счётчик операций (нужен вк. в шаблон)

# --- тело алгоритма ---
# P: решается за полином – сортировка
a = [5, 2, 8, 1, 9, 3]
for i in range(len(a)):
    for j in range(len(a) - 1):
        ops += 1
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
print(cls, "отсортировано за", ops, "операций:", a)`,
    variables: [
        { name: "cls", role: "имя класса – P, NP, PSPACE, EXP" },
    ],
},

```

```

        hit = False
        for l in cl:
            val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
            if val == 1:
                hit = True
        ok = ok and hit
        print("дизъюнкт", k + 1, cl, "выполнен" if hit else
print("сертификат", "принят" if ok else "отвергнут")`,
        variables: [
            { name: "cert", role: "сертификат – назначение пер"},
            { name: "k", role: "номер дизъюнкта, который пров"},
            { name: "ok", role: "итоговый вердикт: сертификат"},
            { name: "clauses", role: "сама формула 3-SAT – 4"},
        ],
    },
    "complexity-classes#growth": {
        vizTitle: "Рост числа операций: полином или экспонен",
        stepNote: "Шаг = одна точка (n, ops). Чем больше то",
        code: `n = 1                                # размер входа
ops = 0                                             # сколько операций насчитал

# --- тело алгоритма ---
# Полиномиальный алгоритм: перебор всех пар –  $\Theta(n^2)$ .
# Точки (n, ops) сами ложатся на график вкладки «Рост».
for n in range(2, 11):
    ops = 0
    for i in range(n):
        for j in range(i, n):
            ops += 1
    print("n =", n, "ops =", ops)

# Экспоненциальный (полный перебор подмножеств) – раскол
# for n in range(1, 15):
#     ops = 2 ** n
#     print("n =", n, "ops =", ops)`,
        variables: [
            { name: "n", role: "размер входа – по нему строит"},
            { name: "ops", role: "число операций алгоритма на"}

```

```

    { name: "u, v, w", role: "концы и вес рассматривае",
    { name: "parent", role: "DSU-массив: цвета компонент",
    { name: "mst", role: "рёбра, уже взятые в остов",
    { name: "total", role: "вес остова (51 на этом графе)"
    ],
},
"mst#prim": {
    vizTitle: "Остовное дерево: Прим (рост из вершины E",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

n = 9                                # вершин: A...I
adj = {"A": [("B", 2), ("D", 5)], "B": [("A", 2), ("C", 3),
      "C": [("B", 4), ("F", 9)], "D": [("A", 5), ("E", 4),
      "E": [("D", 3), ("B", 6), ("F", 7), ("H", 11)],
      "G": [("D", 8), ("H", 10)], "H": [("G", 10), ("E", 11),
      "I": [("H", 12), ("F", 13)]}]

start = "E"                          # из какой вершины растим дерево
heap = [(0, start, start)]           # (вес, вершина, откуда пришла)
taken = []                           # вершины уже в остовае – кра
mst = []                             # рёбра остова
total = 0                            # вес остова (= 51)
u, v, w = None, None, None          # текущее ребро

# --- тело алгоритма ---
while heap:
    w, x, frm = heapq.heappop(heap)   # самое лёгкое ребро
    if x in taken:                    # устаревшая запись
        continue
    taken.append(x)
    u, v = frm, x
    if frm != x:
        mst.append((w, frm, x))
        total += w
    for to, wt in adj[x]:
        if to not in taken:
            heapq.heappush(heap, (wt, to, x))
print("вес остова:", total, "| вершин захвачено:", len(taken))

```

```

        if ra not in cheapest or w < cheapest[ra][0]:
            cheapest[ra] = (w, a, b)
        if rb not in cheapest or w < cheapest[rb][0]:
            cheapest[rb] = (w, a, b)
    if not cheapest:
        break # граф несвязен
    for (w, a, b) in cheapest.values():
        ra, rb = find(a), find(b)
        if ra == rb:
            continue # две компоненты
        comp[ra] = rb
        mst.append((w, a, b))
        total += w
print("фаз:", phase, "| вес остова:", total)
print(mst)\,
    variables: [
        { name: "comp", role: "DSU: корень компоненты вер",
        { name: "cheapest", role: "корень компоненты -> с",
        { name: "phase", role: "номер фазы; на этом графе",
        { name: "u, v, w", role: "ребро-кандидат, которое",
        { name: "mst / total", role: "взятые рёбра и вес "
    ],
},
"sparse-table#2d-build": {
    vizTitle: "Разреженная таблица 2D: построение",
    stepNote: "Шаг = новый уровень k: блоки  $2^k \times 2^k$  доп",
    code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67,
        88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [5
        1, 42, 85, 14]]
r, c, k = 0, 0, 0
st2 = {} # пустой каркас виден до

# --- тело алгоритма ---
# База не отдельный уровень вычислений: блок 1x1 — это
# Записываем её одним шагом, а не проигрываем 64 одноти
# Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(((r, c, 0, 0), A[r][c]) for r in range(2
```

```

S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
    variables: [
        { name: "i, j", role: "текущая ячейка таблицы S (
        { name: "S[i][j]", role: "сумма прямоугольника (0
            },
        { name: "r1, c1, r2, c2", role: "углы запроса; от
    ],
},
};

```

```

/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null, c
    (demoId && chapterId && PAGE_SYNC[`${chapterId}#${demo
    (chapterId && PAGE_SYNC[chapterId]) ||
    { variables: [] } };

```

```

/**
 * Стартовое содержимое редактора для страницы.
 *
 * Формат: короткий комментарий-шапка + исполняемый реф
 * Визуализация читает из его трассы знакомые имена (i,
 * но точно так же примет пользовательский код с теми же
 */

```

```

export function buildSyncTemplate(chapterId: string, ch
    const sync = getPageSync(chapterId, demoId);

```

```

const head = sync.vizTitle
    ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .
    : `# «${chapterTitle}»\n# демо на странице нет – св

```

```

if (!sync.code) return head;

```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 26/111: src/data/tickets/ticket1_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМ

```
import { Chapter } from '../../../types';

/**
 * Билет 1 – «Дерево отрезков с операциями на отрезках»
 *
 * Страница написана по канону остальных билетов (эталон
 * одна главная секция, строгое правило по центру, анализ
 * явная строка сложности, весь код спрятан в <details>
 *
 * Визуализация – src/components/SegmentTreeVisualizer.ts
 * построение / запросы / теория), синхронизация с компонентом
 * (переменные n, i, v, l, r, m) и её подрежимы #build
 */
export const ticket1Segtree: Chapter[] = [
  {
    id: "segment-trees",
    title: "1. Дерево отрезков с операциями на отрезках",
    type: "html",
    description:
      "Структура для запросов на отрезке и обновлений значений",
    category: "Продвинутые структуры",
    content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
```



```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    Аналогия 1: Корпоративная презентация
  </div>
  <p class="text-slate-300 text-sm">Безопасность для всех департаментов: «внутри всем +5 %». Начиная с 2024 года, мы будем использовать только моно-шрифт: font-mono text-emerald-300>lazy[v]</span>) и поддерживать свой баланс. Это и есть <b>отложенное обновление</b>.
```

```

  </div>

  <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
      Аналогия 2: Матрёшки-коробки
    </div>
    <p class="text-slate-300 text-sm">Матрёшки-коробки. Нужно покрасить половину массива в синий? Нет, мы спускаемся только тогда, когда нас спросят.
    (push).</p>
  </div>
</div>

```

```

<div class="bg-slate-800/70 p-5 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <p class="text-sm font-bold text-white">Важно!</p>
  <p class="text-slate-300 text-sm mb-3">Например, на [L, R]» – всё это делается одним спуском.
  <b>ss="font-mono text-emerald-300">tree[v] += 1</b>: перед любым спуском в детей вызываем push.
  <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
    <thead>
      <tr class="text-slate-500 border-bottom">
        <th class="py-1.5 pr-2 font-normal">Левый</th>
        <th class="py-1.5 pr-2 font-normal">Правый</th>
        <th class="py-1.5 font-normal">Сумма</th>
      </tr>
    </thead>
    <tbody class="text-slate-300 font-mono">

```

```

      <tr>
        <td class="py-1.5 pr-2">L</td>
        <td class="py-1.5 pr-2">R</td>
        <td class="py-1.5">L+R</td>
      </tr>
    </tbody>
  </table>

```

в редакторах и базах, сжатие координат + «оить», дерево отрезков по дереву (HLD), пер

```
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
  -b border-slate-700/50">
```

Код: построение и запрос суммы (C

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<pre class="bg-slate-950 p-4 rounded
```

```
nt MAXN = 100005;
```

```
int tree[4 * MAXN];          // 4n – обязательно, 2n не х
```

```
int a[MAXN];
```

```
void build(int v, int l, int r) {
```

```
    if (l == r) { tree[v] = a[l]; return; }    // лист
```

```
    int m = (l + r) / 2;
```

```
    build(2 * v, l, m);                      // левая
```

```
    build(2 * v + 1, m + 1, r);              // права
```

```
    tree[v] = tree[2 * v] + tree[2 * v + 1]; // merge
```

```
}
```

```
int sum(int v, int l, int r, int L, int R) { // сумма
```

```
    if (r < L || R < l) return 0;           //
```

```
    if (L <= l && r <= R) return tree[v];   //
```

```
    int m = (l + r) / 2;
```

```
    return sum(2 * v, l, m, L, R)           // части
```

```
        + sum(2 * v + 1, m + 1, r, L, R);
```

```
}
```

```
void update(int v, int l, int r, int i, int x) { // a[i
```

```
    if (l == r) { tree[v] += x; return; }
```

```
    int m = (l + r) / 2;
```

```
    if (i <= m) update(2 * v, l, m, i, x);
```

```
    else update(2 * v + 1, m + 1, r, i, x);
```

```
    tree[v] = tree[2 * v] + tree[2 * v + 1]; // перес
```

```
}</pre>
```

</details>

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">

Код: итеративное дерево ($2n$, снизу)

</summary>

<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
 # для удобства

```
tree = [0] * (2 * n)
for i in range(n):                # листья: tree[n + i] = a[i]
    tree[n + i] = a[i]
for v in range(n - 1, 0, -1):    # предки снизу вверх
    tree[v] = tree[2 * v] + tree[2 * v + 1]
```

```
def query(l, r):                  # сумма на полуинтервале [l, r)
    res = 0
    l += n; r += n
    while l < r:
        if l & 1: res += tree[l]; l += 1    # l — правая граница
        if r & 1: r -= 1; res += tree[r]    # r — правая граница
        l //= 2; r //= 2
    return res
```

<p class="text-slate-400 text-xs">Иногда это решение кажется вам не самым лучшим, но на самом деле оно действительно заметно хитрее — на экзамене безопаснее использовать именно этот вариант.
 </div>

</details>

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">

Вопросы, которые любят задавать (и которые, к сожалению, часто остаются без ответа)

</summary>

<div class="p-5 text-sm text-slate-300">

<p>• Почему $4n$, а не $2n$? — при n , нечетном, количество листьев равно $n + 1$.
 пример: $n = 6$).</p>

<p>• Сколько вершин посещает запрос

```
<h3 class="text-xl font-bold text-emerald-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-emerald-300 italic">
    <p class="text-xl font-mono text-white">
  </div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейкстра  
ершины <span class="font-mono text-emerald-300">  
требование к графу — неотрицательные вес  
е, и на каждом шаге забирает в него ту неп  
n class="font-mono text-emerald-300">d[u]</
```

```
<p class="text-slate-300 text-sm mb-4">Почему  
ext-emerald-300">u</span> обязан выйти из о  
х</span>, а у неё <span class="font-mono text-emerald-300">  
бавляет</b> неотрицательные веса, значит кор  
ся механика: извлечь минимум → зафиксировать  
ниже.</p>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-slate-800">
    <p class="text-[10px] uppercase tracking-wider">
    <p class="font-mono text-emerald-300">
  </div>
  <div class="bg-slate-900 border border-slate-800">
    <p class="text-[10px] uppercase tracking-wider">
    <p class="font-mono text-emerald-300">
  </div>
  <div class="bg-slate-900 border border-slate-800">
    <p class="text-[10px] uppercase tracking-wider">
    <p class="font-mono text-emerald-300">
  </div>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4">
```



```
<h3 class="text-xl font-bold text-indigo-400">
```

```
<p class="text-slate-300 text-sm mb-4"><b>И  
pan> извлекается из кучи, <span class="font  
nt-mono text-emerald-300">s → u</span>. Отс  
00">dist[v] = δ(s, v)</span> для всех дости  
ext-emerald-300">dist[v] = ∞</span> для нед
```

```
<details class="bg-slate-900/40 rounded-lg l  
  <summary class="p-4 font-bold text-slate  
-b border-slate-700/50">» Доказательство ин
```

```
  <div class="p-5 text-sm text-slate-300  
    <p>Пусть <span class="font-mono tex  
<span class="font-mono text-emerald-300">di  
text-emerald-300">s → u</span> и пусть <sp  
не извлечена, а <span class="font-mono tex  
nt-mono text-emerald-300">y</span> не перва  
    <p>Так как <span class="font-mono t  
pan class="font-mono text-emerald-300">(x, y  
δ(s, x) + w(x, y) = δ(s, y)</span>. Путь д  
font-mono text-emerald-300">u</span>, веса  
&lt; dist[u]</span>.</p>
```

```
    <p>Итого <span class="font-mono tex  
text-emerald-300">u</span> – противоречие с  
0">u</span> нет. ■</p>
```

```
    <p class="text-slate-400 text-xs">Г  
u)</span>». С отрицательным ребром хвост п  
о контрпример из карточки «Ограничения».</p>
```

```
  </div>  
</details>
```

```
<details class="bg-slate-900/40 rounded-lg l  
  <summary class="p-4 font-bold text-slate  
-b border-slate-700/50">» Откуда берётся сл
```

```
  <div class="p-5 text-sm text-slate-300  
    <ul class="list-disc pl-5 space-y-2  
      <li><b>Куча.</b> Каждая вершина  
span class="font-mono">pop</span>), каждое
```

```

<tr class="align-top"><td class="t-mono">O(E + V · C)</td><td class="py-2">C
</tbody>
</table>
</div>

<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
    -b border-slate-700/50">» Код: Дейкстра на
  <div class="p-5 text-sm text-slate-300"
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300"
  INF = float("inf")

def dijkstra(graph, start):
    """graph[v] = [(сосед, вес), ...]; веса >= 0."""
    dist = {v: INF for v in graph}
    prev = {v: None for v in graph}
    dist[start] = 0
    done = set()
    pq = [(0, start)]                # (расстояние, вершина)

    while pq:
        du, u = heapq.heappop(pq)
        if u in done:                # устаревшая информация
            continue
        done.add(u)                  # dist[u] окончательное
        for v, w in graph[u]:
            if du + w < dist[v]:     # релаксация
                dist[v] = du + w
                prev[v] = u
                heapq.heappush(pq, (dist[v], v))
    return dist, prev

def restore(prev, start, target):
    if prev[target] is None and target != start:
        return None                 # вершина недосягаемая
    path = [target]

```

```

void dijkstra(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    priority_queue<pair<long long,int>,>
        vector<pair<long long,int>,>
        greater<pair<long long,int>,>
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue; // устарев
        for (auto [v, w] : g[u]) {
            if (du + w < d[v]) {
                d[v] = du + w;
                p[v] = u;
                pq.push({d[v], v});
            }
        }
    }
}

vector<int> path(int s, int t) { // восст
    vector<int> res;
    if (d[t] == INF) return res; // недостижим
    for (int v = t; v != -1; v = p[v]) res.push_back(v)
    reverse(res.begin(), res.end());
    return res;
}

```

<p class="text-slate-400 text-xs">0
 уть из 10^5 рёбер веса 10^9 даёт 10^{14} – <span
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">» Код: $O(V^2)$ без куд
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 ти $n \times n$, $w[i][j] = \text{INF}$ если ребра нет
 $d = [\text{INF}] * n$

С разными весами FIFO ломается, нужна куча

- «Одна пара».** Алгоритм счи

ий выход при извлечении цели (билет 17), но

ов ценой скорости $O(V^3)$; б
после перевзвешивания потенциалов). Предст

id: "bellman-ford",

title: "15. Графы. Поиск кратчайшего пути. Форд-Бел

type: "html",

category: "Графы. Пути",

content: `

<section id="bellman-ford-content" class="mb-12 scroll-m

<div class="flex items-center mb-6 flex-wrap gap-3">

<span class="bg-blue-600 text-white px-4 py-1 r

<h2 class="text-2xl sm:text-3xl font-bold text-r

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl bord

<h3 class="text-xl font-bold text-rose-400

<div class="bg-slate-900 p-4 rounded-lg mb-

<p class="text-lg text-rose-300 italic

<p class="text-xl font-mono text-white"

есть отрицательный цикл.</p>

</div>

<p class="text-slate-300 text-sm mb-4">Форд-

ьных весов. Плата – скорость: вместо <s

овно **1** выходной в году

, когда он отде-
 -ю ночь Бэтмен делает контрольный проезд: е-
 о-то района снова уменьшилось – значит, Джо
 </div>

```

    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
        Отрицательный цикл: временная п-
      </div>
      <p class="text-slate-300 text-sm mb-4">
        меньше: <span class="font-mono text-emerald-300">
        и ответ «-∞». Контрольная итерация – это де-
      .</p>
    </div>
  </div>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500/50">
  <h3 class="text-xl font-bold text-blue-400">
    <p class="text-slate-300 text-sm mb-4">Демон-
    ь 2 D, Лес E, Склад F) и 9 односторонних
    00">B → C = -2</span> (сплав по течению – б-
    >, которая в обычном режиме весит <span cla
    no text-emerald-300">-6</span>. В компиляторе
    span> внутри <span class="font-mono text-em

```

```

<div class="bg-slate-800/70 p-5 rounded-lg"
  <p class="text-sm font-bold text-white">
  <table class="w-full text-xs text-left"
    <thead>
      <tr class="text-slate-500 border-rose-500">
        <th class="py-1.5 pr-2 font-mono">
        <th class="py-1.5 pr-2 font-mono">
        <th class="py-1.5 font-bold">
      </tr>
    </thead>

```

```

        <tr class="border-b border-slate-
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>

```

```

        <tr class="border-b border-slate-
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>

```

```

        <tr class="align-top"><td class=
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
</tbody>

```

```

    </table>

```

```

    <p class="text-slate-400 text-xs mt-3">
остижимые из цикла вершины (A, C, D, E, F).
х, до которых из него есть путь.</p>
</div>

```

```

</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-indigo-400

```

```

<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate-
-b border-slate-700/50">» Доказательство (и

```

```

    <div class="p-5 text-sm text-slate-300
        <p><b>Утверждение.</b> После <span class=
ss="font-mono text-emerald-300">v</span> зн
кратчайшего пути из <span class="font-mono
nt-mono text-emerald-300">i</span> рёбер.</p>

```

```

        <p><b>База.</b> <span class="font-mono
"font-mono">s</span>, и <span class="font-mono

```

```

        <p><b>Шаг.</b> Пусть утверждение вер
font-mono">P</span> из  $\leq$  <span class="font-mono
o – <span class="font-mono">(u, v)</span>,
– 1</span> рёбер и сам является кратчайшим
font-mono">i – 1</span> итераций <span clas
an>-й итерации ребро <span class="font-mono
 $\leq \text{dist}[u] + w(u, v) \leq |P|$ </span> ■</p>

```

```

        <p><b>Следствие 1.</b> Если отрицат

```

```

        </div>
    </div>

    <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-800">
        <h3 class="text-xl font-bold text-emerald-400">Алгоритм Форда-Беллмана
            <div class="bg-slate-900/60 p-4 rounded-lg border-1 border-slate-800">
                <p class="text-xs font-bold text-amber-500">Время:  $O(n^3)$ 
                :</p>
<pre class="bg-slate-950 p-3 rounded text-xs font-mono">
    d = [float("inf")] * n
    d[start] = 0
    # Бэтмен патрулирует n - 1 ночей
    for _ in range(n - 1):
        for u, v, w in edges:
            if d[u] + w < d[v]:
                d[v] = d[u] + w

# На n-ю ночь проверяем отрицательный цикл ("бесконечная кормёжка")
for u, v, w in edges:
    if d[u] + w < d[v]:
        return True # Бэтмен нашёл бесконечную кормёжку
return False # Всё тихо, отрицательных циклов нет</pre>
    </div>

    <details class="bg-slate-900/40 rounded-lg border-1 border-slate-800">
        <summary class="p-4 font-bold text-slate-200">Сложность:  $O(n^3)$ 
        -b border-slate-700/50">> Код: Форд–Беллман
        <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-xs font-mono">
def bellman_ford(n, edges, s):
    """edges = [(u, v, w), ...] – ориентированные рёбра
    dist = [INF] * n
    prev = [-1] * n
    dist[s] = 0
    changed_vertex = -1

```

```

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D", i], INF) for i in range(len(nodes))])
print([nodes[i] for i in cyc2]) # ['D', ...]
print([dist2[i] for i in range(len(nodes))]) # все вершины, кроме 'D', имеют конечный вес
    <p class="text-slate-400 text-xs">Примечание: если вершина с >INF</span>, то она не принадлежит циклу.
    </div>
</details>

<details class="bg-slate-900/40 rounded-lg p-4">
    <summary class="p-4 font-bold text-slate-400">Код: C++ (структура данных)
    <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-400">
    t;
#include <iostream>
using namespace std;
const long long INF = 4e18;

struct Edge { int u, v; long long w; };

int n; // число вершин
vector<Edge> edges; // плоский список рёбер
vector<long long> d;
vector<int> p;

// возвращает вершину, улучшенную на «лишней» итерации,
int bellman_ford(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    int x = -1;
    for (int it = 0; it < n; ++it) { // n-1 итераций
        x = -1;
        for (const auto &e : edges) {
            if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
                d[e.v] = d[e.u] + e.w;
                p[e.v] = e.u;
                x = e.v;
            }
        }
    }
    return x;
}
    </div>
    </details>

```

```
def spfa(n, adj, s):
    """adj[u] = [(v, w), ...]. Релаксируем только «живые» вершины.
    INF = float("inf")
    dist = [INF] * n
    in_queue = [False] * n
    counter = [0] * n # сколько раз вершина попала в очередь
    dist[s] = 0
    q = deque([s])
    in_queue[s] = True
    while q:
        u = q.popleft()
        in_queue[u] = False
        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if not in_queue[v]:
                    q.append(v)
                    in_queue[v] = True
                    counter[v] += 1
                    if counter[v] >= n:
                        return None # отрицательный цикл
    return dist
```

- Ранний выход. Если зациклился, то в графах экономит кратно.
- SPFA (Shortest Path Faster Algorithm), у которых только что улучшился dist[v], но худший случай остаётся «зациклился» из рёбер) SPFA ловит TL. Критерий цикла — количество итераций, против течения — за O(V + E) итераций даёт финальные dist.
- DAG. Если граф ациклический, то dist можно посчитать за O(V + E).

```

    title: "16. Графы. Поиск кратчайшего пути. Флойд",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">16</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы. Поиск кратчайшего пути. Флойд</h2>
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
      <h3 class="text-xl font-bold text-indigo-400">Задача</h3>

      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Дана неориентированная графа с  $V$  вершинами и  $E$  рёбрами. Найти кратчайшие пути от каждой вершины до каждой вершины (но не с отрицательными циклами) и пишется  $V \times V$  матрица кратчайших расстояний.
        <p class="text-xl font-mono text-white">Вывод: матрица кратчайших расстояний</p>
      </div>

      <p class="text-slate-300 text-sm mb-4">Флойд-Уоршелл — алгоритм поиска кратчайших путей во всех парах вершин графа. Он работает на графах с  $V$  вершинами и  $E$  рёбрами. Алгоритм работает за  $O(V^3)$  и пишется  $V \times V$  матрица кратчайших расстояний.
      <span class="text-emerald-300"> $O(V^3)$ </span> и <span class="font-mono"> $V \times V$ </span> размера <span class="font-mono"> $V \times V$ </span> матрицы кратчайших расстояний, <span class="font-mono text-emerald-300"> $O(V^3)$ </span> в остальных клетках.</p>

      <div class="my-5 p-5 rounded-xl bg-slate-900">
        <div class="mb-4 rounded-xl overflow-hidden">
          
        </div>
        <div class="flex items-center gap-3">
          <span class="text-2xl">16</span>
          <div>
            <h4 class="text-purple-300 font-mono">«Все против всем» (Все против всех)</h4>
            <p class="text-slate-300 text-sm">Алгоритм Флойда-Уоршелла</p>
          </div>
        </div>
      </div>
    </div>
  </div>
`

```

```

        <p class="text-[10px] uppercase tra
        <p class="font-mono text-emerald-300
    </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2
    <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
                Аналогия 1: Таблица пересадок в
            </div>
            <p class="text-slate-300 text-sm mb
це только соседние станции. Затем говорите:
        class="font-mono">через 1</span> там, где
плакат окончательный. Порядок «разрешений»
an>.</p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
            border-rose-500 shadow-md">
                Аналогия 2: Почему k обязан быть
            </div>
            <p class="text-slate-300 text-sm mb
s="font-mono">k</span> <b>один раз</b> и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞</span>, х
пользует «поздние» посредники). Проверка пер
    </div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400
        <div class="grid grid-cols-1 xl:grid-cols-2
            <p class="text-slate-300 text-sm mb-4">Демон
            <p class="text-slate-300 text-sm mb-4">вершины 0...6, <span class="font-mono text-em
            <p class="text-slate-300 text-sm mb-4">merald-300">(i, j)</span>, которая улучшает
```



```

<p class="text-sm font-bold text-white">
<table class="w-full text-xs text-left"
  <thead>
    <tr class="text-slate-500 border"
      <th class="py-1.5 pr-2 font"
      <th class="py-1.5 pr-2 font"
      <th class="py-1.5 font-bold"
    </tr>
  </thead>
  <tbody class="text-slate-300">
    <tr class="border-b border-slate-
y-2 pr-2 font-mono">конечное</td><td class="
    <tr class="border-b border-slate-
y-2 pr-2 font-mono">INF</td><td class="py-2"
    <tr class="border-b border-slate-
y-2 pr-2 font-mono">0</td><td class="py-2">
    <tr class="align-top"><td class="
se-300">&lt; 0</td><td class="py-2 text-rose-400"
делены</td></tr>
  </tbody>
</table>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border"
  <h3 class="text-xl font-bold text-indigo-400">

  <details class="bg-slate-900/40 rounded-lg"
    <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">» Формальная динамика
    <div class="p-5 text-sm text-slate-300"
      <p>Обозначим <span class="font-mono text-slate-300"
o">i → j</span>, все промежуточные вершины
font-mono">i</span> и <span class="font-mono text-slate-300"
      <p><b>База:</b> <span class="font-mono text-slate-300"
ли 0.</p>
      <p><b>Переход:</b> путь из <span class="font-mono text-slate-300"
class="font-mono text-slate-300">{0...k}</span> либо не исп

```

```

        return d

good = floyd(M, "kij")
bad  = floyd(M, "ijk")
print(good[2][1], bad[2][1])    # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good[i][j] != bad[i][j]))
# 4 клетки отличаются; при порядке (i,k,j) их 6</code><
        <p class="text-slate-400 text-xs">В
        угих «почти работают» (большинство клеток с
        </div>
    </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border
    <h3 class="text-xl font-bold text-emerald-400
        <details class="bg-slate-900/40 rounded-lg
            <summary class="p-4 font-bold text-slate
                -b border-slate-700/50">> Код: Флойд–Уоршелл
            <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te

def floyd_warshall(n, edges, directed=True):
    d = [[INF] * n for _ in range(n)]
    nxt = [[-1] * n for _ in range(n)]
    for i in range(n):
        d[i][i] = 0                                # диагональ об

    for u, v, w in edges:
        if w < d[u][v]:                            # кратчайше
            d[u][v] = w
            nxt[u][v] = v
        if not directed and w < d[v][u]:
            d[v][u] = w
            nxt[v][u] = u

    for k in range(n):                            # k – ВНЕШНИЙ
        for i in range(n):

```

```

    t;
#include <algorithm>
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
    for (int i = 0; i < n; ++i) d[i][i] = 0;
    for (int k = 0; k < n; ++k) // k
        for (int i = 0; i < n; ++i) {
            if (d[i][k] == INF) continue; // защищаем себя от переполнения
            for (int j = 0; j < n; ++j) {
                if (d[k][j] == INF) continue;
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
}

bool has_negative_cycle() {
    for (int i = 0; i < n; ++i) if (d[i][i] < 0) return true;
    return false;
}

long long diameter() { // самая длинная дистанция
    long long res = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (d[i][j] < INF) res = max(res, d[i][j]);
    return res;
}

int center() { // вершина с минимальной суммой расстояний
    int best = -1; long long bestEcc = INF;
    for (int i = 0; i < n; ++i) {
        long long ecc = 0;
        for (int j = 0; j < n; ++j) {
            if (d[i][j] < INF) ecc += d[i][j];
        }
        if (ecc < bestEcc) best = i, bestEcc = ecc;
    }
    return best;
}

```

```

    > при r[i][k]
     $V^3$ )</span>.</p>
    </div>
  </details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-1" style="border-color: #f1f3f4">
  <h3 class="text-xl font-bold text-amber-400">

```

```

    <div class="bg-rose-950/30 p-5 rounded-lg border-1" style="border-color: #f1f3f4">
      <p class="text-sm font-bold text-rose-300">
        <ul class="list-disc pl-5 space-y-2 text-rose-300">
          <li><b>Порядок циклов.</b> <span class="font-mono">(i, j, k)</span> на демо-графе 4 клетки
            ] = 999</span> вместо 4).</li>
          <li><b>Диагональ.</b> <span class="font-mono">d[i][j]</span> «починит» её только при наличии цикла, а по
          <li><b>Переполнение INF.</b> <span class="font-mono">d[i][j]</span> уходит в минус и «улучшает» всё подряд. Пр
            no">long long</span>.</li>
          <li><b>Параллельные рёбра.</b> Во в
            no">d[u][v]</span> стартует с худшего веса.
          <li><b>Отрицательные циклы.</b> Форм
            ых с циклом, ответ <span class="font-mono">d[i][j]</span>
            t-mono">d[i][j] = -∞</span>, если существует
            <span class="font-mono">d[i][k] < ∞</span>
          <li><b>Недостижимость ≠ 0.</b> В ор
            таких пар из 49). Выводить 999 или 0 как «р
          <li><b>Память.</b> <span class="font-mono">d[i][j]</span>
            ачений (≈100 МБ на <span class="font-mono">d[i][j]</span>
            истока берите Дейкстру.</li>
          <li><b>Сам путь.</b> Матрица <span class="font-mono">nxt[i][j]</span> (как
        </ul>
      </div>
    </div>

```

```

<p class="text-slate-300 text-sm mt-4"><b>Самый короткий путь от одного истока и дешевле, если все пары н

```

```
order border-emerald-500 shadow-md">
```

Аналогия 1: Такси с навигатором

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
  ы одинаково. А* — тот же таксист, но с навигатором.
  /span> прибавляется оценка остатка пути по маршруту.
  т туда, где сумма меньше. Навигатор обязан выбирать
  /span>): занижать можно, завышать нельзя — это Дейкстра.
  0">h = 0</span> это снова Дейкстра.</p>
```

```
</div>
```

```
<div class="bg-slate-800 p-6 rounded-lg">
```

```
<div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
```

Аналогия 2: Две бригады туннеля

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
  пускают две: от А и от В. Каждая зона роста имеет
  я <b>не в момент встречи</b>, а когда сумма стоимостей
  — иначе можно пропустить более дешёвый объект.
  ирует.</p>
```

```
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left">
```

```
<thead>
```

```
<tr class="text-slate-500 border-b">
```

```
<th class="py-2 pr-3 font-bolder">А
```

```
<th class="py-2 pr-3 font-bolder">В
```

```
<th class="py-2 pr-3 font-bolder">С
```

```
<th class="py-2 font-bolder">Дейкстра
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-500">
  )</td><td class="py-2 pr-3">разрешает Дейкстра
  g V)</td><td class="py-2">нет отрицательных
  <tr class="border-b border-slate-500">
```

Дейкстра (билет 14) быстрая, но
ленный. Алгоритм Джонсона делает финт: пере
ь теми же.

</p>

<ol class="text-sm text-slate-300 s
Добавляем фиктивную верш
Запускаем от s Форда-Бел
ажный факт:</i> Джонсон НЕ «удаляет» отрица
кой и прекращает работу, так как при отрица
Перевзвешиваем: <span class
тво следует из того, что $h(v) \leq h(u) + w(u,$
Магия: для любого пу
) - $h(\text{финиш})$, то есть все пути между фик
еняется.

Запускаем быструю Дейкстр
lass="font-mono text-emerald-300"> $d(u, v) =$

<p class="text-slate-400 text-xs mt
еженном графе ($E \approx V$) это $O(V^2 \log V)$ проти
ойд – для маленьких или очень плотных.</p>

</div>

</div>

<h3 class="text-lg font-bold text-white mt-
<p class="text-slate-300 text-sm mb-3">Если
азад из t (по перевёрнутым рёбрам). Волны р
: в «широком» графе (дорожная сеть) это эконо
огда волны «встретились» в одной вершине; н
огда <span class="font-mono text-emerald-300
более короткий путь.</p>

<h3 class="text-lg font-bold text-white mt-
<p class="text-slate-300 text-sm mb-3">Дейк
по $g(v)$ (уже
(v) = $g(v) + h(v)$, где $h(v)$ – оценка
а требования: допустимость ($h(v) \leq$ и
онотонность/согласованность ($h(u) \leq w(u$
вырождается ровно в Дейкстру – это хороший

м числом раскрытий, 0-1 BFS `0(V + E)`, Dial `0(E + V · C)`, Дейкстра с кучей

```
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-
-b border-slate-700/50">
```

Код: Джонсон целиком (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

```
class="bg-slate-950 p-4 rounded" style="background-color: #1e293b; color: white; padding: 10px; border-radius: 10px;">

```

heapq

```
INF = float("inf")
```

```
def johnson(n, edges):
    # 1) потенциалы: Форд–Беллман от фиктивной вершины
    h = [0] * (n + 1)
    for _ in range(n):
        changed = False
        for u, v, w in edges:
            if h[u] + w < h[v]:
                h[v] = h[u] + w
                changed = True
        if not changed:
            break
    else:
        return None
```

2) перевзвешивание: все $w' \geq 0$

```
a = [[] for _ in range(n)]
```

for u, v, w in edges:

```
g[u].append((v, w + h[u] - h[v]))
```

```

        if dist > df.get(v, INF): continue
        for to, w in g[v]:
            nd = dist + w
            if nd < df.get(to, INF):
                df[to] = nd; heapq.heappush(qf, (nd, to))
            if to in db:
                mu = min(mu, nd + db[to])    # встреча:
dist, v = heapq.heappop(qb)
if dist > db.get(v, INF): continue
for to, w in rg[v]:
    nd = dist + w
    if nd < db.get(to, INF):
        db[to] = nd; heapq.heappush(qb, (nd, to))
    if to in df:
        mu = min(mu, nd + df[to])
return mu

# --- A*: приоритет f = g + h ---
def astar(g, s, t, h):                                # h(v) — допустим
    INF = float("inf")
    gdist = {s: 0}
    heap = [(h(s), 0, s)]                             # (f, g, v)
    closed = set()
    while heap:
        f, gv, v = heapq.heappop(heap)
        if v == t:
            return gv                                  # ранний выход:
        if v in closed:
            continue
        closed.add(v)
        for to, w in g[v]:
            nd = gv + w
            if nd < gdist.get(to, INF):
                gdist[to] = nd
                heapq.heappush(heap, (nd + h(to), nd, to))
    return INF

# --- 0-1 BFS: веса 0 и 1, дека вместо кучи ---

```



```
    },  
  ];  
  

```

ФАЙЛ 28/111: src/data/tickets/ticket18_20_mst.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 231 | РАЗМ

```
import { Chapter } from '../types';
```

```
/**
```

```
 * Билеты 18 · 19 · 20 – одна страница на три алгоритма  
 * минимального остовного дерева (MST). Так же, как в с  
 * KruskalSimulator, три алгоритма живут на одном графе  
 * переключаются вкладками «Краскал / Прима / Боровка».
```

```
 *  
 * Синхронизация с компилятором: PAGE_SYNC["mst"] + под  
 * "mst#kruskal", "mst#prim", "mst#boruvka" (вкладка со  
 * через emitVizDemo, код в панели следует за ней).
```

```
 */
```

```
export const tickets18to20Mst: Chapter[] = [
```

```
  {
```

```
    id: "mst",
```

```
    title: "18–20. Графы. Остовное дерево: Краскал, При
```

```
    type: "html",
```

```
    description:
```

```
      "Три способа построить минимальное остовное дерев  
      .",
```

```
    category: "Графы. Остовные",
```

```
    content: `
```

```
<section id="mst" class="mb-12 scroll-mt-10">
```

```
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
    <span class="bg-blue-600 text-white px-4 py-1 r
```

```
    <h2 class="text-2xl sm:text-3xl font-bold text-v
```

```
  </div>
```

```
  <div class="space-y-8">
```

```

        <tr class="border-b border-slate-200">
          <td class="py-2 pr-3 font-bolder">Рёбра</td>
          <td class="py-2 pr-3">Рёбра</td>
          <td class="py-2 pr-3 font-mono">Рёбра</td>
          <td class="py-2 pr-3 font-mono">Рёбра</td>
          <td class="py-2">рёбра даны</td>
        </tr>
        <tr class="border-b border-slate-200">
          <td class="py-2 pr-3 font-bolder">Растёт</td>
          <td class="py-2 pr-3">Растёт</td>
          <td class="py-2 pr-3 font-mono">Растёт</td>
          <td class="py-2 pr-3 font-mono">Растёт</td>
          <td class="py-2">плотный гр</td>
        </tr>
        <tr class="align-top">
          <td class="py-2 pr-3 font-bolder">Растёт</td>
          <td class="py-2 pr-3">Все к</td>
          <td class="py-2 pr-3 font-mono">Растёт</td>
          <td class="py-2 pr-3 font-mono">Растёт</td>
          <td class="py-2">параллельны</td>
        </tr>
      </tbody>
    </table>
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-3">
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
        Аналогия к билету 18 • Краск
      </div>
      <p class="text-slate-300 text-sm">С
    >. По всей карте растут независимые кусочки
      дерево. Порядок рёбер важен, порядок верши
    </div>

    <div class="bg-slate-800 p-6 rounded-lg"

```

```

        бесплатный); Прим с двоичной кучей <code class="bg-slate-950 px-2 py-0.5 rounded text-xs font-mono text-emerald-300 border border-emerald-300 border-dashed text-emerald-300 font-mono text-emerald-300 border-emerald-300" style="border: 1px dashed #008000; padding: 2px 0.5px; color: #008000; font-family: monospace; font-size: 0.8em; border-radius: 4px; background-color: #a6c9ec; width: fit-content; margin: 10px auto; text-align: center;">0(E log V)</code>, на плотн
    text-xs font-mono text-emerald-300 border border-emerald-300 border-dashed text-emerald-300 font-mono text-emerald-300 border-emerald-300" style="border: 1px dashed #008000; padding: 2px 0.5px; color: #008000; font-family: monospace; font-size: 0.8em; border-radius: 4px; background-color: #a6c9ec; width: fit-content; margin: 10px auto; text-align: center;">
    ex <code class="bg-slate-950 px-2 py-0.5 rounded text-xs font-mono text-emerald-300 border border-emerald-300 border-dashed text-emerald-300 font-mono text-emerald-300 border-emerald-300" style="border: 1px dashed #008000; padding: 2px 0.5px; color: #008000; font-family: monospace; font-size: 0.8em; border-radius: 4px; background-color: #a6c9ec; width: fit-content; margin: 10px auto; text-align: center;">
    /p>

    <details class="bg-slate-900/40 rounded-lg border border-slate-700/50" style="background-color: #1a2b3c; color: #e0e0e0; padding: 10px; border-radius: 10px; border: 1px solid #444; margin: 10px auto; width: 90%;">
        <summary class="p-4 font-bold text-slate-300 border-bottom border-slate-700/50" style="border-bottom: 1px solid #444; padding: 5px 0 5px 10px; margin-bottom: 10px;">
            Код 18: Краскал + DSU (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300" style="padding: 5px 0 5px 10px;">
            <pre class="bg-slate-950 p-4 rounded text-emerald-300 font-mono text-emerald-300 border border-emerald-300 border-dashed text-emerald-300 font-mono text-emerald-300 border-emerald-300" style="border: 1px dashed #008000; padding: 2px 0.5px; color: #008000; font-family: monospace; font-size: 0.8em; border-radius: 4px; background-color: #a6c9ec; width: fit-content; margin: 10px auto; text-align: center;">
                lt;pair<int, pair<int,int>>>>
                vector<int> p, rnk;

                int find(int v) { return p[v] == v ? v : p[v] = find(p[v]); }
                bool unite(int a, int b) {
                    a = find(a); b = find(b);
                    if (a == b) return false; // уже в одной компоненте
                    if (rnk[a] < rnk[b]) swap(a, b);
                    p[b] = a;
                    if (rnk[a] == rnk[b]) rnk[a]++;
                    return true;
                }

                sort(edges.begin(), edges.end()); // O(E log E)
                long long cost = 0; int taken = 0;
                for (auto [w, uv] : edges) {
                    if (unite(uv.first, uv.second)) { // не замыкает цикл
                        cost += w;
                        if (++taken == n - 1) break; // V - 1 ребро
                    }
                }
            </pre>
        </div>
    </details>

```

```

<div class="p-5 text-sm text-slate-300" style="background-color: #f0f0f0; border: 1px solid #ccc; border-radius: 10px; padding: 10px;">
    <pre class="bg-slate-950 p-4 rounded" style="background-color: #2e3436; color: #eeeeec; border: 1px solid #1f2f3d; border-radius: 5px; padding: 10px;">
        = list(range(n))
def find(v):
    while parent[v] != v:
        parent[v] = parent[parent[v]]
        v = parent[v]
    return v

comps = n
total = 0
while comps > 1:
    best = {} # компонента
    for w, u, v in edges:
        cu, cv = find(u), find(v)
        if cu == cv:
            continue # уже внутри
        if cu not in best or w < best[cu][0]:
            best[cu] = (w, u, v)
        if cv not in best or w < best[cv][0]:
            best[cv] = (w, u, v)
    if not best:
        break # граф несвя
    for w, u, v in best.values():
        if find(u) == find(v):
            continue # дубль: два
        parent[find(u)] = find(v)
        comps -= 1
        total += w
print(total)</pre>

```

<p class="text-slate-400 text-xs">3. Ну), значит фаз не больше $\log_2 V$.</p>

</div>
</details>

<details class="bg-slate-900/40 rounded-lg" style="background-color: #2e3436; border: 1px solid #1f2f3d; border-radius: 10px; padding: 10px;">
 <summary class="p-4 font-bold text-slate-400" style="background-color: #2e3436; border: 1px solid #1f2f3d; border-radius: 5px; padding: 5px 10px 5px 10px; margin-bottom: 5px;">
 -b border-slate-700/50">

```

<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
        сом, оканчивающимся в позиции i.</p>
    </div>

    <div class="bg-slate-900 border border-indigo-500">
      <h4 class="text-indigo-300 font-bold text-sm">
        Префикс-функция  $\pi[i]$  – смотрим  $C[i]$ 
      </h4>
      <p class="text-slate-300 text-sm mb-4">
        Чтобы узнать значение <span class="font-mono text-indigo-300">
        )</b> – в уже вычисленный префикс <span class="font-mono text-indigo-300">
        м символом <span class="font-mono text-indigo-300">
      </p>
      <ol class="list-decimal pl-5 space-y-1.5">
        <li>Считаем значение <span class="font-mono text-indigo-300">
        <li>Сравниваем следующий символ префикса с <span class="font-mono text-indigo-300">
        – <span class="font-mono text-indigo-300">
      </ol>
      <div class="bg-slate-950 p-3 rounded-lg">
        <div class="inline-flex items-center justify-center">
          <span class="text-indigo-300 font-mono">
            <span class="text-slate-600">|<span class="bg-indigo-600 text-white text-sm">
          </div>
        <p class="text-xs text-indigo-300 margin-top: 10px">
          - Рамка смотрит ВЛЕВО: проверяет префикс
        </p>
      </div>
    </div>
  </div>

```

```

    pi[i] = j;
} </pre>

```

```

    </div>
</details>

```

```

<h3 class="text-lg font-bold text-white mt-4">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-xl font-mono text-white">

```

```

p>

```

```

</div>

```

```

<p class="text-slate-300 text-sm mb-4">Префикс
для шаблона (pattern) один раз, затем идём
пало –  $j += 1$ . Не совпало – откатываемся на
у заново сравнивать его не нужно. Когда  $j$  до
ждения), делаем  $j = \pi[j - 1]$ .</p>

```

```

<p class="text-slate-300 text-sm mb-4"><b>П
ждый откат  $j$  уменьшает. Значит суммарно отк
классическая амортизационная оценка, её люб.

```

```

<div class="bg-slate-900/40 rounded-lg border"

```

```

    <p class="text-slate-300 text-sm mb-2">

```

```

    <p class="text-slate-400 text-sm mb-1">

```

```

    льшего собственного префикса, равного суффи

```

```

    <p class="text-slate-400 text-sm mb-1">

```

```

    ый алгоритм.</p>

```

```

    <p class="text-slate-400 text-sm mb-1">

```

```

    т онлайн (символы могут приходить потоком),

```

```

    <p class="text-slate-400 text-sm">• <b>

```

```

    5] = 4, период 6 – 4 = 2 («ab»)</p>

```

```

</div>

```

```

<details class="bg-slate-900/40 rounded-lg"

```

```

    <summary class="p-4 font-bold text-slate-300"

```

```

    -b border-slate-700/50">

```

```

        Код: поиск подстроки КМР (Скрыто)

```

```

    </summary>

```

```

    <div class="p-5 text-sm text-slate-300"

```

```

        <pre class="bg-slate-950 p-4 rounded"

```

```

        fix_function(s):

```

```

        n = len(s)

```

```

    },
    {
      id: "string-z-func",
      title: "22. Z-функция — Сдвиг и наложение",
      type: "html",
      category: "Строки",
      content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        са 0) и её суффиксом, начинающимся с i.</p>
      </div>

      <div class="bg-slate-900 border border-emerald-500">
        <h4 class="text-emerald-300 font-bold text-sm">
          Z-функция Z[i] — смотрим ВПРАВО (
        </h4>
        <p class="text-slate-300 text-sm mb-4">
          Буква <b>Z</b> запоминается как <b>
        <span class="text-emerald-300 font-mono text-emerald-300">i</span> и подглядываем <b>ВПРАВО (ПОЗАДИ
        <span class="text-emerald-300 font-mono text-emerald-300">s[0...]</span>.
        </p>
        <div class="bg-slate-950 p-3 rounded-lg">
          <div class="inline-flex items-center justify-center">
            <span class="text-slate-500">[0
            <span class="bg-emerald-600 text-white">
            <span class="text-slate-600">|<
            <span class="text-emerald-300 font-mono text-emerald-300">
          </div>
        </div>
      </div>
    </div>
  </div>
`
    }
  ],
  ...
}

```

```

        </div>
      </details>
    </div>
  </div>
</section>`
  },
  {
    id: "aho-corasick",
    title: "23. Алгоритм Ахо-Корасик",
    type: "html",
    category: "Строки",
    content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-l">
      <h3 class="text-xl font-bold text-blue-400 mb-4">
        <div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">
          <p class="text-sm text-blue-300 italic mb-2">Основа
          <p class="text-lg font-mono text-white">Бор (Тренировка)
        </div>
        <p class="text-slate-300 text-sm">Позволяет найти
          го один проход.</p>
      </div>

      <!-- Аналогии -->
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
          <div class="absolute -top-3 left-4 bg-slate-700 text-emerald-500 shadow-md">
            Аналогия 1: Паутина (Бор)
          </div>
          <p class="text-slate-300 text-sm mb-4">Представ

```



```

</section>`
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description:
      "P, NP, co-NP, NP-трудные и NP-полные, PSPACE и E. Если ли задача NP-полна.",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded">24
    <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">Сведение задач
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Пример: Сведение задачи A к задаче B.
        <p class="text-xl font-mono text-white">Если A ∈ P, то B ∈ P. Если A ∉ P, то B ∉ P.
        ат). Сведение A ≤p B – преобразовать вход A в вход B так, чтобы решение для B существовало тогда и только тогда, когда оно существует для A.
        .</p>
      </div>

      <p class="text-slate-300 text-sm mb-4">Класс сложности (например, P) – это класс задач, которые можно решить за полиномиальное время на машине Тьюринга) с выбранным бюджетом ресурсов (например, временем). Мы говорим, что задача A сводится к задаче B (A ≤p B), если существует алгоритм, который преобразует любой экземпляр задачи A в экземпляр задачи B за полиномиальное время, так что решение для B существует тогда и только тогда, когда оно существует для A. Мы говорим, что задача A является NP-трудной, если для нее существует сведение к NP-полной задаче. Мы говорим, что задача A является NP-полной, если она является NP-трудной и принадлежит к классу NP.
      <b>NP-полные задачи</b> – это задачи, которые являются NP-трудными и принадлежат к классу NP. Если задача A является NP-полной, то для нее существует сведение к любой NP-полной задаче B. Это означает, что если мы сможем решить задачу B, то сможем решить любую NP-полную задачу.
      <b>NP-трудные задачи</b> – это задачи, которые являются NP-трудными, но не обязательно принадлежат к классу NP. Например, задача о минимальном пути (MST) является NP-трудной, но не принадлежит к классу NP.
    </div>
  </div>
`
  }
]

```

```

        <tr class="border-b border-slate-200 py-2 pr-3">сертификат проверяется за полиномиальную
        ешающая версия), коммивояжёр «есть ли тур ≤ k»</td></tr>
        <tr class="border-b border-slate-200 py-2 pr-3">сертификат есть у ответа «НЕТ» (не
        ель)</td></tr>
        <tr class="border-b border-slate-200 py-2 pr-3">не проще любой задачи из NP-полной
        фа, TSP-оптимизация (не решающая версия)</td></tr>
        <tr class="border-b border-slate-200 py-2 pr-3">NP-трудные И из NP</td><td class="border-b
        сумм</td></tr>
        <tr class="border-b border-slate-200 py-2 pr-3">полиномиальная память, время не
        й на полиномиальном поле</td></tr>
        <tr class="align-top"><td class="border-b border-slate-200 py-2 pr-3">за  $2^{(n^k)}$ </td><td class="py-2">обобщённые
        </tbody>
    </table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-10">3. Сертификат полиномиального размера и проблема
<p class="text-slate-300 text-sm mb-3">Стандартная задача: дан граф G, требуется ли
ать сертификат полиномиального размера и проверить, что сертификат верен.
    — «все вершины по разу и каждая соседняя пара вершин встречается ровно один раз»
ь полиномиальное сведение  $A \leq_p B$ . Тогда B — NP-полна.
<p class="text-slate-300 text-sm mb-3"><b>Теорема 3.1. Проблема SAT NP-полна. Доказывается «в лоб»: вычислительная
ево́й формулой размера  $\text{poly}(n)$ , где переменные  $x_1, \dots, x_n$  принимают значения 0 или 1.
ильна принятию входа. Дальше всё катится целиком к задаче о гамильтоновом цикле.
гамильтонов цикл  $\rightarrow$  TSP.</p>

```

```

<div class="bg-slate-800/70 p-5 rounded-lg">
    <p class="text-sm font-bold text-white">Алгоритмы для NP-полных задач
    <ul class="list-disc list-inside text-slate-300">
        <li><b>Точный перебор с отсечениями (branch-and-bound, meet-in-the-middle).</b></li>
        <li><b>Приближённые алгоритмы</b>:
    </ul>

```

```

<div class="p-5 text-sm text-slate-300
  <pre class="bg-slate-950 p-4 rounded
    ьтонов цикл NP-полон: сертификат = порядок
def verify_hamilton(n, edges, cert):
    if sorted(cert) != list(range(n)):          # каждая
        return False
    allowed = {(u, v) for u, v in edges} | {(v, u) for
    for i in range(n):
        a, b = cert[i], cert[(i + 1) % n]      # замык
        if (a, b) not in allowed:
            return False
    return True

# 3-SAT ≤p КЛИКА: формула с k дизъюнктов → граф из 3k в
def reduce_3sat_to_clique(clauses):
    # вершина = {номер дизъюнкта, литерал}; ребро, если
    verts = [(ci, lit) for ci, cl in enumerate(clauses)
    edges = []
    for i in range(len(verts)):
        for j in range(i + 1, len(verts)):
            ci, li = verts[i]; cj, lj = verts[j]
            if ci == cj:                          # из од
                continue
            if li.lstrip("-") == lj.lstrip("-") and (li
                continue                          # x и n
            edges.append((i, j))
    return verts, edges, len(clauses)            # клика

# Проверка задачи из P для контраста: двудольность = 2-р
def is_bipartite(n, graph):
    from collections import deque
    col = [-1] * n
    for s in range(n):
        if col[s] != -1:
            continue
        col[s] = 0
        dq = deque([s])
        while dq:

```

ФАЙЛ 30/111: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 576 | РАЗМ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
```

```
{
```

```
  id: "graph-dfs-bfs",
```

```
  title: "6. Графы. Представление графов, DFS, BFS",
```

```
  type: "html",
```

```
  description: "Представление графов и базовые обходы
```

```
  category: "Графы. База",
```

```
  content: `
```

```
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
```

```
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
```

```
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
</div>
```

```
<div class="space-y-8">
```

```
  {/* База представлений */}
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
    <h3 class="text-xl font-bold text-slate-300">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
      <p class="text-lg text-slate-300 italic">
```

```
      <p class="text-xl font-mono text-white">
```

```
      + E).</p>
```

```
    </div>
```

```
    <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
      <div class="bg-slate-800 p-6 rounded-lg">
```

```
        <div class="absolute -top-3 left-4 border"
```

```
        rder border-emerald-500 shadow-md">
```

```
          Аналогия: Матрица = Таблица
```

```
        </div>
```

```
        <p class="text-slate-300 text-sm mb-4">
```

```
        глупо, но проверка "знает ли Вася Петю" м
```

```

<div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
    BFS (Поиск в ширину) = Волна
</div>
<p class="text-slate-300 text-sm mb-4">
    <b>Что делает:</b> Распространение по соседям. Работает через Очередь (FIFO). Гарантирует минимальное количество шагов.
</p>
<ul class="text-xs text-emerald-300 list-disc">
    <li><b>Где используется:</b></li>
    <li>Кратчайший путь в невзвешенном графе</li>
    <li>Проверка двудольности графа</li>
    <li>Алгоритм Ахо-Корасик (построение автомата)</li>
    <li>Двунаправленный BFS для поиска кратчайшего пути (в узлах)</li>
</ul>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
    <summary class="p-4 font-bold text-slate-300">
        Код обходов C++ (Основа)
    </summary>
    <div class="grid grid-cols-1 md:grid-cols-2">
        <pre class="bg-slate-950 p-3 rounded">
// DFS (Рекурсия с возвратом)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}</pre>

        <pre class="bg-slate-950 p-3 rounded">
// BFS (Очередь FIFO)
vector<int> q;
vector<bool> vis;
vector<vector<int>>> adj;

void bfs(int start) {
    q.push_back(start);
    vis[start] = true;
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : adj[v]) {
            if (!vis[u]) {
                vis[u] = true;
                q.push_back(u);
            }
        }
    }
}</pre>

```

</div>

```
<div class="space-y-4 text-slate-300 text-s<
  <p><b>Главные неравенства на число рёбер</b>
  <ul class="list-disc list-inside space-y-4">
    <li><b>Для простого планарного графа</b>
      <i>Доказательство:</i> Каждая грань
    </li>
    <li><b>Для простого двудольного план</b>
  </ul>
```

```

    <div class="bg-slate-950 p-4 rounded-lg">
      <p class="text-amber-300 font-bold">
      <p class="text-slate-300 text-xs le
        1. <b>K53,3
```

```

    <p><b>Теорема Куратовского (1930) / Вагн</b>
    фа, являющегося <b>подразделением (subdivis
  </p>
</div>
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-purple-400">
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-purple-300 italic">
    <p class="text-xl font-mono text-white">
```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
          <p class="text-xl font-mono text-white">
            рой вершин есть путь. Внешний цикл «для каж
            » даёт и разметку, и количество компонент.</p>
          </div>

          <p class="text-slate-300 text-sm mb-4">Два
            ной запускаем DFS или BFS и присваиваем все
            mp[v] = c</span>. Счётчик c увеличивается р
            Время <b> $O(V + E)$ </b>, память  $O(V)$ . <b>Чере
            каждое ребро делаем union(u, v), в конце чи
            ту</b> и после каждого добавления спрашиваю

          <div class="grid grid-cols-1 xl:grid-cols-2"
            <div class="bg-slate-800 p-6 rounded-lg"
              <div class="absolute -top-3 left-4 l
                rder border-emerald-500 shadow-md">
                  Аналогия 1: Острова
                </div>
                <p class="text-slate-300 text-sm">Т
                  ались серые (неисследованные) зоны? Прыгаеш
                  и компонент. Краска — это comp[v], прыжок ч
                </div>
                <div class="bg-slate-900 p-6 rounded-lg"
                  <div class="absolute -top-3 left-4 l
                    border border-emerald-500 shadow-md">
                      Аналогия 2: Группы друзей (DSU)
                    </div>
                    <p class="text-slate-300 text-sm">К
                      омпании в одну (union). Кто представитель к
                      биты на компании, и вопрос «в одной ли комп
                      рут, только если концы в разных компонентах
                    </div>
                  </div>
                </div>

```

```
-----
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
р>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Ограничения: Циклы
          </div>
          <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
        </div>
      </div>
    <div class="bg-rose-950/40 p-4 rounded-lg border border-rose-500">
      <p class="text-rose-300 font-bold text-sm">
      <p class="text-slate-300 text-xs leading-5">
        Обычный массив <span class="font-mono">
ли в графе есть ориентированный цикл, код с
```



```

fill(color, color + n + 1, 0);
has_cycle = false;
ans.clear();
for (int i = 1; i &lt;= n; ++i) {
    if (color[i] == 0) dfs(i);
}
if (has_cycle) return false; // Граф содержит цикл!
reverse(ans.begin(), ans.end()); // Разворачиваем р
return true;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
        category: "Графы. Продвинутое",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Теория
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
            <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
        </div>
        <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">Макро-вершиной называют вершину
            <div class="text-left font-mono text-sm">
                <p><strong class="text-white">Компонентой сильной связности (SCC)
                н <i>ориентированного</i> графа, в котором

```

```

<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-rose-300 italic">
      <p class="text-xl font-mono text-white">
      <p class="text-xs text-slate-400 mt-1">
    </div>

    <div class="bg-slate-950 p-4 rounded-lg border">
      <p class="text-amber-300 font-bold text">
      <p class="text-slate-300 text-xs leading">
        Проверка <span class="font-mono text-
тиграфе между u и v может быть 2 параллельными
азуют цикл длиной 2.<br>
        <b>Решение:</b> передавать в DFS не v
rald-300">p_edge_id</span> (индекс ребра, по
      </p>
    </div>

    <details class="bg-slate-900/40 rounded-lg border">
      <summary class="p-4 font-bold text-slate-
-b border-slate-700/50">
        Код поиска мостов C++ (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
// Для мультиграфа с edge_id:
vector<pair<int, int>>> adj[MAXN]; // {to, edge_id}
int tin[MAXN], low[MAXN], timer;

void dfs_bridges(int v, int p_edge_id = -1) {
  tin[v] = low[v] = ++timer;
  for (auto [to, edge_id] : adj[v]) {

```

```

        <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
        <div class="p-3 bg-slate-950 rounded
            <span class="text-rose-400 font
            <p class="text-xl font-bold tex
            <p class="text-xs text-slate-400
        </div>
    </div>
</div>

<p class="text-slate-300 text-sm">
    <b>Особый случай:</b> Корень дерева DFS
мого ребёнка</b> в дереве DFS.
</p>

<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
    -b border-slate-700/50">
        Код поиска точек сочленения (Скры
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
void dfs_cutpoints(int v, int p = -1) {
    tin[v] = low[v] = ++timer;
    int children = 0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (tin[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs_cutpoints(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1) {
                is_cutpoint[v] = true;
            }
            ++children;
        }
    }
}

```

```

        <li><b>Эйлеров путь:</b> связан
о 2 – путь начинается в одной нечётной и за
    </ul>

```

```

</div>

```

```

<!-- Ориентированные графы -->

```

```

<div class="bg-slate-800 p-5 rounded-lg

```

```

    <p class="text-indigo-300 font-bold

```

```

    <ul class="list-disc list-inside sp

```

```

        <li><b>Эйлеров цикл:</b> слабос
ald-300">in[v] == out[v]</span>.</li>

```

```

        <li><b>Эйлеров путь:</b> слабос
s] = 1</span> (старт), у одной <span class=
an class="font-mono text-emerald-300">in[v]

```

```

    </ul>

```

```

</div>

```

```

</div>

```

```

<h3 class="text-lg font-bold text-white mt-

```

```

<p class="text-slate-300 text-sm mb-3">Испо

```

```

    → рекурсивный переход. При возврате (на вы
ты: <span class="font-mono text-emerald-300

```

```

<details class="bg-slate-900/40 rounded-lg l

```

```

    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Код построения Эйлерова цикла C++

```

</summary>

```

```

<div class="p-5 text-sm text-slate-300

```

```

    <pre class="bg-slate-950 p-4 rounde

```

```

vector<unordered_multiset<int>>> adj;

```

```

vector<int> path;

```

```

void find_euler(int v) {

```

```

    while (!adj[v].empty()) {

```

```

        int u = *adj[v].begin();

```

```

        adj[v].erase(adj[v].find(u));

```

```

        adj[u].erase(adj[u].find(v)); // Убираем ребро

```

```

const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
  { id: 1, x: 100, y: 150, label: "1" },
  { id: 2, x: 250, y: 250, label: "2" },
  { id: 3, x: 400, y: 150, label: "3" },
  { id: 4, x: 550, y: 150, label: "4" },
  { id: 5, x: 700, y: 50, label: "5" },
  { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
  { u: 3, v: 4 }, // 3 and 4 are articulation points (3
  { u: 4, v: 5 },
  { u: 5, v: 6 },
  { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
  { length: Object.keys(nodes).length },
  () => [],
);

edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

```

```

recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);
for (const to of adj[v]) {
    if (to === p) continue;

    if (visited.has(to)) {
        currentLow[v] = Math.min(currentLow[v], currentLow[to]);
        recordStep(
            `Обратное ребро ${v}-${to}. Обновляем low[${v}]`,
            v,
        );
    } else {
        children++;
        dfs(to, v);
        currentLow[v] = Math.min(currentLow[v], currentLow[to]);

        recordStep(
            `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
            v,
        );

        if (currentLow[to] >= currentTin[v] && p !== -1) {
            ap.add(v);
            recordStep(
                `Найдена точка сочленения: вершина ${v}`,
                v,
            );
        }
    }
}

if (p === -1 && children > 1) {
    ap.add(v);
    recordStep(
        `Корень дерева DFS ${v} имеет >1 детей. Это`,
        v,
    );
} else if (p === -1) {

```

```

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIndex(0);
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        aria-label="Сбросить обход"
        title="Сбросить обход"
        className="flex items-center justify-center
        ors"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>

```

```

        : isVisited
        ? "#10b981"
        : "#1e293b";
const stroke = isCurrent
? "#60a5fa"
: isAp
? "#f43f5e"
: isVisited
? "#34d399"
: "#334155";
const r = isAp ? 24 : 20;

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r={r}
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-all duration-300"
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events-none"
    >
      {node.label}
    </text>

    {isVisited && (
      <

```



```

    <p className="text-white font-medium min-h-
      {currentStep.desc}
    </p>
  </div>

```

```

<div className="space-y-4 flex-1 overflow-y-a
  <div className="mb-4">
    <h5 className="text-xs text-slate-500 fon
    <div className="flex items-center gap-2 t
      <div className="w-3 h-3 rounded-full bg
        Точка сочленения
      </div>
    <div className="flex items-center gap-2 t
      <div className="w-3 h-3 rounded-full bg
        Посещена
      </div>
    <div className="flex items-center gap-2 t
      <div className="w-3 h-3 rounded-full bg
        Текущая
      </div>
    <div className="flex items-center gap-2 t
      <div className="w-8 h-1 bg-rose-500 bor
        Мост
      </div>
    </div>
  </div>

```

```

<div>
  <h5 className="text-xs text-slate-500 fon
    ЛОГ ДЕЙСТВИЙ:
  </h5>
  <div className="space-y-2">
    {steps
      .slice(0, currentStepIndex + 1)
      .reverse()
      .map((step, idx) => (
        <div
          key={idx}
          className={`text-xs p-2 rounded $

```

```

checkingEdge: { u: string; v: string; w: number } | null;
distances: { [key: string]: number };
previous: { [key: string]: string | null };
log: string;
hasNegativeCycle?: boolean;
activeCodeLine: number;
}

```

```

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
    и листать вручную.";

```

```

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState(false);

```

```

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

```

```

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
      { u: 'C', v: 'E', w: 4 },
      { u: 'D', v: 'F', w: 2 },

```

```

        iteration: 0, checkingEdge: null,
        distances: { ...dist }, previous: { ...prev },
        log: '    Фура заведена. Начинаем с Базы (S). Расст
        activeCodeLine: 2,
    });

```

```

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
    let anyUpdate = false;
    for (const edge of edges) {
        const uDist = dist[edge.u];
        const vDist = dist[edge.v];

        let updated = false;
        if (uDist !== 999 && uDist + edge.w < vDist) {
            dist = { ...dist, [edge.v]: uDist + edge.w };
            prev = { ...prev, [edge.v]: edge.u };
            updated = true;
            anyUpdate = true;
        }
    }

```

```

    simulationSteps.push({
        iteration: iter, checkingEdge: { u: edge.u, v
        distances: { ...dist }, previous: { ...prev }
        log: `Итерация ${iter}: Проверяем ${edge.u} →
            updated ? ` Релаксация успешна! dist[${edge
        }`,
        activeCodeLine: updated ? 6 : 5,
    });
}

```

```

if (!anyUpdate && iter < vCount - 1) {
    simulationSteps.push({
        iteration: iter, checkingEdge: null, distance
        log: `    Ранний выход после итерации ${iter}.
        activeCodeLine: 10
    });
    break;
}

```

```

        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Зарплата...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null || liveV !== null || liveW !== null || liveDist !== null;
const currentStep: Step = compilerLinked
    ? {
        ...baseStep,
        iteration: liveI ?? 0,
        checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
        distances: Object.fromEntries(nodes.map((node) => [node, liveDist[node]])),
        log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU ?? "-"} - ${liveV ?? "-"}, вес ${liveW ?? "-"}, расстояние ${liveDist[liveU ?? "-"] ?? "-"}
    }
    : baseStep;

return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
        <div className="flex flex-wrap items-center justify-center gap-3">
            <div className="flex items-center gap-3">
                <div className="p-2.5 bg-blue-600 text-white rounded-md">
                    <span className="text-2xl"> </span>
                </div>
                <div>
                    <h3 className="text-2xl font-bold text-white">
                        <p className="text-sm text-blue-300">Полный
                    </p>
                </div>
            </div>
        </div>

        <div className="flex items-center gap-3">

```

```

        ▲ Отрицательный цикл!
    </div>
  })

<svg className="w-full h-auto max-h-[500px]" >
  <defs>
    <marker id="arrow-bf-normal" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
    <marker id="arrow-bf-active" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
    <marker id="arrow-bf-cycle" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
  </defs>

  {edges.map((edge, idx) => {
    const uNode = nodes.find((n) => n.id === u);
    const vNode = nodes.find((n) => n.id === v);
    const isChecking = currentStep.checkingEdges;

    const isNeg = edge.w < 0;
    const isCyclePart = hasCycle && ((edge.u === u && edge.v === 'D'));

    let strokeColor = '#475569';
    let marker = 'url(#arrow-bf-normal)';
    if (isChecking) { strokeColor = '#f59e0b'; }
    else if (currentStep.hasNegativeCycle && isCyclePart) {
      strokeColor = '#e11d48';
      marker = 'url(#arrow-bf-cycle)';
    }

    return (
      <g key={idx}>
        <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} stroke-width={isChecking && isCyclePart ? 4 : 2} marker={isChecking && isCyclePart ? '4,4' : 'none'} />
        <circle cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)} r={2} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} stroke={strokeColor} stroke-width={2} />
        <text x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Negative' : 'Cycle'} fontSize="11" fontWeight="bold" />
      </g>
    )
  })}
</svg>

```

```

        return (
          <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
          {edge.v} ({edge.w})
          </span>
        )
      }}}}
    </div>
  )}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-300">
      <div className="grid grid-cols-4 gap-2 text-rose-300">
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge === n.id;
          return (
            <div key={n.id} className={`p-2 rounded
: 'border-slate-700/60 bg-slate-800/40'}`}>
              <div className="text-xs text-slate-400">
                <div className={`font-mono font-bo
          v>
            </div>
          );
        }}}}
      </div>
    </div>
  </div>
</div>

  {/* Код алгоритма */}
  <div className="w-full lg:w-1/2 bg-slate-900/60">
    <div>

```

```
{ from: 1, to: 3 }, { from: 1, to: 4 },
{ from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
  activeLine: number;
  queue: number[];
  visited: number[];
  currentNode: number | null;
  logs: string;
}

const generateBfsSteps = (): Step[] => {
  const steps: Step[] = [];
  const queue: number[] = [];
  const visited = new Set<number>();

  // Init
  steps.push({
    activeLine: 1,
    queue: [],
    visited: [],
    currentNode: null,
    logs: "Инициализация: начинаем с корня (0).",
  });

  queue.push(0);
  visited.add(0);
  steps.push({
    activeLine: 2,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: "Добавляем стартовую вершину 0 в очередь и по"
```

```

        visited: Array.from(visited),
        currentNode: curr,
        logs: `Помечаем ${n} как посещенную и добавляем в очередь.`
    ));
    }
  }
} else {
  steps.push({
    activeLine: 6,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: curr,
    logs: `Соседей нет или все уже посещены.`
  });
}
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
  }, []);

```



```

        >
        {isPlaying ? <Pause className="w-5 h-5" />
        </button>
        <button
            onClick={handleNext}
            disabled={stepIdx >= STEPS.length - 1}
            className="p-2 text-emerald-400 hover:text-
            ed: hover: bg-transparent"
            title="Шаг вперед"
        >
            <StepForward className="w-5 h-5" />
        </button>
    </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
    { /* Визуал графа */ }
    <div className="bg-slate-800 border border-slate
        50px]">
        <svg className="w-full h-full min-h-[350px]" >
            { /* Рёбра */ }
            { EDGES.map((e, idx) => {
                const n1 = NODES.find(n => n.id === e.from)
                const n2 = NODES.find(n => n.id === e.to)
                const isVisited = step.visited.includes(n
                return (
                    <line
                        key={idx}
                        x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
                        className={`transition-all duration-500
                    `} `)
                </line>
            )
            })
        </svg>
        { /* Вершины */ }
        { NODES.map(n => {
            const isCurrent = step.currentNode === n.id
            return (
                <div
                    key={n.id}
                    className={`w-5 h-5 rounded-full
                    `} `)
            </div>
        )
        }
    }

```

```

        className="fill-white font-bold text-slate-400"
      >
        {n.label}
      </text>
    </g>
  );
  }}}
</svg>
</div>

```

{/* Код и Очередь */}

```

<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono se tracking-wider">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">Mar {step}</span>
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visited.add(start)' },
        { line: 3, code: '' },
        { line: 4, code: 'while not queue.isEmpty()' },
        { line: 5, code: '  node = queue.pop()' },
        { line: 6, code: '  for neighbor in graph[node]:' },
        { line: 7, code: '    if neighbor not in visited:' },
        { line: 8, code: '      visited.add(neighbor)' },
        { line: 9, code: '      queue.push(neighbor)' },
      ]}.map((cl) => {
        const isActive = step.activeLine === cl.line
        return (
          <div key={cl.line} className={`px-2 py-2 border-l-2 border-blue-400 -ml-0.5` : `text-slate-400`} >
            <span className="opacity-40 w-6 inline-block"> </span>
            <span className="whitespace-pre">{cl.code}</span>
          </div>
        )
      })
    </div>
  </div>

```

```
import dijkstraImg from '../assets/dijkstra-freakazoid.'
import bellmanImg from '../assets/bellman-ford-batman.jpg'
import floydImg from '../assets/floyd-network.jpg';

const imageMap: Record<string, { src: string; alt: string;
  dijkstra: {
    src: dijkstraImg,
    alt: 'Декстер Дуглас и Фриказоид',
    caption: '✂ Декстер Дуглас и Фриказоид – волна виру',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Бэтмен на машине Генри Форда',
    caption: '    Бэтмен на машине Форда – n-1 ночей патр',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Fortnite Все против Всех (Флойд–Уоршелл)',
    caption: '    Fortnite: Все против Всех – Флойд-Уорше',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 ${info.borderColor}>
        <img src={info.src} alt={info.alt} className="w-full h-full"/>
        <div>
          <p>{info.alt}</p>
          <p>{info.caption}</p>
        </div>
      </div>
    </div>
  );
}
```

```

        >
        <ChevronLeft className="w-4 h-4" />
        <span className="hidden sm:inline">Назад</span>
    </button>
    <span className="text-slate-500 font-mono tabular-nums">
        {index + 1} / {total}
    </span>
    <button
        disabled={!next}
        onClick={() => next && onGo(next.id)}
        title={next ? next.title : "Это последняя тема"}
        className="flex items-center gap-1 px-2.5 py-2 rounded-xl
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
    >
        <span className="hidden sm:inline">Вперёд</span>
        <ChevronRight className="w-4 h-4" />
    </button>
</div>
);
}

return (
    <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4">
        {prev ? (
            <button
                onClick={() => onGo(prev.id)}
                className="group text-left p-4 rounded-xl bg-white
                    transition-all"
            >
                <div className="flex items-center gap-1.5 text-slate-600">
                    <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая
                </div>
                <div className="text-sm font-bold text-slate-900">
                    {prev.title}
                </div>
            </button>
        ) : (
            <div className="hidden sm:block" />
        )}
    </nav>
);

```

* демонстрации: прочитал → посмотрел, как работает → протестировал
*
* В текстовой версии (`?lite=1`) и в PDF те же вопросы
* с ответами и разбором – там интерактив не нужен.
*/

```
export const ChapterQuiz: React.FC<Props> = ({ chapterId }) => {  
  const list = quizzes[chapterId];  
  const [idx, setIdx] = useState(0);  
  const [picked, setPicked] = useState<number | null>(null);  
  const [score, setScore] = useState(0);  
  const [finished, setFinished] = useState(false);
```

```
  // другая тема – другой блиц с нуля
```

```
  useEffect(() => {  
    setIdx(0);  
    setPicked(null);  
    setScore(0);  
    setFinished(false);  
  }, [chapterId]);
```

```
  if (!list || list.length === 0) return null;
```

```
  const q = list[idx];  
  const answered = picked !== null;  
  const correct = answered && picked === q.correctIndex;
```

```
  const pick = (i: number) => {  
    if (answered) return;  
    setPicked(i);  
    if (i === q.correctIndex) setScore((s) => s + 1);  
  };
```

```
  const next = () => {  
    if (idx + 1 < list.length) {  
      setIdx(idx + 1);  
      setPicked(null);  
    } else {  
      setFinished(true);  
    }  
  };
```

```

        className="inline-flex items-center gap-1
        -bold text-indigo-200 hover:bg-indigo-500/20
      >
        <RotateCcw className="w-3.5 h-3.5" /> Прогресс
      </button>
    </div>
  ) : (
    <>
      <p className="text-sm font-semibold text-white">
        <div className="space-y-2">
          {q.options.map((option, i) => {
            const isRight = i === q.correctIndex;
            const chosen = picked === i;
            const tone = !answered
              ? "border-slate-600 bg-slate-800/70 text-white"
              : isRight
                ? "border-emerald-500 bg-emerald-500/20 text-emerald-500"
                : chosen
                  ? "border-rose-500 bg-rose-500/15 text-rose-500"
                  : "border-slate-700 bg-slate-800/70 text-white";
            return (
              <button
                key={i}
                type="button"
                onClick={() => pick(i)}
                disabled={answered}
                className={`w-full text-left flex items-center justify-between
                  disabled:cursor-default ${tone}`}
              >
                {answered && isRight && <CheckCircle className="h-5 w-5" />}
                {answered && chosen && !isRight && <X className="h-5 w-5" />}
                <span>{option}</span>
              </button>
            );
          })}
        </div>
      </div>
    {answered && (

```

```

import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermPopover";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";
import ChapterImage from "../ChapterImage";
import MnemonicCards from "../MnemonicCards";

```

```

interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
  /** Перейти во вкладку компилятора – там уже подставлен код */
  onOpenCompiler?: () => void;
}

```

```

const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;

```

```

export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator, onOpenCompiler }) => {
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
  const [saving, setSaving] = useState<"idle" | "work">("idle");

```

```

  useTermHints(contentRef, [chapter.id]);

```

```

  /**

```

```

    * Упоминания «билет 15» становятся переходами на стр. 15

```

```

    * Разметка идёт ПОСЛЕ терминов: подсказки глоссария

```

```

    */

```

```

window.addEventListener("beforeprint", openAll);
window.addEventListener("afterprint", restore);
return () => {
  window.removeEventListener("beforeprint", openAll);
  window.removeEventListener("afterprint", restore);
};
}, [chapter.id]));

const saveHtml = useCallback(async () => {
  setSaving("work");
  try {
    await downloadChapterHtml(chapter);
    setSaving("done");
    window.setTimeout(() => setSaving("idle"), 2500);
  } catch {
    setSaving("idle");
  }
}, [chapter]);

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
    cancelHide();
    setAnchor({ termId, rect: el.getBoundingClientRect() });
  },
  [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
  if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>(".term");

```



```

        <Loader2 className="w-3.5 h-3.5 animate-s
    ) : saving === "done" ? (
        <Check className="w-3.5 h-3.5 text-emeral
    ) : (
        <Download className="w-3.5 h-3.5" />
    )}
    {saving === "done" ? "Файл сохранён" : "Ска
</button>
<button
    onClick={() => window.print()}
    title="Распечатать или сохранить эту тему в
страция, без шапки, содержания и панели ком
    className="flex items-center gap-1.5 text-x
    300 hover:text-white hover:border-indigo-500
>
    <Printer className="w-3.5 h-3.5" /> PDF это
</button>
<a
    href={`?lite=1&topic=${encodeURIComponent(c
    target="_blank"
    rel="noreferrer"
    title="Текстовая версия темы без навигации,
еры"
    className="flex items-center gap-1.5 text-x
    300 hover:text-white hover:border-sky-500 t
>
    <FileText className="w-3.5 h-3.5" /> Текст
</a>
    <span className="text-[11px] text-slate-500">
</div>

<div
    ref={contentRef}
    className="prose prose-invert max-w-none"
    onMouseOver={handlePointerOver}
    onMouseOut={handlePointerOut}
    onClick={handleClick}
    onFocus={handleFocus}

```

```

        title="Открыть Python: код выполняе
рации"
        className="flex items-center gap-1.5
t-emerald-400 hover:text-white hover:border
    >
        <Terminal className="w-3.5 h-3.5" />
    </button>
    })
</div>
</div>
{viz.hint && <p className="text-xs text-sla
<VizChapterContext.Provider value={chapter.
    <VizLiveStatus demoTitle={viz.title} clas
    <viz.Component />
</VizChapterContext.Provider>
</section>
}}

{/* прочитал → посмотрел, как работает → провер
<ChapterQuiz chapterId={chapter.id} />

    <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
  anchor={anchor}
  onClose={() => setAnchor(null)}
  onOpenSimulator={(id) => {
    setAnchor(null);
    onOpenSimulator(id);
  }}
  onCardEnter={cancelHide}
  onCardLeave={scheduleHide}
/>
</>
);
};

```

```
const CLASSES = [
  { id: "EXP", label: "EXP", sub: "2^{n^k}", color: "#0000FF",
    {кванторные формулы}} ],
  { id: "PSPACE", label: "PSPACE", sub: "полиномиальная
    с полной информацией"} ],
  { id: "NP", label: "NP", sub: "сертификат проверяется
    AT", "гамильтонов цикл", "раскраска в 3 цвета", "клика"},
  { id: "P", label: "P", sub: "решается за O(n^k)", color: "red",
    "ид", "MST", "2-SAT, двудольность"} ],
];
```

```
const NPHARD_NOTE = "NP-трудные — всё, к чему сводится
  гамильтонов цикл. Проблема остановки NP-трудна, но в NP."
```

```
/* ----- Сведение -----
```

```
const REDUCE_STEPS = [
  { title: "Дано: задача A, для которой быстрого алгоритма
    — то есть B NP-трудная.", boxes: ["A"], active: 0 },
  { title: "Берём вход x задачи A", note: "x — конкретный элемент
    "x"]}, active: 1 },
  { title: "Строим f(x) = y за полиномиальное время", note: "f(x)
    шинами из разных дизъюнктов, если литералы не противоречат",
    ve: 2 },
  { title: "Решаем B на входе y", note: "Гипотетический алгоритм
    — но допустим, что он есть.", boxes: ["A", "x", "f(x)", "y"]},
  { title: "Переводим ответ обратно: A(x) = B(f(x))", note: "A(x)
    ⇔ формула выполнима. Полиномиальное время сохранилось",
    5 },
  { title: "Вывод: B не проще A", note: "Раз A ≤p B и A NP-полна,
    -полна.", boxes: ["A", "x", "f", "y", "B", "ans"]},
];
```

```
const BOX_LABEL: Record<string, { t: string; c: string }> = {
  A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
  x: { t: "вход x\n|x| = n", c: "#64748b" },
  f: { t: "f: сведение\nO(poly(n))", c: "#f59e0b" },
  y: { t: "вход y\n|y| = poly(n)", c: "#64748b" },
};
```

```

const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;
const [tab, setTab] = useState<Tab>("hierarchy");
const [step, setStep] = useState(0);
const [playing, setPlaying] = useState(false);
const [samples, setSamples] = useState<Sample[]>([]);

useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);

const maxStep = tab === "reduce" ? REDUCE_STEPS.length : 0;
useEffect(() => {
  setStep(0);
  setPlaying(false);
}, [tab]);

useEffect(() => {
  if (!playing) return;
  const t = window.setInterval(() => {
    setStep((s) => {
      if (s >= maxStep) {
        setPlaying(false);
        return s;
      }
      return s + 1;
    });
  }, 1400);
  return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? (vizNumber(vars.step) ?? vizNumber(0)) : null;
useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep))
    setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

```

```

    const q = pts.slice(half);
    if (q.length < 2) return slope;
    const lx = q.map((p) => Math.log(p.n));
    const ly = q.map((p) => Math.log(p.ops));
    const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
    const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
    let nn = 0;
    let dd = 0;
    for (let i = 0; i < lx.length; i++) {
      nn += (lx[i] - ax) * (ly[i] - ay);
      dd += (lx[i] - ax) ** 2;
    }
    return dd === 0 ? slope : nn / dd;
  })());
const cls =
  slopeTail > slope + 0.8 || slopeTail > 4.5
    ? { name: "экспонента / суперполином", color: "#f59e0b",
      chi: true }
    : slope <= 1.25
    ? { name: "≈ линейный,  $O(n)$ ", color: "#10b981",
      chi: false }
    : slope <= 1.8
    ? { name: "≈  $O(n \log n)$ ", color: "#06b6d4", hint: true,
      chi: false }
    : slope <= 2.4
    ? { name: "≈ квадратичный,  $O(n^2)$ ", color: "#f59e0b",
      chi: false }
    : slope <= 3.4
    ? { name: "≈ кубический,  $O(n^3)$ ", color: "#fb9238",
      chi: false }
    : { name: "полином высокой степени", color: "#fb9238",
      chi: false };
return { slope, slopeTail, ...cls };
}, [samples]);

```

```

const reduceFrame = REDUCE_STEPS[Math.min(step, REDUCE_STEPS.length - 1)];
const verifyIdx = Math.min(step, VERIFY_STEPS.length);
const verifyDone = verifyIdx >= VERIFY_STEPS.length;
const allOk = VERIFY_STEPS.every((s) => s.ok);

```

```

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl" style={style} >
    <div className="flex flex-wrap items-center justify-center" >

```

```

    <span className="text-[11px] text-slate-500 m
  </div>
})

{/* ----- ИЕРАРХИЯ -----
{tab === "hierarchy" && (
  <div className="space-y-4">
    <div className="bg-slate-950 rounded-xl p-3 b
      <svg viewBox="0 0 500 380" className="w-full
        {CLASSES.map((c) => {
          const on = liveCls ? liveCls.toUpperCas
          return (
            <g key={c.id}>
              <ellipse cx={c.x} cy={c.y} rx={c.rx
                {on ? 3 : 1.5} strokeDasharray={on ? undefi
                  <text x={c.x} y={c.y - c.ry + 18} t
                  <text x={c.x} y={c.y - c.ry + 32} t
                </g>
              );
            )}}
          {/* примеры задач – внутри своих овалов */
          {CLASSES.map((c) =>
            c.tasks.map((t, i) => {
              <text key={t} x={c.x} y={c.y + (c.id :
                tAnchor="middle" fill="#e2e8f0" fontSize="8
                  · {t}
                </text>
              ))
            ))
          {/* NP-полные = пересечение */}
          <text x={250} y={352} textAnchor="middle"
          <text x={250} y={368} textAnchor="middle"
          рхии времени)</text>
        </svg>
      </div>
    <p className="text-xs text-slate-400 leading-
    <p className="text-[11px] text-slate-500">
      Подсказка: выполни в компиляторе <code clas

```

```

    })

    { /* ----- ВЕРИФИКАТОР -----
    {tab === "verify" && (
      <div className="space-y-3">
        <div className="bg-slate-950 rounded-xl p-4 b
          <p className="text-[10px] uppercase tracking
            <div className="flex flex-wrap gap-2 mb-3">
              {CERT.map((v, i) => {
                <span key={i} className={"font-mono tex
                  ld-600/50 text-emerald-300" : "bg-slate-900
                    x{i + 1} = {liveCert && typeof liveCe
                  </span>
                )))
              {liveCert && <span className="font-mono t
                le-950/40">cert из кода</span>}
            </div>
            <div className="space-y-2">
              {CLAUSES.map((cl, i) => {
                const checked = liveK !== null ? i < li
                const current = liveK !== null ? i ===
                const st = VERIFY_STEPS[i];
                return (
                  <div
                    key={i}
                    className={"flex items-center gap-3
                      amber-950/20" : checked ? {st.ok ? "border-
                        slate-800 bg-slate-900/40"}}
                    >
                      <span className="text-[10px] font-b
                      <span className="font-mono text-xs
                    span>
                      <span className="font-mono text-[10
                        " )}</span>
                      <span className={"text-[11px] font-l
                        0" )}>
                        {checked ? {st.ok ? "✓ истинно" :
                      </span>

```

```

<text x={16} y={110} fill="#64748b" fontS
{GROWTH_LINES.map((g) => {
  const pts: string[] = [];
  for (let i = 0; i <= 20; i++) {
    const n = 1 + i * 0.8;
    const v = g.fn(n);
    const x = 44 + (Math.log(n) / Math.lo
    const y = 200 - (Math.log(Math.max(1,
    pts.push(`${x.toFixed(1)} ${Math.max(
  }
  return <polyline key={g.id} points={pts
acity={0.55} />;
  )))
  {samples.map((s) => {
    const x = 44 + (Math.log(Math.max(1, s.n
    const y = 200 - (Math.log(Math.max(1, s
    return <circle key={` ${s.n}`} cx={Math.n
strokeWidth={2} />;
  )))
  {GROWTH_LINES.map((g, i) => (
    <g key={g.id + "l"}>
      <rect x={330 + (i % 3) * 58} y={16 + 1
      <text x={343 + (i % 3) * 58} y={24 + 1
    </g>
  )))
</svg>
{samples.length === 0 && (
  <p className="text-xs text-slate-500 text
    Пусто. Запусти в компиляторе код с пере
t-mono text-purple-300">ops</code> – напри
  </p>
)}
</div>
<div className={"rounded-xl border p-3.5 " +
  /60"}>
  <p className="text-sm font-bold text-white
    {verdict ? `Похуже на ${verdict.name}` :
  </p>

```


ФАЙЛ 39/111: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";
```

```
interface CodeLine {
  line: string;
  highlight: string;
}
```

```
const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
  { line: "    visited[v] = true;", highlight: "normal" },
  { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "normal" },
  { line: "        if (to == p) continue;", highlight: "normal" },
  { line: "        if (visited[to]) {", highlight: "normal" },
  { line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
  { line: "        } else {", highlight: "normal" },
  { line: "            dfs(to, v);", highlight: "normal" },
  { line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "        if (low[to] > tin[v]) {", highlight: "normal" },
  { line: "            // ЭТО МОСТ!", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "    }", highlight: "normal" },
  { line: "}", highlight: "normal" },
];
```

```
interface DfsStep {
  currentNode: number | null;
}
```

```

    activeCodeLine: [0,1,2]
},
{
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridge: null,
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 6, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: [1,6], backEdge: null,
    log: "Из B идём в G. tin[G]=3, low[G]=3.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 1, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: null, backEdge: null,
    log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
    activeCodeLine: [13,14,15]
},
{
    currentNode: 2, visitedIndices: [0,1,6,2],
    tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
    activeEdge: [1,2], backEdge: null,
    log: "Из B идём в C. tin[C]=4, low[C]=4.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 2, visitedIndices: [0,1,6,2],
    tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,6,2],
    activeEdge: null, backEdge: [2,0],
    log: "Видим соседа A (уже посещён)! Обратное ребро ",
    activeCodeLine: [7,8,9]
},
{

```

```

    log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост",
    activeCodeLine: [11,13,16,17]
  },
  {
    currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост",
    activeCodeLine: [13,14,15,16,17]
  },
  {
    currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост",
    activeCodeLine: [11,13,16,17]
  },
  {
    currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D)",
    activeCodeLine: [11,13,16,17,18]
  },
  {
    currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм DFS завершён",
    activeCodeLine: []
  },
];

export function DfsBridgesSimulator() {
  const [dfsIdx, setDfsIdx] = useState(0);
  useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length - 1);
  const rawV = vizString(vars.v) ?? (vizNumber(vars.v) ?? 0);
  const rawTo = vizString(vars.to) ?? (vizNumber(vars.to) ?? 0);

```

```

        className="p-1.5 hover:bg-slate-700 rounded
        <Undo className="h-3.5 w-3.5" />
    </button>
    <button onClick={() => setDfsAuto(!dfsAuto)}
        className={`px-2.5 py-1 rounded text-[10px]
            dfsAuto ? "bg-rose-600 text-white" : "bg-
        }`>
        {dfsAuto ? <><Pause className="h-3 w-3" />
    </button>
    <button onClick={() => { setDfsAuto(false); s
        disabled={dfsIdx === DFS_STEPS.length-1}
        aria-label="Шаг вперёд" title="Шаг вперёд"
        className="p-1.5 hover:bg-slate-700 rounded
        <SkipForward className="h-3.5 w-3.5" />
    </button>
    <button onClick={() => { setDfsAuto(false); s
        aria-label="Сбросить DFS" title="Сбросить D
        className="p-1.5 hover:bg-slate-700 rounded
        <RefreshCw className="h-3.5 w-3.5" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 g
    {/* Graph visualization */}
    <div className="md:col-span-3 bg-slate-950 roun
        <div className="absolute inset-0 bg-[radial-g
            >
        <svg className="w-full h-[280px] z-10 relativ
            {GRAPH_EDGES.map(e => {
                const u = GRAPH_NODES.find(n => n.id ===
                const v = GRAPH_NODES.find(n => n.id ===
                const bridge = step.bridges.includes(e.ke
                const activeTree = step.activeEdge && (
                    (step.activeEdge[0]===e.u && step.active
                    (step.activeEdge[0]===e.v && step.active
                ));
                const activeBack = step.backEdge && {

```

```

        </g>;
      }}}}
    </svg>
    <div className="flex items-center justify-between" >
      <span>Шаг {dfsIdx+1}/{DFS_STEPS.length}</span>
      <div className="flex-1 mx-2 bg-slate-950 h-10" >
        <div className="bg-indigo-600 h-full transition-all duration-200" >
          <div>
            </div>
          </div>
        </div>
      </div>

    {/* Code panel + stack */}
    <div className="md:col-span-2 space-y-3">
      {/* Code panel */}
      <div className="bg-slate-950 rounded-lg border border-slate-800" >
        <div className="bg-slate-900 px-3 py-1.5 text-slate-300" >
          <div className="p-2 overflow-x-auto" >
            <pre className="text-[10px] font-mono leading-relaxed" >
              {CODE_LINES.map(({cl, i}) => {
                const hl = step.activeCodeLine.includes(cl)
                return <div key={i}
                  className={`px-1 transition-all duration-200
                    ${hl ? "bg-indigo-600/20 text-indigo-300" : "text-slate-400"}
                  >
                <span className="text-slate-600 mr-1" >
                  {cl.line || ' '}
                </div>;
              })}
            </pre>
          </div>
        </div>

      {/* Stack */}
      <div className="bg-slate-900/50 rounded-xl border border-slate-800" >
        <div className="text-[10px] font-bold text-slate-400" >
          <span>Стек DFS</span>
        </div>
      </div>
    </div>
  )}
}

```

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';
```

```
interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
    и листать вручную.";
```

```
export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];
```

```
  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
```

```

const prev: Record<string, string | null> = { A: null };

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ` Дейкстра готов к доставке. Начинаем из ПИ.`,
  activeCodeLine: 2,
});

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }

  if (!u) break;

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ` Выбираем вершину ${u} с мин. непосещённой`,
    activeCodeLine: 5,
  });

  const neighbors = edges.filter(e => e.u === u);
  for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
      currentNode: u, checkingEdge: { u, v },
      distances: { ...dist }, visited: { ...visited },
      log: ` Смотрим соседа ${v}. Текущее расстояние: ${dist[u]} + ${edge.w} = ${dist[v]}`,
      activeCodeLine: 6,
    });
  }
}

```

```

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(var
const compilerLinked = liveU !== null || liveV !== nu
const currentStep: Step = compilerLinked
  ? {
    ...baseStep,
    currentNode: liveU,
    checkingEdge: liveU && liveV ? { u: liveU, v: l
    distances: Object.fromEntries(nodes.map((node) =>
    previous: Object.fromEntries(nodes.map((node) =>
    visited: Object.fromEntries(nodes.map((node) =>
    log: `Компилятор: u=${liveU ?? "-"}, v=${liveV
  }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl bor
    { /* Шапка симулятора */ }
    <div className="flex flex-wrap items-center justi
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-whit
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-whit
            <p className="text-sm text-indigo-300">Полн
          </div>
        </div>
      </div>

    <div className="flex flex-wrap items-center jus

```



```

/>

```

```

<div className="flex flex-col lg:flex-row gap-6 m
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-indigo-950/20
    justify-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-indigo-950/20
      font-bold shadow-sm">
      Шаг {currentStepIndex + 1} из {steps.length}
    </div>

    <svg className="w-full h-auto max-h-[500px]" >
      <defs>
        <marker id="arrow-dh-normal" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#47556b" />
        </marker>
        <marker id="arrow-dh-active" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
        </marker>
        <marker id="arrow-dh-opt" markerWidth="6"
          <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
        </marker>
      </defs>
      {/* Рёбра */}
      {edges.map((edge, idx) => {
        const uNode = nodes.find((n) => n.id === edge.u)
        const vNode = nodes.find((n) => n.id === edge.v)
        const isChecking =
          currentStep.checkingEdge &&
            (currentStep.checkingEdge.u === edge.u &&
              currentStep.checkingEdge.v === edge.v)

        const isOptimal = currentStep.previous[edge.u] === edge.v

        let marker = 'url(#arrow-dh-normal)';
        if (isChecking) marker = 'url(#arrow-dh-active)';
        else if (isOptimal) marker = 'url(#arrow-dh-opt)';

        return (

```

```

        stroke={isCurrent ? '#a5b4fc' : isV
        strokeWidth={isCurrent ? 4 : 2}
        className="transition-all duration-1
    />
    <text x={node.x} y={node.y + 5} fill=
        {node.id}
    </text>
    <text x={node.x} y={node.y - 28} fill=
    "middle" className="bg-indigo-950 px-1 py-0
        {dist === 999 ? '∞' : `d=${dist}`}
    </text>
    <text x={node.x} y={node.y + 35} fill=
        {node.name.split(' ')[0]}
    </text>
    </g>
    );
    }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 ro
    <p className="font-semibold text-amber-400
    <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <div className="space-y-2.5 flex-1 overflow-y
        {Object.keys(adjList).map((u) => {
            const isCurrentNode = currentStep.current
            return (
                <div key={u} className={`p-3 rounded-lg
                    isCurrentNode ? 'bg-emerald-900/60
                t-slate-300'
                }`>
                <div className="flex items-center gap
                    <span className={`px-2 py-0.5 round
                te-300'}`>

```

```

        </tr>
      </thead>
      <tbody className="text-sm">
        {nodes.map((n) => {
          const dist = currentStep.distances[n.id]
          const prev = currentStep.previous[n.id]
          const vis = currentStep.visited[n.id]
          const isCurr = currentStep.currentNode === n.id

          return (
            <tr key={n.id} className={`border-1
              isCurr ? 'bg-indigo-950/60 font-medium' : ''}>
              <td className="py-2.5 px-3 flex items-center">
                <span className={`w-2 h-2 rounded-full ${vis ? 'bg-indigo-500' : 'bg-gray-300'} mr-2`} />
                {n.name}
              </td>
              <td className="py-2.5 px-3 font-medium">
                {dist === 999 ? '∞' : dist}
              </td>
              <td className="py-2.5 px-3 text-sm">
                {prev || '-'}
              </td>
            </tr>
          );
        })}
      </tbody>
    </table>
  </div>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
  <div>
    <h4 className="text-lg font-bold text-slate-200">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map((item) => {

```

```
const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "    if (n in memo) return memo[n];",
  /* 3 */ "    if (n <= 2) return 1;",
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo",
  /* 5 */ "    return memo[n];",
  /* 6 */ "}"
];
```

```
export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |
```

```
useEffect(() => {
  const generatedSteps: DPStep[] = [];
  let stepId = 0;
  const memoMap: { [key: number]: number } = {};
```

```
function simulateFib(n: number): number {
  generatedSteps.push({
    id: stepId++,
    n,
    action: 'call',
    desc: `Вызов fibMemo(${n}). Проверяем наличие в
    codeLine: 2,
    memoState: { ...memoMap }
  });
```

```
if (n in memoMap) {
  generatedSteps.push({
    id: stepId++,
    n,
```

```

        codeLine: 5,
        memoState: { ...memoMap }
    });

    return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
  id: stepId++,
  n: targetN,
  action: 'done',
  desc: `  Расчет завершен! fibMemo(${targetN}) = $
  codeLine: 6,
  memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => {
      setCurrentStepIndex(prev => prev + 1);
    }, speed);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]));

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n

```

```

        onChange={(e) => setTargetN(Number(e.target.value))}
        disabled={isPlaying}
        className="bg-transparent text-white font-mono"
      >
        <option value={4}>N = 4</option>
        <option value={5}>N = 5</option>
        <option value={6}>N = 6</option>
        <option value={7}>N = 7</option>
      </select>
    </div>

    <div className="flex items-center bg-slate-950" >
      <button
        onClick={() => { setIsPlaying(false); setTargetN(0); }}
        className="p-2 text-slate-400 hover:text-white"
        title="Сбросить"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
      <button
        onClick={() => { setIsPlaying(false); setTargetN(0); }}
        disabled={currentStepIndex === 0}
        className="p-2 text-slate-400 hover:text-white"
        title="Шаг назад"
      >
        <ChevronLeft className="w-5 h-5" />
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1.5 px-2 py-2
          ${isPlaying
            ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
            : 'bg-emerald-600 text-white hover:bg-emerald-700'}
        `}
      >
        {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
        {isPlaying ? 'Пауза' : 'Пуск'}
      </button>
    </div>
  </div>
</div>

```

```

        });
      }}}
    </div>

    {currentStep && (
      <div className="mt-6 pt-4 border-t border-slate-400">
        <div className="text-sm font-medium text-slate-400">
          <span className="inline-block w-2 h-2 rounded-full bg-slate-400 mr-2"></span>
          <span>{currentStep.desc}</span>
        </div>
        <div className="text-xs font-mono bg-slate-400 p-2">
          Текущий N: <strong className="text-emerald-400">{currentStep.n}</strong>
        </div>
      </div>
    )}
  </div>

  {/* Tabs */}
  <div className="flex items-center gap-2 border-b border-slate-400">
    <button
      onClick={() => setActiveTab('table')}
      className={`flex items-center gap-2 px-6 py-3 border ${activeTab === 'table' ? 'border-emerald-500 text-emerald-400 bg-slate-400' : 'border-transparent text-slate-400 hover:text-slate-300'} rounded`}
    >
      <Eye className="w-4 h-4" />
      Шаг-бай-шаг состояние
    </button>
    <button
      onClick={() => setActiveTab('code')}
      className={`flex items-center gap-2 px-6 py-3 border ${activeTab === 'code' ? 'border-emerald-500 text-emerald-400 bg-slate-400' : 'border-transparent text-slate-400 hover:text-slate-300'} rounded`}
    >
      Код
    </button>
  </div>

```

```

        isActive
        ? 'bg-emerald-600/20 border-l-4
        : 'text-slate-300 hover:bg-slate-
    }` }
    >
    <span className="w-8 text-xs text-s
    <span>{line}</span>
  </div>
  );
  }}}
</pre>
<div className="p-6 bg-slate-900/50 border-
  <span className="p-1.5 bg-emerald-500/20
  <span>{currentStep?.desc}</span>
</div>
</div>
  })
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-8
  <div className="flex items-center justify-between
    <span>Прогресс выполнения</span>
    <span>№ {currentStepIndex + 1} из {steps.length}</span>
  </div>
  <div className="h-2 bg-slate-950 rounded-full overflow-hidden
    <div
      className="h-full bg-gradient-to-r from-emerald-500 to-emerald-600
      style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%`
    ></div>
  </div>
</div>
</div>
  );
}

```

```

*/
export function EulerSimulator() {
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;

  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Кликни на вершину
  const [c1Done, setC1Done] = useState(false);

  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, "walk");
  }, [chapterId]);

  const resetC1 = () => {
    setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
  };

  const clickC1 = (vId: number) => {
    if (c1Done && c1Path.length > 0) return;
    if (c1Path.length === 0) {
      setC1Path([vId]);
      setC1Msg("Старт! Кликай соседние вершины, чтобы п
      return;
    }
    const last = c1Path[c1Path.length - 1];
    const edge = C1_EDGES.find(e => (e.u === last && e.v
    if (!edge) {
      setC1Msg("Нет ребра между этими вершинами! Выбери
      return;
    }
    const key = ekey(last, vId);
    if (c1VisitedEdges.includes(key)) {
      setC1Msg("Это ребро уже пройдено! Эйлер не прощае
      setC1Done(true); return;
    }
    const nextEdges = [...c1VisitedEdges, key];

```

```

    if (path.length === 0 && usedFlags.length === 0 &&
        // рёбра маршрута: пары соседних вершин
        const keys: string[] = [];
        for (let i = 0; i + 1 < path.length; i++) keys.push
        usedFlags.forEach((f, i) => {
            if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i]).
        }));
        return {
            path,
            keys: Array.from(new Set(keys)),
            deg,
            odd,
            last: last !== null && last >= 0 && last < C1_NODES,
            step,
            complete: keys.length >= C1_EDGES.length,
        };
    }, [vars]));

const path = live && live.path.length ? live.path : c
const visitedKeys = live ? live.keys : c1VisitedEdges
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[
const done = live ? live.complete : c1Done && visited
const msg = live
    ? live.complete
    ? `Компилятор прошёл все ${C1_EDGES.length} рёб
        шин ${odd.length} → это ${odd.length === 0 ? "ц
    : `Маршрут из кода: ${live.path.length ? live.pat
        ys.length}/${C1_EDGES.length}`
    : c1Msg;

const verdict =
    odd.length === 0
    ? { text: "0 нечётных → ЭЙЛЕРОВ ЦИКЛ есть (старт
        }
    : odd.length === 2
    ? { text: `2 нечётные (${odd.map((id) => C1_NODES

```

```

        </g>;
    }}}
    {C1_NODES.map(n => {
      const isLast = lastId === n.id;
      const visited = path.includes(n.id);
      const isOdd = odd.includes(n.id);
      return <g key={n.id} className="cursor-point
        {isLast && <circle cx={n.x} cy={n.y} r={11}
        <circle cx={n.x} cy={n.y} r={11}
          fill={isLast ? "#6366f1" : visited ? "#
          stroke={isLast ? "#fff" : visited ? "#6
          strokeWidth={2.5}
          className="transition-all duration-200
        <text x={n.x} y={n.y + 4} textAnchor="mid
        label}</text>
        <text x={n.x} y={n.y - 16} textAnchor="mi
        one">deg {deg[n.id]}{isOdd ? " • нечёт" : "
      </g>;
    }}}
  </svg>
  <div className="absolute top-2 left-2 bg-slate-
    Ребёр: {visitedKeys.length}/{C1_EDGES.length}
  </div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
  done && visitedKeys.length === C1_EDGES.length
    ? "bg-emerald-950/40 border-emerald-500/30 te
    : c1Done && !live
    ? "bg-rose-950/40 border-rose-500/30 text-ros
    : "bg-slate-900/50 border-slate-700 text-slate
  `}>
  {msg}
</div>

```

/* Критерий Эйлера */

```

<div className={`p-3 rounded-xl border text-xs fo
  Критерий Эйлера (для связного графа): {verdict.

```

```
-----
import { useState, useEffect } from 'react';
import { liveMatrix, pickLiveVar, useVizRuntime, vizNum

interface Step {
  k: number; i: number; j: number;
  matrix: (number | null)[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
    ва листать вручную.";

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  // k/i/j/d приходят напрямую; номер строки произвольн
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];

  const initialMatrix = [
    [0, 4, 999, 2, 999, 999, 999],
    [999, 0, 3, 999, 999, 3, 999],
    [999, 999, 0, 999, 2, 999, 999],
    [999, 999, 999, 0, 4, 2, 999],
    [999, 999, 999, 999, 0, 999, 1],
    [999, 999, 999, 999, 999, 0, 5],
```

```

    log: `+ Внешний цикл. Разрешаем пути через пром
    activeCodeLine: 3,
  });

  for (let i = 0; i < N; i++) {
    for (let j = 0; j < N; j++) {
      if (i === j || i === k || j === k) continue;

      const oldVal = dist[i][j];
      const viaVal = dist[i][k] + dist[k][j];
      if (dist[i][k] !== 999 && dist[k][j] !== 999 &
        dist[i][j] = viaVal;
      simulationSteps.push({
        k, i, j, matrix: dist.map(r => [...r]), u
        log: ` Улучшение пути [${i}]→[${j}] чере
        ∞'.`,
        activeCodeLine: 7,
      });
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
  log: ` Все пары отработаны. Алгоритм Флойда завер
  activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => setCurrentStepIndex((p)

```



```

    {nodesSvg.map((node) => {
      const isK = currentStep.k === node.id;
      const isI = currentStep.i === node.id;
      const isJ = currentStep.j === node.id;
      let fill = '#1e293b'; let stroke = '#64748f';
      if (isK) { fill = '#4f46e5'; stroke = '#a8a2d8'; }
      else if (isI) { fill = '#b45309'; stroke = '#c99e3b'; }
      else if (isJ) { fill = '#047857'; stroke = '#2e8b57'; }
      return (
        <g key={node.id} className="transition-colors">
          <circle cx={node.x} cy={node.y} r={isK ? 4 : 2} className="transition-colors duration-200"/>
          <text x={node.x} y={node.y + 5} fill="white" font-size="10px">{node.k}</text>
          <text x={node.x} y={node.y + 32} fill="white" font-size="10px">{node.i}</text>
        </g>
      );
    })}
  </svg>

  <div className="w-full bg-slate-950/80 p-4 rounded">
    <p className="font-semibold text-amber-400">Список смежности</p>
    <p className="min-h-[40px] flex items-center"></p>
  </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50">
  <h4 className="text-lg font-bold text-emerald-400">Список смежности</h4>
  <div className="space-y-2 text-sm font-mono">
    {Object.keys(adjList).map((uStr) => {
      const u = parseInt(uStr);
      const isI = currentStep.i === u;
      const isK = currentStep.k === u;
      return (
        <div key={u} className={`flex flex-wrap items-center justify-between p-2 border border-1 rounded` +
          (isI ? 'bg-amber-900/40 border-amber-400' :
            isK ? 'bg-indigo-900/40 border-indigo-400' :
            'bg-slate-900/40 border-slate-400')}>

```

```

        labelOf(idx)) {idx}</th>
      )))
    </tr></thead>
    <tbody className="text-xs font-mono">
      {currentStep.matrix.map((row, i) => (
        <tr key={i} className="border-b border-
          <td className={`p-2 bg-rose-950/40
            tStep.k === i ? 'text-amber-400' : ''}`}>
              {labelOf(i)} {i}
            </td>
              {row.map((val, j) => {
                const isUpd = currentStep.i === i
                const isVia = currentStep.k !== -
            tep.j === j));
                let bg = 'bg-slate-900/40 text-slate-
                if (isUpd) bg = 'bg-emerald-900/80
                else if (isVia) bg = 'bg-indigo-900/80
                else if (currentStep.i === -1 &&
                  bg = 'bg-indigo-950/60 text-indigo-
                else if (i === j) bg = 'bg-slate-900/40
                const shown = val === null ? '.' : val
                return (<td key={j} className={`p-2
              })))
            </tr>
          })))
        </tbody>
      </table>
    </div>
  </div>

```

```

  {/* Код алгоритма */}
  <div className="w-full lg:w-1/2 bg-slate-900/60
    <div>
      <h4 className="text-lg font-bold text-slate-900">
        <div className="bg-slate-950/80 rounded-lg p-4">
          {codeLines.map(item => (
            <div key={item.line} className={`py-1.5
              e === item.line ? 'bg-blue-900/80 text-amber-400' :

```



```

*/

type Mode = "dfs" | "dsu";

interface Node {
  id: number;
  label: string;
  x: number;
  y: number;
}

/** Граф из 10 вершин и трёх компонент — ровно как в те
const NODES: Node[] = [
  { id: 0, label: "0", x: 90, y: 80 },
  { id: 1, label: "1", x: 175, y: 45 },
  { id: 2, label: "2", x: 205, y: 140 },
  { id: 3, label: "3", x: 105, y: 175 },
  { id: 4, label: "4", x: 330, y: 70 },
  { id: 5, label: "5", x: 420, y: 45 },
  { id: 6, label: "6", x: 380, y: 145 },
  { id: 7, label: "7", x: 130, y: 290 },
  { id: 8, label: "8", x: 230, y: 320 },
  { id: 9, label: "9", x: 330, y: 275 },
];

const EDGES: [number, number][] = [
  [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
  [4, 5], [5, 6], [4, 6],                // компонента
  [7, 8], [8, 9],                        // компонента
];

const ADJ: number[][] = NODES.map(() => []);
EDGES.forEach(([u, v]) => {
  ADJ[u].push(v);
  ADJ[v].push(u);
});
ADJ.forEach((a) => a.sort((x, y) => x - y));

```

```

        continue;
    }
    comp[next] = c;
    stack.push(next);
    push(next, stack, c, `${v} → ${next}: красим в цвет ${c}, кладём ${next} в стек.`, [v, next]);
}
push(null, [], c, `Компонента ${c} готова`, `Стек пуст. Ищем следующую серую вершину.`);
}
push(null, [], c, "Готово: компонент = " + c, `Внешние вершины. Вязности.`);
return frames;
}

```

```

interface DsuFrame {
  title: string;
  note: string;
  parent: number[];
  comps: number;
  edge: [number, number] | null;
  merged: boolean;
}

```

```

function buildDsuFrames(): DsuFrame[] {
  const frames: DsuFrame[] = [];
  const parent = NODES.map((n) => n.id);
  const find = (v: number): number => (parent[v] === v ? v : find(parent[v]));
  let comps = NODES.length;
  const push = (title: string, note: string, edge: [number, number] | null) => {
    frames.push({ title, note, parent: [...parent], comps, edge });
    push("Старт: каждая вершина сама по себе", `parent[v] = v`, null, false);
    EDGES.forEach(([u, v], i) => {
      const ru = find(u);
      const rv = find(v);
      if (ru === rv) {
        push(`Ребро ${i}: (${u}, ${v}) – корни совпали`, null, null, true);
      }
    });
  };
  return frames;
}

```

```

/* — вкладка сообщает компилятору, какой код подстав.
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, mode);
}, [chapterId, mode]);

/* — автопроигрывание — */
useEffect(() => {
  if (!playing) return;
  timer.current = window.setInterval(() => {
    setStep((s) => {
      if (s >= maxStep) {
        setPlaying(false);
        return s;
      }
      return s + 1;
    });
  }, 900);
  return () => {
    if (timer.current) window.clearInterval(timer.cur
  };
}, [playing, maxStep]);

const goTo = useCallback((n: number) => setStep(Math.r

/* — переменные компилятора, которые рисуются НАПРЯМ.
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя р
const liveComp = useMemo(() => {
  const arr = vars ? (vizArray(vars.comp) ?? vizArray
  if (!arr) return null;
  return arr.slice(0, NODES.length).map((v) => (typeo
}, [vars]);
/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
  const arr = vars ? (vizArray(vars.parent) ?? vizArr
  if (!arr || arr.length < NODES.length) return null;
  return arr.slice(0, NODES.length).map((v) => vizNum

```

```

const edge = frame.edge;
const title = runtime && vars ? `${frame.title} • Контент` : 'Контент';

const switchMode = (m: Mode) => {
  setMode(m);
  setStep(0);
  setPlaying(false);
};

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">
    <div className="flex flex-wrap items-center justify-between">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white rounded-2xl">
          <Network className="w-5 h-5" />
        </div>
        <div>
          <h3 className="text-lg sm:text-xl font-bold">Дерево поиска
          <p className="text-xs text-emerald-300">10 шагов
        </div>
      </div>
      <div className="flex items-center bg-slate-950 p-2.5 rounded-2xl">
        <button
          onClick={() => switchMode("dfs")}
          className={"flex items-center gap-1.5 px-3 py-2 bg-slate-600 text-white shadow" : "text-slate-400 hover:bg-slate-700"}
        >
          <Layers className="w-3.5 h-3.5" /> 06ход (DFS)
        </button>
        <button
          onClick={() => switchMode("dsu")}
          className={"flex items-center gap-1.5 px-3 py-2 bg-slate-600 text-white shadow" : "text-slate-400 hover:bg-slate-700"}
        >
          <Network className="w-3.5 h-3.5" /> DSU (пёрышко)
        </button>
      </div>
    </div>
  </div>
);

```

```

return (
  <line
    key={i}
    x1={a.x} y1={a.y} x2={b.x} y2={b.y}
    stroke={active ? "#fbbf24" : sameComp ? "black" : "black"}
    strokeWidth={active ? 4 : sameComp ? 2 : 1}
    strokeDasharray={sameComp ? undefined : [5, 5]}
    className="transition-all duration-300"
    opacity={sameComp || active ? 1 : 0.55}
  />
);
}}}
{NODES.map((n) => {
  const c = compOf(n.id);
  const color = c > 0 ? COLORS[(c - 1) % COLORS.length] : "black";
  const isCur = cur === n.id;
  const inFrontier = frontier.includes(n.id);
  return (
    <g key={n.id} onClick={() => goTo(step)} >
      {isCur && <circle cx={n.x} cy={n.y} r={13} fill={color} />}
      {inFrontier && !isCur && <circle cx={n.x} cy={n.y} r={13} fill="none" />}
      <circle cx={n.x} cy={n.y} r={13} fill={color} />
      <text x={n.x} y={n.y + 4} textAnchor="middle">
        {n.label}</text>
      {c > 0 && <text x={n.x} y={n.y - 19} textAnchor="middle">
        c{c}</text>}
    </g>
  );
}})
{ /* стек/очередь и DSU-таблица справа */ }
<g>
  <text x={470} y={24} textAnchor="end" fill="black">
    {mode === "dfs" ? "стек DFS" : "parent[]"}
  </text>
  {mode === "dfs"
    ? frontier.slice(-8).map((v, i) => (

```

```

    )
  ) : (
    <span className="text-[11px] text-slate-500">
      открой панель Python – переменные <code cla
      erald-400">parent</code>, <code className="
      comps</code> покрасят этот граф сами
    </span>
  )}
</div>
</div>
);
};

```

```
export default GraphComponentsViz;
```

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';

```

```

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```

```

const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];

```

```

        .stack], visitedNodes: Array.from(vis) }));
        dfs(u);
    }
}

stack.pop();
steps.push({ type: 'backtrack', node: v, desc:
    ueOrStack: [...stack], visitedNodes: Array.fr
});

dfs(start);
return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начн
        ray.from(vis)] }));

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Д
            q], visitedNodes: [...Array.from(vis)] }));
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, de
                    tack: [...q], visitedNodes: [...Array.from(
                }
            }
        }
    }
}

```

```

        setStepIdx(p => {
            if (p >= steps.length - 1) {
                setAutoPlay(false);
                return p;
            }
            return p + 1;
        });
    }, 1200);
    return () => clearInterval(t);
}
}, [autoPlay, steps.length]));

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
    <div className="space-y-4">
        <div className="flex bg-slate-900 border border-slate-800 p-4">
            <button onClick={() => setMode("dfs")}
                className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
                DFS (В глубину)
            </button>
            <button onClick={() => setMode("bfs")}
                className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
                BFS (В ширину / Волна)
            </button>
        </div>

        <div className="flex flex-col md:flex-row gap-4">
            {/* SVG Graph View */}
            <div className="bg-slate-900 border border-slate-800 p-4 flex flex-grow-1 min-h-[300px]">
                <svg className="w-full h-full max-w-[1000px] max-h-[500px]">
                    {/* Edges */}

```



```

        className={`stroke-rose-500`}
        <text x={n.x} y={n.y}
            className={`text-rose-500 font-mono font-weight-normal`}
            {n.label}
        </text>

        {/* Ripple for BFS */}
        {current && mode === 'bfs' ? (
            <circle cx={n.x} cy={n.y}
                className="stroke-rose-500 stroke-width-2"
            >
        )}
    </g>
)
}}
</svg>
</div>

{/* Structure View (Stack / Queue) */}
<div className="bg-slate-900 border border-rose-500 p-10"
    r">
    <div className={`absolute -top-3 left-3 right-3 bottom-3
        ${mode === 'dfs' ? 'bg-emerald-500' : 'bg-rose-500'}
        border border-emerald-500`}
        {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
    </div>

    <div className="flex flex-col gap-2 mt-10"
        {step.queueOrStack && step.queueOrStack.length > 0 ? (
            <div key={idx} className={`w-full flex justify-between items-center
                ${mode === 'dfs' ? 'bg-rose-500' : 'bg-emerald-500'}
                text-emerald-200`}
                >
                <div>
                    {idx === (mode === 'dfs' ? step.queue.length : step.stack.length)
                        ? 'Вызван' : ''}
                </div>
                <div>
                    Узел: {item}
                </div>
            </div>
            <div key={idx} className={`w-full flex justify-between items-center
                ${mode === 'dfs' ? 'bg-rose-500' : 'bg-emerald-500'}
                text-emerald-200`}
                >
                <div>
                    {idx === (mode === 'dfs' ? step.queue.length : step.stack.length)
                        ? 'Вызван' : ''}
                </div>
                <div>
                    Узел: {item}
                </div>
            </div>
        ) : (
            <div key={0} className={`w-full flex justify-between items-center
                ${mode === 'dfs' ? 'bg-rose-500' : 'bg-emerald-500'}
                text-emerald-200`}
                >
                <div>
                    {idx === (mode === 'dfs' ? step.queue.length : step.stack.length)
                        ? 'Вызван' : ''}
                </div>
                <div>
                    Узел: {item}
                </div>
            </div>
        )}
    </div>

```

```
                <div className="text-sm text-slate-500">
                    {step.desc}
                </div>
            </div>
        </div>
    </div>
);
}
```

ФАЙЛ 46/111: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../viz';
```

```
interface Patient {
  id: string;
  name: string;
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  let idx = a.length - 1;
  while (idx > 0) {
    const parent = Math.floor((idx - 1) / 2);
    if (a[parent].priority < a[idx].priority) {
      [a[parent], a[idx]] = [a[idx], a[parent]];
      idx = parent;
    } else break;
  }
}
```

```

    { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
    { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
    { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
    { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
    { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
  ];

```

```

const INITIAL_PATIENTS: Patient[] = (() => {
  let arr: Patient[] = [];
  const start = [
    { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
    { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
    { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
    { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
    { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
  ];
  start.forEach((p, i) => {
    arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
  });
  return arr;
})();

```

```
let uid = 300;
```

```

export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>();
  const runtime = useVizRuntime();
  const liveHeap = vizArray(runtime?.variables?.h) ?? vizArray([]);
  const liveI = vizNumber(runtime?.variables?.i);
  const visualHeap: Patient[] = liveHeap
    ? liveHeap.map((value, index) => {
        const tuple = vizArray(value);
        const priority = vizNumber(tuple?.[0]) ?? vizNumber(0);
        return {
          id: `python-${index}`,
          name: tuple ? String(tuple[1] ?? value) : String(index),
        };
      }) : [];

```

```

    };

    const nextHeap = heapInsert(heap, newPatient).map(x => {
      // Mark specifically this new patient as 'in' in the heap
      const target = nextHeap.find(x => x.priority === pr.priority);
      const marked = nextHeap.map(x => x.id === (target?.id));

      setHeap(marked);
      addLog(` Поступил пациент: ${finalName} с тяжестью ${pr.priority}`);

      setTimeout(() => {
        setHeap(nextHeap);
        setBusy(false);
      }, 500);
    }, [busy, heap]);

    const handleRandomInsert = () => {
      const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
      const randomShift = Math.floor(Math.random() * 10);
      const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
      insertPatient(preset.name, prio);
    };

    const handleCustomInsert = (e: React.FormEvent) => {
      e.preventDefault();
      if (!customName.trim()) return;
      insertPatient(customName, customPriority);
      setCustomName('');
    };

    // --- EXTRACT MAX: Направить самого тяжелого в реанимацию
    const extractMax = useCallback(() => {
      if (busy || heap.length === 0) {
        setShake(true);
        addLog('⚠ Нет пациентов для экстренной госпитализации');
        setTimeout(() => setShake(false), 400);
        return;
      }
    }, [busy, heap]);
  }

```

```

    setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
    setLog([' База данных скорой помощи перезагружена.']);
    setHealing(null);
    setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 } ))));
  });

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
  const res: React.ReactElement[] = [];
  const p = nodePos(idx);
  const l = 2 * idx + 1;
  const r = 2 * idx + 2;
  if (l < visualHeap.length) {
    const lp = nodePos(l);
    res.push(
      <line key={`el${idx}`}
        x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
        x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  if (r < visualHeap.length) {
    const rp = nodePos(r);
    res.push(
      <line key={`er${idx}`}
        x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
        x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  return res;
});

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
  </div>
);

```

```

    />
    <div className="flex flex-col items-center min-w-[300px]">
      <span className="text-[9px] font-mono text-slate-700">
        <input
          type="range"
          min="10"
          max="99"
          value={customPriority}
          onChange={e => setCustomPriority(Number(e.target.value))}
          className="w-20 accent-emerald-500 cursor-pointer"
        />
      </div>
      <button
        type="submit"
        disabled={busy || !customName.trim() || heap.length === 0}
        className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all cursor-pointer"
      >
        Везти
      </button>
    </form>

    {/* Забрать самого тяжелого */}
    <button
      onClick={extractMax}
      disabled={busy || heap.length === 0}
      className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-mono transition-all cursor-pointer flex items-center justify-center"
    >
      <span> В реанимацию (EXTRACT MAX)</span>
    </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white cursor-pointer border border-slate-700"
    >

```

```

        width: isRoot ? 60 : 52,
        height: isRoot ? 60 : 52,
      }}
    >
    <span className="text-2xl leading-none">
    <span className="text-[10px] font-mono">
      {patient.priority}%
    </span>

    {/* Tooltip */}
    <div className="absolute bottom-full r
      <div className="bg-slate-950 border
0 shadow-xl">
      <div className="font-bold text-wh
      <div className="text-emerald-400"
      </div>
    </div>
  </div>
  );
  }}}
</div>

  <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 round
  <div className="absolute top-4 left-4 text-[10px]
    Операционная койка
  </div>

  <div className="flex-1 flex flex-col justify-
    {healingPatient ? (
      <div className="flex flex-col items-cente
        <div className="relative">
          <span className="text-6xl block anim-l
          <span className="absolute -top-4 -right
          <span className="absolute -bottom-2 -

```

```
        </div>
      )})
    </div>
  </div>
);
};
```

ФАЙЛ 47/111: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { createContext, useContext, useEffect, use
```



```
/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше
export const MOVE_MS = 800;
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу – мож
export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
  const [step, setStep] = useState(0);
  const paused = useContext(DemoPauseContext);
  const period = Math.round(ms * SPEED);
```



```
};
```

```
export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}
      {label !== undefined && (
        <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
          {label}
        </text>
      )}
    </g>
  );
```

```
export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    strokeWidth={width}
    strokeDasharray={dashed ? "4 3" : undefined}
    style={{ transition: MOVE }}
  />
);
```

```

        4
      </text>

      <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
      <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
      <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
    </Svg>
  );
};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    ["A", "B"], ["B", "C"],
    ["D", "E"],
    ["F", "G"], ["G", "H"], ["F", "H"],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) => e[0] === v);

  return (
    <Svg caption={step === 0 ? "Граф распался на острова" : ""}>
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => (
          <Edge key={` ${ci} - ${i} `} a={pos[u]} b={pos[v]} />
        ))
      )}
      {Object.entries(pos).map(([k, [x, y]]) => {
        const ci = compOf(k);
        return <Node key={k} x={x} y={y} r={12} label={k} fill={colors[ci]} />
      })}
    </Svg>
  );
};

```

```

    )))
    {Object.entries(pos).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k} fill={C.idle} />
    })}
  </Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => {
        <g key={i}>
          <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
          <text x={startX + i * (boxW + 4) + boxW / 2} y={35}>
            {v}
          </text>
        </g>
      })}

      {step >= 1 &&
        sorted.map((v, i) => {
          <g key={v}>

```

```

        key={i}
        x1={f.pos[a][0]}
        y1={f.pos[a][1]}
        x2={f.pos[b][0]}
        y2={f.pos[b][1]}
        stroke={C.edge}
        strokeWidth={1.8}
        style={{ transition: MOVE }}
      />
    )))
    {Object.entries(f.pos).map(([id, [x, y]]) => (
      <g key={id} style={{ transform: `translate(${x}, ${y})` }>
        <circle r={13} fill={color(id)} stroke={C.idle} />
        <text textAnchor="middle" dominantBaseline="middle">
          {id}
        </text>
      </g>
    ))}
  </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["p – корень, x под ним", "Один поворот: "]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "p", "x": "x"}]},
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p": "p", "x": "x"}]}
    ]}
  />
);

```

```

/** Zig-Zig – x и p с одной стороны, крутим сначала вершину p. */
export const ZigZigDemo: React.FC = () => (
  <SplayFrames
    captions={["Линия g → p → x («бамбук»)", "Поворот В"]}
    frames={[

```

```

const visited = new Set(order.slice(0, step).flat());
const wave = step > 0 && step <= order.length ? order
return (
  <Svg caption={`Уровень ${Math.max(0, Math.min(step,
    {GRAPH_EDGES.map(([u, v], i) => {
      <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
    }}}
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k}
        fill={wave.includes(k) ? C.active : visited.h
        stroke={wave.includes(k) ? "#a5b4fc" : undefi
    }}}
  </Svg>
);
};

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(i
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_I
      }}}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={k ===
      }}}
    </Svg>
  );
};

export const ToposortDemo: React.FC = () => {
  const nodes = [

```

```

return (
  <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
    <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
    <text x={130} y={54} textAnchor="middle" fontSize={16} />
    <Node x={60} y={70} r={20} fill={C.active} label="3" />
    <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
    <text x={60} y={106} textAnchor="middle" fontSize={16} />
    <text x={200} y={106} textAnchor="middle" fontSize={16} />
    <text x={130} y={22} textAnchor="middle" fontSize={16} />
  </Svg>
);
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента"
      {edges.map(([u, v], i) => {
        const isBridge = u === "C" && v === "D";
        if (isBridge && cut) return null;
        return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
      })}
    </Svg>
  );
};

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);

```

```

        <text x={{pos[e.u][0] + pos[e.v][0]} / 2} y=
          textAnchor="middle" fontSize={9} fontWeigh
        </g>
      );
    }}}
    {Object.entries(pos).map(([k, [x, y]]) => (<Node
  </Svg>
);
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step
      })}
    </Svg>
  );
};

```

ФАЙЛ 50/111: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИ

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

```

```

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к"
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6"
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
      <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={20} fill={C.dim} />
          <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};

```

```

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f44336"];
  const root = (i: number): number => (p[i] === i ? i : root(p[i]));
  return (
    <Svg caption="find = «кто твой староста», union = объединение"
      {p.map((par, i) =>
        par !== i ? (
          <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`}
            fill="none" stroke={rootColor[root(i)]} strokeDash={root(i) === 0 ? [4, 4]} : [0]} />
        ) : null
      )}
    </Svg>
  );
};

```



```

        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
    ]}]
    {TRIE_NODES.map((n) => {
        <Node key={n.id} x={n.x} y={n.y} label={n.ch}
        fill={wave.includes(n.id) ? C.active : doneId
    ]}]
    <text x={6} y={12} fontSize={9} fill={C.dim}>очер
</Svg>
    );
};

```

```

export const SuffixLinkDemo: React.FC = () => {
    const links: [number, number][] = [[1, 0], [2, 0], [3
    const step = useLoop(links.length + 1, 900);
    return (
        <Svg caption="Суффиксная ссылка: «куда откатиться,
        {TRIE_NODES.filter((n) => n.parent !== null).map(
            const p = TRIE_NODES[n.parent as number];
            return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
        ]}]
        {links.slice(0, step).map(([from, to], i) => {
            const a = TRIE_NODES[from];
            const b = TRIE_NODES[to];
            return (
                <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
                fill="none" stroke={C.warn} strokeWidth={1.
            );
        ]}]
        {TRIE_NODES.map((n) => {
            <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
        ]}]
        </Svg>
    );
};

```

```

export const SegmentTreeDemo: React.FC = () => {
    const step = useLoop(3, 1000);
    const nodes = [

```

```

    )))
    <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
    {pref.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 33} y={78} width={29} height={10}
          fill={show && i === l ? C.bad : show && i === l ? C.dim : C.idle}
          stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }} />
        <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
      </g>
    ))}
  </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для`}>
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={10}
            fill={C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }} />
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      ))}
      {Array.from({ length: count }).map((_, i) => (
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} style={{ transition: 'fill 0.2s' }} />
      ))}
    </Svg>
  );
};
};

```

```

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);

```

```

    {s.split("").map((ch, idx) => (
      <g key={idx}>
        <rect x={18 + idx * 33} y={26} width={28} height={24}
          fill={step > 0 && (idx < len || (idx > i - 1)) ? C.warn : C.idle}
          style={{ transition: "fill .25s" }} />
        <text x={32 + idx * 33} y={40} textAnchor="middle" fill={C.dim}>{ch}</text>
        <text x={32 + idx * 33} y={70} textAnchor="middle" fill={C.dim}>{ch}</text>
      </g>
    ))}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
    <text x={130} y={106} textAnchor="middle" fontSize={12} fill={C.dim}>
      π[i] – длина самого длинного «префикс = суффикс»
    </text>
  </Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово $
      ${Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>
              <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
                fill={idx === step - 1 ? C.active : filled ? C.warn : C.idle}
                style={{ transition: "fill .25s" }} />
              {filled && (
                <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
                  fontSize={10} fontWeight="700" fill={C.dim}>{ch}</text>
              )}
            </g>
          )
        })}
      </Svg>
    )
  );
};

```

```

    heap: HeapDemo,
    stack: StackDemo,
    queue: QueueDemo,
    dsu: DsuDemo,
    trie: TrieDemo,
    "trie-bfs": TrieBfsDemo,
    "suffix-link": SuffixLinkDemo,
    toposort: ToposortDemo,
    relax: RelaxDemo,
    "prefix-sum": PrefixSumDemo,
    "sparse-table": SparseTableDemo,
    "segment-tree": SegmentTreeDemo,
    bridge: BridgeDemo,
    articulation: ArticulationDemo,
    mst: MstDemo,
    amortized: AmortizedDemo,
    np: NpDemo,
    scc: SccDemo,
    "prefix-function": PrefixFunctionDemo,
    dp: DpDemo,
    floyd: FloydDemo,
    components: ComponentsDemo,
    graph: GraphBasicsDemo,
    "coord-compress": CoordCompressDemo,
    zig: ZigDemo,
    "zig-zig": ZigZigDemo,
    "zig-zag": ZigZagDemo,
  };

```

```

/** Список работающих мини-демо — его проверяет `npm run`
export const MINI_DEMO_KINDS = Object.keys(REGISTRY);

```

```

export const MiniDemo: React.FC<{ kind: DemoKind }> = ({
  const [paused, setPaused] = useState(false);
  const Cmp = REGISTRY[kind];
  if (!Cmp) return null;
  return (
    <DemoPauseContext.Provider value={paused}>

```

```

    return () => {
      window.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
  <div className="fixed inset-0 z-[110] flex items-end">
    <div className="absolute inset-0 bg-slate-950/80">
      <div className="relative w-full sm:max-w-5xl max-h-
        ded-2xl shadow-2xl flex flex-col">
        <div className="flex items-center gap-3 px-4 py-4">
          <div className="min-w-0 flex-1">
            <h3 className="font-bold text-white text-sm">
              {viz.hint && <p className="text-[11px] text-white">
                <VizLiveStatus demoTitle={viz.title} className="text-white">
              </div>
            <button
              onClick={onClose}
              className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
              aria-label="Заккрыть симулятор"
            >
              <X className="w-5 h-5" />
            </button>
          </div>
        </div>
        <div className="overflow-y-auto overscroll-contain">
          {/* Тот же id главы: развёрнутая демонстрация */}
          <VizChapterContext.Provider value={vizId}>
            <Component />
          </VizChapterContext.Provider>
        </div>
      </div>
    </div>,
    document.body
  )
);

```

```

        setPos(null);
        return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300

    let left = anchor.rect.left + anchor.rect.width / 2
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor

    setPos({ top: Math.max(8, Math.min(top, vh - height
    }, [anchor]));

    useEffect(() => {
        if (!anchor) return;
        const onKey = (e: KeyboardEvent) => {
            if (e.key === "Escape") onClose();
        };
        window.addEventListener("keydown", onKey);
        window.addEventListener("scroll", onClose, true);
        return () => {
            window.removeEventListener("keydown", onKey);
            window.removeEventListener("scroll", onClose, true);
        };
    }, [anchor, onClose]);

    if (!anchor) return null;
    const entry = GLOSSARY_BY_ID.get(anchor.termId);
    if (!entry) return null;
    const viz = getViz(entry.simulator);
    const simulatorId = entry.simulator;

    return createPortal(
        <div

```

```

        </div>
      ))

      {viz && simulatorId && (
        <button
          onClick={() => onOpenSimulator(simulatorId)}
          className="w-full flex items-center justify-center
            rounded-lg py-2 transition-colors"
        >
          <Play className="w-3.5 h-3.5" /> Показать
        </button>
      )}
    </div>
  </div>
</div>,
document.body
);
};

```

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
  useVizStepSync(step, setStep, MAX_STEP, (vars) => {
    const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
    return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
  });

  const nodes = [

```

```

    if (e.id === 'AB') {
      if (step === 4) return "stroke-amber-400 stroke-width-2px";
      if (step > 4) return "stroke-emerald-500";
    }
    if (e.id === 'BC') {
      if (step === 5) return "stroke-amber-400 stroke-width-2px";
      if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
      if (step === 6) return "stroke-amber-400 stroke-width-2px";
      if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // otherwise
    return "stroke-slate-500";
  };

const getMarkerUrl = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "";
    if (step === 1) return "url(#arrow-amber)";
    if (step === 2) return "url(#arrow-pink)";
    return "url(#arrow-slate-dim)";
  }
  if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
  if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
  if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
  if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
  if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
  if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

  if (step >= 4) return "url(#arrow-slate-dim)";
  return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
  if (e.u === 'S') return e.weight;

```



```
        Если мы запустим быстрый алгоритм Д
    </div>
);
case 1: return (
    <div>
        <b>Шаг 1: Точка отсчета.</b><br/>
        Добавляем фиктивную вершину S. Соединяем ее с остальными вершинами.
        Это наша база для расчета потенциалов.
    </div>
);
case 2: return (
    <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b><br/>
        Запускаем медленный, но мощный алгоритм для всех вершин:
        <span className="text-pink-300 ml-1">
    </div>
);
case 3: return (
    <div>
        <b>Шаг 3: Магическая формула.</b><br/>
        Всем ребрам графа назначаем новый вес.
        <div className="text-center font-monospace">
            Это повышает или понижает вес ребра.
        </div>
    </div>
);
case 4: return (
    <div>
        <b>Шаг 3.1: Пересчет ребра A → B.</b><br/>
        Старый вес: <span className="text-pink-300">
        Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-pink-300">
    </div>
);
case 5: return (
    <div>
        <b>Шаг 3.2: Пересчет ребра B → C.</b><br/>
        Старый вес: 1. <br/>
```

```

        <marker id="arrow-slate-dim
uto-start-reverse">
            <path d="M 0 0 L 10 5 L
        </marker>
        <marker id="arrow-amber" vie
start-reverse">
            <path d="M 0 0 L 10 5 L
        </marker>
        <marker id="arrow-pink" vie
tart-reverse">
            <path d="M 0 0 L 10 5 L
        </marker>
        <marker id="arrow-emerald"
o-start-reverse">
            <path d="M 0 0 L 10 5 L
        </marker>
    </defs>

    {/* Edges */}
    {edges.map((e, i) => {
        const u = nodes.find(n => n
        const v = nodes.find(n => n

        const isS = e.u === 'S';
        if (isS && (step === 0 || s

        const midX = (u.x + v.x) / 2;
        const midY = (u.y + v.y) / 2;

        const edgeColorClass = getE
        const textHigh = isEdgeText
        const textEmer = isEdgeText

        return {
            <g key={i}>
                <line
                    x1={u.x} y1={u.y}
                    className={`str

```

```

                                <g transform={`
fade-in zoom-in duration-500">
                                <rect x="-1
k-500 stroke-1" />
                                <text x="0"
font-bold fill-pink-400">
                                {pot}
                                </text>
                                </g>
                                )}
                                </g>
                                )
                                }}}
                                </svg>
                                </div>

```

```

<div className="w-full md:w-[320px] flex
  {/* Log Box */}
  <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
    {getDesc()}
  </div>

```

```

  {/* Controls */}
  <div className="flex bg-slate-900 b
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
      <RotateCcw strokeWidth={3}
    </button>
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
      <Undo strokeWidth={3} size=
    </button>
    <button onClick={() => setStep(
      className={`flex-1 font-bol
        ${step === MAX STEP

```

```

    y: number;
    label: string;
}

interface GEdge {
    u: number;
    v: number;
}

const initialNodes: GNode[] = [
    { id: 0, x: 150, y: 100, label: "0" },
    { id: 1, x: 300, y: 100, label: "1" },
    { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
    { id: 3, x: 450, y: 160, label: "3" },
    { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
    { u: 0, v: 1 },
    { u: 1, v: 2 },
    { u: 2, v: 0 },
    { u: 2, v: 3 }, // Connection edge
    { u: 3, v: 4 },
    { u: 4, v: 3 },
];

interface AlgoStep {
    phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
    desc: string;
    visited: Set<number>;
    currentNode: number | null;
    order: number[];
    transposed: boolean;
    sccMap: Record<number, number>; // node -> sccId
    currentSccId: number;
    activeEdges: GEdge[];
}

```

```

        ? initialEdges.map((e) => ({ u: e.v, v: e.u }
          : [...initialEdges],
    ));
};

recordStep(
  "init",
  "Инициализация алгоритма Косарайю. Начинаем первый
  null,
  false,
  -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
  visited.add(u);
  recordStep(
    "dfs1",
    `DFS 1: зашли в вершину ${u}, помечаем посещение
    u,
    false,
    -1,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs1(to);
    }
  }

  order.push(u);
  recordStep(
    "dfs1",
    `DFS 1: вершина ${u} полностью обработана. Запи

```

```

        true,
        currentScc,
    );

    for (const to of adjT[u]) {
        if (!visited.has(to)) {
            dfs2(to, currentScc);
        }
    }
};

// Run DFS2 using the order (reversed from back to
for (let i = order.length - 1; i >= 0; i--) {
    const u = order[i];
    if (!visited.has(u)) {
        sccId++;
        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} комп

```

```

const getSccColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Layers className="w-5 h-5 text-emerald-400"
          Визуализатор Алгоритма Косарайю (Поиск SCC)
        </h3>

      <div className="flex items-center gap-2 bg-slate
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            d-500 text-white"
        >
          {isPlaying ? {
            <Pause className="w-4 h-4 ml-1" />
          } : {
            <Play className="w-4 h-4 ml-1" />
          }}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

        <button
          onClick={reset}
          aria-label="Сбросить обход"
          title="Сбросить обход"
          className="flex items-center justify-center
            ors"
        >
          <RotateCcw className="w-4 h-4" />
        </button>

```

```

        orient="auto-start-reverse"
    >
        <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
    </marker>
</defs>

{/* Edges */}
{initialEdges.map((edge, idx) => {
    const u = initialNodes.find((n) => n.id ==
    const v = initialNodes.find((n) => n.id ==

    // If transposed, draw reversed coordinates
    const x1 = step.transposed ? v.x : u.x;
    const y1 = step.transposed ? v.y : u.y;
    const x2 = step.transposed ? u.x : v.x;
    const y2 = step.transposed ? u.y : v.y;

    const isSccEdge =
        step.sccMap[edge.u] &&
        step.sccMap[edge.v] &&
        step.sccMap[edge.u] === step.sccMap[edge.v];
    let stroke = "#475569";
    let markerId = "arrow";

    if (step.transposed) {
        stroke = isSccEdge ? "#818cf8" : "#4f46e5";
        markerId = "arrow-transposed";
    } else {
        if (
            step.currentNode === edge.u ||
            step.currentNode === edge.v
        ) {
            stroke = "#10b981";
            markerId = "arrow-active";
        }
    }
})

// Adjust slightly for bidirectional link

```



```

        <g key={node.id}>
          <circle
            cx={node.x}
            cy={node.y}
            r="20"
            className={`transition-all duration
              strokeWidth="3"
            />
          <text
            x={node.x}
            y={node.y}
            dy=".3em"
            textAnchor="middle"
            fill="white"
            fontSize="13px"
            fontWeight="bold"
            className="select-none"
          >
            {node.label}
          </text>

          {sccId && (
            <text
              x={node.x}
              y={node.y - 28}
              textAnchor="middle"
              fill="#10b981"
              fontSize="10px"
              fontWeight="bold"
            >
              SCC #{sccId}
            </text>
          )}
        </g>
      );
    }
  }
</svg>

```

```

        </div>
      ))
    })
  </div>
</div>

{/* List of actions log */}
<div>
  <span className="text-[10px] text-slate-500">
    ЛОГ ОПЕРАЦИЙ:
  </span>
  <div className="space-y-1.5 max-h-[150px]">
    {steps
      .slice(0, currentStepIdx + 1)
      .reverse()
      .slice(0, 5)
      .map(({s, idx}) => {
        <div
          key={idx}
          className={`text-[11px] p-1.5 rounded`
        >
          {s.desc}
        </div>
      )}}
    </div>
  </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">
  <span>
    Шаг {currentStepIdx + 1} из {steps.length}
  </span>
  <div className="flex gap-1.5">
    <button
      disabled={currentStepIdx === 0}
      onClick={() => setCurrentStepIdx((prev) => prev - 1)}
      className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
    >
      Предыдущий
    </button>
    <button
      disabled={currentStepIdx === steps.length - 1}
      onClick={() => setCurrentStepIdx((prev) => prev + 1)}
      className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
    >
      Следующий
    </button>
  </div>
</div>

```

```

    id: string;
    u: number;
    v: number;
    weight: number;
    status: 'pending' | 'inspecting' | 'included' | 'reje
    color: string;
}

```

```

interface StepLog {
    title: string;
    description: string;
    type: 'info' | 'success' | 'warning' | 'error';
}

```

// 9 Vertices layout

```

const initialVertices: Vertex[] = [
    { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
    { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
    { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
    { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
    { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
    { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
    { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
    { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
    { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

// 12 Edges connecting the 9 vertices

```

const initialEdges: Edge[] = [
    { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
    { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
    { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
    { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
    { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
    { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
    { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
    { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
    { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
];

```

```

        title: 'Успех: Ребро A-B в осто́ве',
        description: 'Вершины A and B окрашены в красный цвет',
        type: 'success'
    }, ...prev));
},
// Step 3: Inspect D-E (3)
() => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
        description: 'Оно соединяет две другие бесцветные вершины',
        type: 'info'
    }, ...prev]));
},
// Step 4: Include D-E
() => {
    setVertices(prev => prev.map(v => v.id === 3 || v.id === 4 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-E в осто́ве',
        description: 'Вершины D и E окрашены в синий цвет',
        type: 'success'
    }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Берем ребро B-C (вес 4)',
        description: 'Оно соединяет красную вершину B и синюю вершину C',
        type: 'info'
    }, ...prev]));
},
// Step 6: Include B-C
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{

```

```

        title: 'Отказ: Цикл обнаружен!',
        description: 'Мы выбрасываем ребро B-E. Иначе по-прежнему не получится',
        type: 'error'
    }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Берем ребро E-F (вес 7)',
        description: 'Оно соединяет фиолетовую вершину E с фиолетовой вершиной F',
        type: 'info'
    }, ...prev]));
},
// Step 12: Include E-F
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро E-F в осто́ве',
        description: 'Вершина F окрашена в фиолетовый цвет',
        type: 'success'
    }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Берем ребро D-G (вес 8)',
        description: 'Оно соединяет фиолетовую вершину D с фиолетовой вершиной G',
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{

```

```

        description: 'Вершина H перекрашена в фиолетовый',
        type: 'success'
    }, ...prev]);
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Шаг 10: Берем ребро E-H (вес 11)',
        description: 'Оба конца E и H уже фиолетовые. Соединяет их!',
        type: 'warning'
    }, ...prev]));
},
// Step 20: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро E-H.',
        type: 'error'
    }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Берем ребро H-I (вес 12)',
        description: 'Соединяет фиолетовую H и последнюю I',
        type: 'info'
    }, ...prev]));
},
// Step 22: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Успех: MST построено Краскалом!',
        description: 'Все 9 вершин окрашены в единый фиолетовый цвет'
    }

```

```

        type: 'success'
    }, ...prev]);
},
// Step 4: Pop A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
        description: 'Плесень расширяется в сторону верш
        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в куч
        type: 'success'
    }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к верши
        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));

```

```

() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
  setLogs(prev => [{
    title: 'Отказ: Игнорируем внутреннее ребро',
    description: 'Ребро B-E отброшено. Плесень не р
    type: 'error'
  }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (E) тянет
    type: 'info'
  }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
    eap' } : e));
  setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
  setLogs(prev => [{
    title: 'Успех: Вершина F захвачена',
    description: 'Ребро F-I(13) добавлено в кучу. Р
    type: 'success'
  }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
    description: 'Плесень расширяется вниз к G.',
    type: 'info'
  }, ...prev]));
}

```



```

        type: 'info'
    }, ...prev]);
},
// Step 19: Include G-H, add H's edges
() => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
        eap' } : e));
    setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
    setLogs(prev => [{
        title: 'Успех: Вершина H захвачена',
        description: 'В кучу добавлено H-I(12).',
        type: 'success'
    }, ...prev]);
},
// Step 20: Pop E-H (11) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Куча выдает E-H (вес 11)',
        description: 'Обе вершины E и H уже в плесени.',
        type: 'warning'
    }, ...prev]);
},
// Step 21: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setHeapQueue(['H-I (12)', 'F-I (13)']));
    setLogs(prev => [{
        title: 'Отказ: Игнорируем E-H',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]);
},
// Step 22: Pop H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{

```

```

        description: 'Каждая из 9 деревень одновременно
            А выбирает A-B(2), D выбирает D-E(3), а G вы
            !',
        type: 'info'
    }, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Component)
() => {
    // Concurrently build chosen roads. Merge into:
    // Component 1: {A, B, C} (colored red)
    // Component 2: {D, E, F, G, H, I} (colored gold/
    setVertices(prev => prev.map(v =>
        [0, 1, 2].includes(v.id) ? { ...v, color: 'red'
    }));
    setEdges(prev => prev.map(e => {
        if (['0-1', '1-2'].includes(e.id)) {
            return { ...e, status: 'included', color: 'red'
        }
        if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
            return { ...e, status: 'included', color: 'blue'
        }
        return e;
    }));
    setLogs(prev => [{
        title: 'Слияние: Деревни объединились в колхозы
        description: 'Все выбранные дороги построены ра
        оз {D, E, F, G, H, I}.',
        type: 'success'
    }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between
() => {
    // Now both kolkhozes look at all outgoing roads
    // Outgoing roads between {A, B, C} and {D, E, F,
    // A-D (5), B-E (6), C-F (9).
    // Cheapest is A-D (5).
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{

```

```

        setStepIndex(stepIndex + 1);
    }
};

const introLog = {algo: 'kruskal' | 'prim' | 'boruvka'
  if (algo === 'kruskal') {
    return [{
      title: 'Введение в раскраску (Краскал)',
      description: 'Представь, что все 9 вершин графа
      type: 'info',
    }]);
  }
  if (algo === 'prim') {
    return [{
      title: 'Введение в Плесень (Прима)',
      description: 'Логика «Плесени»: выбираем старто
      ь (Heap) и захватываем соседей!',
      type: 'info',
    }]);
  }
  return [{
    title: 'Введение в Коллективизацию (Борувка)',
    description: 'Логика «Борувки»: Каждая из 9 дерев
    ы объединиться в колхоз.',
    type: 'info',
  }]);
};

const handleReset = {algo: 'kruskal' | 'prim' | 'boru
  setActiveAlgo(algo);
  setVertices(initialVertices);
  setEdges(initialEdges);
  setStepIndex(0);
  setHeapQueue([]);
  setLogs(introLog(algo));
};

```

/* ————— связь с Python-компилятором —————

```

const c = name.trim().toUpperCase().replace(/[^A-I]/g, '');
return c ? c.charCodeAt(0) - 65 : null;
};

/** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
const parseEdge = (item: unknown): { u: number; v: number; w: string | null } | null {
  if (item === null || item === undefined) return null;
  if (typeof item === 'string') {
    const nums = item.match(/-?\d+(?:\.\d+)?/g);
    const letters = item.match(/[A-Ia-i]/g);
    let u: number | null = null;
    let v: number | null = null;
    if (letters && letters.length >= 2) { u = idOf(letters[0]); v = idOf(letters[1]); }
    else if (nums && nums.length >= 2) { u = idOf(nums[0]); v = idOf(nums[1]); }
    if (u === null || v === null) return null;
    return { u, v, w: nums && nums.length ? Number(nums[0]) : null };
  }
  if (Array.isArray(item)) {
    const letters = item.filter((x) => typeof x === 'string');
    const nums = item.filter((x) => typeof x === 'number');
    if (letters.length >= 2) {
      const u = idOf(letters[0]); const v = idOf(letters[1]);
      if (u === null || v === null) return null;
      return { u, v, w: nums.length ? nums[0] : null };
    }
    if (nums.length >= 2) {
      const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
      if (u === null || v === null) return null;
      return { u, v, w: nums.length > 2 ? nums[0] : null };
    }
  }
  return null;
};

```

```

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → одинаковый цвет
const colorOfComp = (root: string | undefined): string {
  if (!root) return 'none';

```

```

const compOfVertex = new Map<number, string>();
rawComp.forEach((label, id) => {
  if (parentIdx.size === 0) {
    compOfVertex.set(id, label);
    return;
  }
  let cur = id;
  const seen = new Set<number>([id]);
  for (let hop = 0; hop < 16; hop++) {
    const nxt = parentIdx.get(cur);
    if (nxt === undefined || nxt === cur || seen.has(nxt)) {
      seen.add(nxt);
      cur = nxt;
    }
    compOfVertex.set(id, String.fromCharCode(65 + cur));
  });
const takenRaw = vizArray(vars.mst) ?? vizArray(vars.mst);
const takenEdges = (takenRaw ?? []).map(parseEdge).filter(e => e !== null);
const takenVertices = (vizArray(vars.taken) ?? vizArray(vars.taken)).map((x) => idOf(typeof x === 'string' ? x : vizNumber(x))).filter((x): x is number => x !== null);
const curU = idOf(vizString(vars.u) ?? vizNumber(vars.u));
const curV = idOf(vizString(vars.v) ?? vizNumber(vars.v));
const curW = vizNumber(vars.w);
const heapRaw = vizArray(vars.heap) ?? vizArray(vars.heap);
const heapEdges = (heapRaw ?? []).map(parseEdge).filter(e => e !== null);
const cheapestRaw = vizRecord(vars.cheapest) ?? vizRecord(vars.cheapest);
const cheapestEdges = cheapestRaw ? Object.values(cheapestRaw).filter(e => e !== null) : [];
const total = vizNumber(vars.total) ?? vizNumber(vars.total);
const any = compOfVertex.size > 0 || takenEdges.length > 0 || (curU !== null && curV !== null) || heapEdges.length > 0;
return any ? { compOfVertex, takenEdges, takenVertices, total, curU, curV, curW, heapEdges, cheapestEdges } : { compOfVertex, takenEdges, takenVertices, total, curU, curV, curW, heapEdges, cheapestEdges }, [vars]);

```

```

/** Рёбра, которые рисуем: код пользователя важнее ка
const effEdges: Edge[] = useMemo(() => {

```



```

        >
        <Users className="w-3.5 h-3.5" />
        <span>Борувка (Билет 20)</span>
      </button>
    </div>

    <button
      onClick={() => handleReset(activeAlgo)}
      className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
    >
      <RotateCcw className="w-3.5 h-3.5" />
      <span>Сбросить</span>
    </button>

    <button
      onClick={handleNextStep}
      disabled={stepIndex >= currentSteps.length}
      className={
        "flex items-center justify-center gap-2
nitial cursor-pointer " +
        {stepIndex >= currentSteps.length
          ? "bg-slate-800 text-slate-500 border
            : activeAlgo === 'kruskal'
          ? "bg-indigo-600 hover:bg-indigo-500
            : activeAlgo === 'prim'
          ? "bg-emerald-600 hover:bg-emerald-500
            : "bg-rose-600 hover:bg-rose-500 text
        }
      >
      <SkipForward className="w-4 h-4" />
      <span>{stepIndex === 0 ? 'Начать' : stepI
    </button>
  </div>
</div>

{/* Что компилятор сообщил графу прямо сейчас */
<div className="mb-6 flex flex-wrap items-cente

```

```

        вес остова = {live.total}
    </span>
  })
</>
) : (
  <span className="text-[11px] text-slate-500" >
    открой панель Python и запусти код вкладки
    ка' }»: переменные
    { ' ' } <code className="font-mono text-purple-300" >
    e className="font-mono text-purple-300" >u</code>
    mono text-purple-300" >w</code> перекрасят э
  </span>
  })
</div>

<div className="grid grid-cols-1 lg:grid-cols-3" >
  { /* Graph Canvas */ }
  <div className="lg:col-span-2 bg-slate-950 rounded-3xl p-4" >
    active min-h-[480px] overflow-hidden" >

    { /* Legend */ }
    <div className="absolute top-4 left-4 bg-slate-900 p-4" >
      -center gap-3 text-xs" >
        <div className="flex items-center gap-1.5" >
          <div className="w-3 h-3 rounded-full bg-white" >
          </div>
          {activeAlgo === 'kruskal' ? (
            <div className="flex items-center gap-1.5" >
              <div className="w-3 h-3 rounded-full bg-white" >
              </div>
              <div className="flex items-center gap-1.5" >
                <div className="w-3 h-3 rounded-full bg-white" >
                </div>
                <div className="flex items-center gap-1.5" >
                  <div className="w-3 h-3 rounded-full bg-white" >
                  </div>
                </div>
              </div>
            </div>
          ) : (
            <div className="flex items-center gap-1.5" >
              <div className="w-3 h-3 rounded-full bg-white" >
              </div>
              <div className="flex items-center gap-1.5" >
                <div className="w-3 h-3 rounded-full bg-white" >
                </div>
                <div className="flex items-center gap-1.5" >
                  <div className="w-3 h-3 rounded-full bg-white" >
                  </div>
                </div>
              </div>
            </div>
          )
        </div>
      </div>
    </div>
  </div>

```



```

const isInHeap = edge.status === 'in_heap'

return (
  <g key={edge.id}>
    <line
      x1={u.x + "%"}
      y1={u.y + "%"}
      x2={v.x + "%"}
      y2={v.y + "%"}
      stroke={strokeColor}
      strokeWidth={isInspecting ? 4.5 : 1}
      strokeDasharray={isRejected ? '4,4' : ''}
      className={"transition-all duration-300"}
    />
    <text
      x={({u.x + v.x} / 2 + "%")}
      y={({u.y + v.y} / 2 + "%")}
      fill={isInspecting ? '#f59e0b' : '#94a3b8'}
      fontSize="4.5"
      fontFamily="monospace"
      fontWeight="bold"
      textAnchor="middle"
      dominantBaseline="middle"
      className="select-none filter drop-shadow"
      dy="-1.5"
    >
      {edge.weight}
    </text>
  </g>
);
  )}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
  {effVertices.map(vertex => (
    <div

```

```

    <p className="text-[11px] font-bold text-slate-800">
      ✂ Приоритетная очередь (Min-Heap):
    </p>
    <div className="flex flex-wrap gap-1.5">
      {heapQueue.length === 0 ? (
        <span className="text-xs text-slate-800">
          Очередь пуста
        </span>
      ) : (
        heapQueue.map((item, idx) => (
          <span key={idx} className={"px-2 py-1 shadow-md animate-pulse" : "bg-slate-800 text-white"}>
            {item}
          </span>
        ))
      )}
    </div>
  </div>
)}

<div className="flex-1 p-4 overflow-y-auto">
  {logs.map((log, idx) => (
    <div
      key={idx}
      className={
        "p-4 rounded-xl border transition-all" +
        (log.type === 'success'
          ? 'bg-emerald-950/40 border-emerald-500'
          : log.type === 'warning'
          ? 'bg-amber-950/40 border-amber-500'
          : log.type === 'error'
          ? 'bg-rose-950/40 border-rose-500'
          : 'bg-slate-900/60 border-slate-700')
      }
    >
      <div className="flex items-center gap-2">
        {log.type === 'success' && <CheckCircle className="text-emerald-500"/>
        {log.type === 'warning' && <HelpCircle className="text-amber-500"/>
        {log.type === 'error' && <XCircle className="text-rose-500"/>
        {log.type === 'info' && <Play className="text-slate-500"/>
      </div>
      {log.message}
    >
  )}
</div>
)

```

Борувка (Билет 20)

</button>

</div>

</div>

```
{activeCheatTab === 'kruskal' && {
```

```
  <div className="grid grid-cols-1 lg:grid-cols-
```

```
    <div className="lg:col-span-1 space-y-4">
```

```
      <div className="bg-slate-950 p-4 rounded-
```

```
        <p className="text-slate-400 text-xs for
```

```
          <Clock className="w-3.5 h-3.5 text-in
```

```
        </p>
```

```
        <p className="text-2xl font-black text-
```

```
        <p className="text-slate-400 text-xs mt
```

```
      </div>
```

```
      <div className="bg-slate-950 p-4 rounded-
```

```
        <p className="text-slate-400 text-xs for
```

```
          <Award className="w-3.5 h-3.5 text-in
```

```
        </p>
```

```
        <p className="text-2xl font-black text-
```

```
        <p className="text-slate-400 text-xs mt
```

```
      </div>
```

```
      <div className="bg-slate-950 p-4 rounded-
```

```
        <p className="text-slate-400 text-xs for
```

```
ожении:</p>
```

```
        <p className="text-xs text-slate-300 le
```

```
ди и красим вершины в один цвет, бракуя цик
```

```
      </div>
```

```
    </div>
```

```
  <div className="lg:col-span-2">
```

```
    <div className="bg-slate-950 p-4 rounded-
```

```
      <p className="text-slate-400 text-xs for
```

```
        <Code className="w-3.5 h-3.5 text-ind
```

```
      </p>
```

```
      <pre className="text-xs text-emerald-40
```

```
80 flex-1">
```

```
{`# 1. Инициализируем DSU
```

```
def find(i):
```

```

        <p className="text-slate-400 text-xs mt-1">
        </div>
        <div className="bg-slate-950 p-4 rounded-1">
            <p className="text-slate-400 text-xs font-medium">
                ложении:</p>
            <p className="text-xs text-slate-300 leading-5">
                ие чистые вершины через самое дешевое доступное ребро.</p>
            </div>
        </div>
        <div className="lg:col-span-2">
            <div className="bg-slate-950 p-4 rounded-1">
                <p className="text-slate-400 text-xs font-medium">
                    <Code className="w-3.5 h-3.5 text-emerald-400">
                    </p>
                    <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`import heapq

def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst`}</pre>
            </div>
        </div>
    </div>
})

```

```
        cheapest = [-1] * n
        for u, v, w in edges:
            ru, rv = find(u), find(v)
            if ru != rv:
                if cheapest[ru] == -1 or cheapest[ru][2] > w:
                    cheapest[ru] = [u, v, w]
                if cheapest[rv] == -1 or cheapest[rv][2] > w:
                    cheapest[rv] = [u, v, w]
        # Объединяем все колхозы по выбранным мостам
        for bridge in cheapest:
            if bridge != -1 and union(bridge[0], bridge[1]):
                mst.append(bridge)
    </pre>
</div>
</div>
    })
</div>
    })
};
};
```

ФАЙЛ 57/111: src/components/LiteGuide.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
import { getViz } from "../vizRegistry";
import { PAGE_SYNC } from "../data/vizSync";
import { linkifyTicketsHtml } from "../utils/ticketLink";
import { quizzes } from "../data/quizzes";
```

```
/**
```

```
 * «Лёгкая» текстовая версия пособия.
```

```
 *
```

```
 * Открывается по адресу `?lite=1` (весь список тем) или
```

```

    .lite-toc ol { margin: .4rem 0 0 1.2rem; }
    @media print {
      .lite-wrap { max-width: none; padding: 0; }
      h2 { page-break-before: always; }
      h2:first-of-type { page-break-before: avoid; }
      pre, table, details { page-break-inside: avoid; }
    }
  `;

/** Раскрываем все спойлеры и вычищаем служебную разметку
function toPlainTextHtml(html: string): string {
  return html
    .replace(/<details(?:[>]*\bopen\b)/g, "<details open")
    // кнопки/иконки внутри текста не нужны в текстовой
    .replace(/<button[\s\S]*?<\/button>/gi, "")
    .replace(/<svg[\s\S]*?<\/svg>/gi, "")
    // интерактивные подсказки терминов оставляем как есть
    .replace(/\\sdata-term-id="[^\"]*" /g, "")
    .replace(/class="term-hint"[^\"]*" /g, "")
    // ссылки на другие билеты должны вести в текстовую
    .replace(/href="\\?topic=/g, 'href="?lite=1&topic=')
    .replace(/class="ticket-link"/g, "");
}

interface Props {
  /** id одной темы; undefined – все темы подряд. */
  topicId?: string;
}

export const LiteGuide: React.FC<Props> = ({ topicId }) => {
  const list = useMemo(() => {
    if (!topicId) return chapters;
    const one = chapters.filter((c) => c.id === topicId);
    return one.length ? one : chapters;
  }, [topicId]);

  return (
    <div className="lite-wrap">

```

```

        dangerouslySetInnerHTML={{
          __html: linkifyTicketsHtml(toPlainTextH
        }}
      />

    {viz && (
      <div className="lite-viz">
        <h4>Визуализация: {viz.title}</h4>
        {viz.hint && <p>{viz.hint}</p>}
        {syncKeys.map((k) => {
          const sync = PAGE_SYNC[k];
          const demo = k.includes("#") ? k.split("#")[0] : k;
          return {
            <div key={k}>
              <p>
                <b>{demo ? `Вкладка «${demo}»` : ''}</b>
                {sync.vizTitle ? ` — ${sync.vizTitle}` : ''}
                {sync.stepNote ? ` <> Шаг демонстрации` : ''}
              </p>
              {sync.variables.length > 0 && (
                <ul>
                  {sync.variables.map((v) => {
                    <li key={v.name}>
                      <code>{v.name}</code> — {v.value}
                      {v.range ? ` (${v.range})` : ''}
                    </li>
                  )}}
                </ul>
              )}
            </div>
          );
        })}
      <p className="lite-meta" style={{ margin: 0; padding: 0; }}>
        В интерактивной версии эта демонстрация работает с переменными, перерисовывая её напрямую.
      </p>
    </div>
  )}

```

```

-----
import type { DebugValue } from "../data/debugRunner";

interface LiveDistTableProps {
  /** dist из кода пользователя: имя вершины → расстояние */
  dist: Record<string, DebugValue> | null;
  /** prev/p – предок на кратчайшем пути (необязательно) */
  prev?: Record<string, DebugValue> | null;
  /** done/visited – зафиксированные вершины (необязательно) */
  done?: DebugValue[] | null;
  /** Идентификаторы вершин, нарисованных на схеме демо */
  demoIds: string[];
  /** Подпись таблицы (по умолчанию «Ваши расстояния»). */
  title?: string;
}

const fmt = (value: DebugValue | undefined): string => {
  if (value === undefined || value === null) return "—";
  if (typeof value === "number") {
    if (value === Number.POSITIVE_INFINITY) return "∞";
    if (value === Number.NEGATIVE_INFINITY) return "−∞";
    return String(value);
  }
  if (typeof value === "string") {
    const t = value.trim().toLowerCase();
    if (t === "inf" || t === "infinity") return "∞";
    if (t === "-inf" || t === "-infinity") return "−∞";
    return value;
  }
  return String(value);
};

/**
 * Таблица «ваших» расстояний, когда имена вершин в коде
 *
 * Схема демонстрации нарисована на фиксированном наборе
 * S...F у Форда–Беллмана). Если в коде пользователя верши
 * (0...6, v1...v9), картинка физически не может их показат
 * просто пропадали, и казалось, что визуализация живёт

```



```

        ? "border-emerald-500/40 bg-emerald-500"
        : "border-slate-700 bg-slate-900/70 text-emerald-500"
    }` }
    title={fixed ? "вершина зафиксирована (доказано)" : null}
  >
    <b className={fixed ? "text-emerald-300" : "text-slate-500"}>
      <span className="text-slate-500">= </span>
      <span>{fmt(dist[key])}</span>
      {parent && parent !== "-" ? <span className="text-emerald-400">
        {fixed ? <span className="text-emerald-400">
          </span>
        } : null}
      </span>
    </div>
  </div>
);
}

```

ФАЙЛ 59/111: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import dijkstraImg from '../assets/dijkstra-freakazoid.jpg';
import bellmanImg from '../assets/bellman-ford-batman.jpg';
import floydImg from '../assets/floyd-network.jpg';

```

```

const cards = [
  {
    img: dijkstraImg,
    alt: 'Декстер Дуглас и Фриказоид (Дейкстра)',
  },

```

```
export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4">
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt}
              className="w-full h-full object-cover group-hover:scale-110 transition duration-300"/>
          </div>
          <div>
            <h3 className="text-lg font-bold mb-1 ${c.isCurrent ? 'text-white' : 'text-slate-400'}">
              {c.title}
            </h3>
            <p className="text-slate-400 text-sm">
              {c.desc} <span className={`font-bold ${c.isCurrent ? 'text-white' : 'text-slate-400'}">
                {c.status}
              </span>
            </p>
            <div className={`mt-3 text-xs font-mono px-2 ${c.isCurrent ? 'text-white' : 'text-slate-400'}">
              {c.content}
            </div>
          </div>
        </div>
      ))}
    </div>
  );
}
```

ФАЙЛ 60/111: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы – показываем в кнопке, что это текущая */
  currentTitle: string;
  index: number;
```

```

        <span className="block text-[13px] font-bol
    </span>
  </button>
</div>

{open && (
  <div className="lg:hidden fixed inset-0 z-[90]">
    <div className="absolute inset-0 bg-slate-950"
    <div className="absolute inset-y-0 left-0 w-[100%] h-[100%] flex flex-direction-column justify-center items-center text-white font-bol text-[13px] ease-out">
      <div className="flex items-center justify-center"
        <span className="font-bol text-white text-[13px]">
          <button
            onClick={() => setOpen(false)}
            className="p-2 -mr-2 text-slate-400 hover:text-white"
            aria-label="Заккрыть содержание"
          >
            <X className="w-5 h-5" />
          </button>
        </div>
      <div className="flex-1 min-h-0">
        <Sidebar
          selectedChapterId={selectedChapterId}
          setSelectedChapterId={setSelectedChapterId}
          onNavigate={() => setOpen(false)}
        />
      </div>
    </div>
  </div>
)}
</>
);
};

```



```
        rel="noreferrer"
        className="flex items-center gap-2 px-4 py-2 border border-sky-500/30 transition-all"
        title="Скачать полный TXT файл со всеми файлами"
      >
        <FileText className="w-3.5 h-3.5" />
      </a>
    <button
      id="download-html-btn"
      onClick={saveBook}
      disabled={busy}
      className="flex items-center gap-2 w-32 h-10 text-white hover:bg-amber-600/20 border border-amber-500/20 pointer"
      title="Скачать всё пособие одним файлом"
    >
      {busy ? <Loader2 className="w-3.5 h-3.5" /> : <DownloadIcon />}
    </button>
  </div>
</details>
</div>
</div>
</div>
</header>
);
};
```

ФАЙЛ 62/111: src/components/PlanarColoringViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useCallback, useContext, useEffect, useLayoutEffect, useState } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, Rotate } from 'lucide-react';
import { PlanarityDemo } from './PlanarityDemo';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './VizChapterContext';
```



```

    });
    return frames;
}

/** Проверка двудольности BFS-ом: 2 краски    нет нечётных
function bipartiteFrames(g: Graph): Frame[] {
    const frames: Frame[] = [];
    const color = new Array(g.n).fill(0);
    const queue: number[] = [];
    let conflict: [number, number] | null = null;
    frames.push({
        title: "Старт: пробуем покрасить в 2 цвета",
        note: "BFS красит вершину в цвет, противоположный р
            двудольный ( $\chi > 2$ ).",
        color: [...color], v: null, u: null, e: null,
    });
    for (let start = 0; start < g.n && !conflict; start++)
        if (color[start] !== 0) continue;
        color[start] = 1;
        queue.push(start);
        frames.push({
            title: `Запускаем BFS из ${g.names[start]}: цвет
            note: `Вершина ${g.names[start]} ещё не покрашена
                и 2.`,
            color: [...color], v: start, u: null, e: null,
        });
        while (queue.length && !conflict) {
            const v = queue.shift() as number;
            for (const u of g.adj[v]) {
                if (color[u] === 0) {
                    color[u] = 3 - color[v];
                    queue.push(u);
                    frames.push({
                        title: `${g.names[v]} → ${g.names[u]}: цвет
                        note: `Сосед не покрашен: color[${g.names[u]
                            x}).join(", ") || "пуста"}].`,
                        color: [...color], v, u, e: null,
                    });
                }
            }
        }
    }
}

```

```

const usedB = Array.from(inc[b].values());
const used = Array.from(new Set([...usedA, ...usedB]));
let c = 1;
while (used.includes(c)) c += 1;
ecolor[i] = c;
inc[a].set(i, c);
inc[b].set(i, c);
frames.push({
  title: `Ребро (${g.names[a]}, ${g.names[b]}): цве
  note: `В вершине ${g.names[a]} заняты цвета ${us
    y) => x - y).join(", ") || "-"}}. Наименьший св
    да.`
  color: new Array(g.n).fill(0), ecolor: [...ecolor
});
});
const chiE = Math.max(...ecolor);
frames.push({
  title: `Готово:  $\chi' = \chi_E$  при  $\Delta = \Delta$ `,
  note:
    g.key === "bip"
      ? `Двудольный граф:  $\chi' = \Delta = \Delta$  – это теор
        сс 1 по Визингу.`
      : ` $\chi' = \chi_E$ . Теорема Визинга:  $\Delta \leq \chi' \leq \Delta + 1$ 
         $\chi' = \Delta + 1$  – классом 2 (например, K3 и любой
  color: new Array(g.n).fill(0), ecolor: [...ecolor],
});
return frames;
}

```

```

const TABS: { id: Tab; label: string; icon: React.React
  { id: "planar", label: "1 • Планарность: K5 и K3,3",
  { id: "greedy", label: "2 • Покраска вершин ( $\chi \leq \Delta + 1$ )",
  { id: "bipartite", label: "3 • Двудольность (2 краски)",
  { id: "edges", label: "4 • Покраска рёбер (Визинг, Кё
];

```

```

export const PlanarColoringViz: React.FC = () => {
  const chapterId = useContext(VizChapterContext);

```



```

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : 0;
useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep))
    setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveColor = vars ? vizArray(vars.color) ?? vizArray(0) : 0;
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vizArray(0) : 0;
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumber(0) : 0;
const linked = Boolean(liveColor || liveEcolor || (liveV || liveU || liveE || liveChi));

const frame = frames[Math.min(step, maxStep)] ?? ({ t: 0, color: 0, ecolor: 0 });
const nodeColor = (id: number): number => {
  if (liveColor && typeof liveColor[id] === "number")
    return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
  if (liveEcolor && typeof liveEcolor[i] === "number")
    return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV) ? liveV : 0;
const curU = liveU !== null && Number.isFinite(liveU) ? liveU : 0;
const curE = liveE !== null && Number.isFinite(liveE) ? liveE : 0;
const delta = Math.max(...graph.adj.map((a) => a.length));
const usedColors = Array.from(new Set(graph.pos.map((p) => p)));
const usedEcolors = Array.from(new Set(graph.edges.map((e) => e)));

const go = useCallback((d: number) => {
  setPlaying(false);
  setStep((s) => Math.max(0, Math.min(maxStep, s + d)));
}, [maxStep]);

if (tab === "planar") {

```



```

    </div>
    <div className="bg-slate-900/70 border border
      {tab === "greedy" && <>Жадная покраска: <b
        : <b className="text-slate-200"> $\chi \leq \Delta$ </b>,
        в  $\chi \leq 5$  (доказывается через  $E \leq 3V - 6$  и це
        ром).</>}
      {tab === "bipartite" && <>Двудольный граф
        /DFS за  $O(V + E)$ . Распознавание 3-раскрашив
      {tab === "edges" && <>Рёберная покраска: <b
         $\Delta$  (Кёниг). Задача « $\chi' = \Delta$ ?» NP-полна даже
    </div>
  </div>
</div>

<div className="mt-3 flex flex-wrap items-center"
  <span className="text-[10px] uppercase tracking
    /> из компилятора:</span>
  {vars && linked ? (
    ["v", "u", "e", "color", "ecolor", "chi"].map
    vars[name] === undefined ? null : (
      <span key={name} className="text-[11px] f
        y-1">
        {name} = {JSON.stringify(vars[name]).sl
      </span>
    )
  )
  ) : (
    <span className="text-[11px] text-slate-500">
      открой панель Python: массив <code classNam
        igo-300">ecolor</code> для рёбер) и вершина
    </span>
  )}
</div>
</div>
);
};

```

```

const Header: React.FC = () => {

```

```

<div className="space-y-4">
  <h4 className="text-base font-bold text-white">K5
  <p className="text-xs text-slate-400">Просто смотри

<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
  { /* K5 */ }
  <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
    <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
    <svg className="w-full h-full absolute inset-0" >
      {(() => {
        const pts = [
          {id:0,x:160,y:40},
          {id:1,x:270,y:100},
          {id:2,x:230,y:230},
          {id:3,x:90,y:230},
          {id:4,x:50,y:100}
        ];
        const edges = [
          [0,1],[0,2],[0,3],[0,4],
          [1,2],[1,3],[1,4],
          [2,3],[2,4],
          [3,4]
        ];
        function findPt(id:number) { return pts.find(p => p.id === id); }
        function intersect(a:number,b:number,c:number,d:number) {
          if (a===c||a===d||b===c||b===d) return true;
          const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
          function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
          }
          return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
        }
        const crosses: number[] = [];
        for(let i=0;i<edges.length;i++) {
          for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))

```

```

const edges = [
  [0,3],[0,4],[0,5],
  [1,3],[1,4],[1,5],
  [2,3],[2,4],[2,5]
];
function findPt(id:number) { return pts.f
function intersect(a:number,b:number,c:nu
  if (a===c||a===d||b===c||b===d) return
  const p1=findPt(a),p2=findPt(b),p3=find
  if (!p1||!p2||!p3||!p4) return false;
  function ccw(ax:number,ay:number,bx:num
    return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
  }
  return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.
    && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
}
const crosses:number[]=[];
for(let i=0;i<edges.length;i++) for(let j
  if (intersect(edges[i][0],edges[i][1],e
    if (!crosses.includes(i)) crosses.push
    if (!crosses.includes(j)) crosses.push
  }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
  const p1=findPt(edges[i][0]),p2=findPt(
  const xing=crosses.includes(i);
  lines.push(<line key={`l${i}`} x1={p1.x
    stroke={xing?"#f43f5e":"#4f46e5"} str
}
const labels = ["\u04141","\u04142","\u04
for(const p of pts) {
  circles.push(<g key={`c${p.id}`}>
    <circle cx={p.x} cy={p.y} r={8} fill=
    <text x={p.x} y={p.y+3} textAnchor="m
  </g>);
}
return <>{lines}{circles}</>;

```

```

    Loader2,
    Pause,
    Pencil,
    Play,
    Plus,
    RotateCcw,
    SkipBack,
    SkipForward,
    Terminal,
    Unlink,
    Variable,
    X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
    baseVarName,
    buildDebugRunner,
    changedVars,
    DEBUG_STEP_CAP,
    orderVars,
    type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
    /** Страница, с которой синхронизируется редактор. */
    chapterId: string;
    chapterTitle: string;
    /** Размер панели (px) управляется родителем и сохран
    width?: number;
    height?: number | null;

```

```

-----
    if (!runtimePromise) {
        runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_NAME, {
            (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_URL })
        });
    }
    return runtimePromise;
}

function errorText(error: unknown) {
    if (error instanceof Error) return error.message;
    return String(error);
}

/**
 * Предел вывода в панель «Результат».
 *
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), но не по
 * напечатанной строки: print("x" * 10_000_000) или print("x" * 10_000_000)
 * — это один шаг, после которого весь текст уезжал в <div>.
 * Обрезаем по границе строки и честно пишем, сколько э
 */
const OUTPUT_CHAR_CAP = 20_000;

function clipOutput(text: string): string {
    if (text.length <= OUTPUT_CHAR_CAP) return text;
    const cut = text.slice(0, OUTPUT_CHAR_CAP);
    const lastBreak = cut.lastIndexOf("\n");
    const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.slice(0, lastBreak) : cut;
    return `${head}\n... вывод обрезан: показаны первые ${head.length} символов`;
}

/** -----
 * Шаблон синхронизации: короткая шапка + минимальный
 * скелет, шаг-в-шаг с подсветкой визуализации (см. data-visualizer.js)
 */
-----

/** -----
 * Шпаргалка Python: команды сортировки, циклы, структ

```

```

        code: "i = 1\nwhile i < 5:\n    print(f\"итераци\n    },\n    {\n        label: \"for i, x in enumerate(...)\",\n        tip: \"Индекс и элемент одновременно: i — позици\n        code: \"for i, x in enumerate(['a\", 'b\", 'c\n    },\n},\n},\n{\n    group: \"Структуры\",\n    items: [\n        {\n            label: \"Стек (список)\",\n            tip: \"LIFO, как рекурсия DFS: append = push, pop\n            code: 'st = []\nst.append(\"A\")    # push\nst.ap\n        },\n        {\n            label: \"Очередь (deque)\",\n            tip: \"FIFO для BFS: popleft() за O(1). list.pop\n            code: 'from collections import deque\nq = deque\n    t(q))\n',\n        },\n        {\n            label: \"Куча (heapq)\",\n            tip: \"Мин-куча: (вес, вершина) — как очередь пр\n            code: 'import heapq\nh = []\nheapq.heappush(h,\n    w, v)\n',\n        },\n        {\n            label: \"Матрица n × m\",\n            tip: \"Таблица расстояний: dist[i][j], бесконечн\n            code: 'n, m = 3, 4\ndist = [[float(\"inf\")] * m\n        },\n        {\n            label: \"Множество visited\",\n            tip: \"Посещённые вершины: in/add за O(1).\",\n            code: 'visited = set()\nvisited.add(\"A\")\nprint

```



```

    ]],
    [sync]
  );
  const syncVarSet = useMemo(() => new Set(syncVarBases)
  /** Полный эталон: глаз сразу вставляет его в обычный
  const template = useMemo(() => buildSyncTemplate(chapterId)
  /** Дефолт редактора: только инициализация демо; она
  const initTemplate = useMemo(() => buildInitTemplate(chapterId)

  const [code, setCode] = useState(() => {
    const saved = savedEditors.get(chapterId);
    if (!saved) return initTemplate;
    if (saved.code === saved.base || saved.code.trim() === "")
      return saved.code;
  });
  /** «Глаз» – переключатель: эталонный код демо ↔ инициализация
  const [refOn, setRefOn] = useState(() => {
    const saved = savedEditors.get(chapterId);
    return !!saved && saved.code.trim() !== "" && saved.code !== saved.base
  });
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Код выполняется");
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState("");
  /** Принудительный повтор (глаз / Ctrl+Enter), даже если код не
  const [traceRequest, setTraceRequest] = useState(0);
  const [copied, setCopied] = useState(false);
  const [stripTab, setStripTab] = useState<"vars" | "highlight">("vars");
  const [outputOpen, setOutputOpen] = useState(true);
  const editorRef = useRef<HTMLTextAreaElement>(null);
  /**
   * Панель жива? Заккрытие (крестик, «глаз» в шапке, см.
   * компонент, но `runPythonAsync` к тому моменту может
   * этого флага доехавший до конца прогон опубликовал
   * после закрытия: панель исчезла, а визуализация прогона
   * кода, которого на экране больше нет.
   */

```

```
// Смена страницы (или вкладки демо): нетронутый редактор
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
  const prev = prevTplRef.current;
  if (initTemplate === prev.base && template === prev.full) {
    prevTplRef.current = { base: initTemplate, full: template };
    setPlaying(false);
    setEditHold(null);
    setDebug(null); // трасса ссылается на строки старого кода
    setRefOn(false);
    if (code === prev.base || code === prev.full || code === prev.full) {
      setCode(initTemplate);
      setDesyncedFrom(null);
    } else {
      setDesyncedFrom(chapterId);
    }
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
  prevTplRef.current = { base: initTemplate, full: template };
  setCode(initTemplate);
  setRefOn(false);
  setDesyncedFrom(null);
  setStripTab("vars");
  setPlaying(false);
  setEditHold(null);
  setDebug(null);
  setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
```

```

    const changed = changedVars(step?.locals, locals);
    return {
      line: step?.line,
      func: step?.func,
      locals,
      variables,
      changed: Object.keys(changed).filter((name) => ch
    };
  }, []));

const publishDebug = useCallback(
  (result: DebugResult, idx: number, source: string) => {
    if (!aliveRef.current) return; // панель закрыта
    if (!hasViz || result.steps.length === 0) return;
    const snapshot = snapshotAt(result, idx, source);
    emitVizState(chapterId, {
      line: snapshot.line,
      func: snapshot.func,
      variables: snapshot.variables,
      changed: snapshot.changed,
    });
  },
  [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
  if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
  (nextIdx: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps
      return { ...d, idx };
    });
  });

```

```

const source = code;
const input = stdin;
const request = traceRequest;
setRunning(true);
setPlaying(false);
setEditHold(null);
setDebug(null);
clearVizState(chapterId);
setOutputOpen(false);

// Пустой редактор: Pyodide не гоняем, панель в режим
// связь с визуализацией снимаем — демо показывает
if (source.trim() === "") {
  setDebug({ result: { steps: [], error: "", truncated: false } });
  setOutput("В редакторе пусто — выполнять нечего.\n" +
    «глаз».");
  setRuntimeMessage("Пустой код: связь с визуализацией снята");
  setOutputOpen(true);
  setRunning(false);
  return;
}

setRuntimeMessage(runtimeReady ? "Обновляю трассу..." : "Запускаю");

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
  const runtime = await getPythonRuntime();
  setRuntimeReady(true);
  runtime.setStdout({ batched: (message) => stdout.push(message) });
  runtime.setStderr({ batched: (message) => stderr.push(message) });
  runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });

  const raw = await runtime.runPythonAsync(buildDebugCode);
  // Панель могли закрыть, пока Python считал: выходим
  if (!aliveRef.current) return;

```

```

        if (!aliveRef.current) return;
        const message = errorText(error);
        setDebug({ result: { steps: [], error: message, t
        clearVizState(chapterId);
        setOutputOpen(true);
        setOutput(`Ошибка:\n${message}`);
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [chapterId, code, stdin, traceRequest, running, ru

// Код выполняется сам: debounce даёт спокойно допечата
// изменился во время загрузки Pyodide, после завершения
useEffect(() => {
    if (running || editHold === code) return;
    if (debug?.src === code && debug.input === stdin &&
    const timer = window.setTimeout(() => void debugRun
    return () => window.clearTimeout(timer);
}, [code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не стр
useEffect(() => {
    if (!debug || debug.result.steps.length === 0) retu
    const onKey = (e: KeyboardEvent) => {
        const target = e.target as HTMLElement | null;
        if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
        if (e.metaKey || e.ctrlKey || e.altKey) return;
        if (e.key === "ArrowRight") {
            e.stopPropagation();
            e.preventDefault();
            setPlaying(false);
            stepBy(1);
        }
        if (e.key === "ArrowLeft") {
            e.stopPropagation();
            e.preventDefault();
            setPlaying(false);

```

```

// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
  if (!debug || !curDebug?.step) return {};
  return snapshotAt(debug.result, debug.idx, debug.src);
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(curDebug) : false;
const debugOrdered = curDebug?.step ? orderVars(shownLocals) : [];

/** Одна и та же кнопка запуска всегда видна: из редактора
 * проигрывает её, в отладчике переключает play/pause */
const runOrTogglePlayback = useCallback(() => {
  if (running) return;
  if (!debug || debug.result.steps.length === 0) {
    void debugRun(true);
    return;
  }
  if (playing) {
    setPlaying(false);
    return;
  }
  if (debug.idx >= debug.result.steps.length - 1) stepToNext();
  setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string}) => {
  const ta = editorRef.current;
  setCode((current) => {
    if (!ta) return current + (current.endsWith("\n") ? "" : "\n");
    const start = ta.selectionStart ?? current.length;
    const end = ta.selectionEnd ?? start;
    const padded = (start > 0 && !current.slice(0, start).endsWith("\n")) ? "\n" : "";
    const next = current.slice(0, start) + padded + snippet + current.slice(end, current.length);
    requestAnimationFrame(() => {
      ta.focus();
      ta.selectionStart = ta.selectionEnd = start + padded.length;
    });
    return next;
  });
});

```

```

        el.addEventListener("pointerup", onUp as EventListener)
        el.addEventListener("pointercancel", onUp as EventListener)
    })
>
    <div className="absolute left-1/2 top-1/2 h-10 w-10
        dingo-400 transition-colors" />
</div>
})

{/* Левый край меняет ширину на десктопе. */}
{onWidthChange && (
    <div
        role="separator"
        aria-orientation="vertical"
        aria-label="Потяните, чтобы изменить ширину панели"
        title="Потяните, чтобы изменить ширину панели"
        className="hidden lg:block absolute top-0 bottom-0
        onPointerDown={(e) => {
            const el = e.currentTarget;
            el.setPointerCapture(e.pointerId);
            const startX = e.clientX;
            const startW = width ?? 560;
            const onMove = (ev: PointerEvent) => {
                const next = Math.round(startW + (startX - ev.clientX) * 2);
                onWidthChange(Math.max(380, Math.min(next, 1000)));
            };
            const onUp = () => {
                el.removeEventListener("pointermove", onMove);
                el.removeEventListener("pointerup", onUp);
            };
            el.addEventListener("pointermove", onMove);
            el.addEventListener("pointerup", onUp as EventListener);
        })
    >
        <div className="absolute left-1/2 top-1/2 -transform-generate
            dingo-500 transition-colors" />
        </div>
    )}
})

```

```

        </button>
    </Tooltip>
    <Tooltip side="bottom" content="Вернуть коммент" >
        <button
            type="button"
            onClick={resync}
            className="p-1.5 rounded-lg text-slate-400"
            aria-label="Сбросить к шаблону страницы"
        >
            <RotateCcw className="w-4 h-4" />
        </button>
    </Tooltip>
    <Tooltip side="bottom" content="Скрыть панель" >
        <button
            type="button"
            onClick={onClose}
            className="p-1.5 rounded-lg text-slate-400"
            aria-label="Заккрыть компилятор"
        >
            <X className="w-4 h-4" />
        </button>
    </Tooltip>
    <Tooltip
        side="bottom"
        content={
            refOn
                ? "Убрать эталонный код: редактор станет
обственному встроенному демо. Кнопка ↺ вернёт
                : "Вставить полный эталонный код этого
удет заменён."
            }
    >
        <button
            type="button"
            onClick={() => {
                const next = !refOn;
                setRefOn(next);
                // «Убрать эталон» — значит убрать код

```



```

        <option key={speed} value={speed}>{speed}
      </option>
    </select>
  </label>
</div>
</div>

```

{/* — Мини-вкладки: переменные страницы / шпарга.

```

<div className="shrink-0 border-b border-slate-800" >
  <div className="flex items-center gap-1 px-2 pt-2" >
    <button
      type="button"
      onClick={() => setStripTab("vars")}
      className={`inline-flex items-center gap-1.5`}
      stripTab === "vars" ? "bg-slate-800 text-white" : "text-slate-400"
    >
      <Variable className="w-3.5 h-3.5" /> Переменные
    </button>
    <button
      type="button"
      onClick={() => setStripTab("hints")}
      className={`inline-flex items-center gap-1.5`}
      stripTab === "hints" ? "bg-slate-800 text-white" : "text-slate-400"
    >
      <HelpCircle className="w-3.5 h-3.5" /> Подсказки
    </button>
  </div>
  <div className="ml-auto pr-1" >
    <Tooltip
      side="bottom"
      content={
        hasViz
          ? "Связь идёт по именам ниже: реальный тип"
          : "На выбранной странице нет интерактивных элементов"
      }
    >

```

```

т заменён)."
>
  <button
    type="button"
    onClick={resync}
    className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
  >
    <Unlink className="w-3.5 h-3.5 shrink
    Страница сменилась – подставить её пер
  </button>
</Tooltip>
}}
{hasViz && {
  <div className="flex flex-wrap items-centr
    {sync.variables.map((v) => {
      <Tooltip
        key={v.name}
        side="bottom"
        content={
          <span>
            <span className="block font-bol
              <code className="font-mono te
            </span>
            <span className="block">{v.role
              {v.range && <span className="bl
            </span>
          }
        }
      >
        <code
          tabIndex={0}
          className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emeral
        >
          {v.name}
        </code>
      </Tooltip>
    }}}

```

```

        <p className="text-[10px] text-slate-600">H
    </div>
  })
</div>

```

```

{/* — Редактор (всегда на месте) + трасса + ввод
<div className="flex-1 min-h-0 flex flex-col">
  {/* Заголовок редактора: сразу видно, что выполн
  <div className="shrink-0 flex items-center gap-1">
    <Pencil className="w-3.5 h-3.5 text-emerald-400">
    <span className="text-[10px] font-bold uppercase">
      Редактор • выполняется этот текст
    </span>
    {state && (
      <span
        className="inline-flex items-center gap-1
        ont-bold text-amber-300 shrink-0"
        title="Трасса ниже построена по предыдущей"
      >
        <Loader2 className="w-2.5 h-2.5 animate-spin">
      </span>
    )}
    <span className="ml-auto text-[10px] text-slate-600">
      {code.trim() === "" ? "пусто — ничего не выведено" : ""}
    </span>
  </div>

```

```

<textarea
  ref={editorRef}
  value={code}
  onChange={(event) => {
    setEditHold(null);
    setRefOn(false);
    setCode(event.target.value);
  }}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
      event.preventDefault();
    }
  }}

```

```

bled={!curDebug?.prev} className="p-1 rounded-
tion-colors">
    <ChevronLeft className="w-4 h-4" />
  </button>
</Tooltip>
<Tooltip content="Шаг вперёд (→) – автопр
    <button type="button" aria-label="Шаг в
abled={!curDebug?.step || debug.idx >= debug
e hover:bg-slate-800 disabled:opacity-40 tra
    <ChevronRight className="w-4 h-4" />
  </button>
</Tooltip>
<Tooltip content="В конец трассы">
    <button type="button" aria-label="В кон
result.steps.length - 1); }} disabled={!cur
d text-slate-400 hover:text-white hover:bg-
    <SkipForward className="w-4 h-4" />
  </button>
</Tooltip>
<span className="ml-1.5 text-[10px] text-
  {curDebug?.step ? (
    <>
      шаг {debug.idx + 1}/{debug.result.s
      {curDebug.step.func !== "<module>"
    </>
  ) : (
    "шагов нет – исправьте ошибку и начини
  )}
</span>
{debug.result.truncated && (
  <Tooltip content={`Выполнение остановлен
    <span tabIndex={0} className="cursor-l
rounded-full px-1.5 py-0.5">
      ЛИМИТ
    </span>
  </Tooltip>
)}
<Tooltip content={hasViz ? "Визуализация

```

```

        </Tooltip>
    </div>

    {traceOpen && (
    <>
    /* листинг с подсветкой текущей строки */
    <div ref={listRef} className="flex-1 min-h-100"
        {debug.src.split("\n").map((line, i) => {
            const ln = i + 1;
            const active = ln === curDebug?.step?.line;
            return (
                <div
                    key={i}
                    data-line={ln}
                    className={`flex whitespace-pre px-1
                        active ? "bg-emerald-500/15 text-emerald-300" : ""
                    }`}
                >
                    <span
                        className={`w-7 shrink-0 select-none
                            active ? "text-emerald-300" : "text-emerald-500"
                        }`}
                    >
                        {active ? "▶" : ln}
                    </span>
                    <span>{line || " "}</span>
                </div>
            );
        })}
    </div>

    /* панель переменных – «Variables», как в VS Code
    <div className="shrink-0 max-h-[34%] overflow-hidden"
        <div className="flex items-center gap-1.5"
            <Variable className="w-3 h-3 text-indigo-300"
                Переменные на этом шаге
            <span className="normal-case tracking-normal"
                </div>
    </div>

```

```

        py-2 font-mono text-[11px] leading-snug text-slate-600
        {debug.result.error}
      </pre>
    ) : null}
  <div className="shrink-0 border-t border-slate-800" >
    <label htmlFor="python-stdin" className="text-slate-600">
      stdin для input()
    </label>
    <input
      id="python-stdin"
      value={stdin}
      onChange={(event) => {
        setEditHold(null);
        setStdin(event.target.value);
      }}
      spellCheck={false}
      placeholder="строки ввода через ↵"
      className="w-full bg-transparent font-mono text-slate-600"
    />
  </div>
</div>

```

```

{ /* — Вывод ————— */
  <div className="shrink-0 border-t border-slate-800" >
    <button
      type="button"
      onClick={() => setOutputOpen((o) => !o)}
      className="flex items-center justify-between text-slate-600"
    >
      <span className="inline-flex items-center gap-2" >
        <Terminal className="w-3.5 h-3.5 text-emerald-500" />
      </span>
      <span className="inline-flex items-center gap-2" >
        {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" />}
        <span className="text-slate-600">{outputOpen ? "открыто" : "закрыто"}</span>
      </span>
    </button>
  </div>
}

```

```

    рцию борща!' },
    { name: 'Кот Борис',      emoji: ' ', color: 'from-amber',
      bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.' },
    { name: 'Бабушка Люба',   emoji: ' ', color: 'from-rose',
      bubbleText: 'Бабушка Люба просит борща.' },
    { name: 'Инопланетянин',  emoji: ' ', color: 'from-emerald',
      bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.' },
    { name: 'Бизнесмен',      emoji: ' ', color: 'from-slate',
      bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.' },
  ];

```

```
let counter = 3;
```

```

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-amber',
      bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён',   emoji: ' ', color: 'from-emerald',
      bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис',     emoji: ' ', color: 'from-slate',
      bubbleText: 'Пожалуйста, пришлите мне борщ, пожалуйста.', animState: 'idle' },
  ]));

```

```

const runtime = useVizRuntime();
const liveQ = vizArray(runtime?.variables?.q);
const liveV = vizString(runtime?.variables?.v);
const displayQueue: Customer[] = liveQ
  ? liveQ.map((value, index) => ({
      id: `python-${index}`,
      name: String(value),
      emoji: " ",
      color: "from-emerald-500/20 to-emerald-600/20 border-emerald-500",
      bubbleText: `q[${index}] из Python`,
      animState: "idle",
    }))
  : queue;

```

```
const [servedHistory, setServedHistory] = useState<st
```

```

    addLog(`⚠ DEQUEUE: В очереди никого нет! Повар ск
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsServing(true);
  const headGuest = queue[0];

  addLog(`  DEQUEUE: Повар наливает суп первому – ${h

  // Step 1: Head guest gets eating animation
  setQueue(prev => prev.map((c, idx) => idx === 0 ? {

  setTimeout(() => {
    // Step 2: Guest leaves with happy face, queue sh
    setQueue(prev => prev.map((c, idx) => idx === 0 ?

    setServedHistory(prev => [`${headGuest.emoji} ${h

    setTimeout(() => {
      setQueue(prev => prev.slice(1));
      setIsServing(false);
      addLog(`  ${headGuest.name} поел и ушел сытым.
    }, 400);
  }, 900);
}, [queue, isServing]));

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: ' ', c
      т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: ' ', c
      апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: ' ', col
      без сметаны, плиз.', animState: 'in' },
  ]);
  setServedHistory([]);

```



```

        opacity-40 disabled:cursor-not-allowed text
        :scale-95 transition-all cursor-pointer fle
    >
    <span> Налить суп первому (DEQUEUE)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800
      r-pointer border border-slate-700"
  >
    Сброс
  </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

  {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
  <div className={`md:col-span-9 bg-slate-900 rounded
    -[300px] overflow-hidden ${shakeEmpty ? 'anim
  <div className="absolute top-4 left-4 text-[1.2em]
    Очередь голодных гостей (Буфер FIFO / BFS)
    {liveQ && <span className="ml-2 text-emerald-500">
  </div>

  <div className="flex-1 flex flex-col justify-between" >
    <div className="flex items-end justify-between" >

      {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
      <div className="flex flex-col items-center" >
        <div className="relative" >
          <div className="absolute -top-10 left-1/2
            <div className="w-1.5 h-6 bg-slate-800
            <div className="w-2 h-4 bg-slate-400
          </div>
          <span className="text-5xl block animate-pulse">
        </div>
      </div>
    </div>
  </div>

```

```

            <div className={`
                absolute -top-16 left-1/2 -tr
8 text-center text-slate-200 leading-tight
                ${isHead ? 'opacity-100 borde
            `}>
                <div className="font-bold tex
                {customer.bubbleText}
                <div className="absolute -bot
it rotate-45"></div>
            </div>

            {/* Персонаж */}
            <div className={`
                w-14 h-14 rounded-2xl border-
relative
                ${customer.color}
                ${isHead ? 'ring-4 ring-amber
            `}>
                <span className="text-3xl">{c

            {/* Индексы HEAD/TAIL */}
            {(isHead || isTail) && (
                <span className={`
                    absolute -bottom-2 text-[
                    ${isHead ? 'bg-amber-500
                `}>
                    {isHead && isTail ? 'h+t'
                </span>
            )}
            </div>
        </div>
    );
    })
    })
</div>

</div>
</div>

```

```
        {/* Пол */}
        <div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500">
        </div>
    </div>

    {/* ЛОГ ДЕЙСТВИЙ */}
    <div className="bg-slate-900 border border-slate-800 p-4">
        <div className="text-[10px] font-mono text-slate-400">
            {log.map((entry, idx) => (
                <div
                    key={idx}
                    className={`text-xs font-mono flex items-center justify-between p-2`}
                >
                    {idx === 0 ? <span className="text-indigo-500 font-weight-bold">Действие</span> : <span></span>
                    <span>{entry}</span>
                </div>
            ))}
        </div>
    </div>
);
};
```

ФАЙЛ 66/111: src/components/SalmonAutomatonWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 100

```
import React, { useState, useEffect, useContext, useMemo } from 'react';
import { Terminal } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from 'viz';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
  id: number;
  char: string;
  parent: number;
```

```

const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
const liveQueue = vars ? vizArray(vars.queue) : null;
const liveNodes = vars ? vizNumber(vars.nodes) : null;
const liveFail = vars ? vizRecord(vars.fail) : null;

// набор образцов из кода перестраивает бор целиком
useEffect(() => {
  if (!liveWordsSig) return;
  const parsed = (JSON.parse(liveWordsSig) as unknown[])
    .map((w) => (typeof w === "string" ? w : typeof w !== "string" ? "" : w))
    .filter((w) => Boolean(w))
    .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g, ""))
    .filter((w) => w.length > 0)
    .slice(0, 8);
  if (!parsed.length) return;
  if (parsed.join("|") === words.join("|")) return;
  setWords(parsed);
  buildInitialTrie(parsed);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [liveWordsSig]);

// состояние автомата и шаг BFS – подсвечиваем вершины
useEffect(() => {
  if (liveState !== null && liveState >= 0 && nodes[liveState] !== null) {
    if (liveStep !== null && liveStep >= 0) {
      // simulationStep – грубый статус: 0 = бор готов,
      const total = Object.keys(nodes).length;
      const failReady = liveFail ? Object.keys(liveFail).length : 0;
      setSimulationStep(liveStep <= 0 ? 0 : failReady ? failReady : total);
    }
    if (liveQueue) {
      const q = liveQueue.map((x) => (typeof x === "number" ? x : undefined));
      if (q.length) setBfsQueue(q);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [vars]);

```

```

        term_link: 0,
        is_terminal: false,
        words: [],
        depth: newNodes[curr].depth + 1
    };
}
curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
    newNodes[curr].words.push(word);
    newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
        newNodes[u].x = 40 + currentX * paddingX;
        currentX++;
    } else {
        let leftX = -1;
        let rightX = -1;
        childKeys.forEach(v => {
            layoutDFS(v, depth + 1);
            if (leftX === -1) leftX = newNodes[v].x!;
            rightX = newNodes[v].x!;
        });
        newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX / 2;
        if (leftX === -1) currentX++;
    }
};

```

```

    });

    // Запуск BFS
    const startBFS = () => {
        const rootChildren = Object.values(nodes[0].children);
        const queue = [...rootChildren];
        const updatedNodes = { ...nodes };
        rootChildren.forEach((childId) => {
            updatedNodes[childId].fail = 0;
        });

        setNodes(updatedNodes);
        setBfsQueue(queue);
        setSimulationStep(1);
        setCurrentNode(null);
        setSpeechText(
            'Начинаем обход в ширину (BFS)! Все вершины на графике  

            уже суффикса просто нет. Жми «Следующий шаг»!'
        );
    };

    // Один шаг BFS
    const stepBFS = () => {
        if (bfsQueue.length === 0) {
            setSimulationStep(2);
            setCurrentNode(null);
            setSpeechText(
                'Ура! Очередь пуста! Мы успешно построили полный  

                граф и полюбоваться таблицей переходов!'
            );
            return;
        }

        const r = bfsQueue[0];
        const newQueue = bfsQueue.slice(1);
        const updatedNodes = { ...nodes };
        const rNode = updatedNodes[r];
        const childrenIds = Object.values(rNode.children);
    };

```

```

    <div>
      <h4 className="text-2xl font-bold text-white">
        Интерактивный конструктор: Говорящий Эхо-
      </h4>
      <p className="text-sm text-slate-400">
        Построение дерева и расчет суффиксных ссы
      </p>
    </div>
  </div>
  <div className="flex items-center gap-2">
    <button
      onClick={() => buildInitialTrie(words)}
      className="flex items-center gap-1 bg-slate
        ransition-colors"
      title="Сбросить автомат"
    >
      <RotateCcw className="w-4 h-4" /> Сбросить
    </button>
  </div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
  {/* Аватар Лосося (Сеня) с анимацией моргания и
  <div className="flex flex-col items-center just
    <div className="w-40 h-40 relative flex items
      full border border-blue-500/30 shadow-inner
      {/* SVG Лосося */}
    <svg
      viewBox="0 0 200 200"
      className={`w-36 h-36 transition-transform
        isTalking ? 'scale-105 drop-shadow-[0_0_
        ]`}
    >
      {/* Тело лосося */}
      <ellipse cx="100" cy="100" rx="70" ry="45"
      <path d="M 40 85 Q 100 55 160 85 Q 100 115

```

```

        {/* Декоративные пузыри */}
        <div className="absolute -top-1 right-4 text-slate-400">
          <div className="absolute top-8 right-1 text-slate-400">
            </div>
          <span className="mt-3 bg-rose-600/30 border border-slate-400 px-2 py-1">
            Эхо-Лосось Сеня
          </span>
        </div>

      {/* Пузырь с текстом (Баббл) */}
      <div className="md:col-span-2 bg-slate-800 border border-slate-400 rounded-2xl p-4
        transition min-h-[140px]">
        {/* Треугольник баббла */}
        <div className="absolute -left-3 top-1/2 -transform rotate(180deg)
          bg-slate-800 border-b-[12px] border-b-transparent border-l-[12px]
            border-l-transparent border-r-[12px] border-r-transparent">
          </div>

        <div className="flex items-center space-x-2 text-slate-400">
          <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}` />
          <span>Прямое вещание из аквариума:</span>
        </div>

        <p className="text-slate-200 text-base leading-relaxed">
          {speechText}
        </p>

        <div className="mt-4 pt-3 border-t border-slate-400">
          <div className="text-xs text-slate-400 flex justify-between">
            <Info className="w-3.5 h-3.5 text-blue-400">
              Статус: {simulationStep === 0 ? 'Бор готов к бою!' : 'Бор готов к бою!'}
            </div>
          </div>

          <div className="flex gap-2">
            {simulationStep === 0 && (
              <button
                type="button"
                onClick={startBFS}
                className="bg-emerald-600 hover:bg-emerald-500 text-white px-2 py-1">
                  Начать
                </button>
            )}
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```



```

Object.values(nodes).forEach(n => {
  if (n.x && n.x > maxX) maxX = n.x;
  if (n.y && n.y > maxY) maxY = n.y;
});
const svgWidth = Math.max(maxX + 80, 400);
const svgHeight = Math.max(maxY + 60, 200);

return (
  <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
    <defs>
      <marker id="arrowhead" markerWidth="6"
        <polygon points="0 0, 6 3, 0 6" fill=
      </marker>
      <marker id="fail-arrow" markerWidth="6"
        <polygon points="0 0, 6 3, 0 6" fill=
      </marker>
    </defs>

    {/* Draw Edges */}
    {Object.values(nodes).map(node => {
      if (node.id === 0) return null;
      const parent = nodes[node.parent];
      if (!parent.x || !parent.y || !node.x || !node.y) return null;

      const isCurrent = currentNode === node.id;

      return (
        <g key={`edge-${node.id}`}>
          <line
            x1={parent.x}
            y1={parent.y + 16}
            x2={node.x}
            y2={node.y - 16}
            stroke={isCurrent ? '#60a5fa' : '#ccc'}
            strokeWidth="2"
            markerEnd="url(#arrowhead)"
          />
          <text

```

```

    { /* Draw Nodes */ }
    {Object.values(nodes).map(node => {
      if (!node.x || !node.y) return null;
      const isRoot = node.id === 0;
      const isCurrent = currentNode === node;
      const isInQueue = bfsQueue.includes(node);

      let fill = '#1e293b';
      let stroke = '#475569';

      if (isCurrent) {
        fill = '#2563eb';
        stroke = '#60a5fa';
      } else if (isInQueue) {
        fill = '#b45309';
        stroke = '#fbbf24';
      } else if (node.is_terminal) {
        fill = '#059669';
        stroke = '#34d399';
      }

      return (
        <g key={`node-${node.id}`}>
          <circle
            cx={node.x}
            cy={node.y}
            r="16"
            fill={fill}
            stroke={stroke}
            strokeWidth="2"
          />
          <text
            x={node.x}
            y={node.y + 4}
            fill="white"
            fontSize={isRoot ? "10" : "12"}
            fontWeight="bold"

```

```

        onClick={() => removeWord(w)}
        className="text-slate-500 hover:text-slate-400"
        title="Удалить"
      >
        ✕
      </button>
    </span>
  )})
</div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
  <input
    type="text"
    value={newWord}
    onChange={(e) => setNewWord(e.target.value)}
    placeholder="НОВОЕ СЛОВО..."
    maxLength={8}
    className="bg-slate-800 border border-slate-700 border-radius-4px outline-none focus:border-blue-500"
  />
  <button
    type="submit"
    className="bg-slate-700 hover:bg-slate-600 text-white font-medium py-2 px-4 transition-colors"
  >
    <Plus className="w-3.5 h-3.5" /> Добавить
  </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
  <h5 className="text-white font-bold mb-3 flex" >
    <span className="flex items-center gap-1.5" >
      <span> </span> Таблица переходов и fail-
    </span>
    <span className="text-xs text-slate-400 font" >

```

```

: isCurrent
? 'bg-blue-400 animate-pi
: isInQueue
? 'bg-amber-400'
: 'bg-slate-600'
} ` }
></span>
<span>#{node.id}</span>
{node.id === 0 && <span className
</td>
<td className="py-2.5 px-3">
  <span className="bg-slate-800 b
    {node.char}
  </span>
</td>
<td className="py-2.5 px-3 text-s
  {node.id === 0 ? '-' : `#{node
</td>
<td className="py-2.5 px-3">
  {node.id === 0 ? {
    <span className="text-slate-5
  } : {
    <span className="bg-indigo-95
      #{node.fail}
    </span>
  }}
</td>
<td className="py-2.5 px-3">
  {node.id === 0 || node.term_lin
    <span className="text-slate-5
  } : {
    <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Ck
      #{node.term_link}
    </span>
  }}
</td>
<td className="py-2.5 px-3 text-s

```

```

-2 py-1">шаг BFS = {liveStep}</span>
{liveQueue && <span className="text-[11px]
-1">queue = {JSON.stringify(liveQueue).slice
</>
) : (
  <span className="text-[11px] text-slate-500">
    открой панель Python: массив <code className=
nt-mono text-cyan-300">state</code>/<code c
  </span>
  )}
</div>
</div>
);
};

```

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMe
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRi
import { useVizRuntime, vizNumber, vizArray } from '../

// ===== TYPES =====
interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>;           // node ind
  highlightNodes: number[];                    // current
  mergeChildren?: [number, number];            // pair be
  arrayHighlight?: [number, number];           // [L, R]
  result?: number;                             // partial
}

// ===== PURE LOGIC: build tree =====

```

```

        steps.push({ id: id++, desc: `build(${v}, ${l}, ${r},
            , highlightNodes: [v], arrayHighlight: [l, r] });
        tree[v] = arr[l];
        steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]},
            l, r] });
        return;
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r},
        { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v, l, m);
    steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r},
        highlightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1],
        tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень: tree[1],
    Highlight: [0, n - 1] });
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
    const n = arr.length;
    const tree = buildTreeFull(arr);
    const steps: Step[] = [];
    let id = 0;

    function go(v: number, l: number, r: number, ql: number, qr: number) {
        if (ql > r || qr < l) {
            steps.push({ id: id++, desc: `query(${v}, [${l}, ${r}], [${ql}, ${qr}],
                odeLine: 2, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
            return 0;
        }
        if (ql <= l && r <= qr) {
            steps.push({ id: id++, desc: `query(${v}, [${l}, ${r}], [${ql}, ${qr}],
                odeLine: 2, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
            return 1;
        }
        const m = (l + r) >> 1;
        go(2 * v, l, m, ql, qr);
        go(2 * v + 1, m + 1, r, ql, qr);
        tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
        steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1],
            tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
    }
    go(1, 0, n - 1, ql, qr);
    return steps;
}

```

```

const highlights: number[] = [];
if (l & 1) {
  res += tree[l] ?? 0;
  steps.push({ id: id++, desc: `l=${l} нечётный → l
    0}. Сумма = ${res}. l++.`, codeLine: 5, treeSta
  highlights.push(l);
  l++;
}
if (r & 1) {
  r--;
  res += tree[r] ?? 0;
  steps.push({ id: id++, desc: `r=${r + 1} нечётный
    , treeState: tree, highlightNodes: [r], arrayHi
  highlights.push(r);
}
l >>= 1;
r >>= 1;
if (l < r) {
  steps.push({ id: id++, desc: `Поднимаемся: l = ${
    , arrayHighlight: [qL, qR], result: res }));
}
}
steps.push({ id: id++, desc: `  Запрос sum([${qL}..${
  ighlightNodes: [1], arrayHighlight: [qL, qR], resul
return steps;
}

// ===== TREE RENDERING HELPER =====
interface VNode { v: number; l: number; r: number; val:

function buildVisualTree(treeState: Record<number, number>) {
  if (l === r) return { v, l, r, val: treeState[v] ?? n
  const m = (l + r) >> 1;
  return { v, l, r, val: treeState[v] ?? null, left: bu
    v + 1, m + 1, r) };
}

// ===== CODE SNIPPETS =====

```

```

-----
const [playing, setPlaying] = useState(false);
const [speed, setSpeed] = useState(700);

useEffect(() => { setIdx(0); setPlaying(false); }, [s

useEffect(() => {
  if (!playing || idx >= steps.length - 1) { if (idx :
    const t = setTimeout(() => setIdx(i => i + 1), speed
    return () => clearTimeout(t);
  }, [playing, idx, steps, speed]);

const step = steps[idx] || steps[0];
if (!step) return null;

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
  border-indigo-500 bg-indigo-600', childRing: 'ring-
  -900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
  30 border-emerald-500 bg-emerald-600', childRing: '
  -slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
  der-amber-500 bg-amber-600', childRing: 'ring-rose-
  [color];

  return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
  | 'emerald' | 'amber'; arr: number[] }) {
  const p = Player({ steps, color });
  const runtime = useVizRuntime();
  const liveV = vizNumber(runtime?.variables?.v);
  const liveTree = vizArray(runtime?.variables?.tree);
  const liveI = vizNumber(runtime?.variables?.i);

  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed
  const n = arr.length;

```



```

        </div>
      </div>
    })
  </div>
);
}, [step, clr, fullTree, liveV, liveTree]);

return (
  <div className={`rounded-2xl border overflow-hidden
    ? 'border-emerald-500/30' : 'border-amber-500/30'
    /* Header */
    <div className={`px-4 py-2.5 flex items-center justify-between
      -emerald-950/50' : 'bg-amber-950/50'} border-b border-white
      <h4 className="font-bold text-sm text-white flex-1
      <span className="text-[10px] font-mono text-slate-400"
    </div>

    /* Array strip */
    <div className="px-3 py-2 bg-slate-950 border-b border-white
      <div className="text-[11px] font-bold text-slate-400"
        <span className="text-amber-400">Исходный массив</span>
        <span className="text-slate-400 font-mono text-slate-400">
      </div>
      <div className="flex flex-wrap gap-1 justify-center"
        {arr.map((v, i) => {
          const inRange = step.arrayHighlight && i >= step.start;
          const isLiveI = liveI === i;
          return (
            <div key={i} className={`flex flex-col items-center justify-center
              -amber-400 ring-2 ring-amber-400/40' : inRange
              d' ? 'bg-emerald-950 border-emerald-500' : 'bg-slate-950'
              '}}>
              <span className="text-[8px] text-amber-400">{v}</span>
              <span className="text-[12px] font-extrabold text-slate-400">{i}</span>
            </div>
          );
        })}
      </div>
    </div>
  );
}
}
}
</div>

```

```

    <div className="flex items-center bg-slate-900" style={styles.container}>
      <button onClick={() => { setPlaying(false); setSpeed(1200); }}
        rounded title="Сброс" ><RotateCcw className="w-3.5 h-3.5" /></button>
      <button onClick={() => { setPlaying(false); setSpeed(1200); }}
        title="Шаг назад" className="p-1.5 text-slate-400" ><ChevronLeft
        className="w-3.5 h-3.5" /></button>
      <button onClick={() => setPlaying(!playing)}
        white transition-all ${playing ? 'bg-rose-500' : 'bg-slate-900'}
        {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}
      </button>
      <button onClick={() => { setPlaying(false); setSpeed(1200); }}
        title="Шаг вперёд" className="p-1.5 text-slate-400" ><ChevronRight
        className="w-3.5 h-3.5" /></button>
    </div>
    <select value={speed} onChange={e => setSpeed(Number(e.target.value))}>
      <option value={1200}>Медл.</option>
      <option value={700}>Норма</option>
      <option value={350}>Быстро</option>
      <option value={120}>Турбо</option>
    </select>
  </div>

  {/* Progress bar */}
  <div className="h-1 bg-slate-950"><div className="bg-rose-500" style={{width:
    (currentSpeed / total) * 100 : 0}%` }} /></div>
</div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const runtime = useVizRuntime();
  const liveI = vizNumber(runtime?.variables?.i);
  const liveA = vizArray(runtime?.variables?.A) || vizArray([5, 8, 3, 12]);
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);

```

```

const queryTDSteps = useMemo(() => genQueryTopDownSteps)
const queryBUSteps = useMemo(() => genQueryBottomUpSteps)

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border border-amber-200" >
    {/* ---- TOP BAR: array editor + query range ---- */}
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4">
        {/* Array input */}
        <form onSubmit={handleApply} className="flex flex-col">
          <div className="flex items-center justify-between">
            <label className="text-[10px] font-bold uppercase">
              label>
            <button type="button" onClick={randomize} className="text-white">
              "><RefreshCw className="w-3.5 h-3.5" /></button>
            </div>
            <div className="flex gap-2">
              <input
                value={customInput}
                onChange={e => setCustomInput(e.target.value)}
                className="flex-1 bg-slate-900 border border-amber-200 focus:border-indigo-500"
                placeholder="5, 8, 3, 12, 7, 2"
              />
              <button type="submit" className="bg-indigo-600 text-white px-4 py-2 transition-colors">Применить</button>
            </div>
          </form>
          <div className="flex flex-wrap gap-1.5 pt-1">
            {arr.map((v, i) => (
              <span key={i} className={`border px-2 py-1 text-amber-200 ring-2 ring-amber-400/30` : "border-amber-200 ring-2 ring-amber-400/30"}>
                <span className={liveI === i ? "text-white" : "text-amber-200"}>
                  {v}</span>
                </span>
            ))}
            <span className="text-xs text-slate-500 font-medium">total: {totalSum}</span>
          </div>
        </div>
      </div>
    </div>
  )
)

```

```

{ /* ---- TABS ---- */
<div className="flex flex-wrap items-center gap-1"
  <button onClick={() => setView('build')} className="
    -colors ${view === 'build' ? 'border-indigo-500
    -slate-200'}`">
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} className="
    on-colors ${view === 'queries' ? 'border-emerald-500
    er:text-slate-200'}`">
    <Search className="w-3.5 h-3.5" /> Запрос: св
  </button>
  <button onClick={() => setView('theory')} className="
    n-colors ${view === 'theory' ? 'border-blue-500
    te-200'}`">
    <Info className="w-3.5 h-3.5" /> Почему / Спра
  </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && (
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon=" "
    steps={buildSteps}
    code={BUILD_CODE}
    color="indigo"
    arr={arr}
  />
)}

```

```

{view === 'queries' && (
  <div className="grid grid-cols-1 xl:grid-cols-2"
    <SimPanel
      key={`qtd-${arr.join('-')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Сверху
      icon="↑ "

```

```
</div>
```

```
{/* Comparison cards */}
```

```
<div className="grid grid-cols-1 xl:grid-cols-
```

```
  <div className="bg-slate-950 p-5 rounded-xl
```

```
    <div className="absolute -top-3 left-4 bg-
      e border border-emerald-500 shadow-md">
```

```
      † Запрос Сверху-вниз (Рекурсия)
```

```
    </div>
```

```
    <p className="text-xs text-slate-300 mb-3
```

```
      Начинаем с корня. Если отрезок узла полн
      наче спускаемся в обоих детей.
```

```
    </p>
```

```
    <div className="space-y-1.5 text-xs">
```

```
      <div className="flex justify-between bo
        span className="text-rose-400 font-mono">0(
```

```
        <div className="flex justify-between bo
        ><span className="text-rose-400 font-mono">
```

```
        <div className="flex justify-between"><
        d-400 font-bold">✓ легко</span></div>
```

```
      </div>
```

```
    </div>
```

```
<div className="bg-slate-950 p-5 rounded-xl
```

```
  <div className="absolute -top-3 left-4 bg-
    rder border border-amber-500 shadow-md">
```

```
    † Запрос Снизу-вверх (Итерация)
```

```
  </div>
```

```
  <p className="text-xs text-slate-300 mb-3
```

```
    Стартуем с двух указателей (l и r) на л
    ём его левого соседа. Поднимаемся пока l {"
```

```
  </p>
```

```
  <div className="space-y-1.5 text-xs">
```

```
    <div className="flex justify-between bo
      span className="text-emerald-400 font-mono":
```

```
    <div className="flex justify-between bo
    ><span className="text-emerald-400 font-mono
```

```
    <div className="flex justify-between"><
    00 font-bold">x сложно</span></div>
```

```

export const Sidebar: React.FC<SidebarProps> = ({ selectQuery }) => {
  const [query, setQuery] = useState("");

  /**
   * Группы – это разделы пособия (SECTIONS), а не случаи.
   * связанные темы стоят рядом, порядок разделов не важен.
   * страница встретилась первой.
   */
  const groups = useMemo(() => {
    const q = query.trim().toLowerCase();
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
    return SECTIONS.map((section) => ({
      section,
      items: filtered.filter((c) => sectionOf(c.id).id === section.id).filter((g) => g.items.length > 0);
    })), [query]);

  return (
    <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:rounded-r-2xl" >
      <div className={compact ? "p-2 pb-1.5 shrink-0" : "p-2 pb-1.5 shrink-0"} >
        {compact && (
          <h3 className="text-[10px] font-bold text-slate-400" >
            <Compass className="w-3.5 h-3.5 text-indigo-400" />
          </h3>
        )}
        <h3 className={`text-xs font-bold text-slate-400`} >
          <Compass className="w-4 h-4 text-indigo-400" />
        </h3>
        {!compact && (
          <div className="relative" >
            <Search className="w-4 h-4 text-slate-500 absolute left-0 top-0" />
            <input
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="Поиск по темам.."
              className="w-full bg-slate-800/70 border border-slate-700"
            />
          </div>
        )}
      </div>
    </aside>
  );
};

```

```

        return (
          <button
            key={chapter.id}
            onClick={() => {
              setSelectedChapterId(chapter.id);
              onNavigate?.();
              window.scrollTo({ top: 0, behavior: 'smooth' });
            }}
            title={chapter.title}
            aria-current={isSelected ? "page" : undefined}
            className={`w-full text-left rounded-2xl
              ${isSelected ? "bg-indigo-600/15 border-indigo-600/20" : "bg-slate-800/40 border-transparent"}
            `}
          >
            <span className="shrink-0 mt-px rounded-2xl font-medium text-indigo-300">
              {num ? num[1] : "•"}
            </span>
            <span className={`font-semibold leading-4`}>
              {label}
              {hasViz && (
                <span className="ml-1.5 inline-block text-indigo-300">
                  визуализация
                </span>
              )}
            </span>
            {!compact && (
              <ChevronRight
                className={`w-4 h-4 shrink-0 mt-px text-indigo-400`}
                isSelected={isSelected}
              />
            )}
          </button>
        );
      });
    });
  }
}

```

```
// Stack state
const [stack, setStack] = useState<Plate[]>([
  { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
  { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
  { id: '3', name: 'Блюдец от торта', icon: ' ' },
]);
const [popMessage, setPopMessage] = useState<string | null>('');

// Queue state
const [queue, setQueue] = useState<Person[]>([
  { id: '1', name: 'Студент Вася', icon: ' ' },
  { id: '2', name: 'Голодный кодер', icon: ' ' },
  { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<string | null>('');

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', icon: ' ' },
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"', icon: ' ' },
  { id: '3', text: 'Кандидат: "Привет, я испанский интеллект"', icon: ' ' },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.5);
const [heapMessage, setHeapMessage] = useState<string | null>('');

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: '☞' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon,
  };
  setStack([newPlate, ...stack]);
};

const popPlate = () => {
  if (stack.length > 0) {
    setStack(stack.slice(1));
    setPopMessage('Поплавок удален');
  }
};

const enqueuePerson = () => {
  const newPerson = {
    id: Date.now().toString(),
    name: 'Новый человек',
    icon: ' ',
  };
  setQueue([newPerson, ...queue]);
};

const dequeuePerson = () => {
  if (queue.length > 0) {
    setQueue(queue.slice(1));
    setDequeueMessage('Человек удален');
  }
};

const addHypothesis = () => {
  const newHypothesis = {
    id: Date.now().toString(),
    text: 'Новая гипотеза',
    icon: ' ',
  };
  setHeap([newHypothesis, ...heap]);
};

const removeHypothesis = () => {
  if (heap.length > 0) {
    setHeap(heap.slice(1));
    setHeapMessage('Гипотеза удалена');
  }
};

const newCustomHypothesis = () => {
  if (newCustomText.trim() > 0) {
    const newHypothesis = {
      id: Date.now().toString(),
      text: newCustomText,
      icon: ' ',
    };
    setHeap([newHypothesis, ...heap]);
    setNewCustomText('');
    setNewCustomProb(0.5);
  }
};

const setCustomProb = (prob: number) => {
  setNewCustomProb(prob);
};
```



```

        icon: randomPreset.icon
    };
    setQueue(prev => [...prev, newPerson]);
    setDequeueMessage(` Встал в конец очереди: ${newPe
};

const servePerson = () => {
    if (queue.length === 0) {
        setDequeueMessage("⚠ Очередь пуста! Суп ждет нов
        return;
    }
    const firstPerson = queue[0];
    setQueue(prev => prev.slice(1));
    setDequeueMessage(` Выдан суп первому в очереди (F
};

const resetQueue = () => {
    setQueue([
        { id: '1', name: 'Студент Вася', icon: ' ' },
        { id: '2', name: 'Голодный кодер', icon: ' ' },
        { id: '3', name: 'Кот Борис', icon: ' ' },
    ]);
    setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
    e.preventDefault();
    if (!newCustomText.trim()) return;
    const item: Hypothesis = {
        id: Date.now().toString(),
        text: `Кандидат: "${newCustomText}"`,
        probability: Number(newCustomProb)
    };
    setHeap(prev => [...prev, item]);
    setNewCustomText('');
    setHeapMessage(` Добавлена гипотеза с вероятностью
};

```

```

        Проверь работу принципов LIFO, FIFO и Приоритет
      </p>
    </div>

```

```

    { /* Tabs */ }
    <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
      <button
        onClick={() => setActiveTab('stack')}
        className={`flex items-center justify-between gap-2
          ${activeTab === 'stack'
            ? 'bg-blue-600/20 border-blue-500 text-blue-600'
            : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
        `}
      >
        <div className="flex items-center gap-3">
          <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-600' : 'text-slate-400'}
            <div className="text-left">
              <div className="font-bold text-sm text-white">Stack
              <div className="text-xs opacity-80">LIFO
            </div>
          </div>
        </div>
        <span className="text-xs bg-blue-500/20 text-blue-600 border border-blue-500/20 padding-2">
          DFS
        </span>
      </button>

      <button
        onClick={() => setActiveTab('queue')}
        className={`flex items-center justify-between gap-2
          ${activeTab === 'queue'
            ? 'bg-indigo-600/20 border-indigo-500 text-indigo-600'
            : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
        `}
      >
        <div className="flex items-center gap-3">
          <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-indigo-600' : 'text-slate-400'}
            <div className="text-left">
              <div className="font-bold text-sm text-white">Queue
              <div className="text-xs opacity-80">FIFO
            </div>
          </div>
        </div>
        <span className="text-xs bg-indigo-500/20 text-indigo-600 border border-indigo-500/20 padding-2">
          BFS
        </span>
      </button>
    </div>
  </div>

```

```

        </h3>
        <p className="text-slate-400 text-xs mt-1">
            Новые тарелки ставятся наверх. Мыть можн
        </p>
    </div>
    <div className="flex flex-wrap items-center">
        <button
            onClick={addPlate}
            className="flex-1 md:flex-none flex item
            rounded-xl text-sm font-bold shadow-lg tra
        >
            <Plus className="w-4 h-4" /> Поставить
        </button>
        <button
            onClick={popPlate}
            className="flex-1 md:flex-none flex item
            border-blue-500/40 px-4 py-2.5 rounded-xl
        >
            Помыть верхнюю
        </button>
        <button
            onClick={resetStack}
            title="Сбросить"
            className="p-2.5 bg-slate-800 hover:bg-
            tion-colors cursor-pointer"
        >
            <RotateCcw className="w-5 h-5" />
        </button>
    </div>
</div>

{popMessage && {
    <div className="bg-blue-950/60 border borde
    imate-fade-in">
        <span className="inline-block w-2 h-2 roun
        {popMessage}
    </div>
}}

```

```

        isTop ? 'bg-blue-500 text-white
    }`}>
        {isTop ? 'Готово к помывке' : `
    </span>
  </div>
    );
  }}}
</div>
})
<div className="mt-6 text-xs text-slate-500
  ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
})

{/* QUEUE TAB */}
{activeTab === 'queue' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-il
        /span>
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Кто первым пришел, тот первым получил с
        </p>
        </div>
        <div className="flex flex-wrap items-center
          <button
            onClick={addPerson}
            className="flex-1 md:flex-none flex ite
          -2.5 rounded-xl text-sm font-bold shadow-lg
          >
            <Plus className="w-4 h-4" /> Встать в о

```

```

<div className="flex flex-col items-center justify-center gap-300">
  <span className="text-3xl"> </span>
  <span className="text-xs font-bold font-mono">
</div>

<div className="text-indigo-500 flex items-center justify-center">
  <ArrowRight className="w-6 h-6 animate-pulse">
</div>

{queue.map((person, index) => {
  const isFirst = index === 0;
  return (
    <div
      key={person.id}
      className={`flex flex-col items-center justify-center gap-3
        ${isFirst
          ? 'bg-gradient-to-b from-indigo-500 to-white'
          : 'bg-slate-900/90 border-slate-500'}
        `}
    >
      {isFirst && (
        <span className="absolute -top-10 left-0 right-0 text-center">
          Первый (Head)
        </span>
      )}
      <span className="text-4xl mb-2">{person.name}</span>
      <span className={`font-bold text-2xl font-mono`}
        >{person.name}</span>
      <span className={`text-[10px] font-mono`}
        >{isFirst ? 'bg-indigo-500/30 text-white' : 'bg-white text-slate-900'}</span>
      <span>{isFirst ? 'Получает сун' : 'Входит'}</span>
    </div>
  )}
)

```

```

        Взять кандидата с макс. вероятностью
    </button>
    <button
      onClick={resetHeap}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
  <div className="md:col-span-6">
    <label className="block text-xs font-bold">
      <input
        type="text"
        value={newCustomText}
        onChange={e => setNewCustomText(e.target
placeholder='например: "Привет, я лучший"'
        className="w-full bg-slate-900 border b
rder-emerald-500 transition-colors"
      />
    </div>
    <div className="md:col-span-3">
      <label className="block text-xs font-bold">
        Вероятность: <span className="text-emera
      </label>
      <input
        type="range"
        min="0.01"
        max="0.99"
        step="0.01"
        value={newCustomProb}
        onChange={e => setNewCustomProb(Number(e

```

```

    </div>

    {sortedHeap.map(({item, index}) => {
      const isTop = index === 0;
      const percent = (item.probability * 100).toFixed(
      return {
        <div
          key={item.id}
          className={`flex items-center justify-between
            isTop
              ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]'
              : 'bg-slate-900/80 border-slate-800'
            }`}
        >
          <div className="flex items-center justify-between" >
            <span className={`flex items-center justify-between
              isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900/80 text-slate-400'
            }`>
              #{index + 1}
            </span>
            <div>
              <div className={`font-bold text-white`} >
                {item.text}
              </div>
              {isTop && (
                <span className="text-[10px] text-white" >
                  Вершина Кучи (Root) • Будущее
                </span>
              )}
            </div>
          </div>
        </div>

        <div className="flex items-center justify-between" >
          {/* Custom progress bar */}
          <div className="hidden sm:block" >
            <div>

```

```
* Страница splay – три вкладки по нарастанию сложности
* 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
*   (прицел → поворот, призраки старых позиций, обход)
* 2. «Большое дерево» – те же случаи внутри настоящего
*   поворот – кадры «прицел → отстёгиваем среднее по
* 3. «Собери сам» – песочница: подними узел кликами,
*/
type Tab = "cases" | "big" | "play";

export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({
  const [tab, setTab] = useState<Tab>(defaultTab);
  const chapterId = useContext(VizChapterContext);

  // вкладка = подрежим страницы: панель Python подстав
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab);
  }, [chapterId, tab]);

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border">
      <div className="flex flex-wrap items-center justify-between">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-indigo-600 text-white">
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">
              <p className="text-sm text-indigo-300">Один
            </div>
          </div>
        </div>
      </div>

      <div className="flex flex-wrap gap-2 mb-5">
        {[
          { id: "cases" as Tab, label: "1 · Механика: Zig-Zig-Zag" },
          { id: "big" as Tab, label: "2 · Большое дерево" },
          { id: "play" as Tab, label: "3 · Собери сам" }
        ]}.map((t) => {
```


*

* Здесь ничего не подсказывается: дано настоящее дерево

* поднять отмеченный узел в корень. Клик по узлу = один

* Порядок поворотов выбираешь сам; правильный ли он по

* только после того, как узел окажется наверху (или по

*/

```
export interface TNode {  
  key: number;  
  left: TNode | null;  
  right: TNode | null;  
}
```

```
const leaf = (key: number): TNode => ({ key, left: null
```

```
const clone = (t: TNode | null): TNode | null =>  
  t ? { key: t.key, left: clone(t.left), right: clone(t
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
```

```
export const PRESETS: { name: string; hintCase: string;
```

```
{
```

```
  name: "Линия влево",
```

```
  hintCase: "Zig-Zig",
```

```
  target: 1,
```

```
  build: () => {
```

```
    const n1 = leaf(1);
```

```
    const n2: TNode = { key: 2, left: n1, right: leaf
```

```
    const n4: TNode = { key: 4, left: n2, right: leaf
```

```
    return { key: 6, left: n4, right: leaf(7) };
```

```
  },
```

```
},
```

```
{
```

```
  name: "Змейка",
```

```
  hintCase: "Zig-Zag",
```

```
  target: 3,
```

```
  build: () => {
```

```
    const n3 = leaf(3);
```

```
    const n2: TNode = { key: 2, left: leaf(1), right:
```

```

const path = findPath(root, key);
if (path.length < 2) return root;

const x = path[path.length - 1];
const p = path[path.length - 2];
const gg = path.length >= 3 ? path[path.length - 3] : null;

if (p.left === x) {
  p.left = x.right;
  x.right = p;
} else {
  p.right = x.left;
  x.left = p;
}

if (!gg) return x;
if (gg.left === p) gg.left = x;
else gg.right = x;
return root;
}

```

```

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number) {
  const path = findPath(root, key);
  if (path.length < 2) return null;
  const x = path[path.length - 1];
  const p = path[path.length - 2];
  const g = path.length >= 3 ? path[path.length - 3] : null;

  if (!g) return { node: x.key, kind: "Zig" };

  const xLeft = p.left === x;
  const pLeft = g.left === p;
  if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
  return { node: x.key, kind: "Zig-Zag (сначала нижняя)" };
}

```

```

/* ----- раскладка -----

```

```

const [presetIdx, setPresetIdx] = useState(0);
const preset = PRESETS[presetIdx];

const [tree, setTree] = useState<TNode>(() => preset.l
const [history, setHistory] = useState<TNode[]>([]);
const [moves, setMoves] = useState<{ node: number; co
const [showAnswer, setShowAnswer] = useState(false);

const target = preset.target;
const solved = tree.key === target;

const reset = useCallback(
  (idx = presetIdx) => {
    setPresetIdx(idx);
    setTree(PRESETS[idx].build());
    setHistory([]);
    setMoves([]);
    setShowAnswer(false);
  },
  [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, tar
const expected = useMemo(() => (solved ? null : canon

const clickNode = (key: number) => {
  if (solved) return;
  const p = findPath(tree, key);
  if (p.length < 2) return; // корень крутить не с ке

  const want = canonicalNext(tree, target);
  setHistory((h) => [...h, clone(tree) as TNode]);
  setMoves((m) => [...m, { node: key, correct: want?.l
  setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {

```

```

        {p.name}
      </button>
    )))
  </div>

  <div className="bg-slate-900 rounded-xl border bo
    <svg
      viewBox={`0 0 ${width} ${height}`}
      className="h-auto block mx-auto"
      style={{ minWidth: Math.min(width * 1.5, 420)
      role="img"
    >
      {edges.map(([a, b], i) => {
        const na = nodes.find((n) => n.key === a);
        const nb = nodes.find((n) => n.key === b);
        if (!na || !nb) return null;
        const onPath = path.has(a) && path.has(b);
        return (
          <line
            key={i}
            x1={na.x}
            y1={na.y}
            x2={nb.x}
            y2={nb.y}
            stroke={onPath ? "#6366f1" : "#475569"}
            strokeWidth={onPath ? 3 : 1.8}
            style={{ transition: "all 700ms cubic-b
          />
        );
      })}
    {nodes.map((n) => {
      const isTarget = n.key === target;
      const clickable = !solved && n.depth > 0;
      return (
        <g
          key={n.key}
          onClick={() => clickable && clickNode(n

```

```

        <RotateCcw className="w-3.5 h-3.5" /> Заново
    </button>
    <button
      onClick={() => setShowAnswer((s) => !s)}
      className={`flex items-center gap-1.5 px-3 py-2
        showAnswer
          ? "bg-amber-600/20 border-amber-500 text-amber-500"
          : "bg-slate-800 border-slate-700 text-slate-200"}
    >
      {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
    </button>

    <span className="text-[11px] text-slate-500 ml-2">
      поворотов: {moves.length} • минимум: {minimalMoves}
    </span>
  </div>

  {showAnswer && !solved && expected && (
    <div className="mt-3 text-[12px] bg-amber-950/40 p-3">
      Сейчас splay крутил бы узел{" "}
      <span className="font-mono font-bold">{expected}
    </div>
  )}

  {solved && (
    <div
      className={`mt-3 rounded-lg border px-3 py-2.5
        wrong === 0
          ? "bg-emerald-950/40 border-emerald-700/60 text-emerald-500"
          : "bg-rose-950/30 border-rose-800/60 text-rose-500"}
    >
      <div className="flex items-center gap-2 font-mono">
        {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <X className="w-4 h-4" />}
        {wrong === 0
          ? `Узел ${target} в корне за ${moves.length}

```

```
* * у узлов настоящие числа (20 / 40 / 60), а не абстр
* сразу видно, что дерево остаётся деревом поиска;
* * под деревом печатается обход слева направо: он ОД
* это и есть доказательство, что поворот ничего не
* * кадр «прицел» ничего не двигает, только подсвечив
* * на кадре «результат» остаются призраки – пунктирн
* местах и стрелки «откуда → куда»;
* * по умолчанию анимация НЕ крутится сама: пользоват
* идёт в своём темпе. Автопрокрутка включается кноп
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел».
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – рисуем призрак и стр
  moved?: NodeId[];
  ms: number;
}
```

```
interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах.
  inorder: string[];
  /** Что означают поддеревья A..D. */
  ranges: string;
```

```

    {
      pos: { x: [160, 50], A: [80, 130], p: [245, 130],
      edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
      kind: "result",
      title: "Стало: 20 – корень",
      text: "40 стало правым сыном двадцатки. Поддеревов
        ева от 40 – это одно и то же место.",
      moved: [{"x", "p", "B"},
      ms: RESULT_MS,
    },
  ],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
  pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
  edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
  pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
  edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
  key: "zig-zig",
  label: "Zig-Zig",
  when: "x и p – дети с ОДНОЙ стороны (оба слева или об
  rotations: "2 поворота, первым – верхний",
  keys: { x: 20, p: 40, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZZ_S0,
      kind: "start",
      title: "Было: прямая линия 60 → 40 → 20",
      text: "Три узла выстроились в «бамбук»: каждый сл

```

```

    },
  ],
};

/* ----- Zig-Zag -----
const ZG_S0 = {
  pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
  edges: [ ["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
  pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
  edges: [ ["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
  key: "zig-zag",
  label: "Zig-Zag",
  when: "x и p – дети с РАЗНЫХ сторон (змейка)",
  rotations: "2 поворота, первым – нижний",
  keys: { x: 40, p: 20, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZG_S0,
      kind: "start",
      title: "Было: 60 → влево на 20 → вправо на 40",
      text: "Узлы идут «змейкой»: сначала шаг влево, по
        ",
      ms: RESULT_MS,
    },
    {
      ...ZG_S0,
      kind: "aim",
      title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
      text: "Дед (60) стоит на месте и в этом повороте
      focus: ["p", "x"],
    }
  ]
};

```



```
};
```

```
const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "х — ищем его",
  p: "р — родитель",
  g: "г — дед",
};
```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```
const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])
```

```
return (
  <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
    <defs>
      <marker id="splay-arrow" viewBox="0 0 10 10" re
        <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
      </marker>
    </defs>
```

```
/* призраки: где узел стоял до поворота */
```

```
{ghosts.map((id) => {
  const from = prev[id]!;
  const to = pos[id]!;
  const r = isSubtree(id) ? 15 : 20;
  const dx = to[0] - from[0];
  const dy = to[1] - from[1];
  const len = Math.hypot(dx, dy) || 1;
  const pad = r + 8;
  return (
    <g key={`ghost-${id}`} opacity={0.55}>
      <circle cx={from[0]} cy={from[1]} r={r} fil
      {len > pad * 2 + 6 && (
        <line
          x1={from[0] + (dx / len) * pad}
```

```

const r = sub ? 15 : 20;
const hot = Boolean(focus && focus.includes(id))
return (
  <g
    key={id}
    style={{
      transform: `translate(${p[0]}px, ${p[1]}px)`,
      transition: MOVE,
      opacity: dim(id) ? 0.28 : 1,
    }}
  >
    {hot && <circle r={r + 7} fill="none" stroke="red" />}
    <circle r={r} fill={c.fill} stroke={c.stroke} />
    <text
      textAnchor="middle"
      dominantBaseline="central"
      fontSize={sub ? 12 : 15}
      fontWeight="700"
      fill={sub ? "#94a3b8" : "#fff"}
    >
      {sub ? id : keys[id]}
    </text>
    {!sub && (
      <text textAnchor="middle" y={-r - 7} font-size={10}
        {id}
      </text>
    )}
  </g>
);
}}
</svg>
);
};

```

```

/** Имя случая из кода пользователя → вкладка демонстрация
const CASE_BY_NAME: Record<string, number> = {
  zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-zigzig": 1,
};

```

```
const idx = Math.min(frame, total - 1);
const current = active.frames[idx];
const prevPos = active.frames[Math.max(idx - 1, 0)].pos;
const duration = useMemo(() => Math.round(current.ms - prevPos));
const last = idx === total - 1;

useEffect(() => {
    setFrame(0);
}, [caseIdx]);

useEffect(() => {
    if (!playing) return;
    const t = setTimeout(() => setFrame((f) => (f + 1)), duration);
    return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
    <div className="w-full bg-slate-950 rounded-2xl border" style={{borderColor: "#808080"}}>
        <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
            {CASES.map((c, i) => (
                <button
                    key={c.key}
                    onClick={() => setCaseIdx(i)}
                    className={`px-3 sm:px-4 py-2 rounded-lg text-white transition-colors
                        ${i === caseIdx ? "bg-indigo-600 text-white"}
                        `}
                >
                    {c.label}
                </button>
            ))}
        </div>

        <div className="mb-3 text-xs sm:text-[13px] text-white">
            <span className="font-bold text-indigo-300">Корреляция</span>
            <span className="text-slate-500">({active.rotations})</span>
        </div>

        <div>
            <div>
                <div><div>

```

```

    </div>
    <p className="text-[11px] text-slate-500 mt-1" >
  </div>
</div>

{/* индикатор автопрокрутки */}
<div className="h-1 w-full bg-slate-800 rounded-f
  <div
    key={` ${caseIdx}- ${frame}- ${playing}- ${slow}`
    className="h-full bg-indigo-500"
    style={
      playing
        ? { animation: `demoProgress ${duration}m
          : { width: "100%", background: "#334155"
        }
    }
  />
</div>

<div className="flex flex-wrap items-center gap-2" >
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f - 1 + total) % total);
    }}
    disabled={idx === 0}
    className="px-3 py-2 rounded-lg bg-slate-800 l
      d:hover:text-slate-300 text-xs font-bold tra
  >
    ← Назад
  </button>
  <button
    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f + 1) % total);
    }}
    className="px-4 py-2 rounded-lg bg-indigo-600
      w-indigo-900/40"
  >

```

```

    )
  })
</div>
</div>

```

```

{/* что сейчас сообщил компилятор */}
<div className="mt-3 pt-3 border-t border-slate-800">
  <span className="text-[10px] uppercase tracking-wider">
    <Terminal className="w-3.5 h-3.5" /> из компилятора
  </span>
  {liveCase || liveX !== null || liveStep !== null ? (
    <>
      {liveCase && (
        <span className="text-[11px] font-mono bg-slate-900 text-white">
          case = {liveCase}
        </span>
      )}
      {liveX !== null && <span className="text-[11px] font-mono bg-slate-900 text-white py-1">x = {liveX}</span>}
      {liveP !== null && <span className="text-[11px] font-mono bg-slate-900 text-white py-1">p = {liveP}</span>}
      {liveG !== null && <span className="text-[11px] font-mono bg-slate-900 text-white py-1">g = {liveG || "нет (Zig)"}</span>}
      {liveStep !== null && <span className="text-[11px] font-mono bg-slate-900 text-white py-2 py-1">step = {liveStep}</span>}
    </>
  ) : (
    <span className="text-[11px] text-slate-500">
      запусти код из панели Python: переменная <code>case</code> случай,
      а <code>liveStep</code> шаг
    </span>
  )}
</div>

<p className="mt-2 text-[11px] text-slate-500">
  Листай кнопкой «Дальше» в своём темпе – кадры не пролетят мимо
</p>

```

```

    n.left = l;
    l.parent = n;
  }
  if (r) {
    n.right = r;
    r.parent = n;
  }
  return n;
}

export function bstInsert(root: SN | null, key: number) {
  if (!root) return { key, left: null, right: null, parent: null };
  const r = cloneWithParents(root)!;
  let cur: SN = r;
  for (;;) {
    if (key < cur.key) {
      if (!cur.left) {
        cur.left = { key, left: null, right: null, parent: cur };
        return r;
      }
      cur = cur.left;
    } else {
      if (!cur.right) {
        cur.right = { key, left: null, right: null, parent: cur };
        return r;
      }
      cur = cur.right;
    }
  }
}

```

/** Поднять child над его родителем (один атомарный поворот)

```

export function rotateUp(child: SN): void {
  const p = child.parent;
  if (!p) return;
  const g = p.parent;
  if (p.left === child) {
    p.left = child.right;
  }
}

```

```

    return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
  kind: SplayStepKind;
  root: SN | null;
  aimEdge?: [number, number];
  /** Кадр cut: ребро, по которому отстёгивается среднее */
  cutEdge?: [number, number];
  cutLabel?: string;
  /** Куда β переедет: сторона у верхнего узла. */
  cutDest?: string;
  pathKeys?: number[];
  caseLabel?: string;
  note: string;
  rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
  rootIn: SN | null,
  key: number,
  isInsert = false,
): { steps: SplayStep[]; found: boolean } {
  let root: SN | null = cloneWithParents(rootIn);
  const steps: SplayStep[] = [];
  if (!root)
    return { steps: [{ kind: "done", root: null, note:

  // --- спуск ---
  let cur: SN | null = root;
  const pathKeys: number[] = [];
  let found = false;
  let last: SN = root;
  while (cur) {
    last = cur;
    pathKeys.push(cur.key);

```

```

        kind: "cut",
        root: cloneWithParents(root)!,
        cutEdge: [child.key, mid ? mid.key : child.key],
        cutLabel: mid ? `β = поддерезо ${mid.key}` : "β п",
        cutDest: mid ? (p.left === child ? "справа у p" :
        caseLabel: label,
        note: mid
            ? `Отстёгиваем среднее поддерезо β = ${mid.key}
              й у них общий — после поворота переедет ${p.l
            : `Среднего поддереза нет (β пусто): ${child.ke
        rotCount,
    });
const before = collectEdges(root);
rotateUp(child);
fixRoot();
rotCount += 1;
const diff = diffEdges(before, collectEdges(root));
steps.push({
    kind: "rot",
    root: cloneWithParents(root)!,
    caseLabel: label,
    note: `Поворот: ${child.key} встал над ${p.key}.
    rotCount,
});
};

while (target.parent) {
    const p = target.parent;
    const g = p.parent;
    if (!g) {
        emitRotation(
            target,
            "Zig",
            [p.key, target.key],
            `Zig: у ${target.key} нет деда — хватит одного
        );
        continue;
    }
}

```



```

        note: `Второе ребро: ${target.parent.key}-${target.key}
        rotCount,
    });
    emitRotation(
        target,
        "Zig-Zag",
        [target.parent.key, target.key],
        `Доводим: ${target.key} над ${target.parent.key}
    );
}
}

steps.push({
    kind: "done",
    root: cloneWithParents(root)!,
    note: `    ${target.key} в корне за ${rotCount} ${rotCount}
    rotCount,
});
return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number =>
    let d = 0;
    let cur = t;
    while (cur) {
        if (key === cur.key) return d;
        cur = key < cur.key ? cur.left : cur.right;
        d += 1;
    }
    return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [
    ...inorderKeys(t.left),
    t.key,
    ...inorderKeys(t.right)
] : []);

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 7, 6, 5, 4, 3, 2, 1];

```

```

        stroke={isCut ? "#f59e0b" : "#64748b"}
        strokeWidth={isCut ? 3 : 2}
        strokeDasharray={isCut ? "5,3" : "none"}
    />
    <circle cx="55" cy="46" r="15" fill="#1e293b"
    <text x="55" y="50" textAnchor="middle" fill=
    <circle cx="30" cy="86" r="15" fill="#78350f"
    <text x="30" y="90" textAnchor="middle" fill=
    <circle cx="12" cy="126" r="13" fill="#0f172a"
    <text x="12" y="130" textAnchor="middle" fill=
    <circle
        cx="48" cy="126" r="13"
        fill={isCut ? "#78350f" : "#0f172a"}
        stroke={isCut ? "#f59e0b" : "#64748b"}
        strokeWidth={isCut ? 2.5 : 1.5}
        strokeDasharray={isCut ? "4,3" : "none"}
    />
    <text x="48" y="130" textAnchor="middle" fill=
    <circle cx="80" cy="126" r="13" fill="#0f172a"
    <text x="80" y="130" textAnchor="middle" fill=
    <text x="98" y="92" fill={isCut ? "#f59e0b" :
        {isCut ? " β отстёрнута" : "β при x"}}
    </text>
    <text x="16" y="160" fill="#64748b" fontSize=
</g>
<text x="163" y="96" fill="#10b981" fontSize="20"
<g transform="translate(186,6)">
    <text y="10" fill="#94a3b8" fontSize="11" font=
    <line x1="70" y1="46" x2="45" y2="86" stroke=
    <line x1="70" y1="46" x2="95" y2="86" stroke=
    <line x1="45" y1="86" x2="27" y2="126" stroke=
    <line
        x1="45" y1="86" x2="63" y2="126"
        stroke={isRot ? "#10b981" : "#64748b"}
        strokeWidth={isRot ? 3 : 2}
    />
    <circle cx="70" cy="46" r="15" fill="#059669"
    <text x="70" y="50" textAnchor="middle" fill=

```

```

    walk(n.left, d + 1);
    pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d });
    slot += 1;
    walk(n.right, d + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

const SplayWalkViz: React.FC = () => {
  const initial = useMemo(buildInitial, []);
  const [steps, setSteps] = useState<SplayStep[] | null>();
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [input, setInput] = useState("");
  const [treeVers, setTreeVers] = useState(0);
  const baseRef = useRef<SN>(initial);
  const timer = useRef<ReturnType<typeof setTimeout> | null>();

```

```

  const runKey = (key: number, isInsert = false) => {
    const { steps: st } = splaySteps(baseRef.current, key, isInsert);
    setSteps(st);
    setIdx(0);
    setPlaying(true);
  };

```

```

  useEffect(() => {
    if (!playing || !steps) return;
    if (idx >= steps.length - 1) {
      setPlaying(false);
      baseRef.current = steps[steps.length - 1].root ?? null;
      setTreeVers((v) => v + 1);
      return;
    }
    timer.current = setTimeout(() => setIdx((v) => v + 1), 100);
    return () => {
      if (timer.current) clearTimeout(timer.current);
    };
  });

```

```

const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;

const doInsert = () => {
  const v = parseInt(input, 10);
  if (Number.isNaN(v)) return;
  const withNew = bstInsert(baseRef.current, v);
  baseRef.current = withNew;
  setTreeVers((t) => t + 1);
  runKey(v, true);
  setInput("");
};

const stepColor = (k: number): string => {
  if (!step) return "#0f172a";
  if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
  if ((step.kind === "aim" || step.kind === "cut") &&
    const e = step.aimEdge ?? step.cutEdge!;
    if (e[0] === k) return "#1e293b";
    if (e[1] === k) return "#78350f";
  )
  if (step.kind === "rot" && step.pathKeys?.includes(k)) return "#0f172a";
  return "#0f172a";
};

const strokeColor = (k: number): string => {
  if (!step) return "#64748b";
  if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
  if ((step.kind === "aim" || step.kind === "cut") &&
    const e = step.aimEdge ?? step.cutEdge!;
    if (e.includes(k)) return "#f59e0b";
  )
  if (step.pathKeys?.includes(k)) return "#38bdf8";
  return "#64748b";
};

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border"
    {/* Wanka */}

```

```

        >
            Исходное
        </button>
        <button
            onClick={() => {
                const shuffled = [...INITIAL_KEYS].sort(() =>
                    Math.random() - 0.5);
                let r: SN | null = null;
                for (const k of shuffled) r = bstInsert(r, k);
                baseRef.current = r!;
                setSteps(null);
                setIdx(0);
                setPlaying(false);
                setTreeVers((v) => v + 1);
            }}
            className="px-3 py-2 rounded-lg text-xs font-medium"
        >
            Перемешать
        </button>
    </div>
</div>

{/* Плеер */}
{steps && (
    <div className="flex flex-wrap items-center gap-2" >
        <button onClick={() => { setIdx(0); setPlaying(false); }}
            className="px-3 py-2 rounded-lg bg-slate-700 text-white disabled:opacity-50"
            > В начало
        </button>
        <button onClick={() => { setPlaying(false); setTreeVers((v) => v + 1); }}
            className="px-3 py-2 rounded-lg text-xs font-bold bg-slate-700 text-white"
            > ⏮
        </button>
        <button onClick={() => setPlaying(!playing)}
            className="px-3 py-2 rounded-lg text-xs font-medium bg-emerald-600 text-white"
            > {playing ? "Пауза" : "▶ Пуск"}
        </button>
        <button onClick={() => { setPlaying(false); setTreeVers((v) => v + 1); }}
            className="px-3 py-1.5 rounded-lg text-xs font-medium"
            > ⏭
        </button>
    </div>
)
}

```

```

        strokeDasharray={isAim ? "6,4" : isCut ? "5,5" : ""}
        className="transition-all duration-500"
      />
      {isCutEdge && (
        <text x={({a.px + b.px} / 2 + 14} y={({a.py + b.py} / 2 + 14)}
        nospace" className="transition-all duration-500"
        {step?.cutLabel} → {step?.cutDest}
      </text>
      )}
    </g>
  );
}
}
}
{nodes.map((n) => {
  const aimE = step?.aimEdge ?? (step?.kind === "aim" ? step?.aimEdge : null);
  const isAimNode = aimE?.includes(n.key);
  return (
    <g
      key={`n-${n.key}-${treeVers}`}
      transform={`translate(${n.px}, ${n.py})`}
      onClick={() => {
        if (playing) return;
        const has = inorderKeys(baseRef.current).includes(n.key);
        if (has) runKey(n.key);
      }}
      className="cursor-pointer"
    >
      <circle
        r={isAimNode ? 24 : 20}
        fill={stepColor(n.key)}
        stroke={strokeColor(n.key)}
        strokeWidth={isAimNode ? 4 : 2}
        className="transition-all duration-300"
      />
      <text y={4} fill="white" fontSize="13">
        {n.key}
      </text>
    </g>
  );
})}

```

```

<div className="w-full lg:w-1/2 bg-rose-950/10" style={{backgroundImage: 'linear-gradient(to right, #f8d7da 49%, #fff 49% 51%, #f8d7da 51%)'}}>
  <h4 className="text-sm font-bold text-rose-300">Шаги
  {steps ? (
    <table className="w-full text-sm">
      <tbody className="font-mono">
        <tr className="border-b border-slate-800">
          <td className="py-2 text-slate-400">г.
          <td className="py-2 text-right text-slate-400">г.
        </tr>
        <tr className="border-b border-slate-800">
          <td className="py-2 text-slate-400">в.
          <td className="py-2 text-right text-slate-400">в.
        </tr>
        <tr>
          <td className="py-2 text-slate-400">п.
          <td className="py-2 text-right text-slate-400">п.
        </tr>
      </tbody>
    </table>
  ) : (
    <p className="text-xs text-slate-400">Запись
  )}
  <p className="text-xs text-slate-500 mt-3">Од
    щений дешевлеет.</p>
</div>
<div className="w-full lg:w-1/2 bg-slate-900/60" style={{backgroundImage: 'linear-gradient(to right, #34495e 49%, #fff 49% 51%, #34495e 51%)'}}>
  <h4 className="text-sm font-bold text-slate-200">Шаги
  <div className="bg-slate-950/80 rounded-lg p-4">
    {CODE_LINES.map((l) => {
      const zig = step?.caseLabel === "Zig" &&
      const zz1 = step?.caseLabel === "Zig-Zig"
      const zz2 = step?.caseLabel === "Zig-Zig"
      const za1 = step?.caseLabel === "Zig-Zag"
      const za2 = step?.caseLabel === "Zig-Zag"
      const active = zig || zz1 || zz2 || za1 |
      return (
        <div key={l.line} className={`py-1 px-2
          border-400" : "text-slate-400"}`>

```

```
-----
  { name: 'Тарелка из-под суши', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20', isDirty: false, animState: 'idle' },
  { name: 'Сковорода от глазуньи', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20', isDirty: false, animState: 'idle' },
];
```

```
let counter = 0;
```

```
export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>(
    [
      { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20', isDirty: true, animState: 'idle' },
      { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20', isDirty: true, animState: 'idle' },
      { id: 'p3', name: 'Блюдец от торта', emoji: '🍰', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500/20', isDirty: true, animState: 'idle' },
    ]
  );
```

```
  const runtime = useVizRuntime();
  const liveStack = vizArray(runtime?.variables?.st) ?? [];
  const displayStack: Plate[] = liveStack
    ? liveStack.map((value, index) => ({
        id: `python-${index}`,
        name: String(value),
        emoji: " ",
        color: "from-indigo-500/20 to-indigo-600/20 border-indigo-500/20",
        isDirty: true,
        animState: "idle",
      }))
    : dirtyStack;
```

```
  const [cleanRack, setCleanRack] = useState<Plate[]>([]);
  const [isWashing, setIsWashing] = useState(false);
  const [showSoap, setShowSoap] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>([]);
```

```
  const addLog = (msg: string) => setLog(prev => [...prev, msg]);
```

```
  // PUSH: Положить грязную тарелку сверху стопки
```



```

const topPlate = dirtyStack[dirtyStack.length - 1];

addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);

// Step 1: Start washing animation (sponge and soap)
setDirtyStack(prev => prev.map(p => p.id === topPlate.id ? {
  ...p,
  isDirty: false,
  animState: 'clean',
} : p));

setShowSoap(true);

setTimeout(() => {
  // Step 2: Wash complete. Plate becomes clean and
  const cleanPlate: Plate = {
    ...topPlate,
    isDirty: false,
    animState: 'clean',
  };

  setCleanRack(prev => [cleanPlate, ...prev].slice(0, 1));
  setDirtyStack(prev => prev.slice(0, prev.length - 1));
  setShowSoap(false);
  setIsWashing(false);
  addLog(`  Готово! Чистая тарелка ${topPlate.emoji}`);
}, 850);
}, [dirtyStack, isWashing]));

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', animState: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', animState: 'in' },
    { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', animState: 'in' },
  ]);
  setCleanRack([]);
  setIsWashing(false);
  setShowSoap(false);
  setLog(['  Стол сброшен. Посуда снова грязная!']);
};

```

```
      <span> Помыть верхнюю (POP)</span>
    </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800
        r-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

  {/* ИНТЕРАКТИВНАЯ КУХНЯ */}
  <div className="grid grid-cols-1 md:grid-cols-12" >

    {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
    <div className="md:col-span-7 bg-slate-900 rounded-3xl
      [380px]">
      <div className="absolute top-4 left-4 text-[10px] font-mono">
        Грязная стопка (Стек вызовов / LIFO)
      </div>

      {/* Визуализация раковины */}
      <div className="absolute top-4 right-4 flex items-center
        r-slate-800/80 text-[10px] font-mono">
        <span className="w-1.5 h-1.5 rounded-full bg-slate-700" />
        <span>РАКОВИНА </span>
      </div>

      {/* Мыльные пузыри при мытье */}
      {showSoap && {
        <div className="absolute inset-0 pointer-events-none">
          <div className="absolute w-6 h-6 rounded-full bg-slate-700
            opacity-50"></div>
          <div className="absolute w-4 h-4 rounded-full bg-slate-700
            opacity-50"></div>
          <div className="absolute w-5 h-5 rounded-full bg-slate-700
            opacity-50"></div>
        </div>
      }
    }
  </div>
```

```

        ${plate.animState === 'washing' ?
        ${isTop ? 'ring-4 ring-blue-500/50' : ''}
      `}
    >
    <span className="text-lg">{plate.emblem}</span>
    <span className="text-slate-800 text-sm">
      {isTop && (
        <span className="text-[9px] bg-blue-500/50 p-1">
          LIFO
        </span>
      )}
    </div>
  );
  }}}
</div>
))

  { /* Столешница раковины */ }
  <div className="w-full h-4 bg-gradient-to-r from-slate-500 to-slate-800">
</div>

  { /* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */ }
  <div className="md:col-span-5 bg-slate-900 rounded-2xl p-4">
    <div className="absolute top-4 left-4 text-[12px] font-mono">
      Сушилка для чистой посуды
    </div>

    <div className="absolute top-4 right-4 flex items-center justify-end">
      <div className="order-emerald-800/80 text-[9px] font-mono">
        <Sparkles className="w-3.5 h-3.5 animate-pulse">
          <span>ЧИСТО </span>
        </div>
      </div>

      <div className="flex-1 flex flex-col justify-end">
        {cleanRack.length === 0 ? (
          <div className="flex-1 flex flex-col item">
            <span className="text-5xl mb-2"> </span>
            <p className="text-xs font-bold font-mono">

```

```
};
```

ФАЙЛ 75/111: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef, useMemo } from "react";
import { useVizRuntime, vizArray, vizNumber, vizString } from "viz-runtime";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";
```

```
function generateKmpSteps(pattern: string, text: string) {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];
```

```
  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });
```

```
  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];
```

```
    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
    ;
```

```
    while (j > 0 && s[i] !== s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
      // яд назад: откат j = pi[j - 1] = pi[j - 1];
      j = pi[j - 1];
    }
  }
```

```
  if (s[i] === s[j]) {
    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
    // ска увеличивается.` });
    j++;
  } else {
    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
  }
```

```

        падают! Окно растёт.` }));
        z[i]++;
    }

    if (i + z[i] < n) {
        steps.push({ i, l, r, zTarget: z[i], zSource: z[i] })` разные. Стоп.` });
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Упёр` });
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксируем` });

    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
        steps.push({ i, l, r, z: [...z], desc: `0` });
    }

    if (z[i] === pattern.length) {
        steps.push({ i, l, r, z: [...z], found: true });
    }
}
return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
    const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
    const [pattern, setPattern] = useState("aba");
    const [text, setText] = useState("abacaba");

    // Ensure inputs are valid dynamically
    const safePattern = pattern || " ";
    const safeText = text || " ";

    const { s: demoString, steps, patternLength: demoPatternLength } = generateKmpSteps(safeText, safePattern);
    if (mode === "kmp") return generateKmpSteps(safeText, safePattern);
    else return generateZSteps(safePattern, safeText);
}

```

```

const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveVal
const step = compilerLinked
  ? {
    ...baseStep,
    i: liveI ?? baseStep.i,
    j: liveJ ?? baseStep.j,
    l: liveL ?? baseStep.l,
    r: liveR ?? baseStep.r,
    [mode === "kmp" ? "pi" : "z"]: hasLiveValues
    desc: `Компилятор: i=${liveI ?? "-"}${mode ===
  }
  : baseStep;

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>

    <div className="flex items-center gap-2
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p

```

```

-900/40");
    }
}

return {
    <div key={index} className=
        {/* L/R markers for Z */
        {mode == "z" && step.l
            <div className="abs
        }}
        {mode == "z" && step.r
            <div className="abs
        }}

        {/* Index */}
        <span className={`text-

        {/* Char Box */}
        <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
        {char}
        </div>

        {/* Value array box */}
        <div className="text-[1
slate-700/50 font-mono font-bold">
            {(mode == "kmp" ?
        </div>

        {/* KMP fingers */}
        {mode == "kmp" && step
            <div className="abs
z-20">
                <span classNam
            </div>
        }}
        {mode == "kmp" && step
            <div className="abs

```

```

    )
  }
}
</div>
</div>

<div className="flex flex-col sm:flex-row gap-4"
  {/* Controls */>
  <div className="flex bg-slate-900 border-1 border-slate-800 rounded-lg p-4"
    <button onClick={prevStep} disabled={steps.length === 0}
      -400 hover:text-white hover:bg-slate-800 rounded-lg border-1 border-slate-800
      <Undo strokeWidth={3} size={16} />
    </button>
    <button onClick={playPause} className="flex-1"
      x items-center justify-center min-w-[80px] text-white
      {autoplay ? <><Pause size={16} /> : <><Play size={16} />}
    </button>
    <button onClick={nextStep} disabled={steps.length === steps.length}
      e="p-2 text-slate-400 hover:text-white hover:bg-slate-800 rounded-lg border-1 border-slate-800
      <SkipForward strokeWidth={3} size={16} />
    </button>
    <button onClick={reset} aria-label="Reset"
      :bg-slate-800 rounded-lg ml-2 border-1 border-slate-800
      <RefreshCw strokeWidth={3} size={16} />
    </button>
  </div>

  {/* Description Log */>
  <div className="flex-1 bg-slate-900/50 border-1 border-slate-800 p-4"
    overflow-hidden">
    <div className="absolute top-0 right-0 text-white font-mono text-sm"
      <span className="font-mono text-sm" />
    </div>
    <div className="text-[10px] text-slate-400"
      {steps.length}</div>
    <div className="text-sm text-slate-400"
      {step.desc}
    </div>
  </div>
</div>

```



```
        </div>
      </div>
    );
  }
}
```

ФАЙЛ 76/111: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";

interface TooltipProps {
  /** Содержимое подсказки (можно JSX). */
  content: React.ReactNode;
  /** Элемент, на который наводят курсор / фокус. */
  children: React.ReactNode;
  /** С какой стороны показывать. По умолчанию – сверху
  side?: "top" | "bottom";
  className?: string;
}

/**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
export const Tooltip: React.FC<TooltipProps> = ({ content, children }) => {
  const [open, setOpen] = useState(false);

  const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

  return (
    <span
      className={`relative inline-flex ${className ?? ""}
      onMouseEnter={() => setOpen(true)}
      onMouseLeave={() => setOpen(false)}
      onFocus={() => setOpen(true)}
    >
```

```

    id: number;
    x: number;
    y: number;
    label: string;
    name: string;
}

interface TEdge {
    u: number;
    v: number;
}

const dagNodes: TNode[] = [
    { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (0)" },
    { id: 1, x: 280, y: 70, label: "1", name: "Дискретка" },
    { id: 2, x: 280, y: 230, label: "2", name: "Линейный" },
    { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы" },
    { id: 4, x: 640, y: 150, label: "4", name: "Машинное" },
];

const dagEdges: TEdge[] = [
    { u: 0, v: 1 },
    { u: 0, v: 2 },
    { u: 1, v: 3 },
    { u: 2, v: 3 },
    { u: 3, v: 4 },
];

interface TStep {
    desc: string;
    visited: Set<number>;
    fullyProcessed: Set<number>;
    currentNode: number | null;
    topoList: number[];
    dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
```

```

const adj: number[][] = Array.from({ length: 5 }, (
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
  visited.add(u);
  dfsStack.push(u);
  recordStep(
    `DFS: зашли в вершину ${u} (${dagNodes[u].name
    u,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs(to);
    } else if (!fullyProcessed.has(to)) {
      // Explaining potential cycle detected (not in
      recordStep(
        `⚠ Внимание: Рёбра в серую вершину ${to} о
        u,
      );
    }
  }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in
recordStep(
  `DFS: закончили обработку '${dagNodes[u].name}'
  ,
  u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    recordStep(
      `Запускаем новый обход DFS от нетронутой верши

```

```

    setCurrentStepIdx(0);
  };

  const getTopoListString = () => {
    return [...step.topoList]
      .reverse()
      .map((id) => dagNodes[id].name)
      .join(" → ");
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-
      <div className="bg-slate-800 p-4 border-b border-
        <h3 className="font-bold text-white flex items-
          <Clock className="w-5 h-5 text-indigo-400" />
          Топологическая сортировка (Зависимости обучени
        </h3>

      <div className="flex items-center gap-2 bg-slate
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            500 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

        <button
          onClick={reset}
          aria-label="Сбросить сортировку"
          title="Сбросить сортировку"
          className="flex items-center justify-center
            ors"

```

```

    const v = dagNodes.find((n) => n.id === e.v);

    const isActive =
      (step.currentNode === edge.u || step.currentNode ===
      step.visited.has(edge.v));

    return (
      <line
        key={`edge-${idx}`}
        x1={u.x}
        y1={u.y}
        x2={v.x}
        y2={v.y}
        stroke={isActive ? "#818cf8" : "#334155"}
        strokeWidth={isActive ? "3" : "2"}
        markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow"})`}
        className="transition-all duration-300"
      />
    );
  });
}

{/* Nodes */}
{dagNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited = step.visited.has(node.id);
  const isProcessed = step.fullyProcessed.has(node.id);

  let fill = "#1e293b";
  let stroke = "#334155";
  let textColor = "text-slate-400";

  void textColor;

  if (isCurrent) {
    fill = "#312e81";
    stroke = "#818cf8";
    textColor = "text-white font-bold";
  } else if (isProcessed) {
    fill = "#022c22";
  }
})}

```

```

        className="pointer-events-none"
      >
        {node.name.split(" ").slice(1).join(" ")}
      </text>
    </g>
  );
}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800 p-4">
    <p className="text-xs text-indigo-300 min-h-[100px]">
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto">
    {/* Topological List output */}
    <div>
      <span className="text-[10px] text-slate-500">
        РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
      </span>
      <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
        {step.topoList.length === 0 ? (
          <span className="text-slate-600 text-sm">
            ожидаем обработку узлов...
          </span>
        ) : (
          getTopoListString()
        )}
      </div>
    </div>
  </div>
</div>

```

```
        onClick={() => setCurrentStepIdx((prev) => prev + 1)}
        className="bg-slate-900 border border-slate-800 p-2 text-white"
      >
        Вперед
      </button>
    </div>
  </div>
</div>
</div>
</div>
);
};
```

ФАЙЛ 78/111: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useContext, useEffect, useMemo, useState } from "react";
import { Terminal } from "lucide-react";
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from "../context";
```

```
/**
```

```
 * Интерактивный Треар — шесть режимов:
```

```
 *   build   — сборка: ВИДИМАЯ сортировка точек (и чем сортировать, так и сортируем)
```

```
 *           координатная плоскость НАВЕРХУ (ассоциативная сортировка)
```

```
 *           чек-лист инвариантов (автопроверка, как у бинарного поиска)
```

```
 *   split   — разрез по x с двумя корзинами (грамотная сортировка)
```

```
 *   merge    — склейка двух деревьев по y.
```

```
 *   erase    — удаление ПРОИЗВОЛЬНОЙ точки: найти по x, удалить по y.
```

```
 *   layers    — миф «дети в массиве по  $2k/2k+1$ »: работает, но неэффективно
```

```
 *   search    — «отсортирую бинарным поиском»: поиск не сортирует, но сортировка платит  $\geq n \cdot \log n$  сравнений, сортировка платит  $\geq n \cdot \log n$  сравнений, сортировка платит  $\geq n \cdot \log n$  сравнений,
```

```
 *           сортировка платит  $\geq n \cdot \log n$  сравнений, сортировка платит  $\geq n \cdot \log n$  сравнений,
```

```
 *           сортировка платит  $\geq n \cdot \log n$  сравнений, сортировка платит  $\geq n \cdot \log n$  сравнений,
```

```
 * Канонический набор: все x и все y различны  $\Rightarrow$  дерево — это бинарное дерево
```

```
 * Узел идентифицируется парой id = "x:y".
```

```

-----
    { key: 12, grade: 0 },
  ];
// (3,5) дубль ключа/приоритета – уберём: ввод без повт
export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p) => !INPUT_ORDER.some(
  (q) => q.key === p.key);

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

/* ----- чистые алгоритмы (экспортируются для тестов) ----- */

export function insert(root: TNode | null, p: Pair): TNode | null {
  if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null };
  if (p.key <= root.key) root.left = insert(root.left, p);
  else root.right = insert(root.right, p);
  return root;
}

export function buildTree(ps: Pair[]): TNode | null {
  let root: TNode | null = null;
  for (const p of sortedByY(ps)) root = insert(root, p);
  return root;
}

export interface SplitResult {
  l: TNode | null;
  r: TNode | null;
  trace: { atId: NodeId; goLeft: boolean; note: string };
}

export function splitTree(root: TNode | null, x: number): SplitResult {
  const trace: SplitResult["trace"] = [];
  const go = (t: TNode | null): { l: TNode | null; r: TNode | null } | null {
    if (!t) return { l: null, r: null };
    if (t.key <= x) {
      const sub = go(t.right);
      trace.push({ atId: t.id, goLeft: true, note: `(${t.key} <= ${x})  
о правое поддерево.` });
      t.right = sub.l;
      return { l: t, r: sub.r };
    }
    return go(t.left);
  }
  const result = go(root);
  return result ? result : { l: null, r: null, trace };
}

```



```

const go = (t: TNode | null): TNode | null => {
  if (!t) return null;
  if (key < t.key) {
    trace.push({ atId: t.id, note: `${key} < ${t.key}` });
    t.left = go(t.left);
    return t;
  }
  if (key > t.key) {
    trace.push({ atId: t.id, note: `${key} > ${t.key}` });
    t.right = go(t.right);
    return t;
  }
  trace.push({ atId: t.id, note: `Нашли (${t.key}), ${t.key}` });
  const m = mergeTree(t.left, t.right);
  trace.push(...m.trace.map((s) => ({ atId: s.chosenId })));
  return m.root;
};
return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
  t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

```

/* ----- валидация treap (для чек-листа и стресс-теста) ----- */

```

export function checkTreap(root: TNode | null): { bst: boolean, heap: boolean } {
  let bst = true;
  let heap = true;
  let prev: number | null = null;
  const walk = (n: TNode | null, min: number, max: number, parentGrade: number) => {
    if (!n) return;
    if (n.key <= min || n.key > max) bst = false;
    if (parentGrade !== null && n.grade > parentGrade) heap = false;
    walk(n.left, min, n.key, n.grade);
    if (prev !== null && n.key < prev) bst = false;
    prev = n.key;
    walk(n.right, n.key, max, n.grade);
  };
  walk(root, -Infinity, Infinity, null);
  return { bst, heap };
}

```

```

edgeColor?: (from: NodeId, to: NodeId) => string;
edgeDash?: (from: NodeId, to: NodeId) => boolean;
pulseId?: NodeId | null;
subtitle?: string;
}

const TreeView: React.FC<TreeViewProps> = ({ root, width, height, pulseId }) => {
  const pos = useMemo(() => layoutTree(root), [root]);
  const nodes = useMemo(() => [...pos.entries()].map(([id, n]) => n), [pos]);
  const edges = useMemo(() => treeEdges(root), [root]);
  const autoW = Math.max(420, ...nodes.map((n) => n.px));
  const autoH = Math.max(260, ...nodes.map((n) => n.py));
  const w = width ?? autoW;
  const h = height ?? autoH;
  const byId = (id: NodeId) => nodes.find((n) => n.id === id);
  return (
    <div>
      <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full" style={style}>
        {edges.map((e) => {
          const a = byId(e.from);
          const b = byId(e.to);
          if (!a || !b) return null;
          return (
            <line
              key={`te-${e.from}-${e.to}`}
              x1={a.px} y1={a.py} x2={b.px} y2={b.py}
              stroke={edgeColor ? edgeColor(e.from, e.to) : 'black'}
              strokeWidth={2.5}
              strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
              className="transition-all duration-500"
            />
          );
        })}
        {nodes.map((n) => {
          const p = pairParts(n.id);
          const pulse = pulseId === n.id;
          return (
            <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`}

```

```

    )) => {
const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.key) })
return (
  <svg
    viewBox={`0 0 ${PL_W} ${PL_H}`}
    className={`w-full ${interactive ? "touch-none select-none" : ""}`}
    onPointerMove={onMove}
    style={interactive ? { cursor: "crosshair" } : undefined}
  >
    <defs>
      <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
        <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
      </pattern>
    </defs>
    <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1"/>
    <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1"/>
    <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H - 16} stroke="black" stroke-width="1"/>
    {[0, 10, 20, 30, 40].map((k) => {
      <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} fill="black" font-size="10" font-weight="bold"/>
    })}
    {[{-4, 0, 4, 8].map((g) => {
      <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} fill="black" font-size="10" font-weight="bold"/>
    })}
    <text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8" font-size="10" font-weight="bold"/>
    <text x={PX(1) - 26} y="14" fill="#94a3b8" font-size="10" font-weight="bold"/>

    {drawnEdges.map((e, i) => {
      const a = byPair(e.a);
      const b = byPair(e.b);
      const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.b.key === e.b.key && badEdge.grade === e.b.grade) || (badEdge.a.key === e.b.key && badEdge.b.key === e.a.key && badEdge.grade === e.a.grade));
      return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y} stroke="black" stroke-width="2" strokeDasharray={isBad ? "4,3" : "none"}/>
    })}

    </svg>
  )
}

/* временная линия игры */
{interactive && selected && cursor && (

```

Универсальное пособие • полный исходный код

/* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родите

```
export const CANON_GAME: GEdge[] = (() => {
  const es: GEdge[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ a: { key: n.key, grade: n.g
    if (n.right) { es.push({ a: { key: n.key, grade: n.
  };
  walk(buildTree(POINTS));
  return es;
})();
```

```
const sameP = (p: Pair, q: Pair) => p.key === q.key &&
const gpid = (p: Pair) => `${p.key}:${p.grade}`;
```

```
/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge |
  edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pa
  let cur: Pair | undefined = from;
  const seen = new Set<string>();
  while (cur) {
    if (sameP(cur, target)) return true;
    if (seen.has(gpid(cur))) return false;
    seen.add(gpid(cur));
    cur = parentOf(edges, gpid(cur))?.a;
  }
  return false;
};
```

```
/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 спр
export function forestIsBst(edges: GEdge[], all: Pair[])
  const kids = new Map<string, Pair[]>();
  for (const e of edges) kids.set(gpid(e.a), [...(kids.
  const hasParent = new Set(edges.map((e) => gpid(e.b))
```



```

        reason:
            x.grade > y.grade ? undefined : "линия в treap во
    };
}

export function checkGameState(
    points: Pair[],
    edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
    const n = points.length;
    if (n < 2) return { heapOk: true, singleParent: true,
    const heapOk = edges.every((e) => e.a.grade > e.b.grade);
    const parentCount = new Map<string, number>();
    for (const e of edges) parentCount.set(pid(e.b), (parentCount.get(pid(e.b)) || 0) + 1);
    const singleParent = [...parentCount.values()].every(v => v === 1);
    // связность: компоненты через union-find
    const comp = new Map<string, string>(points.map((p) => {
    const find = (x: string): string => {
        while (comp.get(x) !== x) {
            comp.set(x, comp.get(x)!);
            x = comp.get(x)!;
        }
        return x;
    };
    for (const e of edges) {
        const ra = find(pid(e.a));
        const rb = find(pid(e.b));
        if (ra !== rb) comp.set(ra, rb);
    }
    const comps = new Set(points.map((p) => find(pid(p))));
    const connected = comps.size === 1;
    const countOk = edges.length === n - 1;
    // BST: дети по x, inorder не убывает; корень один; э
    const children = new Map<string, Pair[]>();
    for (const e of edges) children.set(pid(e.a), [...(children.get(pid(e.a)) || []), e.b]);
    const roots = points.filter((p) => !parentCount.has(pid(p)));
    let bstOk = roots.length === 1;
    if (bstOk) {

```

```

}

/** Дерево из проведённых линий – для показа результата
export function edgesToTree(edges: GameEdge[]): TNode | null {
  const parentOf = new Map<string, GameEdge>();
  for (const e of edges) parentOf.set(pid(e.b), e);
  const roots = edges.filter((e) => !parentOf.has(pid(e.b)));
  if (!roots.length && edges.length) return null;
  const nodes = new Map<string, TNode>();
  const make = (p: Pair): TNode => ({
    id: pid(p),
    key: p.key,
    grade: p.grade,
    left: null,
    right: null,
  });
  const rootPair = roots.length ? roots[0].a : edges[0].a;
  if (!rootPair) return null;
  const build = (p: Pair): TNode => {
    const node = make(p);
    nodes.set(pid(p), node);
    const kids = edges.filter((e) => pid(e.a) === pid(p));
    if (kids.length > 2) return node; // невалидно – по
    kids.sort((u, v) => u.key - v.key);
    const left = kids.filter((k) => k.key <= p.key); //
    const right = kids.filter((k) => k.key > p.key);
    if (left.length > 1 || right.length > 1) return node;
    if (left[0]) node.left = build(left[0]);
    if (right[0]) node.right = build(right[0]);
    return node;
  };
  return build(rootPair);
}

const BuildMode: React.FC = () => {
  const [raw, setRaw] = useState(DEMO_RAW);
  const live = useTreapLive();

```

```

useEffect(() => {
  if (!playing || phase === "game") return;
  const t = setTimeout(() => {
    if (phase === "sort") {
      if (algo === "none") { setPlaying(false); return; }
      if (sortIdx < 3) setSortIdx(sortIdx + 1);
      else setPlaying(false);
    } else {
      const total = order === "y-desc" ? canonEdges.length;
      if (connIdx < total) setConnIdx(connIdx + 1);
      else setPlaying(false);
    }
  }, 900);
  return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, canonEdges]);

const startConnect = (ord: ConnectOrder) => {
  setOrder(ord);
  setPhase("connect");
  setConnIdx(0);
  setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? Math.floor(n / 2) : 0) : 0;
const connEdgesAll = order === "y-desc" ? canonEdges : canonEdges.reverse();
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && connIdx >= connEdges.length;

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>([]);
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>({ x: 0, y: 0 });
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {

```



```

        attemptConnect(sel, p);
        setSel(null);
    }
};
const pointUp = (p: Pair) => {
    if (phase !== "game" || !sel) return;
    if (sel.key === p.key && sel.grade === p.grade) return;
    attemptConnect(sel, p);
    setSel(null);
    setCursor(null);
};
const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
    if (!sel) return;
    const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
    setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
};

const undoEdge = () => {
    setGameEdges((prev) => prev.slice(0, -1));
    setGameMsg(null);
};
const showSolution = () => {
    setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
    setHintUsed(true);
    setGameMsg("Решение показано – собери сам в следующем наборе");
};

return (
    <div className="space-y-5">
        {/* Произвольный список */}
        <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-700">
            <div className="flex flex-wrap items-center justify-center">
                <h4 className="text-sm font-bold text-slate-300">Задача
                <button onClick={() => { setRaw(DEMO_RAW); setRawIndex(0); }}
                    py-1 rounded-lg text-xs font-bold bg-slate-700 text-slate-200
                    демо-набор
                </button>
            </div>
        </div>
    </div>
);

```

```

        className={`px-3 py-1.5 rounded-lg text-white`} : "bg-slate-700 text-slate-400"
      >
        {a === "quicksort" ? "quicksort (сравнение)" : "быстрая сортировка"}
      </button>
    )}}
  </>
) : (
  <>
    <span className="text-xs text-slate-400">
      <button
        onClick={() => startConnect("y-desc")}
        className={`px-3 py-1.5 rounded-lg text-white`} : "bg-emerald-600 text-white" : "bg-slate-700 text-slate-400"
      >
        по y ↓ (алгоритм)
      </button>
      <button
        onClick={() => startConnect("left-right")}
        className={`px-3 py-1.5 rounded-lg text-white`} : "bg-emerald-600 text-white" : "bg-slate-700 text-slate-400"
      >
        слева направо
      </button>
      <button
        onClick={startGame}
        disabled={n < 2}
        className={`px-3 py-1.5 rounded-lg text-white`} : "bg-indigo-500/80 text-white" : "bg-emerald-600 text-white"
      >
        соединить сам
      </button>
    </>
  )}
</div>
</div>
<Plane
  points={userPairs}

```

```

        </b>: точки прибиты, дерево одно. А теперь
    ) : (
        <>Линий проведено: {connIdx} из {connEdgesA
    )}
</div>
</div>

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
  er-slate-700 bg-slate-900/40`} `>
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    {phase === "game" ? (
      <>
        <li>{verdict.heapOk ? " " : " "} каждое р
        <li>{verdict.singleParent ? " " : " "} у
        <li>{verdict.connected ? " " : " "} все т
        <li>{verdict.countOk ? " " : " "} линий р
        <li>{verdict.bstOk ? " " : " "} обход сле
      </>
    ) : (
      <>
        <li>{connDone ? " " : " "} каждое ребро в
        <li>{connDone ? " " : " "} обход слева-на
        <li>{connDone ? " " : " "} линий ровно <b
      </>
    )}
  </ul>
  {phase === "game" && (
    <div className="flex flex-wrap gap-2 mt-3">
      <button onClick={undoEdge} disabled={!gameE
        white disabled:opacity-30">↶ Убрать линию</
      <button onClick={resetGame} disabled={!gameE
        ate-700 text-white disabled:opacity-30"> С
      <button onClick={showSolution} disabled={ga
        te disabled:opacity-30"> Показать решение<
    </div>
  )}
}

```

```

    setX0(live.mid);
    setStepIdx(0);
    // eslint-disable-next-line react-hooks/exhaustive-
  }, [live.mid]));
  useEffect(() => {
    if (live.step === null) return;
    setStepIdx(Math.max(0, Math.min(steps.length - 1, l
    // eslint-disable-next-line react-hooks/exhaustive-
  }, [live.step, steps.length]));
  const done = idx >= steps.length - 1;

  const inL = new Set<NodeId>();
  const inR = new Set<NodeId>();
  steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? in

  const lTree = split.l;
  const rTree = split.r;

  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center gap-3">
        <span className="text-sm text-slate-300">Пазpez
        <select value={x0} onChange={(e) => { setX0(Num
          -700 rounded-lg px-3 py-1.5 text-sm font-mono
          {[2, 3, 6, 7, 9, 12, 45].map((k) => <option k
        </select>
        <button onClick={() => setStepIdx(Math.max(0, i
          bg-slate-700 text-white disabled:opacity-30">
        <button onClick={() => setStepIdx(Math.min(step
          font-bold bg-indigo-600 text-white disabled:op
        <span className="text-xs text-slate-400">war {i
      </div>

      <div className="bg-slate-900/40 rounded-xl p-4 bo
        <h4 className="text-sm font-bold text-slate-300
          , <span className="text-rose-400">розовые → R
        <TreeView
          root={fullTree}

```

```

const steps = merged.trace;

// step из кода листает кадры слияния
useEffect(() => {
  if (live.step === null) return;
  setStepIdx(Math.max(0, Math.min(steps.length - 1, live.stepIdx)));
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [live.step, steps.length]);
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;
const cur = steps[idx];

return (
  <div className="space-y-4">
    <div className="flex items-center gap-3">
      <button onClick={() => setStepIdx(Math.max(0, live.stepIdx - 1))}
        bg-slate-700 text-white disabled:opacity-30">←
      <button onClick={() => setStepIdx(Math.min(stepIdx + 1, steps.length - 1))}
        font-bold bg-indigo-600 text-white disabled:opacity-30">→
      <span className="text-xs text-slate-400">mar {cur.mar}
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
      <div className="bg-sky-950/20 rounded-xl p-3 border border-slate-700">
        <h5 className="text-sm font-bold text-sky-300">Левое дерево
        <TreeView root={l} width={480} height={220} nodeSize={100} />
      </div>
      <div className="bg-rose-950/20 rounded-xl p-3 border border-slate-700">
        <h5 className="text-sm font-bold text-rose-300">Правое дерево
        <TreeView root={r} width={480} height={220} nodeSize={100} />
      </div>
    </div>

    <div className="text-xs text-slate-300 bg-slate-900 p-3">
      {cur?.chosenId ? (
        <>⊗ {cur.note}</>
      ) : (
        <>Готово: {done ? "деревья склеены обратно в" : "деревья не склеены"}
      )}
    </div>
  </div>
)

```

```

    setStepIdx(Math.max(0, Math.min(steps.length - 1, live.stepIdx)), [live.step, steps.length]);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [live.step, steps.length]);
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

  const pathIds = useMemo(() => {
    const ids: NodeId[] = [];
    for (const s of steps.slice(0, idx + 1)) if (s.atId)
      return ids;
  }, [steps, idx]);
  const pathSet = new Set(pathIds);

  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center gap-3">
        <span className="text-sm text-slate-300">Удалить</span>
        <select value={key} onChange={(e) => { setKey(NodeId(e.target.value)) }}>
          {POINTS.map((p) => <option key={p.key} value={p.key}>{p.key}</option>)}
        </select>
        <button onClick={() => setStepIdx(Math.max(0, idx - 1))}>←</button>
        <button onClick={() => setStepIdx(Math.min(stepIdx + 1, steps.length - 1))}>→</button>
        <span className="text-xs text-slate-400">шаг {idx + 1}</span>
      </div>

      {!done ? (
        <div className="bg-slate-900/40 rounded-xl p-4">
          <TreeView
            root={fullTree}
            nodeColor={(id) => (pathSet.has(id) ? "#783f3f" : "#f5f5f5")}
            nodeStroke={(id) => (pathSet.has(id) ? "#f5f5f5" : "#783f3f")}
            subtitle="Куча сама умеет удалять только вершины"
          />
        </div>
      ) : (

```

```

    walk(full, 1);
    return arr;
  }, [full]));
const nodeCount = slots.filter(Boolean).length;

return (
  <div className="space-y-4">
    <div className="flex gap-2">
      <button onClick={() => setPick("heap")} className="text-white" : "bg-slate-700 text-slate-300"`>
      <button onClick={() => setPick("treap")} className="text-white" : "bg-slate-700 text-slate-300">
    </div>

    {pick === "heap" ? (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-slate-300">
          4>
        <div className="flex gap-2 overflow-x-auto mb-2">
          {heapArr.slice(1).map((v, i) => (
            <div key={i} className="px-3 py-2 rounded">
              <div className="text-slate-500 text-[10px]>
                <div className="text-white font-bold">{v}</div>
                <div className="text-emerald-400 text-[10px]>{i}</div>
              </div>
            </div>
          ))}
        </div>
        <p className="text-xs text-slate-400">Полное дерево
          →  $2i/2i+1$  попадает в узел.</p>
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4">
        <h4 className="text-sm font-bold text-rose-300">
          5>
        <TreeView root={full} subtitle="Попробуй «посчитать»>
          </div>
        <div className="flex gap-2 overflow-x-auto mt-2">
          {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]}
        </div>
      </div>
    )}
  </div>
);

```

```

    ];
    const target = 7;
    const sortedByKey = [...shuffled].sort((a, b) => a.key < b.key);
    const [stage, setStage] = useState<0 | 1 | 2>(0); // 0 - поиск, 1 - результат, 2 - ошибка
    const [bidx, setBidx] = useState(0);

    // шаги бинарного поиска на отсортированном массиве
    const bsSteps = useMemo(() => {
      const steps: { lo: number; hi: number; mid: number; found: boolean; }[] = [];
      let lo = 0, hi = sortedByKey.length - 1;
      while (lo <= hi) {
        const mid = Math.floor((lo + hi) / 2);
        if (sortedByKey[mid].key === target) { steps.push({ lo, hi, mid, found: true }); }
        if (sortedByKey[mid].key < target) { steps.push({ lo, hi, mid, found: false }); lo = mid + 1; }
        else { steps.push({ lo, hi, mid, found: false }); hi = mid - 1; }
      }
      return steps;
    }, [sortedByKey]);

    const wrongMid = Math.floor((shuffled.length - 1) / 2);
    const comparisons = stage === 2 ? bidx + 1 : 0;

    return (
      <div className="space-y-4">
        <div className="flex gap-2">
          {stage < 2 && (
            <button onClick={() => { setStage(stage === 0 ? 1 : 2); }}>
              {stage === 0 ? "Найти y для x=7 бинарным поиском" : "Поиск не удался"}
            </button>
          )}
          {stage === 2 && (
            <button onClick={() => setBidx(Math.min(bsSteps[comparisons - 1].mid, shuffled.length - 1.5))}>
              Шаг поиска →
            </button>
          )}
        </div>
      </div>
    );
  }
}

```


ля маленьких целых у (вкладка «Собрать», кноп
 задать порядок соединения.

```

        </div>
    </div>
  );
};

```

/* ===== РЕЖИМ GAME ===== */

```

const GameMode: React.FC = () => {
  const [edges, setEdges] = useState<GEdge[]>([]);
  const [sel, setSel] = useState<Pair | null>(null);
  const [hover, setHover] = useState<Pair | null>(null);
  const [msg, setMsg] = useState<string>("Кликни точку-");
};

```

```

const v = gameVerdict(edges, POINTS);

```

```

const click = (p: Pair) => {
  if (!sel) {
    setSel(p);
    setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кликай!`);
    return;
  }
  if (sameP(sel, p)) {
    setSel(null);
    return;
  }
  const res = tryConnect(edges, sel, p);
  if (res.ok) {
    const parent = sel.grade > p.grade ? sel : p;
    const child = sel.grade > p.grade ? p : sel;
    setEdges([...edges, { a: parent, b: child }]);
    setMsg(`Линия (${parent.key}; ${parent.grade})`);
    setSel(null);
  } else {
    setMsg("x " + res.reason);
    setSel(null);
  }
};

```

```

    {POINTS.map((p) => {
      const isSel = sel && sameP(sel, p);
      const isHover = hover && sameP(hover, p) &&
      return {
        <g key={`gp-${p.key}:${p.grade}`} onClick=
l)} className="cursor-pointer">
          <circle cx={PX(p.key)} cy={PY(p.grade)}>
          <circle cx={PX(p.key)} cy={PY(p.grade)}
"#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8"
transition-all duration-200" />
          <text x={PX(p.key)} y={PY(p.grade) + 3.5
space">{p.key}</text>
          <text x={PX(p.key)} y={PY(p.grade) + 24
}</text>
        </g>
      );
    })}
  </svg>
</div>

```

```

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
ate-900/40`} >
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    <li>{v.hear ? " " : " "} каждое ребро – вниз
    <li>{v.oneParent ? " " : " "} у каждой точки
    <li>{v.acyclic ? " " : " "} циклов нет</li>
    <li>{v.bst ? " " : " "} обход слева-направо –
    <li>{v.complete ? " " : " "} линий  $n-1 = \{POI$ 
  </ul>
  {win && (
    <p className="mt-3 text-sm font-bold text-еме
      ения просто не существует.</p>
  )}
</div>

```

```

const key = vars ? vizNumber(vars.key) : null;
const step = vars ? vizNumber(vars.step) ?? vizNumber(
const height = vars ? vizNumber(vars.height) ?? vizNumber(
const n = vars ? vizNumber(vars.n) : null;
return { vars, pairsText, mid, key, step, height, n }
};

```

```

/* ===== КОРПУС ===== */

```

```

type Tab = "build" | "split" | "merge" | "erase" | "layers"

```

```

const TABS: { id: Tab; label: string }[] = [
  { id: "build", label: "Собрать" },
  { id: "split", label: "Split ≈ " },
  { id: "merge", label: "Merge  " },
  { id: "erase", label: "Erase   " },
  { id: "layers", label: "Миф: 2k/2k+1" },
  { id: "search", label: "Поиск ≠ сортировка" },
  { id: "game", label: "Игра    " },
];

```

```

export const TreapBuildViz = () => {
  const [tab, setTab] = useState<Tab>("build");
  const chapterId = useContext(VizChapterContext);
  const live = useTreapLive();

```

```

  // вкладка = подрежим страницы: панель Python подстав.
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab);
  }, [chapterId, tab]);

```

```

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border">
      <div className="flex flex-wrap items-center justify">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-emerald-600 text-white">
            <span className="text-2xl"> </span>
          </div>

```

```

        x-2 py-1">step = {live.step}</span>
      </>
    ) : (
      <span className="text-[11px] text-slate-500">
        открой панель Python: переменные <code clas
        merald-300">mid</code>, <code className="for
        step</code> управляют этой демонстрацией
      </span>
    )}
  </div>

  {tab === "build" && <BuildMode />}
  {tab === "split" && <SplitMode />}
  {tab === "merge" && <MergeMode />}
  {tab === "erase" && <EraseMode />}
  {tab === "layers" && <LayersMode />}
  {tab === "search" && <SearchMode />}
  {tab === "game" && <GameMode />}
</div>
);
};

```

ФАЙЛ 79/111: src/components/VizLiveStatus.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";

interface VizLiveStatusProps {
  /** Название демонстрации – подписывается в режиме вс
  demoTitle?: string;
  className?: string;
}

/** Имена, по которым визуализации обычно читают матрицу

```

ФАЙЛ 80/111: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphComponentsViz } from "./GraphComponentsViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { ComplexityViz } from "./ComplexityViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import { PlanarColoringViz } from "./PlanarColoringViz";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumVisualizer";
import { SparseTableViz } from "./visualizers/SparseTableVisualizer";
import { TreapBuildViz } from "./TreapBuildViz";
```

```

    },
    "sparse-table": {
        title: "Разреженная таблица (1D и 2D)",
        hint: "Наведите курсор на любую ячейку – подсветится",
        Component: SparseTableViz,
    },
    treap: {
        title: "Treap: плоскость → дерево → Split/Merge/Era
        hint: "7 вкладок: Собрать (сортировка + соединение
            собери дерево мышкой, судья-инварианты объясняет к
        Component: TreapBuildViz,
    },
    "prefix-sums-2d": {
        title: "Префиксные суммы (1D и 2D)",
        hint: "Наведите на число – увидите, из чего оно сло
        Component: PrefixSumViz,
    },
    "dynamic-programming": {
        title: "Динамическое программирование",
        hint: "Таблица подзадач заполняется на глазах.",
        Component: DPVisualizer,
    },
    "splay-tree": {
        title: "Splay: механика → большое дерево → собери с
        hint: "Три вкладки: покадровая механика поворотов в
            песочница «подними узел сам» с проверкой порядка.
        Component: SplayPlayground,
    },
    "graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
    "top-sort": { title: "Топологическая сортировка", Component:
    "scc-kosaraju": { title: "Компоненты сильной связности", Component:
    "graph-articulation": { title: "Точки сочленения", Component:
    "bridges-code": { title: "Мосты: DFS + tin/low", Component:
    "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component:
    "planarity-euler-formula": {
        title: "Планарность и покраска:  $K_5/K_{3,3}$ ,  $\chi \leq \Delta+1$ , д
        hint: "Четыре вкладки: непланарные  $K_5$  и  $K_{3,3}$ , жадна
            ашего кода красит вершины напрямую.",

```

```
    Component: () => (  
      <div className="space-y-10">  
        <WaterfallAnimationWidget />  
        <SalmonAutomatonWidget />  
      </div>  
    ),  
  },  
  intro: {  
    title: "Интерактивный симулятор базовых структур: с  
    hint: "Проверь LIFO, FIFO и приоритетную очередь на  
    Component: Simulator,  
  },  
  "everyday-basics": {  
    title: "Бытовой тренажёр: стек, очередь, куча",  
    hint: "Тарелки, очередь в столовой и мешок гипотез  
    Component: Simulator,  
  },  
};  
  
export const getViz = (id?: string): VizEntry | undefined
```

ФАЙЛ 81/111: src/components/VizSourceBadge.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { Cpu, Film } from "lucide-react";  
import { Tooltip } from "../Tooltip";  
  
interface VizSourceBadgeProps {  
  /**  
   * true – картинку рисуют переменные из выполненного кода  
   * false – компонент показывает собственную встроенную картинку  
   */  
  linked: boolean;  
  /**  
   * Компилятор уже выполнил код и прислал шаг, но переключатель  
   */  
}
```

```

        className={`cursor-help inline-flex items-center
        linked
        ? "border-emerald-500/50 bg-emerald-500/10
        : awaiting
        ? "border-sky-500/50 bg-sky-500/10 text
        : "border-slate-600 bg-slate-800/80 tex
        }`}
      >
      {linked ? <Cpu className="h-3 w-3" /> : awaiting
      {linked ? "данные из компилятора" : awaiting
        встроенное демо"}
      {linked && detail ? <span className="font-mon
      </span>
    </Tooltip>

    {!linked && !awaiting && demoTitle ? (
      <span className="rounded-full border border-sla
      сценарий: {demoTitle}
      </span>
    ) : null}

    {mismatch ? (
      <Tooltip side="bottom" content={mismatch}>
        <span
          tabIndex={0}
          className="cursor-help rounded-full border l
          0"
        >
          размерность не совпадает
        </span>
      </Tooltip>
    ) : null}
  </div>
);
}

```



```
};
```

```
// Нормальные переходы бора
```

```
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
  0: { H: 1, S: 7 },
  1: { E: 2, I: 5 },
  2: { R: 3 },
  3: { S: 4 },
  4: {},
  5: { S: 6 },
  6: {},
  7: { H: 8 },
  8: { E: 9 },
  9: {},
};
```

```
export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training'>('training');
```

```
  /* — связь с Python-компилятором (PAGE_SYNC["aho-corasick"])
     text — сканируемый текст, state — вершина автомата
     matches — найденные вхождения (индекс конца, слово)
  */
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
```

```
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, "aho");
  }, [chapterId]);
```

```
  const liveText = vars ? vizString(vars.text) ?? vizString('') : '';
  const liveState = vars ? vizNumber(vars.state) ?? vizNumber(0) : 0;
  const liveI = vars ? vizNumber(vars.i) ?? vizNumber(0) : 0;
  const liveMode = vars ? vizString(vars.mode) : null;
  const liveMatchesRaw = vars ? vizArray(vars.matches) : [];
  const liveMatchesSig = useMemo(() => {liveMatchesRaw.map(m => m[0])});
  const liveFail = vars ? vars.fail : undefined;
```

```

    }, [vars]));

    const compilerLinked = Boolean(liveText || (liveState

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>(
const [currentIndex, setCurrentIndex] = useState<number>(
const [currentNode, setCurrentNode] = useState<number>(
const [isSearchDone, setIsSearchDone] = useState<boolean>(
const [searchLog, setSearchLog] = useState<string>(
    'Режим поиска: Введите текст и жмите «Следующий шаг»
);
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>(
const [foundMatches, setFoundMatches] = useState<string[]>(
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const clean = e.target.value.toUpperCase().replace(/ /g, '');
    setTextToScan(clean);
    resetSearchSimulation();
};

const resetSearchSimulation = () => {
    setCurrentIndex(0);
    setCurrentNode(0);
    setIsSearchDone(false);
    setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
    setJumpPath(null);
    setFoundMatches([]);
    setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
    if (currentIndex >= textToScan.length) {

```

```

        curr = nextNode;
    }

    const currentMatches: { index: number; word: string }[] = [];
    const matchedWordsForLog: string[] = [];

    let temp = curr;
    while (temp !== 0) {
        if (WATERFALL_NODES[temp].is_terminal) {
            WATERFALL_NODES[temp].words.forEach((word) => {
                currentMatches.push({ index: currentIndex, word });
                matchedWordsForLog.push(word);
            });
        }
        temp = WATERFALL_NODES[temp].term_link;
    }

    if (matchedWordsForLog.length > 0) {
        logMsg += ` БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}\n`;
        setActiveSearchCodeLine(4);
    }

    setCurrentNode(curr);
    setCurrentIndex(currentIndex + 1);
    setSearchLog(logMsg);
    if (currentMatches.length > 0) {
        setFoundMatches((prev) => [...prev, ...currentMatches]);
    }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Другие вершины сразу инициализируются с fail = ROOT',
        codeLine: 1,
    },

```

```

{
  targetNodeId: 5,
  title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
  desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
    "H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
  codeLine: 3,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'I',
  checkingBranchesFrom: 0,
},
{
  targetNodeId: 8,
  title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТА
  desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
    ). Рыба плавает в ROOT и ищет ветку "H". У корня
    суффикс "H"!',
  codeLine: 4,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'H',
  checkingBranchesFrom: 0,
},
{
  targetNodeId: 3,
  title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
  desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
    е подходят. fail[HER] = ROOT.',
  codeLine: 3,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'R',
  checkingBranchesFrom: 0,
},
{
  targetNodeId: 6,
  title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
  desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель

```

```

<div className="flex flex-wrap items-center justify-center" >
  <div className="flex items-center space-x-3">
    <span className="text-3xl"> </span>
    <div>
      <h4 className="text-2xl font-bold text-white">
        Анимация водопада: Полное обучение автома
      </h4>
      <p className="text-sm text-slate-400">
        Визуализация пошагового построения всех f
      </p>
    </div>
  </div>
</div>

```

/* Переключатель режимов */

```

<div className="bg-slate-900 p-1.5 rounded-xl b
  <button
    onClick={() => setActiveMode('training')}
    className={`flex items-center space-x-1.5 p-2
      activeMode === 'training'
        ? 'bg-indigo-600 text-white shadow-lg sl
        : 'text-slate-400 hover:text-white'
    }`>
  >
    <GitBranch className="w-4 h-4" />
    <span>Режим 1: Полное обучение (BFS от корн
  </button>
  <button
    onClick={() => setActiveMode('search')}
    className={`flex items-center space-x-1.5 p-2
      activeMode === 'search'
        ? 'bg-blue-600 text-white shadow-lg sha
        : 'text-slate-400 hover:text-white'
    }`>
  >
    <Play className="w-4 h-4" />
    <span>Режим 2: Поиск в тексте</span>
  </button>
</div>

```



```

    ) : (
      /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
      <div className="grid grid-cols-1 lg:grid-cols-3"
        <div className="bg-slate-900/60 p-5 rounded-x
          <div>
            <h5 className="text-white font-bold mb-2"
              <span>» </span> Текст для сканирования
            </h5>
            <p className="text-xs text-slate-400 mb-4"
              Введи текст, чтобы лосось просканировал
            </p>
            <input
              type="text"
              value={textToScan}
              onChange={handleTextChange}
              maxLength={15}
              placeholder="ВВЕДИТЕ ТЕКСТ..."
              className="w-full bg-slate-800 border b
            cus:outline-none focus:border-blue-500"
            />
          </div>

          <div>
            <div className="mb-4 flex flex-wrap gap-1"
              <span className="text-xs text-slate-400"
                {textToScan.split('').map((char, idx) =>
                  <span
                    key={idx}
                    className={`w-7 h-8 flex items-centr
                      idx === currentIndex
                        ? 'bg-blue-600 text-white shadow
                        : idx < currentIndex
                        ? 'bg-emerald-950/60 text-emera
                        : 'bg-slate-800 text-slate-500'
                    }`}
                  >
                    {char}
                  </span>

```

```

                <span>2. while (curr != root && !trie
                    {activeSearchCodeLine === 2 && <span <
                ржок по fail!</span>}
            </div>
            <div className={`p-1.5 rounded flex item
                border-blue-400 text-blue-200 font-bold` :
                <span>3. curr = trie[curr].get(char)
                    {activeSearchCodeLine === 3 && <span <
                ем по течению</span>}
            </div>
            <div className={`p-1.5 rounded flex item
                l-4 border-emerald-500 text-emerald-300 fon
                <span>4. if (is_terminal[curr]) report
                    {activeSearchCodeLine === 4 && <span <
                овпадение!</span>}
            </div>
        </div>
    </div>

    <div>
        <h5 className="text-slate-300 font-bold m
            <AlertCircle className="w-3.5 h-3.5 tex
        </h5>
        <div className="bg-slate-800 p-3.5 rounded
            tems-center">
            {searchLog}
        </div>
    </div>
</div>
</div>
)}

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
    -hidden shadow-2xl">
    {/* Водопадные фоновые эффекты */}
    <div className="absolute inset-0 opacity-10 bg-
        o-900 to-transparent pointer-events-none"></d

```



```

        stroke="#334155"
        strokeWidth="1"
        strokeDasharray="4,4"
        opacity="0.4"
      />
      <rect x="25" y={level.y - 12} width="14" height="24" />
    />
    <text x="32" y={level.y + 4} fill="#94a3b8" font-size="12">
      {level.label}
    </text>
  </g>
)}}

```

```

{/* Рендер прямых переходов (потоков воды) */}
{Object.entries(TRIE_TRANSITIONS).map(([fromId, children]) => {
  const fromNode = WATERFALL_NODES[Number(fromId)];
  return Object.entries(children).map(([childId, toId]) => {
    const toNode = WATERFALL_NODES[toId];

```

```

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ||
    type === 'normal';

```

```

    // Подсветка в режиме обучения (когда ищем совпадения)
    let isTrainingExamined = activeMode === 'training' ||
    let isMatchBranch = isTrainingExamined && !isHighlighted;

```

```

    return (
      <g key={`edge-${fromNode.id}-${toNode.id}`} >
        {/* Линия течения воды */}
        <line
          x1={fromNode.x}
          y1={fromNode.y}
          x2={toNode.x}
          y2={toNode.y}
          stroke={isHighlighted ? '#3b82f6' : '#334155'}
          strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
          className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
        />

```

```

const stepIdxForNode = TRAINING_STEPS_INF
if (activeMode === 'training' && trainStep

let isHighlighted = activeMode === 'search
e === 'fail';
let isJustEstablished = activeMode === 't
-ссылку

// Вычисляем изогнутую кривую для красоты
const dx = targetNode.x - node.x;
const dy = targetNode.y - node.y;
const cx = node.x + dx / 2 + (node.id % 2
const cy = node.y + dy / 2 - 30;

return (
  <g key={`fail-${node.id}`}>
    <path
      d={`M ${node.x} ${node.y} Q ${cx} $
      fill="none"
      stroke={isHighlighted ? '#f43f5e' :
      strokeWidth={isHighlighted || isJus
      strokeDasharray="6,6"
      className={isHighlighted || isJustE
      opacity={isHighlighted || isJustEst
    />
    {isJustEstablished && (
      <text x={cx} y={cy - 10} fill="#10b
  >
    fail[{node.label}] = {targetNode.
    </text>
  )}
</g>
);
}})

```

```

{/* Рендер вершин (бассейнов водопада) */}
{Object.values(WATERFALL_NODES).map((node) :
  const isCurrent = activeFishPosNode.id ===

```

```

        <g transform={`translate(${node.x},
            <rect x="-35" y="-12" width="70" h="10" fill="black" />
            <text x="0" y="2" fontSize="10" font-family="monospace" />
                ИЩЕМ FAIL
            </text>
        </g>
    )}

    {/* Индикатор словарных слов */}
    {hasWords && {
        <g transform={`translate(${node.x} +
            <circle cx="0" cy="0" r="9" fill="black" />
            <text x="0" y="3" fontSize="10" font-family="monospace" />
                *
            </text>
        </g>
    )}
    </g>
    );
}}

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
<g>
    {/* Круг-подложка под рыбу */}
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b572d' : 'black'}
        stroke="black"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bezier(0.25, 0.8, 0.25, 0.75)',
            className="shadow-lg"
        }}
    />
    {/* Сама рыба */}
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
    />

```

```

        </div>
      })
    </div>
  })

  { /* что сейчас сообщил компилятор */
  <div className="mt-6 flex flex-wrap items-center" >
    <span className="text-[10px] uppercase tracking
      <Terminal className="w-3.5 h-3.5" /> из компи
    </span>
    {compilerLinked ? (
      <>
        {liveText && <span className="text-[11px] f
        >text = {liveText}</span>}
        {liveI !== null && <span className="text-[1
        py-1">i = {liveI}{liveText ? ` (${liveText.
        {liveState !== null && (
          <span className="text-[11px] font-mono bg
          state = {liveState}{WATERFALL_NODES[liv
          </span>
        })
        {liveMatchesSig && <span className="text-[1
        d px-2 py-1">matches: {foundMatches.map((m)
        {liveFail !== undefined && <span className=
        ed-md px-2 py-1">fail = {JSON.stringify(liv
      </>
    ) : (
      <span className="text-[11px] text-slate-500">
        открой панель Python и запусти Ахо–Корасик:
        "font-mono text-cyan-300">state</code>, <co
        an-300">matches</code> поведут лосося по эт
      </span>
    })
  </div>
</div>
);
};

```

```

];

const cellBase =
  "flex items-center justify-center rounded-md font-monospace"
  "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  const chapterId = useContext(VizChapterContext);

  // Вкладка 2D = своё демо (свой скелет): компилятор по
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, mode === "2d"
  }, [chapterId, mode]);

  return (
    <div className="w-full bg-slate-950 rounded-2xl border"
      <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar"
        {(
          [
            ["1d", "1. Одномерные суммы"],
            ["2d", "2. Двумерные суммы"],
          ] as const
        ).map(([key, label]) => {
          <button
            key={key}
            onClick={() => setMode(key)}
            className={`px-3 py-2 rounded-lg text-xs sm:text-sm
              mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
            }`
          >
            {label}
          </button>
        ))}
      </div>
      {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
    </div>
  );
};

```



```

        });
    }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border border-slate-800" >
  {shownDriven !== null && !hover ? (
    <p className="text-slate-300">
      <b className="text-emerald-400">Компилятор </b>
      <span className="font-mono text-white">i =
      <span className="font-mono text-emerald-300">
        {hasLiveP && <> = <span className="font-mono text-slate-500">
          <span className="ml-1 text-slate-500">Пустой массив
        </span>
      </span>
    </p>
  ) : hover?.kind === "pref" ? (
    hover.i === 0 ? (
      <p className="text-slate-300">
        <b className="text-emerald-400">P[0] = 0</b> —
        отрезка, начинающегося с нуля.
      </p>
    ) : (
      <p className="text-slate-300">
        <b className="text-emerald-400">
          P[{hover.i}] = {pref[hover.i]}
        </b>{" "}
        — это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
          a[0..{hover.i - 1}] = {A1.slice(0, hover.i)}
        </span>
      </p>
    )
  ) : hover?.kind === "a" ? (
    <p className="text-slate-300">
      <b className="text-indigo-400">
        a[{hover.i}] = {A1[hover.i]}
      </b>{" "}
    </p>
  )
  )

```

```

const liveJ = vizNumber(vars?.j);

const P = useMemo(() => {
  const p = Array.from({ length: n + 1 }, () => Array<number>()
  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= m; j++) {
      p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1];
    }
  }
  return p;
}, [n, m]);

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

/**
 * Матрицы из кода пользователя рисуются В СВОЕЙ размерности
 *
 * Раньше значения S[i][j] протискивались в демо-сетку
 * размере матрицы часть ячеек становилась «пустыми»,
 * шаги компилятора не совпадали с картинкой.
 */
const livePrefSource = pickLiveVar(vars, ["S", "P", "g"]);
const liveS = liveMatrix(livePrefSource?.value);
const hasLiveS = liveS !== null;

const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);

const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
  ? Array.from({ length: prefRows }, (_, i) =>
    Array.from({ length: prefCols }, (_, j) => liveS[i][j])
  )
  : P;

const srcRows = liveA?.rows ?? n;

```



```

const total = Aa !== null && B !== null && Cc !== null;
const fmt = (v: number | null) => (v === null ? "-" : v);

const cornerOf = (i: number, j: number) => {
  if (i === r2 + 1 && j === c2 + 1) return "A";
  if (i === r1 && j === c2 + 1) return "B";
  if (i === r2 + 1 && j === c1) return "C";
  if (i === r1 && j === c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    {sizeMismatch ? (
      <p className="rounded-lg border border-amber-500">
        {sizeMismatch}
      </p>
    ) : null}
    <div className="grid gap-5 lg:grid-cols-2">
      /* Исходная матрица */
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">
          Исходная матрица {liveA ? `${liveArrSource?}` : ""}
          чтобы задать прямоугольник
        </div>
        <div className="inline-block">
          {srcGrid.map((row, i) => (
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => (
                const inRect = i >= r1 && i <= r2 &&
                return (
                  <div
                    key={j}
                    onClick={() =>
                      setRect((r) =>
                        i < r.r1 || j < r.c1
                          ? { r1: i, c1: j, r2: Math.max(r.r2, i), c2: Math.max(r.c2, j) }
                          : { ...r, r2: i, c2: j }
                    )
                  </div>
                )
              )}
            </div>
          )}
        </div>
      </div>
    </div>
  )}

```

```

<div
  key={j}
  onMouseEnter={() => setHover({ i,
  onMouseLeave={() => setHover(null
  className={`${cellBase} cursor-he
    isCompilerCell
      ? "bg-emerald-600 text-white
      : isHover
        ? "bg-amber-500 text-white
        : blank
          ? "bg-slate-950 text-slat
          : cornerColor
    `}
  title={hasLiveS ? `${livePrefSour
>
  {blank ? "." : String(shownValue)
  {corner && !isHover && {
    <span className="absolute -top-
ht">
      {corner}
    </span>
  }}
</div>
  );
  }}}
</div>
  )))
</div>
</div>
</div>

```

```

<div className="min-h-[54px] bg-slate-900 border
  {hover ? {
    <p className="text-slate-300">
      <b className="text-amber-400">
        {liveS ? livePrefSource?.name ?? "S" : "P
      </b>{" "}
    }
  }
  — сумма всего прямоугольника от левого верх

```

```

    VizChapterContext,
  } from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
  }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({ length: 8 }, () => []);

```

```

const chapterId = useContext(VizChapterContext);

// Каждая вкладка = своё демо с собственным скелетом:
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, tab === "2d_build", [chapterId, tab]);
}, [chapterId, tab]);

return (
  <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5" >
    <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800" >
      <button
        onClick={() => setTab("1d")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-white bg-slate-800 text-slate-400`} >
        1. Сворачивание 1D
      </button>
      <button
        onClick={() => setTab("2d_build")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-white bg-slate-800 text-slate-400`} >
        2. Сворачивание 2D (Построение)
      </button>
      <button
        onClick={() => setTab("2d_query")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-white bg-slate-800 text-slate-400`} >
        3. Запрос 2D (Формула)
      </button>
    </div>

    <AnimatePresence mode="wait">
      {tab === "1d" ? (
        <Viz1D key="1d" />
      ) : tab === "2d_build" ? (
        <Viz2DBuild key="2d_build" />
      ) : null}
    </AnimatePresence>
  </div>
);

```

```

: hasLiveSource && liveSource && liveSource.length
  ? `Демо-массив из ${N} элементов, а в вашем коде
    ся демо-размера ${N}×${LOG}.`
  : null;

```

```

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    computeSteps.push({ i, j });
  }
}

```

```

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

```

```

return (
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-500">
          Построение Sparse Table (Min)
        </h3>
        <p className="text-sm text-slate-400">
          Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$ 
        </p>
        {dimMismatch ? (
          <p className="mt-2 max-w-[560px] rounded-lg">
            {dimMismatch}
          </p>
        ) : null}
      </div>
      <div className="flex gap-2">
        <button
          onClick={() => setStep(Math.max(0, step - 1))}

```



```

let isActive = !!liveTarget && liveTarget.i === i
let isSrc1 = false;
let isSrc2 = false;

if (compilerLinked && liveTarget && liveTarget.i === i) {
  isSrc1 = i === liveTarget.i && j === liveTarget.j
  isSrc2 = i === liveTarget.i + (1 << liveTarget.j) && j === liveTarget.j
} else if (!compilerLinked && step > 0) {
  const currentStep = computeSteps[step]
  isActive = currentStep.i === i && currentStep.j === j
  if (currentStep.j - 1 === j) {
    if (currentStep.i === i) isSrc1 = true
    if (currentStep.i + (1 << (currentStep.j - 1)) === i) isSrc2 = true
  }
}

// Готовые вычисленные уровни демо не вычисляем
// уровень j=0 всегда берём из уже скомпьютеризованного
const shownValue = compilerLinked ? liveTarget : undefined
const displayValue =
  shownValue === undefined || shownValue === null
    ? "."
    : typeof shownValue === "object"
      ? JSON.stringify(shownValue)
      : String(shownValue);

return (
  <div key={j} className="relative flex flex-direction: column align-items: center">
    {!isValid ? (
      <div className="w-[clamp(30px,90px,100px)] h-[clamp(30px,90px,100px)]" />
    ) : (
      <motion.div
        onMouseEnter={() => (compilerLinked ? setHover(1) : setHover(0))}
        onMouseLeave={() => setHover(0)}
        onClick={() => (compilerLinked ? setCompilerLinked(!compilerLinked) : setCompilerLinked(true))}
        style={{
          initial: { opacity: 0, scale: 1.1 },
          animate: {
            opacity: 1,
            scale: 1,
          },
        }}
      />
    )}
  </div>
)

```

```

        }}
      >
        <path
          d={`M 0 0 C 30 0, 30 ${isSrc1 ? "0" : "1"} 30 0`}
          fill="none"
          stroke={isSrc1 ? "#10b981" : "#f87171"}
          strokeWidth="3"
        />
      </motion.svg>
    )}
  </div>
);
}}}
</React.Fragment>
)}}
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-800 p-4" >
  {compilerLinked && liveTarget ? (
    <p className="text-base text-slate-300">
      <span className="text-emerald-400 font-bold">Step</span>
      <span className="font-mono text-white">i = 0</span>
      – активна ячейка <span className="font-mono text-white">ST[{c.i}][{c.j}]</span>
      {(hasLiveTable || hasLiveSource) && <> = <span className="font-mono text-white">ST[{c.i}][{c.j}]</span>
      ?.[liveTarget.j] ?? {liveTarget.j === 0 ? 1 : 0}</span>
    </p>
  ) : step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">ST[{c.i}][{c.j}]</span>
          <span>{" "}</span>
        </p>
      )
    })
  ) : null
  )
}

```



```

* Шаг, закреплённый кликом по вкладке. Без него live
* перетирал выбор пользователя и вкладки «Шаг 1/2/3»
* клик есть, а матрица не меняется. Закрепление живё
*/

```

```

const [pinnedK, setPinnedK] = useState<number | null>
const prevLiveK = useRef<number | null>(liveK);
useEffect(() => {
  if (liveK !== prevLiveK.current) {
    prevLiveK.current = liveK;
    setPinnedK(null);
  }
}, [liveK]);
const liveR = vizNumber(vars?.r);
const liveC = vizNumber(vars?.c);
const liveTable = vizRecord(vars?.st2);
const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'source');
const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'table');
const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
const codeCell = liveR !== null && liveC !== null ? {
  const compilerLinked = codeCell !== null || liveK !== null;
  const shownK = pinnedK !== null
    ? pinnedK
    : liveK !== null
      ? Math.max(0, Math.min(3, Math.trunc(liveK)))
      : hasLiveSource
        ? 0
        : k;
}

```

```

/** 2D-демонстрация нарисована на фиксированной сетке
const liveSource2D = liveMatrix(vars?.A);
const dimMismatch2D = liveSource2D
  ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
  : `Демо 2D построено на матрице 8x8, а в вашем коде
    не её не отрисовываются, значения вашей матрицы
  : null
  : null;

```

```

// Уровень и клетка берутся прямо из k/r/c; номер строки

```

Шаг закреплён вручную: когда `<span className=`
демонстрация вернётся к шагу кода.

```

</p>
  })

  <div className="flex flex-col items-center gap-1
    scrollbar">
    {[...Array(shownK + 1)].map((_, i) => shownK
      const stepL = 1 << step;
      const stepSize = 8 - stepL + 1;
      const isCurr = step === shownK;

      return (
        <React.Fragment key={step}>
          {idx > 0 && <div className="text-slate
            <div className={`flex flex-col items-c
              <h4 className={`font-bold mb-3 flex
            }>
              <span>Матрица блоков {stepL}x{step
              <span className={`text-[10px] font
                rder-indigo-800/40 text-indigo-300' : 'bg-s
              </h4>
              <div className="max-w-full overflow-
                <div className={`grid gap-[2px] p-1
                go-900/40 shadow-[0_0_25px_rgba(99,102,241,0.5)]
                eColumns: `repeat(${stepSize}, max-content)
                {Array.from({length: stepSize * step
                  const r = Math.floor(idx / step
                  const c = idx % stepSize;

                  const liveValue = liveTable?.[`
                  // Базовый слой (блоки 1x1) – э
                  // одного элемента равен ему сам
                  // показываем из A сразу, как A
                  // цикла построения st2. Уровни
                  const sourceValue = step === 0
                  const shownValue = compilerLink
                  const isBlank = shownValue ===

```



```

e="text-amber-300">1</span>],          <span
    &nbsp;&nbsp;&nbsp;<span className="text-blue
ext-amber-300">1</span>][k-<span className=
pan><br/>
    &nbsp;&nbsp;&nbsp;<span className="text-emer
="text-amber-300">1</span>][k-<span classNa
span><br/>
    &nbsp;&nbsp;&nbsp;<span className="text-ambe
white">c + pL</span>][k-<span className="te
ame="text-slate-500">// Правый Низ</span><b
    {'}}''));
</div>

```

```

{activeCell ? (
  <div className="bg-slate-950 p-5 rounded
.2)] mt-2 transition-all">
    <p className="text-slate-300 mb-3 t
    <span>Пример для ячейки:</span>
    {locked && <span className="text
ded border border-rose-500/20"><Lock size={
    </p>
    <p className="text-indigo-300 borde
    <span className="font-bold text-w
}}[{{shownK}}][{{shownK}}</span>
    <span className="text-slate-400">
    </p>

```

```

    <div className="space-y-2 pt-3 pl-2
    {((() => {
      const prevL = 1 << (shownK -
      return (
        <>
          <div className="flex ite
            <span className="flex
hadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
            <span className="text
tiveCell.r)][activeCell.c][shownK-1][shownK-
          </div>

```

```

        </div>
      </div>
    </div>
  );
};

const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);
  const runtime = useVizRuntime();

  // Координаты запроса – те же захардкоженные имена,
  useEffect(() => {
    const vars = runtime?.variables;
    const nr1 = vizNumber(vars?.r1);
    const nc1 = vizNumber(vars?.c1);
    const nr2 = vizNumber(vars?.r2);
    const nc2 = vizNumber(vars?.c2);
    if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
    if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
    if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
    if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
  }, [runtime]);

  const queryVars = runtime?.variables;
  const liveCoords = ["r1", "c1", "r2", "c2"].map((na
  const queryMismatch = liveCoords.some((v) => v !== r
    ? `Координаты запроса в вашем коде выходят за дем
      ] ?? "-"}, r2=${liveCoords[2] ?? "-"}, c2=${live
    : null;

  const H = Math.max(1, r2 - r1 + 1);
  const W = Math.max(1, c2 - c1 + 1);
  const kx = Math.floor(Math.log2(H));
  const ky = Math.floor(Math.log2(W));
  const Lx = 1 << kx;

```

```

        top: `${(r1 / 8) * 100}%`,
        left: `${(c1 / 8) * 100}%`,
        marginTop: '8px',
        marginLeft: '8px',
        height: 'calc(' + (H/8*100) +
        width: 'calc(' + (W/8*100) +
      }}
    />

    {/* Показываем 4 угла в самой матрице */}
    <div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
    <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
    <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(
    <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(
    </div>

    <div className="bg-slate-950 p-4 rounded-lg shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]" style={{
    <label className="flex items-center justify-between" style={{ width: 100%; }}>
      <span className="text-slate-400">Регулировка размера</span>
      <div className="flex gap-2">
        <input type="number" min={0} max={100} value={100} />
        <input type="number" min={0} max={100} value={100} />
      </div>
    </label>
    <label className="flex items-center justify-between" style={{ width: 100%; }}>
      <span className="text-slate-400">Регулировка цвета</span>
      <div className="flex gap-2">

```

```

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
В запроса)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-rose-900">
r1</span>][<span className="text-white bg-rose-900">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
во, чтобы правый край уперся в c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-blue-600">
r1</span>][<span className="text-white bg-blue-600">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
px, чтобы нижний край уперся в r2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-emerald-500">
nded">r2 - Lx + 1</span>][<span className="text-white bg-emerald-500">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
и влево от c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-amber-500">
">r2 - Lx + 1</span>][<span className="text-white bg-amber-500">
        {'}}''));
</div>

<div className="w-full flex justify-center">
  <div className="flex flex-col items-center">
    <h4 className="text-slate-400 font-bold">
      Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]
    </h4>
    <div className="grid gap-[2px] p-2.5">
Columns: `repeat(${matCols}, 1fr)`}}>
      {Array.from({length: matRows * matCols}, (v, idx) => {
        const r = Math.floor(idx / matCols);
        const c = idx % matCols;
        let isTL = r === r1 && c === c1;
        let isTR = r === r1 && c === c2;
        let isBL = r === r2 - Lx + 1 && c === c1;
        let isBR = r === r2 - Lx + 1 && c === c2;
      })}
    </div>
  </div>
</div>

```

```

            <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
            </div>
        </div>
    </div>
}

```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 85/111: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 137 | РАЗМЕР

```

        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').cla
            ? 'px-3 py-1 rounded text-sm font-bold l
            : 'px-3 py-1 rounded text-sm font-bold l

        document.getElementById('btn-mode-wtf').cla
            ? 'px-3 py-1 rounded text-sm font-bold l
            : 'px-3 py-1 rounded text-sm font-bold l

        const aside = document.querySelector('aside
        const headerMain = document.querySelector('
        const questionsContainer = document.getElem

```



```

        n-colors';
        document.getElementById('btn-theme-light').onmouseover = function() {
            document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
        };
        document.getElementById('btn-theme-notebook').onmouseover = function() {
            document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
        };
    }

    // Initialize theme and mode
    setTheme('dark');

    // Setup intersection observer for TOC highlighting
    document.addEventListener('DOMContentLoaded', () => {
        const mobileMenuBtn = document.getElementById('mobile-menu-btn');
        const mobileDrawer = document.getElementById('mobile-drawer');
        const closeDrawerBtn = document.getElementById('close-drawer');
        const drawerOverlay = document.getElementById('drawer-overlay');

        function toggleDrawer() {
            const isClosed = mobileDrawer.classList.contains('hidden');
            if (isClosed) {
                mobileDrawer.classList.remove('hidden');
                if (drawerOverlay) {
                    drawerOverlay.classList.remove('hidden');
                    // Timeout allows display:block to take effect
                    setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
                }
                document.body.style.overflow = 'hidden';
            } else {
                mobileDrawer.classList.add('hidden');
                if (drawerOverlay) {
                    drawerOverlay.classList.add('hidden');
                    setTimeout(() => drawerOverlay.classList.add('hidden'), 10);
                }
                document.body.style.overflow = '';
            }
        }

        mobileMenuBtn.addEventListener('click', toggleDrawer);
        closeDrawerBtn.addEventListener('click', toggleDrawer);
        drawerOverlay.addEventListener('click', toggleDrawer);
    });

```

```
        }
      });
    }, observerOptions);

    document.querySelectorAll('section[id^="que
      observer.observe(section);
    });
  });
</script>
</body>
</html>
```

ФАЙЛ 86/111: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js
  <script id="MathJax-script" async src="https://cdn.
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\(', '\\)']]
        displayMath: [['$$', '$$'], ['\\[', '\\
      }
    };
  </script>
  <style>
    @import url('https://fonts.googleapis.com/css2?
    @reference "../src/index.css";
```

```

        -ms-overflow-style: none;
        -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar {
        display: none;
    }

    @media (max-width: 640px) {
        html { font-size: 13px; }

        .uno-card { width: 40px; height: 60px; font-size: 12px; }
        .uno-card::before, .uno-card::after { font-size: 12px; }
        .uno-card::before { top: 1px; left: 2px; }
        .uno-card::after { bottom: 1px; right: 2px; }

        .uno-card.mini { width: 28px !important; height: 40px !important; }
        .uno-card.mini::before, .uno-card.mini::after { font-size: 10px; }

        .uno-equation .text-2xl { font-size: 1.2rem; }
        .uno-equation.gap-4 { gap: 0.5rem !important; }

        /* Allow horizontal scroll for equations in containers */
        .formula-scroll { flex-wrap: nowrap !important; }
    }

    .uno-inner {
        position: absolute; width: 110%; height: 75%;
        border-radius: 50%; transform: rotate(-30deg);
        box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
    }

    .uno-val { position: relative; z-index: 2; }
    .card-red { background-color: #e52521; } .card-blue { background-color: #004dcf; } .card-green { background-color: #339e35; } .card-yellow { background-color: #ffc400; } .card-black { background-color: #222; }
    .card-black::before, .card-black::after { text-align: center; }

```

```

        box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
    }
    body.theme-notebook .text-slate-200, body.theme
        ant; }
    body.theme-notebook .text-slate-400 { color: #5
    body.theme-notebook .border-slate-600, body.the
        : #94a3b8 !important; }
    body.theme-notebook .font-mono { font-family: "
    body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1);
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
    transform-origin: center;
    animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
    <div class="sticky top-0 z-50 w-full bg-slate-950/9
        center items-center gap-4 sm:gap-8 transition-col

```

```

        <path d="M0,0 L6,3 L0,6 Z" fill="#818cf"
    </marker>
    <marker id="arrow-lime" markerWidth="6" mar
        <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63"
    </marker>
    <marker id="arrow-green" markerWidth="6" ma
        <path d="M0,0 L6,3 L0,6 Z" fill="#4ade8"
    </marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
    <rect x="0" y="0" width="40" height="10"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
    <path d="M 10 40 L 15 20 Q 25 15 35 20
    <rect x="5" y="40" width="40" height="20"
    <!-- Ручка -->
    <path d="M 45 50 Q 60 50 60 30 L 75 30"
    <circle cx="75" cy="30" r="4" fill="#33
    <!-- Выход -->
    <path d="M 5 45 L -5 45 L -5 55 L 5 55
    <circle cx="25" cy="50" r="15" fill="#f

```

```
    </h3>
    <button id="close-drawer-btn" class="
      <svg xmlns="http://www.w3.org/20
        <path stroke-linecap="round
      </svg>
    </button>
  </div>
  <nav class="space-y-3 text-sm font-medium">

    <details class="group" open>
      <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
        <a href="#question-1" class="text-blue-500 pl-3 font-bold text-blue-300">
          1. Алгебраические структуры
        </a>
      </summary>
      <div class="pl-6 space-y-2 text-sm">
        <a href="#section-1-1" class="text-blue-500 pl-3 font-medium text-blue-300">
        <a href="#section-1-2" class="text-blue-500 pl-3 font-medium text-blue-300">
        <a href="#section-1-3" class="text-blue-500 pl-3 font-medium text-blue-300">
        <a href="#section-1-4" class="text-blue-500 pl-3 font-medium text-blue-300">
      </div>
    </details>

    <details class="group">
      <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
        <a href="#question-2" class="text-indigo-500 pl-3 font-bold text-indigo-300">
          2. Комплексные числа
        </a>
      </summary>
      <div class="pl-6 space-y-2 text-sm">
        <a href="#section-2-1" class="text-indigo-500 pl-3 font-medium text-indigo-300">
        <a href="#section-2-2" class="text-indigo-500 pl-3 font-medium text-indigo-300">
        <a href="#section-2-3" class="text-indigo-500 pl-3 font-medium text-indigo-300">
      </div>
    </details>
```

```

        <a href="#q5-def" class="block ho
        <a href="#q5-group" class="bloc
        <a href="#q5-prim" class="block
        <a href="#q5-crit" class="block
        <a href="#q5-cyclic" class="blo
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-6" class=
er-lime-500 pl-3 font-bold text-lime-300">
        6. Делимость, НОД и Евклид
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q6-1" class="block ho
        <a href="#q6-2" class="block ho
        <a href="#q6-3" class="block ho
        <a href="#q6-4" class="block ho
        <a href="#q6-5" class="block ho
        <a href="#q6-6" class="block ho
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
        7. Взаимно простые числ
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q7-1" class="bloc
        <a href="#q7-2" class="bloc
        <a href="#q7-3" class="bloc
        <a href="#q7-4" class="bloc
```

```
<summary class="list-none cursor-pointer-rose-500 pl-3 font-bold text-rose-400">
    10. Делимость и Схема Г
</a>
</summary>
<div class="pl-6 space-y-2 text-rose-400">
    <a href="#q10-1" class="block">
    <a href="#q10-2" class="block">
    <a href="#q10-3" class="block">
a>
        <a href="#q10-4" class="block">
        <a href="#q10-5" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer-cyan-500 pl-3 font-bold text-cyan-400">
        11. НОД и НОК многочленов
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-cyan-400">
        <a href="#q11-1" class="block">
        <a href="#q11-2" class="block">
        <a href="#q11-3" class="block">
        <a href="#q11-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer-blue-500 pl-3 font-bold text-blue-400">
```



```
            border border-rose-500/30"> К
        </div>
    </details>
```

```
    <details class="group">
        <summary class="list-none cursor-pointer text-teal-500 font-medium">
            <a href="#question-15" class="text-teal-500 font-medium">
                15. Производная. Критер
            </a>
        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q15-1" class="block text-teal-500 pl-3 font-bold text-cyan-400">
            <a href="#q15-2" class="block text-teal-500 pl-3 font-bold text-cyan-400">
            <a href="#q15-3" class="block text-teal-500 pl-3 font-bold text-cyan-400">
            <a href="#q15-4" class="block text-teal-500 pl-3 font-bold text-cyan-400">
        </div>
    </details>
```

```
    <details class="group">
        <summary class="list-none cursor-pointer text-teal-500 font-medium">
            <a href="#question-16" class="text-teal-500 font-medium">
                16. Неприводимые многоч
            </a>
        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q16-1" class="block text-teal-500 pl-3 font-bold text-teal-500">
            <a href="#q16-2" class="block text-teal-500 pl-3 font-bold text-teal-500">
            <a href="#q16-3" class="block text-teal-500 pl-3 font-bold text-teal-500">
        </div>
    </details>
```

```
    <div class="mt-4 mb-2 text-xs font-sans">
        <a href="#question-17" class="block text-teal-500 pl-3 text-slate-400">
            <span class="font-bold text-teal-500">
            </a>
```

```

    });

    // Scroll spy logic
    document.addEventListener('DOMContentLoaded', () => {
        const sections = document.querySelectorAll(
            'h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14, h15, h16, h17, h18, h19, h20, h21, h22, h23, h24, h25, h26, h27, h28, h29, h30, h31, h32, h33, h34, h35, h36, h37, h38, h39, h40, h41, h42, h43, h44, h45, h46, h47, h48, h49, h50, h51, h52, h53, h54, h55, h56, h57, h58, h59, h60, h61, h62, h63, h64, h65, h66, h67, h68, h69, h70, h71, h72, h73, h74, h75, h76, h77, h78, h79, h80, h81, h82, h83, h84, h85, h86, h87, h88, h89, h90, h91, h92, h93, h94, h95, h96, h97, h98, h99, h100'
        );
        const divs = document.querySelectorAll(
            'div[id^="q5-"], div[id^="q6-"], div[id^="q7-"], div[id^="q8-"], div[id^="q9-"], div[id^="q10-"], div[id^="q11-"], div[id^="q12-"], div[id^="q13-"], div[id^="q14-"], div[id^="q15-"], div[id^="q16-"], div[id^="q17-"], div[id^="q18-"], div[id^="q19-"], div[id^="q20-"], div[id^="q21-"], div[id^="q22-"], div[id^="q23-"], div[id^="q24-"], div[id^="q25-"], div[id^="q26-"], div[id^="q27-"], div[id^="q28-"], div[id^="q29-"], div[id^="q30-"], div[id^="q31-"], div[id^="q32-"], div[id^="q33-"], div[id^="q34-"], div[id^="q35-"], div[id^="q36-"], div[id^="q37-"], div[id^="q38-"], div[id^="q39-"], div[id^="q40-"], div[id^="q41-"], div[id^="q42-"], div[id^="q43-"], div[id^="q44-"], div[id^="q45-"], div[id^="q46-"], div[id^="q47-"], div[id^="q48-"], div[id^="q49-"], div[id^="q50-"], div[id^="q51-"], div[id^="q52-"], div[id^="q53-"], div[id^="q54-"], div[id^="q55-"], div[id^="q56-"], div[id^="q57-"], div[id^="q58-"], div[id^="q59-"], div[id^="q60-"], div[id^="q61-"], div[id^="q62-"], div[id^="q63-"], div[id^="q64-"], div[id^="q65-"], div[id^="q66-"], div[id^="q67-"], div[id^="q68-"], div[id^="q69-"], div[id^="q70-"], div[id^="q71-"], div[id^="q72-"], div[id^="q73-"], div[id^="q74-"], div[id^="q75-"], div[id^="q76-"], div[id^="q77-"], div[id^="q78-"], div[id^="q79-"], div[id^="q80-"], div[id^="q81-"], div[id^="q82-"], div[id^="q83-"], div[id^="q84-"], div[id^="q85-"], div[id^="q86-"], div[id^="q87-"], div[id^="q88-"], div[id^="q89-"], div[id^="q90-"], div[id^="q91-"], div[id^="q92-"], div[id^="q93-"], div[id^="q94-"], div[id^="q95-"], div[id^="q96-"], div[id^="q97-"], div[id^="q98-"], div[id^="q99-"], div[id^="q100-"]'
        );
        let isScrolling = false;

        document.querySelectorAll('.group-link').forEach(link => {
            link.addEventListener('click', () => {
                isScrolling = true;
                setTimeout(() => isScrolling = false, 1000);
            });
        });

        const observer = new IntersectionObserver((entries) => {
            if (isScrolling) return;

            let visibleId = null;
            entries.forEach(entry => {
                if (entry.isIntersecting) {
                    visibleId = entry.target.id;
                }
            });

            // Highlight active link
            document.querySelectorAll('a').forEach(a => {
                a.style.fontWeight = 'normal';
                if (a.href.endsWith(visibleId)) {
                    a.style.fontWeight = 'bold';
                }
            });

            // Open the parent details
            const correspondingLink = document.querySelector(`a[href="#${visibleId}"]`);
            if (correspondingLink) {
                const details = correspondingLink.closest('details');
                if (details && !details.open) {
                    details.open = true;
                }
            }
        });
    });

```

```
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('h2');
header.innerText = 'Экспорт в ' + fileName;
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за особенностей браузеров, для сохранения файла необходимо открыть этот документ в новой веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#fff';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
```

```

// Smart markdown parsing based
clone.querySelectorAll('h2').fo
clone.querySelectorAll('h3').fo
clone.querySelectorAll('.analog
clone.querySelectorAll('li').fo
clone.querySelectorAll('svg').fo
    let caption = '';
    if (el.nextElementSibling &
        caption = el.nextElement
    }
    el.textContent = '\n\n[ Илл
});
clone.querySelectorAll('.img-ca

let text = clone.innerText;
text = text.replace(/\n{3,}/g,

if (window !== window.top) {
    fallbackModal(text, 'Markdow
    return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_alge
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

    const body = document.body;
    body.classList.remove('bg-slate

);

```

```

or: #94a3b8 !important; }
    body.bg-white .img-capt

    /* Tweak card blocks */
    body.bg-white .card-red

    body.bg-white aside { b
#334155 !important; }
    body.bg-white .text-whi

    /* Ensure active links
    body.bg-white aside a.t
    body.bg-white aside a.t
    body.bg-white aside a.t
    body.bg-white aside a.t
    body.bg-white aside a.t
    body.bg-white aside a.t
    body.bg-white aside a:h

    body.bg-white table tr.
    body.bg-white table tr.
    body.bg-white .text-tea
    body.bg-white .text-ros
    body.bg-white .text-eme
    body.bg-white .text-cya
    body.bg-white .text-amb
    body.bg-white .text-fuc
    body.bg-white .text-lim
    body.bg-white .text-blur
    body.bg-white .text-gre
    body.bg-white .text-ind

    `;
} else if (theme === 'terminal'
    body.classList.add('bg-black
    document.getElementById('th
    style.innerHTML = `
        body.bg-black .bg-slate

```

```

        </button>
        <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
        <span> </span> HTML
        </button>
    </div>

    <!-- Смена темы -->
    <div>
        <div class="text-xs text-slate-500">
        <div class="flex items-center gap-2">
            <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
            <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
            <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-white
терминала"> </button>
        </div>
    </div>
</div>
</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
-base min-w-0">
    <header class="text-center mb-12 border-b border-slate-700">
        <h1 class="text-4xl md:text-5xl font-bold">
            ВЫСШАЯ АЛГЕБРА
        </h1>
        <p class="text-slate-400 italic text-lg">
            <!-- Print Table of Contents -->

```

```

        <div class="bg-orange-900/40 border-4 border-yellow-500">
ольцо полиномов</div>
        <div class="bg-cyan-900/40 border-4 border-yellow-500">
сые числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setMainQuestion(1)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-yellow-400">База</div>
        <div class="font-bold text-slate-200">База</div>
        </a>
        <a href="#question-9" onclick="setMainQuestion(9)">
-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-400">База</div>
        <div class="font-bold text-slate-200">База</div>
        </a>
        <a href="#question-2" onclick="setMainQuestion(2)">
-4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">База</div>
        <div class="font-bold text-slate-200">База</div>
        </a>

        <!-- Row 2: Базовые операции / Евклид -->
        <a href="#question-6" onclick="setMainQuestion(6)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-yellow-400">Базовые операции</div>
        <div class="font-bold text-slate-200">Базовые операции</div>
        </a>
        <a href="#question-10" onclick="setMainQuestion(10)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-400">Базовые операции</div>
        <div class="font-bold text-slate-200">Базовые операции</div>
        </a>
        <a href="#question-3" onclick="setMainQuestion(3)">
-4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">Базовые операции</div>
        <div class="font-bold text-slate-200">Базовые операции</div>
        </a>

```

```

        </a>
        <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-500"
        <div class="font-bold text-slate-700"
        </a>

        <!-- Row 6: Кратность -->
        <div class="hidden md:block"></div>
        <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <div class="hidden md:block"></div>

        <!-- Row 7: Производная -->
        <div class="hidden md:block"></div>
        <a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <div class="hidden md:block"></div>

        <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
        <div class="col-span-1 md:col-span-1"
b-2 border-b border-emerald-500/30 pb-2">Кл

        <a href="#question-17" onclick="setM
-1-4 border-teal-500 hover:bg-slate-700 tra
        <div class="text-xs text-teal-500"
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-18" onclick="setM
-1-4 border-blue-500 hover:bg-slate-700 tra
        <div class="text-xs text-blue-500"
        <div class="font-bold text-slate-700"

```



```
        <div class="bg-red-500/20 p-3 rounded-2xl">
          <svg class="w-8 h-8 text-red-500"
            rg/2000/svg">
            <path stroke-linecap="round"
              110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
            </svg>
          </div>
          <h2 class="text-3xl pr-4 border"
        </div>
```

```
    <div class="prose prose-invert max-w-4xl">
      <p class="text-lg">
        Хватит тупить. Доказательств
        есь для того, чтобы жать на кнопки. Вот жес
      </p>
```

```
    <h2 class="text-2xl font-bold border"
      (Меметика)</h2>
```

```
    <!-- Прямое -->
    <div class="bg-slate-800/80 p-6">
      <h3 class="text-xl text-blue-400">
      <p class="text-sm text-slate-400">
      <ul class="list-disc pl-5">
        <li><span class="text-white">
          k$).</li>
        <li><span class="text-white">
        <li><span class="text-white">
          ово.</li>
      </ul>
    </div>
```

```
    <!-- От противного -->
    <div class="bg-slate-800/80 p-6">
      <h3 class="text-xl text-fuchsia-400">
      <p class="text-sm text-slate-400">
      <ul class="list-disc pl-5">
        <li><span class="text-white">
```

```
<p class="text-sm text-slate-500">
  <ul class="list-disc pl-5 space-y-2">
    <li><span class="text-w"
вверх по уравнениям).</li>
    <li><span class="text-w"
рху вниз, показывая, что  $\delta \vdots r_i$ 
    <li><span class="text-w"
х. Твой  $d$  больше.</li>
  </ul>
</div>
```

```
<h2 class="text-2xl font-bold border-top-2px solid #ccc padding-top: 10px">
  Алгоритмы экзекуции</h2>
```

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-4 rounded-lg">
    <h4 class="text-red-400">Задача 1
    <p class="text-sm text-slate-500">
    <p class="text-xs font-sans">
со следующим. Если в конце 0 – пациент мертв
  </div>
```

```
<div class="bg-slate-800 p-4 rounded-lg">
  <h4 class="text-red-400">Задача 2
  <p class="text-sm text-slate-500">
  <p class="text-xs font-sans">
 $c$ . Пока получается 0 – корень жив. Как то
корня минус один.</p>
</div>
```

```
<div class="bg-slate-800 p-4 rounded-lg">
  <h4 class="text-red-400">Задача 3
  <p class="text-sm text-slate-500">
  <p class="text-xs font-sans">
ициентов (они делятся на  $p$ ), кроме старше
зложить.</p>
</div>
```

```
        <div>
            <h4 class="text-cyan">
            <p class="text-sm text-cyan">
                есть сложение — есть ноль. Если есть умножение, то
                <code class="text-x">
            </div>
        </li>

        <li class="bg-slate-800/50 p-4">
            <span class="text-2xl">
            <div>
                <h4 class="text-cyan">
                <p class="text-sm text-cyan">
                    авь их драться. Умножение всегда врывается и
                    <code class="text-x">
                </div>
            </li>
        </ul>
    </div>
</section>
</div>
```

```
<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

РАЗДЕЛ: УТИЛИТЫ

ФАЙЛ 87/111: src/utls/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```

        const res = await fetch(href);
        if (res.ok) parts.push(await res.text());
    } catch {
        /* чужой домен – пропускаем */
    }
}
}
return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
    border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65;
.export-shell pre { font-size: 12px; line-height: 1.5;
.export-figs { margin: 0 0 1.6rem; }
.export-figs-title { color: #e2e8f0; font-size: .95rem;
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b;
.export-fig figcaption { color: #94a3b8; font-size: .75rem;
@media print {
    body { background: #fff; color: #0f172a; }

```

```

    });
  } catch {
    /* ссылки – улучшение, а не обязательная часть файла */
  }
}

const ids: string[] = [];
host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
  const id = el.dataset.termId;
  if (!id) return;
  if (!ids.includes(id)) ids.push(id);
  const link = document.createElement("a");
  link.className = "term-hint";
  link.href = `#term-${id}`;
  link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
  link.textContent = el.textContent ?? "";
  el.replaceWith(link);
});

// <details> в файле лучше раскрыть: иначе код и разбросан
host.querySelectorAll("details").forEach((d) => d.setOpen());

return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[], book = false) {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escapeHtml(id)}">
        <h3>${escapeHtml(e.title)}</h3>
        <p>${escapeHtml(e.short)}</p>
        ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
        ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
      </div>`;
    })
    .join("");
}

```

```
${opts.gloss}
```

```
    <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}
```

```
/**
```

```
 * Картинки встраиваются в файл data-URL: автономная ко
 * иллюстрации офлайн (раньше src вел на ассет приложен
 * картинки исчезали – оставались только самые первые,
 */
```

```
async function inlineImages(html: string): Promise<string> {
  const doc = new DOMParser().parseFromString(`<body>${html}</body>`, 'text/html');
  const imgs = Array.from(doc.querySelectorAll("img[src]"));
  await Promise.all(
    imgs.map(async (img) => {
      const src = img.getAttribute("src") ?? "";
      if (!src || src.startsWith("data:")) return;
      try {
        const res = await fetch(src);
        if (!res.ok) return;
        const blob = await res.blob();
        if (blob.size > 4_000_000) return;
        const data = await new Promise<string>((resolve, reject) => {
          const fr = new FileReader();
          fr.onload = () => resolve(String(fr.result));
          fr.onerror = () => reject(fr.error);
          fr.readAsDataURL(blob);
        });
        img.setAttribute("src", data);
      } catch {
        /* картинка недоступна – глава останется со ссы
      }
    })
  );
};
```

```

        return "";
    }
}

function triggerDownload(html: string, filename: string) {
    const blob = new Blob([html], { type: "text/html;charset=utf-8" });
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.href = url;
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    a.remove();
    setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» – без кириллицы и пробелов */
function safeName(title: string, id: string): string {
    const num = title.match(/^\s*(\d+)/)?[1];
    const base = id.replace(/^[^a-z0-9-]+/gi, "-").toLowerCase();
    return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Собрать автономный HTML одной главы. */
export async function buildChapterHtml(chapter: Chapter) {
    const css = await collectCss();
    const { body: rawBody, termIds } = prepareBody(chapter);
    const body = (await vizFigures(chapter.id)) + (await vizGlossary(termIds));
    return wrap({
        title: chapter.title,
        subtitle: chapter.category || "Универсальное пособие",
        css,
        body,
        gloss: glossarySection(termIds),
        origin: window.location.origin,
    });
}

```

```
    body,  
    gloss: glossarySection(allTerms, true),  
    origin: window.location.origin,  
  });  
  return html;  
}  
  
/** Выгрузить всё пособие одним файлом. */  
export async function downloadBookHtml(chapters: Chapter  
  triggerDownload(await buildBookHtml(chapters), "posob  
}
```

ФАЙЛ 89/111: src/utils/ticketLinks.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 165 | РАЗМЕР: 7.4 KB

```
import { useEffect } from "react";  
import { chapterTopics } from "../data/content";  
  
/**  
 * Перекрёстные ссылки между билетами.  
 *  
 * В тексте страниц десятки упоминаний вида «...берите Фор  
 * или «сердце Краскала (билет 18)». Раньше это был про  
 * номер, но попасть на него не мог — ссылка была «битой  
 * в никуда. Здесь такие упоминания превращаются в настоя  
 *  
 *   * в приложении — ` *   `ChapterView` и переключает страницу без перезагру  
 *   * в текстовой версии — ` *   работающий URL, который понимают парсеры и ридеры  
 *   * в PDF — остаётся текст (ссылки там всё равно не  
 *  
 * Номер билета → страница берётся из `chapterTopics`,  
 * «билет 19» корректно ведёт на общую страницу основны  
 * (она закрывает 18, 19 и 20 сразу).
```



```

}

/**
 * Строковая версия – для текстовой (`?lite=1`) версии,
 * через `dangerouslySetInnerHTML`, а эффекты разметки
 * HTML режется на «тег» и «текст», меняются только тек
 * вне `pre/code/a`.
 */
export function linkifyTicketsHtml(html: string, opts:
  let inCode = 0;
  let inLink = 0;
  return html
    .split(/(<[>]*>)/g)
    .map((part) => {
      if (!part) return part;
      if (part.startsWith("<")) {
        const closing = part.startsWith("</");
        const tag = part.slice(closing ? 2 : 1).split(/
        if (tag === "pre" || tag === "code" || tag ===
          inCode = Math.max(0, inCode + (closing ? -1 :
        } else if (tag === "a") {
          inLink = Math.max(0, inLink + (closing ? -1 :
        }
        return part;
      }
      if (inCode > 0 || inLink > 0) return part;
      return part.replace(TICKET_RE, (whole, a: string,
        const from = Number(a);
        const to = b ? Number(b) : from;
        const id = chapterForRange(from, to);
        if (!id || id === opts.selfId) return whole;
        return `<a class="ticket-link" data-goto="${id}
      });
    })
    .join("");
}

/**

```

```

        if (!id || id === opts.selfId) continue;
        if (m.index > last) frag.appendChild(document.createElement("a"));
        const a = document.createElement("a");
        a.className = "ticket-link";
        a.dataset.goto = id;
        a.setAttribute("href", hrefFor(id, opts.lite ?? false));
        a.textContent = m[0];
        frag.appendChild(a);
        last = m.index + m[0].length;
        created += 1;
    }
    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createTextNode(textNode.parentNode?.replaceChild(frag, textNode)));
}
return created;
}

/** Размечает ссылки при смене главы и подстраховывается
export function useTicketLinks(
  ref: React.RefObject<HTMLElement | null>,
  deps: unknown[],
  opts: { selfId?: string; lite?: boolean } = {}
) {
  useEffect(() => {
    const el = ref.current;
    if (el) annotateTicketLinks(el, opts);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, deps);
}

```

=====

РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

=====

```

const html = c.content ?? "";
const analogies = (html.match(/Аналогия/gi) ?? []).length;
return {
  id: c.id,
  title: c.title,
  num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
  analogies,
  hasAnalogy: analogies > 0,
  inlineSvg: /<svg[\\s>]/i.test(html),
  image: /<img[\\s>]/i.test(html),
  interactive: vizIds.has(c.id),
  chars: html.length,
};
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG | image |");
  console.log("| --- | --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} | ${r.interactive ? "да" : "нет"} | ${r.inlineSvg ? "да" : "нет"} | ${r.image ? "да" : "нет"} |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\\n`);
  const dump = (title, list) => {

```

```

-----
const re = new RegExp(
  `(?!<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
    "giu"
  )};
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\\s\\S]*?<\\/\\1>/gi
    .replace(/<[^>]+>/g, " ");

  const perTerm = new Map();
  const perSurface = new Map();
  let highlights = 0;
  const ids = new Set();

  for (const m of text.matchAll(re)) {
    const surface = m[1].toLowerCase();
    const id = map.get(surface);
    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) con
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }

  if (ids.size === 0) bad.push(c.title);

```

```
const GUIDE_PDF_PATH = path.join(OUT_DIR, "guide_lite.pdf");
const SHOTS_DIR = path.join(ROOT, "tmp", "viz-shots");
const ENTRY = path.join(ROOT, "tmp", "guide-entry.ts");
```

/** Векторные схемы графов и таблицы состояний для PDF

```
const VECTOR_VISUALIZERS = {
  dijkstra: {
    title: "Векторный граф релаксации (Дейкстра)",
    nodes: [
      { id: "A", label: "A", x: 40, y: 60, dist: "0", s: "s" },
      { id: "B", label: "B", x: 130, y: 30, dist: "3", s: "s" },
      { id: "C", label: "C", x: 130, y: 90, dist: "2", s: "s" },
      { id: "G", label: "G", x: 220, y: 90, dist: "5", s: "s" },
      { id: "D", label: "D", x: 220, y: 30, dist: "6", s: "s" },
      { id: "E", label: "E", x: 310, y: 90, dist: "7", s: "s" },
      { id: "F", label: "F", x: 400, y: 60, dist: "8", s: "s" },
    ],
    edges: [
      { u: "A", v: "B", w: 4, highlighted: true },
      { u: "A", v: "C", w: 2, highlighted: true },
      { u: "C", v: "B", w: 1, highlighted: true },
      { u: "B", v: "D", w: 5 },
      { u: "C", v: "G", w: 3, highlighted: true },
      { u: "C", v: "E", w: 8 },
      { u: "G", v: "D", w: 1, highlighted: true },
      { u: "G", v: "E", w: 2, highlighted: true },
      { u: "D", v: "F", w: 2 },
      { u: "E", v: "F", w: 1, highlighted: true },
    ],
    table: [
      ["Вершина", "Старт", "Кратчайшее dist", "Предшествующая"],
      ["Пиццерия A", "s = A", "0", "-", "зафиксирована"],
      ["Парк C", "из A", "2", "A", "зафиксирована"],
      ["Дом 1 B", "из C", "3", "C", "зафиксирована"],
      ["Метро G", "из C", "5", "C", "зафиксирована"],
    ],
  },
};
```

```

title: "Матрица кратчайших путей (Фloyd–Уоршелл)",
table: [
  ["из \\ в", "S (0)", "A (1)", "B (2)", "C (3)", "D (4)"],
  ["S (0)", "0", "4", "5", "3", "6"],
  ["A (1)", "∞", "0", "∞", "1", "4"],
  ["B (2)", "∞", "∞", "0", "-2", "1"],
  ["C (3)", "∞", "∞", "∞", "0", "3"],
  ["D (4)", "∞", "∞", "∞", "∞", "0"],
],
},
mst: {
  title: "Минимальный остов (Краскал / Прим / Борувка)",
  nodes: [
    { id: "1", label: "1", x: 50, y: 30, status: "done" },
    { id: "2", label: "2", x: 150, y: 30, status: "done" },
    { id: "3", label: "3", x: 250, y: 30, status: "done" },
    { id: "4", label: "4", x: 50, y: 90, status: "done" },
    { id: "5", label: "5", x: 150, y: 90, status: "done" },
    { id: "6", label: "6", x: 250, y: 90, status: "done" },
    { id: "7", label: "7", x: 350, y: 60, status: "done" },
  ],
  edges: [
    { u: "1", v: "2", w: 2, highlighted: true },
    { u: "2", v: "3", w: 3, highlighted: true },
    { u: "1", v: "4", w: 1, highlighted: true },
    { u: "4", v: "5", w: 4, highlighted: true },
    { u: "2", v: "5", w: 5 },
    { u: "3", v: "6", w: 2, highlighted: true },
    { u: "5", v: "6", w: 6 },
    { u: "6", v: "7", w: 3, highlighted: true },
  ],
  table: [
    ["Ребро", "Вес w", "Краскал (DSU)", "Прим (Куча)", "Борувка"],
    ["(1, 4)", "1", "добавлено (comp 1-4)", "добавлено", "добавлено"],
    ["(1, 2)", "2", "добавлено (comp 1-2-4)", "добавлено", "добавлено"],
    ["(3, 6)", "2", "добавлено (comp 3-6)", "добавлено", "добавлено"],
    ["(2, 3)", "3", "добавлено (объединение)", "добавлено", "добавлено"],
    ["(6, 7)", "3", "добавлено (финиш)", "добавлено", "добавлено"]
  ]
}

```

Универсальное пособие • полный исходный код

const pick = (list) => list.find((f) => fs.existsSync(f

/**

* Убираем символы, которых нет в DeJaVu (эмодзи и пикто

* на их месте появляется «пустой» глиф. Стрелки, матем

* геометрические фигуры DeJaVu знает – их оставляем.

*/

const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2722}]/gu;

/** Русское склонение счётных существительных: 1 страниц

const plural = (n, one, few, many) => {

const mod10 = Math.abs(n) % 10;

const mod100 = Math.abs(n) % 100;

if (mod10 === 1 && mod100 !== 11) return one;

if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 100))

return many;

};

const plain = (text) =>

String(text ?? "")

.replace(UNSUPPORTED, "")

.replace(/\s{2,}/g, " ")

.replace(/\u0000/g, "")

.trim();

/* ----- данные приложения (через esbuild

async function loadAppData() {

ensureDir(path.dirname(ENTRY));

fs.writeFileSync(

ENTRY,

[

'export { chapters, chapterTopics } from "../src/

'export { PAGE_SYNC } from "../src/data/vizSync";

'export { VIZ_REGISTRY } from "../src/components/

'export { quizzes } from "../src/data/quizzes";',

].join("\n")

);

```
class Renderer {
  constructor(doc, fonts) {
    this.doc = doc;
    this.fonts = fonts;
    this.footerTitle = "";
    this.pages = 0;
    this.cy = TOP;
  }

  /**
   * Колонтитул текущей страницы (печатаем до перехода к следующей)
   *
   * Важно: на время печати нижнее поле страницы уменьшаем, чтобы
   * решает, что текст не влез, и молча добавляет пустые строки.
   */
  stampFooter() {
    if (this.pages === 0 || !this.doc.page) return;
    const prevBottom = this.doc.page.margins.bottom;
    this.doc.page.margins.bottom = 16;
    const y = PAGE.height - PAGE.margin + 4;
    this.doc.font(this.fonts.regular).fontSize(7.6).fill('black');
    this.doc.text("Универсальное пособие • алгоритмы и структуры данных",
      width: CONTENT_W * 0.62,
      align: "left",
      lineBreak: false,
      ellipsis: true,
    ));
    this.doc.text(`${this.footerTitle} • ${this.pages}`,
      width: CONTENT_W * 0.38,
      align: "right",
      lineBreak: false,
      ellipsis: true,
    ));
    this.doc.page.margins.bottom = prevBottom;
    this.doc.y = this.cy;
  }

  newPage(footerTitle) {
```



```

        if (current.trim()) chunks.push(current.trim());
        current = "";
    };
    for (const sentence of source.split(/(?<=[.!?...;:])\s/)) {
        const withSentence = current ? `${current} ${sentence}` : sentence;
        if (measure(withSentence) <= max) {
            current = withSentence;
            continue;
        }
        flush();
        if (measure(sentence) <= max) {
            current = sentence;
            continue;
        }
        for (const word of sentence.split(/\s+/)) {
            const withWord = current ? `${current} ${word}` : word;
            if (measure(withWord) <= max) current = withWord;
            else {
                flush();
                current = word;
            }
        }
    }
    flush();
    return chunks.length ? chunks : [source];
}

rule(color = "#c7d2fe", width = 1) {
    const y = this.cy;
    this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin + width, y);
    this.cy = y + 8;
}

h1(rawText, { size = 20, color = "#111827", rule = true }) {
    const text = plain(rawText);
    if (!text) return;
    const h = this.height(text, { font: this.fonts.bold });
    this.ensure(h + (rule ? 14 : 6) + gapBefore, 22);

```

```

        const y0 = this.cy;
        this.doc.font(font).fontSize(size).fillColor(color);
        this.cy = Math.max(y0 + h, this.doc.y) + (last ? 0 : 1);
        if (this.cy > BOTTOM) this.newPage();
    });
}

```

```

bullet(rawText, { ordinal = null, depth = 0, size = 9 }) {
    const text = plain(rawText);
    if (!text) return;
    const indent = 16 + depth * 14;
    const marker = ordinal ? `${ordinal}.` : "*";
    const width = CONTENT_W - indent - 4;
    const opts = { font: this.fonts.regular, size, width };
    const chunks = this.splitToFit(text, opts, BOTTOM - 10);
    chunks.forEach((chunk, index) => {
        const h = this.height(chunk, opts);
        this.ensure(h + 2.5, 6);
        const y0 = this.cy;
        if (index === 0) {
            this.doc.font(this.fonts.regular).fontSize(size)
                .text(marker, { align: "right", lineGap: 1 });
        }
        this.doc.font(this.fonts.regular).fontSize(size).text(
            chunk, { x: indent, y: y0, lineGap: 1 });
        this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
        if (this.cy > BOTTOM) this.newPage();
    });
}

```

```

/** Код: режим на фрагменты по высоте страницы, каждый фрагмент
code(rawText, { size = 8.0 } = {}) {
    const font = this.fonts.mono;
    const inner = CONTENT_W - 22;
    const lines = String(rawText ?? "").replace(/[\n\r]/g, " ");
    // собираем фрагменты, которые влезают в одну рамку
    let i = 0;
    while (i < lines.length) {
        // Если у низа страницы осталось меньше минимальной

```

```

        .fontSize(9.5)
        .fillColor("#1e293b")
        .text(title, PAGE.margin + 12, startY + 8);
    }

    // Рёбра графа
    for (const e of edges) {
        const fromNode = nodes.find((n) => n.id === e.u);
        const toNode = nodes.find((n) => n.id === e.v);
        if (!fromNode || !toNode) continue;

        const x1 = PAGE.margin + fromNode.x;
        const y1 = startY + fromNode.y;
        const x2 = PAGE.margin + toNode.x;
        const y2 = startY + toNode.y;

        const strokeColor = e.highlighted ? "#059669" : "#000000";
        const strokeWidth = e.highlighted ? 2 : 1;

        this.doc
            .moveTo(x1, y1)
            .lineTo(x2, y2)
            .lineWidth(strokeWidth)
            .strokeColor(strokeColor)
            .stroke();

        if (e.w !== undefined) {
            const mx = (x1 + x2) / 2;
            const my = (y1 + y2) / 2;
            this.doc
                .circle(mx, my, 7)
                .fillAndStroke("ffffff", strokeColor);
            this.doc
                .font(this.fonts.bold)
                .fontSize(7)
                .fillColor("#0f172a")
                .text(String(e.w), mx - 7, my - 3.5, { width: 14 });
        }
    }
}

```

```

const cols = Math.max(...rows.map((r) => r.length))
const widths = new Array(cols).fill(CONTENT_W / cols)
const pad = 5;
const header = rows[0];

const rowHeight = (cells) => {
  let max = 0;
  for (let i = 0; i < cols; i++) {
    const h = this.height(cells[i] ?? "", { font: this.font, size: this.size });
    max = Math.max(max, h);
  }
  return max + pad * 2;
};

const drawRow = (cells, y, isHeader, zebra) => {
  const h = rowHeight(cells);
  this.doc.rect(PAGE.margin, y, CONTENT_W, h).fill(zebra ? "#f2f2f2" : "white");
  for (let i = 0; i < cols; i++) {
    const x = PAGE.margin + widths.slice(0, i).reduce((a, b) => a + b, 0);
    this.doc
      .font(isHeader ? this.fonts.bold : this.fonts.regular)
      .fontSize(size)
      .fillColor("#1f2937")
      .text(String(cells[i] ?? ""), x, y + pad, { width: widths[i] });
  }
  this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineWidth(1);
  return h;
};

this.ensure(rowHeight(header) + 8, 14);
let y = this.cy;
rows.forEach((cells, idx) => {
  const row = [];
  for (let i = 0; i < cols; i++) row.push(cells[i]);
  const h = rowHeight(row);
  if (y + h > BOTTOM) {
    this.cy = y;
    this.newPage();
  }
  drawRow(row, y, idx === 0, idx % 2 === 0);
  y += h + 8;
});

```

```

    const w = img.width * scale;
    const h = img.height * scale;
    this.ensure(h + capH + 10, 20);
    const y0 = this.cy;
    const x0 = PAGE.margin + (CONTENT_W - w) / 2;
    this.doc.image(file, x0, y0, { width: w, height: h
    this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeCo
    this.cy = y0 + h + 4;
    if (caption) {
        this.doc.font(this.fonts.regular).fontSize(8).fil
        enter" });
        this.cy = Math.max(this.cy, this.doc.y) + 6;
    }
    return true;
}

```

```

/** Бокс с заголовком и строками; длинные режутся на
box(rawTitle, rawLines, { color = "#eef2ff", border =
    const title = plain(rawTitle);
    const lines = rawLines.map((l) => plain(l)).filter(
    const inner = CONTENT_W - 20;
    let rest = lines.slice();
    let first = true;
    while (rest.length || first) {
        first = false;
        const headH = this.height(title, { font: this.fon
        this.ensure(headH + 24, 16);
        const y0 = this.cy;
        // сколько строк влезет на эту страницу
        const avail = BOTTOM - y0 - headH - 16;
        const taken = [];
        let h = 0;
        while (rest.length) {
            const ch = this.height(rest[0], { font: this.fon
            if (taken.length && h + ch > avail) break;
            taken.push(rest.shift());
            h += ch;
        }
    }

```

```
      Subject: "Текст всех тем + визуализации, без кода",
      CreationDate: new Date(),
    },
  ));
const stream = fs.createWriteStream(GUIDE_PDF_PATH);
doc.pipe(stream);

const R = new Renderer(doc, fonts);
const today = new Date().toISOString().slice(0, 10);
const totalTopics = new Set(Object.values(chapterTopics));
const vizCount = chapters.filter((c) => VIZ_REGISTRY[c]);

/* — обложка — */
R.firstPage("Обложка");
R.cy = TOP + 90;
R.h1("Универсальное пособие", { size: 26, rule: false });
R.h1("Алгоритмы и структуры данных", { size: 17, color: "#4b5563" });
R.para(
  `Билеты 1-${totalTopics} · ${chapters.length} ${plusSign} ${vizCount}, "интерактивная демонстрация", "интерактивные примеры"`,
  { size: 10.5, color: "#4b5563" }
);
R.para(
  "Автоматический компактный сборник: весь текст тем + визуализации. Навигация приложения и код панели Python",
  { size: 9.6, color: "#374151" }
);
R.para(`Дата сборки: ${today}`, { size: 9, color: "#6b728b" });

const missingViz = chapters.filter((c) => !VIZ_REGISTRY[c]);
R.box("Как устроено пособие", [
  "• Страница = один или несколько билетов: текст, анимация, интерактивные примеры",
  "• У каждой страницы есть интерактивная демонстрация, которую можно просматривать, кликая по ней, или открывать её напрямую.",
  "• В этом PDF демонстрации показаны снимками (когда анимация работает, или интерактивные примеры выполняются).",
]);
R.para(`Всего страниц: ${missingViz.length + 1}`);
```

```

    } else if (b.type === "summary") {
      R.para("► " + b.text, { size: 9.8, color: "#373737" });
    } else if (b.type === "li") {
      R.bullet(b.text, { ordinal: b.ordinal, depth: b.depth });
    } else if (b.type === "code") {
      R.code(b.text);
    } else if (b.type === "table") {
      R.table(b.rows);
    } else if (b.type === "quote") {
      R.quote(b.text);
    } else if (b.type === "caption") {
      R.para(b.text, { size: 8.3, color: "#6b7280", italic: true });
    } else {
      R.para(b.text);
    }
  }
}

/* демонстрация: интерактивная схема (векторные графы)
const viz = VIZ_REGISTRY[chapter.id];
const vecViz = VECTOR_VISUALIZERS[chapter.id];
const syncKeys = Object.keys(PAGE_SYNC).filter((k) => k !== "viz");

if (viz || syncKeys.length || vecViz) {
  R.h2("Демонстрация и интерактивная схема");
  if (viz) {
    R.para(viz.title, { size: 10.2, color: "#1e293b" });
    if (viz.hint) R.para(viz.hint, { size: 9.1, color: "#6b7280" });
  }

  if (vecViz) {
    if (vecViz.nodes && vecViz.edges) {
      R.vectorGraph(vecViz.nodes, vecViz.edges, { height: 300 });
    }
    if (vecViz.table) {
      R.table(vecViz.table);
    }
  }
}

```

```
// Самопроверка: страниц в файле должно быть ровно столько, сколько
// рендерер. Расхождение означает, что pdftk создал файл с лишними страницами
// в файл без нижнего колонтитула и без нумерации.
const pdfLib = await import("pdf-lib");
const physical = (await pdfLib.PDFDocument.load(fs.readFileSync(R.PDF_PATH))).getPages().length;
console.log(`Физический документ: ${R.pages} (рендер) / ${physical} (файл)`);
if (R.pages > 100) console.warn(`Документ больше 100 страниц. ВНИМАНИЕ: ${R.PDF_PATH}`);
if (physical !== R.pages) {
  console.error(`Физический документ не соответствует рендеру. ОШИБКА: pdftk добавил ${physical - R.pages} страниц`);
  process.exitCode = 1;
}

main().catch((err) => {
  console.error("Ошибка сборки PDF пособия:", err.message);
  process.exit(1);
});
```

ФАЙЛ 93/111: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, ensureFile } from "../constants";

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };
```



```
    });

    console.log("[3/3] png -> pdf");
    console.log(`    страниц: ${pages.length}`);
    console.log(`    выход:    ${path.relative(ROOT, PDF)}`);
  }

  main().catch((err) => {
    console.error("Ошибка сборки PDF:", err.message);
    process.exit(1);
  });
```

ФАЙЛ 94/111: scripts/export/capture-viz-shots.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 122 | РАЗМ: 1.2 КБ

```
/**
 * Снимки визуализаций для PDF-конспекта: tmp/viz-shots.
 *
 * Headless-браузер открывает каждую страницу с демонстрацией
 * блок #chapter-viz. У страниц с вкладками (sparse-table)
 * снимается каждая вкладка – файл получает имя `<chapter>_<tab>`
 * и PDF подписывает его как «режим «вкладка»».
 *
 *   npm run export:shots                # сервер уже поднят
 *   SHOTS_BASE_URL=http://127.0.0.1:4173 npm run export:shots
 *
 * В CI это делает rebuild-pdf.yml: сборка → vite preview →
 * npm run export:guide. Локально без браузера скрипт протестируй
 * честно пишет, что снимки добавляются при сборке в CI.
 */
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import puppeteer from "puppeteer";
import { ROOT } from "../common.mjs";
```

```
-----
const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями: `);

const browser = await puppeteer.launch({
  headless: true,
  args: ["--no-sandbox", "--disable-setuid-sandbox", "--no-dev-shm-usage"],
});
const page = await browser.newPage();
await page.setViewport({ width: 1400, height: 1000, deviceScaleFactor: 1 });

let done = 0;
let failed = 0;
for (const id of ids) {
  try {
    await page.goto(`${BASE_URL}/?topic=${id}`, { waitUntil: 'load' });
    await page.waitForSelector("#chapter-viz", { timeout: 10000 });
    // демо-анимации: даём кадру собраться до осмысленного состояния
    await new Promise((r) => setTimeout(r, 1600));
    const viz = await page.$("#chapter-viz");
    if (!viz) throw new Error("нет #chapter-viz");
    await viz.screenshot({ path: path.join(OUT_DIR, `${id}.png`), type: 'png' });
    done += 1;

    for (const tab of TABS[id] ?? []) {
      try {
        const clicked = await page.evaluate((label) => {
          const root = document.querySelector("#chapter-viz");
          if (!root) return false;
          const btn = Array.from(root.querySelectorAll("a")).find(
            (b) => b.textContent ?? "" === label.trim().startsWith(label)
          );
          if (!btn) return false;
          btn.dispatchEvent(new MouseEvent("click", { bubbles: true }));
          return true;
        }, tab);
        if (!clicked) continue;
        await new Promise((r) => setTimeout(r, 1200));
        const el = await page.$("#chapter-viz");
      } catch {
        failed += 1;
      }
    }
  } catch {
    failed += 1;
  }
}
```

Универсальное пособие • полный исходный код

```
-----  
* Список файлов берётся из git (чтобы ничего не забыть  
* с фоллбэком на обход файловой системы, если git недо  
*/
```

```
import fs from "node:fs";  
import path from "node:path";  
import { execFileSync } from "node:child_process";  
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,
```

```
/** Расширения, которые считаем «кодом» и включаем в да  
const CODE_EXT = new Set([  
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",  
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y  
]);
```

```
/** Явные исключения: генерируемое, бинарное и просто ш  
const EXCLUDE_FILES = new Set(["package-lock.json", "bu  
const EXCLUDE_DIRS = ["node_modules", "dist", "build",
```

```
/** Человечитаемые категории для оглавления. */  
const CATEGORIES = [  
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],  
  [/^src\/data\/$/, "Контент и данные пособия"],  
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в  
  [/^src\/components\/$/, "Интерактивные компоненты и си  
  [/^src\/layout\/$/, "HTML-разметка (layout)"],  
  [/^src\/utils\/$/, "Утилиты"],  
  [/^src\/$/, "Ядро приложения"],  
  [/^scripts\/$/, "Скрипты сборки и экспорта"],  
  [/^\.github\/$/, "CI / автоматизация"],  
  [/^[^\/]+\$/ , "Конфигурация проекта"],  
];
```

```
const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel))
```

```
/** Порядок разделов в документе. */  
const ORDER = [  
  "Конфигурация проекта",  
  "Ядро приложения",
```

```

    .filter((rel) => fs.existsSync(path.join(ROOT, rel))
    .sort((a, b) => {
        const ia = ORDER.indexOf(categoryOf(a));
        const ib = ORDER.indexOf(categoryOf(b));
        return ia !== ib ? ia - ib : a.localeCompare(b);
    }));
}

/** Достаём заголовки глав из данных пособия (без запус
function extractChapters(files) {
    const chapters = [];
    const seen = new Set();
    for (const rel of files.filter((f) => f.startsWith("s
        const src = fs.readFileSync(path.join(ROOT, rel), "
        const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([
        let m;
        while ((m = re.exec(src)) !== null) {
            const [, id, title] = m;
            if (seen.has(id)) continue;
            seen.add(id);
            chapters.push({ id, title, file: rel });
        }
    }
    return chapters.sort((a, b) => {
        const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
        const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
        return na - nb;
    }));
}

function main() {
    const files = listFiles();
    const chapters = extractChapters(files);
    ensureDir(OUT_DIR);

    const out = [];
    const push = (s = "") => out.push(s);

```

```

current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push(HR);
    push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
    push(HR);
    push();
  }
  const content = fs.readFileSync(path.join(ROOT, rel));
  const lines = content.split("\n");
  push(SUBHR);
  push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
  push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
  push(SUBHR);
  push();
  push(content.replace(/\n+$/, ""));
  push();
  push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`файлов: ${files.length}`);
console.log(`глав: ${chapters.length}`);
console.log(`строк: ${txt.split("\n").length}`);
console.log(`выход: ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

```
    /** Максимум символов в строке (далее – жёсткий пере  
    cols: 138,  
    /** Строк содержимого на странице (+2 служебные: шапка  
    rows: 76,  
};  
  
export const HR = "=".repeat(PAGE.cols);  
export const SUBHR = "-".repeat(PAGE.cols);  
  
export function ensureDir(dir) {  
  fs.mkdirSync(dir, { recursive: true });  
}  
  
export function humanSize(bytes) {  
  if (bytes < 1024) return `${bytes} B`;  
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;  
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;  
}  
  
/** ImageMagick 7 (`magick`) или 6 (`convert`). */  
export function imageMagickCmd() {  
  for (const cmd of ["magick", "convert"]) {  
    try {  
      execFileSync(cmd, ["-version"], { stdio: "ignore" });  
      return cmd;  
    } catch {}  
    /* пробуем следующий */  
  }  
  throw new Error(  
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick  
  );  
}
```

```

-----
export function parseHtml(html) {
  const root = { tag: "#root", attrs: {}, children: [] }
  const stack = [root];
  const re = /<!--[\s\S]*?-->|<\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)\s*/g;
  let m;
  while ((m = re.exec(html)) !== null) {
    const [full, closeTag, openTag, attrsStr, selfClose] = m;
    if (full.startsWith("<!--")) continue;

    if (closeTag) {
      const tag = closeTag.toLowerCase();
      // закрываем до совпадения (прощает незакрытые <br)
      for (let i = stack.length - 1; i >= 1; i--) {
        if (stack[i].tag === tag) {
          stack.length = i;
          break;
        }
      }
      continue;
    }

    if (openTag) {
      const tag = openTag.toLowerCase();
      const attrs = {};
      const attrRe = /([a-zA-Z-_:]+)\s*=\s*"([^"]*)"|'([^']*)'|([a-zA-Z-_:]+)\s*/g;
      let a;
      while ((a = attrRe.exec(attrsStr || "")) !== null) {
        const node = { tag, attrs, children: [] };
        stack[stack.length - 1].children.push(node);
        if (!VOID.has(tag) && !selfClose) stack.push(node);
        continue;
      }

      if (textChunk !== undefined && textChunk.length) {
        stack[stack.length - 1].children.push({ tag: "#text", text: textChunk });
      }
    }
  }
  return root;
}

```

```

    } else if (SKIP.has(c.tag)) {
      continue;
    } else if (BLOCK.has(c.tag)) {
      continue;
    } else if (c.tag === "br") {
      out += "\n";
    } else {
      out += inlineText(c);
    }
  }
  return clean(out);
}

```

```

/** Весь текст поддерева (для ячеек таблиц и подписей).
function collectText(node) {
  if (node.tag === "#text") return clean(node.text);
  if (SKIP.has(node.tag)) return "";
  if (node.tag === "table") return tableRows(node).map(
    return clean(node.children.map(collectText).join(" "))
  )
}

```

```

/** Сырой текст внутри <pre> – переносы строк важны. */
function collectRawPre(node) {
  if (node.tag === "#text") return node.text;
  if (node.tag === "br") return "\n";
  if (SKIP.has(node.tag) && node.tag !== "img") return
  return node.children.map(collectRawPre).join("");
}

```

```

/** Таблица → массив строк из массивов ячеек. */
function tableRows(node) {
  const rows = [];
  const visit = (n) => {
    if (n.tag === "tr") {
      const cells = n.children.filter((c) => c.tag ===
      if (cells.some((c) => c.length)) rows.push(cells)
      return;
    }
  }
}

```



```

const walk = (node, listDepth = 0) => {
  if (node.tag !== "#root" && SKIP.has(node.tag)) return;

  if (node.tag === "table") {
    const rows = tableRows(node);
    if (rows.length) push({ type: "table", rows });
    return;
  }
  if (node.tag === "pre") {
    const text = decodeEntities(collectRawPre(node));
    if (text.trim()) push({ type: "code", text: text });
    return;
  }

  // есть блочные дети: идём по порядку, инлайн-пробе
  if (hasBlockChild(node)) {
    let buf = "";
    const flush = () => {
      const t = clean(buf);
      if (t) push({ type: "p", text: t });
      buf = "";
    };
    for (const c of node.children) {
      if (c.tag === "#text") {
        buf += c.text;
        continue;
      }
      if (SKIP.has(c.tag)) continue;
      if (!BLOCK.has(c.tag)) {
        buf += inlineText(c) + " ";
        continue;
      }
      if (c.tag === "ul" || c.tag === "ol") {
        flush();
        emitList(c, listDepth);
        continue;
      }
      flush();
    }
  }
}

```

```
}  
  
-----
```

ФАЙЛ 98/111: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 3.5 КБ

```
-----  
  
/**  
 * ШАГ 2: TXT -> PNG  
 *  
 * Разбивает full_code_all_pages.txt на страницы формата  
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)  
 *  
 * Результат: tmp/export-pages/page-0001.png, page-0002.png  
 */  
import fs from "node:fs";  
import path from "node:path";  
import os from "node:os";  
import { execFile } from "node:child_process";  
import { promisify } from "node:util";  
import {  
  ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize  
} from "../common.mjs";  
  
const execFileAsync = promisify(execFile);  
  
/** Раскрываем табы и жёстко переносим длинные строки,  
function wrap(line) {  
  const expanded = line.replace(/\t/g, "    ").replace(/  
  if (expanded.length <= PAGE.cols) return [expanded];  
  const parts = [];  
  const indent = "    ".repeat(Math.min((expanded.match(/  
  let rest = expanded;  
  let first = true;  
  while (rest.length > 0) {  
    const width = first ? PAGE.cols : PAGE.cols - indent.length;  
    parts.push((first ? "" : indent) + rest.slice(0, width));  
    rest = rest.slice(width);  
    first = false;  
  }  
  return parts;  
}
```

```
const args = (job) => [
  "-density", String(PAGE.density),
  "-page", PAGE.size,
  "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
  "-pointsize", String(PAGE.pointsized),
  `text:${job.txtFile}`,
  "-background", "white",
  "-flatten",
  "-colorspace", "Gray",
  ...(PAGE.monochrome ? ["-monochrome"] : []),
  "-strip",
  "-define", "png:compression-level=9",
  job.pngFile,
];
```

```
const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;
```

```
async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`отрендерено ${done} страниц\n`);
    }
  }
}
```

```
await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");
```

```
const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);
```

```

    name: string;
    ok: boolean;
    ms: number;
    detail: string;
    consoleErrors: string[];
}

const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;

/* -----
/*  утилиты
/* -----

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));

async function settle(ms = 40) {
  await act(async () => {
    await sleep(ms);
  });
}

function bodyText(): string {
  return (document.body.textContent ?? "").replace(/\s+/g, " ");
}

function allButtons(): string[] {
  return Array.from(document.querySelectorAll("button"))
    .filter((b) => (b.getAttribute("aria-label") || b.getAttribute("title")));
}

/** Кнопки без доступного имени: ни aria-label, ни title
function namelessButtons(scope: ParentNode = document): string[] {
  return Array.from(scope.querySelectorAll("button"))
    .filter((b) => {
      const name = [b.getAttribute("aria-label"), b.getAttribute("title")].filter(Boolean).join(" ");
      return !name;
    });
}

```

```
-----  
function typeInto(ta: HTMLTextAreaElement, value: string) {  
    const setter = Object.getOwnPropertyDescriptor(window, ta.tagName.toLowerCase() + "value")  
    if (!setter) throw new Error("нет нативного setter'a")  
    act(() => {  
        setter.call(ta, value);  
        ta.dispatchEvent(new window.Event("input", { bubble: true }));  
    });  
}
```

```
async function key(k: string, init: KeyboardEventInit = {}) {  
    await act(async () => {  
        document.dispatchEvent(new window.KeyboardEvent("keydown", init));  
        await sleep(20);  
    });  
}
```

```
/** Ждём появления признака в DOM (текст/предикат), перепроверяем каждые 10 мс  
или пока не истечёт таймаут */  
async function waitFor(pred: () => boolean, timeoutMs = 1000) {  
    const deadline = Date.now() + timeoutMs;  
    while (Date.now() < deadline) {  
        if (pred()) return true;  
        await settle(step);  
    }  
    return pred();  
}
```

```
const RUN_DONE = [  
    "Трасса готова автоматически",  
    "Нет шагов трассы",  
    "Пустой код",  
    "Код не выполнялся",  
    "Выполнение остановлено после",  
    "Не удалось выполнить код",  
    "Ошибка:",  
];
```

```
let root: Root | null = null;  
let host: HTMLElement | null = null;
```

```

if (!entry) throw new Error(`в реестре нет визуализации`);
const Comp = entry.Component;
root = createRoot(host);
await act(async () => {
  root!.render(
    <VizChapterContext.Provider value={vizId}>
      <VizLiveStatus demoTitle={entry.title} />
      <Comp />
    </VizChapterContext.Provider>
  );
});
await settle(60);
return host;
}

```

```

const BAD_DOM = [
  { re: /\bNaN\b/, why: "NaN в разметке" },
  { re: /\bundefined\b/, why: "undefined в разметке" },
  { re: /\[object Object\]/, why: "[object Object] в разметке" },
];

```

/**

* Битые подстановки видны в КОРОТКИХ текстовых узлах (листья
 * вершины): «undefined», «NaN», «[object Object]». В длинных
 * псевдокода те же слова законны — их не трогаем.
 */

```

function domDefects(scope: HTMLElement | null): string[] {
  if (!scope) return [];
  const found: string[] = [];
  const walker = document.createTreeWalker(scope, 4 /* NodeFilter.SHOW_TEXT */);
  let node = walker.nextNode() as Text | null;
  while (node) {
    const text = (node.nodeValue ?? "").replace(/\s+/g, " ");
    if (text.length > 0 && text.length < 60) {
      for (const bad of BAD_DOM) {
        if (bad.re.test(text)) found.push(`${bad.why}: ${text}`);
      }
    }
    node = walker.nextNode();
  }
  return found;
}

```

```

-----
const skipGroups = new Set((process.env.HAR_SKIP || "")
/** HAR_SYNC=floyd,dijkstra — прогнать только эти записи
const onlySync = new Set((process.env.HAR_SYNC || "").split(",")).filter(s => s);

export async function runSuite({ py }: { py: Py }): Promise<void> {
  const chapterList = onlyChapters.length ? chapters.filter(ch => ch.id !== "intro") : chapters;
  const want = (group: string) => !skipGroups.has(group);
  // перехват console.error приложения (React пишет сюда)
  console.error = (...args: unknown[]) => {
    consoleErrors.push(args.map(a => (a instanceof Error ? a.message : a)));
  };

  /* — 1. Панель Python: реальный запуск на каждой главе
  for (const chapter of chapterList) {
    const group = "A. страницы";
    await test(group, `${chapter.id}: монтируется, бейджик`);
    await mountApp(chapter.id);
    const txt = bodyText();
    if (!txt.includes(chapter.title.slice(0, 18))) throw new Error("неверно");
    if (!VIZ_REGISTRY[chapter.id]) return "визуализация не найдена";
    if (!/встроенное демо|данные из компилятора/.test(txt)) return "неверно";
    const defects = domDefects(host);
    if (defects.length) throw new Error(`артефакты в DOM: ${defects.join(", ")}`);
    const nameless = namelessButtons(host!);
    if (nameless.length) throw new Error(`кнопок без имени: ${nameless.join(", ")}`);
    return `бейдж «встроенное демо» на месте, кнопок: ${nameless.length}`;
  });

  await test(group, `${chapter.id}: панель открывается`);
  await mountApp(chapter.id);
  const toggle = findButton("Боковая панель Python");
  await click(toggle, "кнопка панели Python в навбаре");
  const ta = textarea();
  if (!ta) throw new Error("редактор (textarea) не найден");
  if (!ta.value.includes("# «")) return `редактор не настроен`;
  const eye = findButton(/эта?лонный код/i);
  if (!eye) throw new Error("нет кнопки-«глаза» (встроенное демо)");
  const run = findButton(/Запустить код|автопроход/);

```

```

    { name: "матрица под именем D и k/i/j вне диапазона
      [10, 11, 12, 0]] }, expect: "4x4" },
  ];

```

```

for (const c of floydCases) {
  await test("В. Флойд: размерности", c.name, async () => {
    await mountViz("floyd");
    await act(async () => {
      emitVizState("floyd", { line: 3, func: "<module
      await sleep(30);
    });
    const txt = (host?.textContent ?? "").replace(/\\s
    if (!/данные из компилятора/.test(txt)) throw new
    if (!txt.includes(c.expect)) throw new Error(`в р
    const defects = domDefects(host);
    if (defects.length) throw new Error(`артефакты в
    return `бейдж: ${txt.match(/данные из компилятора
  });
}

```

```

/* — 3. Префиксные суммы 2D и разреженная таблица —
const gridCases: { viz: string; chapter: string; name
  { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
    9, 10, 11, 12]], S: [[1, 3, 6, 10], [6, 14, 24, 3
    тора/ },
  { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
    pref: [[1, 3], [4, 10]], r1: 0, c1: 0, r2: 1, c2
  { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
    9]], S: [[1, 3, 6], [5, 12, 21], [12, 27, 45]],
  { viz: "sparse-table", chapter: "sparse-table", name
    [3, 3, 1, 1, 0], [1, 1, 0, 0, 0]] }, expect: /не
  { viz: "sparse-table", chapter: "sparse-table#2d-bu
    0, i: 2, j: 3, st: Array.from({ length: 4 }, () :
    pa/ },
  { viz: "sparse-table", chapter: "sparse-table#2d-qu
    30, j2: 40, res: 2 }, expect: /не совпадает|данны
];
for (const c of gridCases) {

```



```

    }
    if (domDefects(host).length) throw new Error(`артеф
    return "оригинал из А с первого шага, вкладка «Шаг
  });

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин
for (const [viz, vars] of [
  ["dijkstra", { u: 0, v: 3, w: 4, dist: [0, 4, 2, 6,
  ["bellman-ford", { n: 5, m: 4, i: 1, u: 2, v: 4, w:
] as [string, Record<string, unknown>][]) {
  await test("D. графы путей", `${viz}: вершины 0...4 в
  await mountViz(viz);
  const before = allButtons().length;
  const playEnabled = (findButton(/^Пуск|Пуск$/)) as
  await act(async () => {
    emitVizState(viz, { line: 5, func: "<module>",
    await sleep(30);
  });
  const txt = (host?.textContent ?? "").replace(/\\s
  if (!/данные из компилятора/.test(txt)) throw new
  if (!/ваши расстояния|dist/i.test(txt)) throw new
  const step = findButton(/^Шаг$|Шаг /) as HTMLButt
  if (step && !step.disabled) throw new Error("кноп
  if (domDefects(host).length) throw new Error(`арт
  await act(async () => {
    clearVizState(viz);
    await sleep(30);
  });
  const step2 = findButton(/^Шаг$|Шаг /) as HTMLBut
  if (step2 && step2.disabled) throw new Error("кноп
  return `кнопок до/после: ${before}/${allButtons()}
});
}

```

```

/* — 5. Все референсные коды: настоящий Python → нас
const syncKeys = Object.keys(PAGE_SYNC).filter((k) =>
for (const key of syncKeys) {
  const sync = PAGE_SYNC[key];

```

```
/* — 6. Поведение редактора: пустой код, ошибки, гла.
const ch = "dijkstra";
```

```
await test("F. редактор", "только комментарии: редакт
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, "# просто комментарий\n# и ещё один\n"
  const done = await waitFor(() => bodyText().include
  if (!done) throw new Error(`не дождались сообщения
  if (!textarea()) throw new Error("редактор исчез по
  if (!/встроенное демо/.test(bodyText())) throw new
  return "редактор на месте, бейдж «встроенное демо»"
});
```

```
await test("F. редактор", "синтаксическая ошибка: ред
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, "def broken(:\n    pass\n");
  const done = await waitFor(() => /Код не выполнен
  if (!done) throw new Error(`ошибка синтаксиса не по
  if (!textarea()) throw new Error("редактор исчез на
  if (!/invalid syntax|SyntaxError/i.test(bodyText()))
  return "ошибка показана, редактор остался";
});
```

```
await test("F. редактор", "гигантский print обрезаетс
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, 'dist = {0: 0}\nprint("x" * 200000)\np
  const ready = await waitFor(() => bodyText().includ
  if (!ready) throw new Error(`программа с огромным p
```

```

await click(findButton("Вставить эталонный код"), "
const trace = await waitFor(() => !!findButton(/Ост
if (!trace) throw new Error(`эталон не запустился.
if (!textarea()) throw new Error("редактор исчез во
if (/Ошибка:/.test(panelText())) throw new Error(`э
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.
if (!ta2?.value.includes(firstRefLine)) {
  throw new Error(`выполняется не тот текст, что ле
    0, 100)}`);
}
const mustBe = [
  { what: "пуск/автопроход", re: /Запустить код|Про
  { what: "копирование кода", re: /Скопировать код|
  { what: "закрыть панель", re: /Закрыть компилятор
  { what: "шаг вперёд", re: /Шаг вперёд|вперёд/i },
  { what: "шаг назад", re: /Шаг назад|назад/i },
  { what: "свернуть листинг", re: /Свернуть листинг
];
const btns = allButtons().join(" | ");
for (const m of mustBe) {
  if (!m.re.test(btns)) throw new Error(`в панели н
}
// На первом шаге переменных ещё нет (отладчик вста
// поэтому связь с демонстрацией проверяем, дойдя д
await click(findButton("В конец трассы"), "в конец
const linked = await waitFor(() => /данные из компи
if (!linked) throw new Error(`эталон выполнен до ко

// — глаз ВЫКЛ: кода нет вовсе, ничего не выполняе
await click(findButton("Убрать эталонный код"), "гла
const cleared = await waitFor(() => bodyText().incl
if (!cleared) throw new Error(`после выключения гла
const ta3 = textarea();
if (!ta3) throw new Error("редактор пропал после вы
if (ta3.value.trim() !== "") throw new Error(`глаз
if (!/встроенное демо/.test(bodyText())) throw new
if (/данные из компилятора/.test(bodyText())) throw

```

```

    if (!textarea()) throw new Error("во время прогона");
    if (!findButton("/Остановить прогон трассы/")) throw new Error("ошибка");
    // Листинг сворачивается (освобождая место редактору кода)
    // и редактор остаются на месте.
    const codeLines = () => panel()?.querySelectorAll("pre");
    if (codeLines() === 0) throw new Error("в раскрытой панели нет кода");
    await click(findButton("Свернуть листинг трассы"), "кнопка");
    await settle(80);
    if (!textarea()) throw new Error("после сворачивания редактора");
    if (codeLines() !== 0) throw new Error("листинг не сворачивается");
    if (!findButton("В конец трассы")) throw new Error("ошибка");
    await click(findButton("Показать листинг трассы"), "кнопка");
    await settle(80);
    if (codeLines() === 0) throw new Error("листинг не сворачивается");
    // Esc: прогон остановлен, трасса убрана, редактор кода на месте
    await key("Escape");
    await settle(150);
    if (!textarea()) throw new Error("Esc убрал редактор кода");
    if (!findButton("/Остановить прогон трассы/")) throw new Error("ошибка");
    if (!/встроенное демо/.test(bodyText())) throw new Error("ошибка");
    return "редактор всегда на месте: трасса рядом, сворачивается";
  });
}

```

```

await test("F. редактор", "закрытие панели во время прогона");
await mountApp("floyd"); // 869 шагов: прогон долгий
await click(findButton("Боковая панель Python"), "кнопка");
await waitFor(() => panel() !== null, 4000);
await click(findButton("Заккрыть компилятор"), "кнопка");
await settle(150);
if (!panel()) throw new Error("панель не закрылась");
const backToDemo = await waitFor(() => /встроенное демо/.test(bodyText()));
if (!backToDemo) throw new Error("после закрытия панели");
await settle(3000); // ждём, пока догоняющий прогон
if (/данные из компилятора/.test(bodyText())) {
  throw new Error("закрытая панель всё-таки опубликована");
}
if (!/встроенное демо/.test(bodyText())) throw new Error("ошибка");
return "панель закрыта во время выполнения — демонстрация";
}

```

```

    if (!linked) throw new Error("в конце трассы визуализации  

    // 4) сняли связь (Esc + пустой код) — снова демо  

    await key("Escape");  

    await settle(80);  

    const ta = textarea();  

    if (!ta) throw new Error("редактор не вернулся по E  

    typeInto(ta, "");  

    const cleared = await waitFor(() => /встроенное демо  

    if (!cleared) throw new Error("после очистки кода б  

    if (/переменных пока нет/.test(bodyText())) throw n  

    return "демо → выполнено/переменных нет → данные из  

  });

```

/* — 7. Текстовый режим для PDF/парсеров —————

/* — 9. Ссылки между билетами, разделы содержания, б.

```

await test("Н. доступность", "содержание сжимается в  

  await mountApp("bridges-code");  

  const root = () => document.querySelector<HTMLElemen  

  const toggle = () => findButton(/Сжать содержание|С  

  if (!root()) throw new Error("нет сайдбара");  

  if (!/lg:w-56/.test(root()!.className)) throw new E  

  await click(toggle(), "сжать");  

  if (!/lg:w-44/.test(root()!.className)) throw new E  

  const items = Array.from(root()!.querySelectorAll("l  

  if (items.length < chapters.length) throw new Error  

  if (root()!.querySelector("input")) throw new Error  

  await click(toggle(), "скрыть");  

  if (!/lg:w-0/.test(root()!.className)) throw new Er  

  await click(toggle(), "развернуть");  

  if (!/lg:w-56/.test(root()!.className)) throw new E  

  return "широкое → узкое (темы на месте, поиска нет)  

});

```

```

await test("J. шапка", "кнопки загрузки не прячутся и  

  await mountApp("segment-trees");  

  const menu = document.querySelector("#download-menu  

  if (!menu) throw new Error("нет меню загрузки в шап

```

```
});
```

```
await test("I. ссылки", "содержание: разделы по поряд
  await mountApp("bridges-code");
  const sidebar = document.querySelector("#sidebar-ro
  if (!sidebar) throw new Error("нет сайдбара содержа
  const headers = Array.from(sidebar.querySelectorAll
    .map((el) => (el.textContent ?? "").trim()))
    .filter((t) => SECTIONS.some((sec) => sec.title === t))
  if (headers.length !== SECTIONS.length) {
    throw new Error(`в содержании ${headers.length} раз
  }
  const expected = SECTIONS.map((sec) => sec.title).join("
  if (headers.join(" | ") !== expected) throw new Error
  // 11 «Мосты» и 12 «Точки сочленения» читаются парой
  const items = Array.from(sidebar.querySelectorAll("
  const iBridges = items.findIndex((t) => t.includes("М
  const iArtic = items.findIndex((t) => t.includes("Т
  if (iBridges < 0 || iArtic < 0) throw new Error(`в
  if (Math.abs(iArtic - iBridges) !== 1) throw new Error
  // 2 «разреженная» и 5 «префиксные» – в одном разделе
  if (sectionOf("sparse-table").id !== sectionOf("pre
    throw new Error("билеты 2 и 5 попали в разные раз
  }
  if (sectionOf("bridges-code").id !== sectionOf("graph
    throw new Error("билеты 11 и 12 попали в разные раз
  }
  // порядок чтения: номера билетов сверху вниз не убыва
  const nums = Array.from(sidebar.querySelectorAll("b
    .map((b) => (b.querySelector("span")?.textContent ?? ""))
    .filter((t) => /^\\d/.test(t))
    .map((t) => Number.parseInt(t, 10));
  const ticketed = chapters.filter((c) => (chapterTop
  if (nums.length !== ticketed) {
    throw new Error(`номера билетов видны у ${nums.length}
  }
  for (let i = 1; i < nums.length; i += 1) {
    if (nums[i] < nums[i - 1]) throw new Error(`порядок
```

```

    if (!shown().includes(list[0].question.slice(0, 24))
      throw new Error(`первый вопрос не показан: ${list[0].question}`);
    }
    const wrong = list[0].options.findIndex((_, i) => i !== list[0].correctIndex);
    await click(ask(list[0].options[wrong].slice(0, 20)));
    await settle(80);
    if (!/Неверно/.test(shown())) throw new Error("на неверный ответ не ждем");
    if (!shown().includes(list[0].explanation.slice(0, 24)))
      throw new Error("после ответа нет разбора – польза от него не очевидна");
    }
    const nextBtn = ask("Следующий вопрос");
    if (!nextBtn) throw new Error("нет кнопки «Следующий вопрос»");
    await click(nextBtn, "дальше");
    await settle(80);
    if (!shown().includes(list[1].question.slice(0, 24)))
      // верный ответ
      await click(ask(list[1].options[list[1].correctIndex].slice(0, 20)));
    await settle(80);
    if (!/Верно/.test(shown())) throw new Error("на верный ответ не ждем");
    if (!/вопрос 2\\./.test(shown())) throw new Error(`с ${list[1].question} не ждем продолжения`);
    return `блиц из ${list.length} вопросов отвечает, продолжайте`;
  });
});

await test("I. факты", "память: разреженная жирная, префиксная", async () => {
  const memoryRow = async (topic: string) => {
    await mountApp(topic);
    const rows = Array.from(document.querySelectorAll("table tbody tr"));
    const row = rows.find((r) => /^\\s*Память/.test(r.textContent));
    if (!row) throw new Error(`на странице «${topic}» не найдено «Память»`);
    return Array.from(row.querySelectorAll("td")).map((c) => c.textContent);
  };
  const sparse = await memoryRow("sparse-table");
  const prefix = await memoryRow("prefix-sums-2d");
  // билет 2: у префиксной N × M, у разреженной – с логарифмом
  if (!/log/.test(sparse.join(" "))) throw new Error("в разреженной таблице не лог");
  const prefixCell = sparse.find((c) => !/log/.test(c));
  if (!prefixCell) throw new Error("в сравнении билетов не найдено «Память»");
});

```

```
});
```

/* — 10. Сквозной путь: свой код чужой размерности, 1

```
await test("К. свой код", "Флойд 4x4 из редактора вед
  await mountApp("floyd");
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(
    ta,
    [
      "n = 4",
      "INF = 999",
      "d = [[0, 3, INF, 7], [8, 0, 2, INF], [5, INF,
      "k = 0",
      "i = 0",
      "j = 0",
      "for k in range(n):",
      "    for i in range(n):",
      "        for j in range(n):",
      "            if d[i][k] + d[k][j] < d[i][j]:",
      "                d[i][j] = d[i][k] + d[k][j]",
      "print(d[0], d[1][0])",
    ].join("\n")
  );
  const ready = await waitFor(() => bodyText().include
  if (!ready) throw new Error(`своя программа не выпол
  if (!bodyText().includes("[0, 3, 5, 6]")) {
    throw new Error(`Флойд на своей матрице 4x4 посчи
  }
  if (!textarea()) throw new Error("редактор исчез, п
  await click(findButton("В конец трассы"), "в конец
  const linked = await waitFor(() => /данные из компи.
  if (!linked) throw new Error("демонстрация не приня
  const page = bodyText();
  if (!/4x4/.test(page)) throw new Error(`демонстраци
  const defects = domDefects(document.querySelector("a
```



```

    await click(findButton("Боковая панель Python"), "п
    if (!textarea()) throw new Error("нет редактора");
    await click(findButton("Вставить эталонный код"), "
    const ready = await waitFor(() => bodyText().include
    if (!ready) throw new Error(`эталон Косарайю не вып
    if (/Ошибка:/.test(panelText())) throw new Error(`э
    const out = panelText();
    if (!/порядок выхода:/.test(out)) throw new Error(`
    // граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SC
    if (!/компонент: 2/.test(out)) throw new Error(`в гр
    if (!textarea()) throw new Error("редактор исчез, п
    await click(findButton("В конец трассы"), "в конец
    const linked = await waitFor(() => /данные из компи.
    if (!linked) throw new Error("демонстрация не приня
    const defects = domDefects(document.querySelector("
    if (defects.length) throw new Error(`артефакты в де
    return "эталон Косарайю: order считается DFS-ом, на
  });

```

```

await test("G. lite-режим", "?lite=1 отдаёт текст без
  if (root) {
    await act(async () => {
      root?.unmount();
    });
    root = null;
  }
  if (host) host.remove();
  host = freshHost();
  window.history.replaceState({}, "", "/?lite=1&topic
  root = createRoot(host);
  await act(async () => {
    root!.render(<App />);
  });
  await settle(120);
  const txt = bodyText();
  if (txt.length < 2000) throw new Error(`lite-режим
  if (/main\.py|Pyodide/.test(txt)) throw new Error("
  if (!/разреженн/i.test(txt)) throw new Error("в lite

```

```
        byGroup.set(r.group, g);
    }
    return { total: results.length, passed: results.filter(
        .fromEntries(byGroup) });
}
```

```
/** Инициализация страницы из vizSync – нужна для проверки
export const initTemplateOf = (id: string) => buildInitTemplateOf(id)
```

ФАЙЛ 100/111: scripts/verify/check-links.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 199 | РАЗМЕР: 1000

```
/**
 * Аудит внутренних ссылок пособия: ничего не должно вешаться
 *
 * npm run verify:links
 *
 * Проверяет (по настоящим модулям приложения, собранным в release):
 * 1. билеты 1–24 закрыты страницами, id в chapterTopic соответствует id в glossary;
 * 2. разделы содержания: каждая страница в своём разделе;
 * 3. упоминания «билет N» в тексте ведут на существующие билеты;
 * 4. все data-goto / href="?topic=..." указывают на существующие билеты;
 * 5. якоря href="#..." есть в той же главе;
 * 6. glossary.simulator существует в реестре и (кроме simulator) имеет свою страницу – иначе модалка не получит данные;
 * 7. glossary.демо – работающая мини-демо;
 * 8. у каждого ключа PAGE_SYNC есть запись в реестре;
 * 9. у каждой страницы есть визуализация;
 * 10. страницы связаны перекрёстными ссылками (темы не исключение);
 */
import * as esbuild from "esbuild";
import { mkdirSync, writeFileSync } from "fs";

const OUT = "tmp/verify/links-entry.mjs";
const FAILS = [];
```

```

-----
const registryIds = Object.keys(VIZ_REGISTRY);

/** Демо без собственной страницы: открываются из подсказки */
const EXTRA_DEMOS = new Set(["stack-dfs", "queue-bfs",

/* 1. билеты 1–24 */
const covered = new Set();
for (const [id, tickets] of Object.entries(chapterTopics)) {
  ok(chapterIds.has(id), `chapterTopics ссылается на не-
  tickets.forEach((n) => covered.add(n));
}
for (let n = 1; n <= 24; n += 1) {
  ok(covered.has(n), `билет ${n} не закрыт ни одной стр
}

/* 2. разделы содержания */
for (const chapter of chapters) {
  const sec = sectionOf(chapter.id);
  ok(!sec, `страница «${chapter.id}» не попала ни в од
  const tickets = chapterTopics[chapter.id] ?? [];
  ok(
    tickets.every((n) => n >= sec.from && n <= sec.to),
    `билеты страницы «${chapter.id}» (${tickets.join(",
  });
}
{
  const bySection = SECTIONS.map((s) => ({
    s,
    nums: chapters.flatMap((c) => (sectionOf(c.id).id =
  }));
  for (let i = 1; i < bySection.length; i += 1) {
    const prevMax = Math.max(...bySection[i - 1].nums);
    const curMin = Math.min(...bySection[i].nums);
    ok(curMin > prevMax, `разделы пересекаются: «${bySe
  }
  const all = bySection.flatMap((x) => x.nums);
  for (let n = all[0]; n <= all[all.length - 1]; n += 1)
    ok(all.includes(n), `в содержании дыра: билет ${n}

```

```

-----
    for (const m of html.matchAll(/href="#([^\"]+)/g)) {
      ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
    }

    if (mentionsHere === 0 && !/data-goto=/.test(html)) w
  }

/* 6–7. глоссарий */
const demoKinds = new Set(MINI_DEMO_KINDS);
for (const term of GLOSSARY) {
  if (term.simulator) {
    ok(registryIds.includes(term.simulator), `глоссарий
    ok(
      chapterIds.has(term.simulator) || EXTRA_DEMOS.has
      `глоссарий «${term.id}»: simulator "${term.simula
      ые компилятора`
    );
  }
  if (term.demo) ok(demoKinds.has(term.demo), `глоссарий
}

/* 8–9. реестр и синхронизация */
for (const key of Object.keys(PAGE_SYNC)) {
  const base = key.split("#")[0];
  ok(registryIds.includes(base), `PAGE_SYNC["${key}"]:
}
for (const id of registryIds) {
  if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
    FAILS.push(`реестр: визуализация "${id}" не являетс
  }
}
for (const chapter of chapters) {
  ok(registryIds.includes(chapter.id), `страница «${cha
}

/* 96. файлы, на которые ведут кнопки шапки, действител
{
  const { existsSync, statSync } = await import("fs");

```

```
-----
* (pyodide из node_modules) под тем же трейсером, что и
* проверяется:
*   1. код не падает и не обрывается по лимиту шагов;
*   2. трасса не пустая (иначе панель показала бы пустую);
*   3. каждая переменная, обещанная странице, реально есть;
*       иначе визуализация не может за ним следовать;
*   4. в снимках нет «мусорных» значений, которые уронят
*/
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
mkdirSync(OUT_DIR, { recursive: true });

writeFileSync(
  `${OUT_DIR}/contract-entry.ts`,
  `import { buildDebugRunner } from "../../src/data/debugRunner";
import { PAGE_SYNC } from "../../src/data/vizSync";
import { chapters } from "../../src/data/content";
export const runnerOf = (src: string) => buildDebugRunner(src);
export const sync = Object.fromEntries(Object.entries(PAGE_SYNC));
export const declared = Object.fromEntries(Object.entries(DECLARED));
export const chapterIds = chapters.map((c) => c.id);
`
);

const built = await esbuild.build({
  entryPoints: [`${OUT_DIR}/contract-entry.ts`],
  bundle: true,
  outfile: `${OUT_DIR}/contract-bundle.mjs`,
  format: "esm",
  platform: "node",
  jsx: "automatic",
  sourcemap: false,
  loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
  logLevel: "warning",
});
```

```

    .filter((a) => !a.some((x) => ident(x) && seen.has(x)))
    .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обеща
/*
 * Обещанная переменная не должна быть пустой на ВСЕХ
 * объявил и не наполнил – визуализации нечего показы
 * «зашитый» результат вместо работы алгоритма. Именно
 * Косарайю: order был дан готовым списком, а comp та
 */
const isEmpty = (v) =>
  Array.isArray(v) ? v.length === 0
  : typeof v === "string" ? v.length === 0
  : typeof v === "object" && v !== null ? Object.keys
  : false;
const neverFilled = (mod.declared[key] ?? [])
  .map(alts)
  .filter((a) => a.some(ident))
  .map((a) => a.find((x) => ident(x) && seen.has(x)))
  .filter(Boolean)
  .filter((name) => {
    let sawValue = false;
    let sawNonEmpty = false;
    for (const step of [...result.steps, { values: re
      const v = step.values?.[name];
      if (v === undefined || v === null) continue;
      sawValue = true;
      if (!isEmpty(v)) sawNonEmpty = true;
    }
    return sawValue && !sawNonEmpty;
  });
if (neverFilled.length) issues.push(`объявлены, но ни

const stderr = printed.filter((p) => p.startsWith("[s
if (stderr.length) issues.push(`предупреждения: ${std

if (issues.length) problems++;
rows.push({ key, steps: result.steps.length, vars: se
}

```

```

virtualConsole.on("jsdomError", (error) => {
  jsdomErrors.push(error?.message || String(error));
  onError?.(error);
});
// console.error/warn приложения собираем отдельно —
const consoleErrors = [];
virtualConsole.on("error", (...args) => consoleErrors.push(...args));
virtualConsole.on("warn", () => {});

const dom = new JSDOM(
  `<!doctype html><html lang="ru"><head><meta charset="utf-8"><title></title></head><body><div id="root"></div></body></html>`,
  { url, pretendToBeVisual: true, runScripts: "outside-only" },
);
const { window } = dom;

window.ResizeObserver = window.ResizeObserver ?? OBSERVER_UNIMPLEMENTED;
window.IntersectionObserver = window.IntersectionObserver ?? OBSERVER_UNIMPLEMENTED;
window.matchMedia =
  window.matchMedia ??
  ((media) => ({
    matches: false,
    media,
    onchange: null,
    addListener() {},
    removeListener() {},
    addEventListener() {},
    removeEventListener() {},
    dispatchEvent: () => false,
  }));
window.scrollTo = () => {};
window.HTMLElement.prototype.scrollIntoView = () => {};
window.HTMLElement.prototype.focus = window.HTMLElement.prototype.blur = () => {};
window.URL.createObjectURL = window.URL.createObjectURL = () => {};
window.URL.revokeObjectURL = window.URL.revokeObjectURL = () => {};
Object.defineProperty(window, "innerWidth", { value: 1000 });
Object.defineProperty(window, "innerHeight", { value: 1000 });
Object.defineProperty(window, "devicePixelRatio", { value: 1 });

```

```
    };
    for (const key of copy) {
      try {
        Object.defineProperty(globalThis, key, { value: w
      } catch {
        /* read-only в Node – не критично */
      }
    }
    globalThis.IS_REACT_ACT_ENVIRONMENT = true;

    return { dom, window, document: window.document, cons
  }
```

ФАЙЛ 103/111: scripts/verify/live-matrix.test.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМ

```
/**
 * Модульные проверки разбора «живых» данных компилятор
 *
 * npm run verify:unit
 *
 * liveMatrix() – то, благодаря чему визуализации матриц
 * суммы 2D, разреженная таблица) принимают данные ЛЮБОГО
 * пользователя вместо захардкоженной демо-сетки.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
writeFileSync(
  `${OUT_DIR}/unit-entry.ts`,
  `export { liveMatrix, pickLiveVar, clampIndex } from
);
await esbuild.build({
```



```
-----
check("список словарей с числовыми ключами", cells([{"0": 1, "1": 2}, {"0": 3, "1": 4}]);
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }]);
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } });
check('ключи-пары "{i, j}" после снапшота tuple', cells({ "0, 1": 5, "1, 2": 6 }));
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5, "0, 1": 6 }));
```

```
/* — liveMatrix: что матрицей НЕ является -----
check("плоский список чисел → null", m([1, 2, 3, 4, 5]));
check("словарь по именам → null", m({ dist: 1, prev: 2 }));
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);
check("null → null", m(null), null);
check("число → null", m(42), null);
check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m(["a", "b"], ["c", "d"]));
```

```
/* — liveMatrix: особые значения -----
check('"inf" → +Infinity', cells([["inf", 1], [2, "-inf"]]));
check('числа-строки "42" → 42', cells([["1", "2"], ["3", "4"]]));
check("булевы значения ячейкой не считаются", m([[true, false], [false, true]]));
check("обрывок до 64×64", shape(Array.from({ length: 8096 }, () => 0), { numbers: true }));
check("hasNumbers=false невозможна для живой матрицы", m({}));
```

```
/* — pickLiveVar -----
check("первое найденное имя по приоритету", pickLiveVar(["a", "b"], ["a", "c"], ["d"]));
check("берёт следующее имя, если первого нет", pickLiveVar(["a", "b"], ["a", "c"], ["d"]));
check("ничего не найдено → null", pickLiveVar({ x: 1 }, { x: 2 }, { x: 3 }));
check("нет снимка → null", pickLiveVar(undefined, ["d"], ["e"]));
check("ключ со значением undefined всё равно считается", pickLiveVar({ x: 1 }, { x: 2 }, { x: 3 }));
```

```
/* — clampIndex -----
check("null остаётся null", clampIndex(null, 5), null);
check("размер 0 → null", clampIndex(2, 0), null);
check("обрезка сверху", clampIndex(99, 5), 4);
check("обрезка снизу", clampIndex(-7, 5), 0);
check("дробный индекс усекается", clampIndex(2.9, 5), 2);
```

```

-----
    return (f?.textContent ?? "").replace(/\s+/g, " ").slice(0, 100);
}
function state(tag: string) {
    const p = panel();
    const ed = editor();
    const txt = (p?.textContent ?? "").replace(/\s+/g, " ");
    const step = txt.match(/шаг[^\s]*\s*\d+\s*\//\s*\d+/i);
    process.stdout.write(
        `    [${tag.padEnd(9)}] редактор: ${ed ? "ДА" : "нет"}
        step ? step[0] : "-" } | футер: ${footer()}\n`
    );
    process.stdout.write(`        бейдж страницы: ${
        /встроенное демо/.test(document.body.textContent ?? "")
    }`);
}
function typeInto(ta: HTMLTextAreaElement, value: string) {
    const setter = Object.getOwnPropertyDescriptor(window, "value")?.set;
    act(() => {
        setter?.call(ta, value);
        ta.dispatchEvent(new window.Event("input", { bubble: true }));
    });
}
async function click(el: Element | null | undefined, label: string) {
    if (!el) throw new Error(`нет кнопки: ${label}`);
    await act(async () => {
        el.dispatchEvent(new window.MouseEvent("click", { bubble: true }));
        await sleep(20);
    });
}
function findButton(match: RegExp): HTMLButtonElement | undefined {
    return Array.from(document.querySelectorAll("button")).find(
        b => match.test([b.getAttribute("aria-label"), b.textContent].join(" "))
    ) as HTMLButtonElement | undefined;
}

export async function probe() {
    const host = document.createElement("div");
    document.body.appendChild(host);
    window.history.replaceState({}, "", `/?topic=dijkstra`);
}

```

```

for (const t of [200, 800, 2000, 6000]) {
  await settle(t);
  state(`t+${t}`);
}

process.stdout.write("\n=== 4. очищаем редактор полностью\n");
const ed4 = editor();
if (ed4) {
  typeInto(ed4, "");
  state("очистили");
  for (const t of [200, 800, 2000]) {
    await settle(t);
    state(`t+${t}`);
  }
} else {
  process.stdout.write("  редактора нет (листинг) – ждём\n");
  await act(async () => {
    document.dispatchEvent(new window.KeyboardEvent(""));
    await sleep(30);
  });
  state("после Esc");
}
}

```

ФАЙЛ 105/111: scripts/verify/README.md

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

Проверки приложения (`npm run verify`)

Три уровня проверок, которые можно гонять локально и в CI. Для этого нужен: DOM даёт `jsdom`, Python – `pyodide` из `node\_modules` 3.12, что грузится в браузере пользователя, только без `node\_modules`.

```
```bash
```

```
npm run verify
```

```
всё сразу: unit → эталонные к...
```

`[object Object]`; нет кнопок без имени |  
| В. Флойд: размерности (10) | 5×5, 10×10 (больше демо-  
| й список, 80×80 (обрезка), пустая матрица, чужое имя  
| С. матричные страницы (6) | префиксные суммы 2D и раз  
| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами  
| грывания гаснут и оживают обратно |  
| Е. эталонный код (46) | прогон кода в CPython → финал  
| е показать мусор |  
| F. редактор (6) | код из одних комментариев, синтаксис  
| ляет эталон и возвращает инициализацию; код не перете  
| G. lite-режим (1) | `?lite=1` отдаёт текст без навига  
| H. доступность (2) | в панели подписаны все кнопки в  
| очника |

Фильтры: `HAR\_CHAPTERS=floyd,dijkstra`, `HAR\_SKIP=A,E`,

## Как это заведено в приложение

- \* `window.\_\_algv0PythonRuntime` — точка подмены интерпретатора  
`PythonCompiler.tsx`. В браузере пользователя поле не  
грузит Pyodide с CDN как раньше; харнес кладёт туда л
- \* Состояние визуализаций передаётся через `emitVizState`  
(`src/data/vizStepBus.ts`) — те же события, что шлёт  
идёт по боевому пути, а не через внутренние API компо

---

ФАЙЛ 106/111: scripts/verify/run-browser-checks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 96 | РАЗМ

---

/\*\*

\* DOM-проверки приложения: jsdom + настоящий Pyodide и  
\*

\* npm run verify:dom  
\*

\* Фильтры:

\* HAR CHAPTERS=floyd,dijkstra только эти страницы

```

define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
},
loader: { ".css": "empty", ".png": "dataurl", ".jpg":
logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/b

process.on("unhandledRejection", (reason) => {
 jsdomErrors.push(`unhandledRejection: ${reason instan
});

log("прогоняю сценарии...\n");
const results = await mod.runSuite({ py, window, document
const sum = mod.summary();

writeFileSync(`${OUT_DIR}/report.json`, JSON.stringify(

const failed = results.filter((r) => !r.ok);
console.log("\n" + "=".repeat(96));
console.log("СВОДКА");
console.log("=".repeat(96));
for (const [group, g] of Object.entries(sum.byGroup)) {
 console.log(` ${group.padEnd(26)} ok: ${String(g.ok)
}
console.log(`\n ВСЕГО ${sum.total}: прошло ${sum.pass
if (failed.length) {
 console.log("\nПРОВАЛЫ:");
 for (const f of failed) console.log(` x [${f.group}]
}
if (jsdomErrors.length) {
 console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченны
 for (const e of [...new Set(jsdomErrors)].slice(0, 10

```

```
const { window, document } = createDom();
window.__algv0PythonRuntime = py;

await esbuild.build({
 entryPoints: ["scripts/verify/probe-compiler.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/probe-bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/probe-bundle.mjs`);
log("зондирую панель...");
await mod.probe();
process.exit(0);
```

---

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

---

---

ФАЙЛ 108/111: [.github/workflows/deploy-pages.yml](#)  
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9 КБ

---

name: Deploy app to GitHub Pages

cache: npm

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Build for the repository subpath  
env:  
VITE\_BASE: /algv0/  
run: npm run build
- name: Make GitHub Pages fallback  
run: cp dist/index.html dist/404.html
- name: Upload Pages artifact  
uses: actions/upload-pages-artifact@v5  
with:  
path: ./dist

deploy:

name: Publish to GitHub Pages  
needs: build  
runs-on: ubuntu-latest  
environment:  
name: github-pages  
url: \${ steps.deployment.outputs.page\_url }

steps:

# Ранняя и понятная ошибка вместо невнятного «Fail»  
# когда в репозитории Pages не переключён на сбор

- name: Preflight – Pages собирается через GitHub  
env:

GH\_TOKEN: \${ secrets.GITHUB\_TOKEN }

run: |

bt=\$(gh api "repos/\${ github.repository }/pages/build\_type")

echo "Pages build\_type: \${bt}"

if [ "\${bt}" != "workflow" ]; then

echo "::error title=Pages не настроен::Setting up Pages environment before starting this deployment.

те этот запуск. См. комментарий в начале деплоя.

exit 1

```
 default: "150"

concurrency:
 group: rebuild-pdf-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[bot: ')"

 steps:
 - name: Checkout
 uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
 run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends imagemagick
 # На Ubuntu политика ImageMagick иногда запрещает
 # разрешаем то, что нужно пайплайну (text -> pdf)
 for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policy.xml; do
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
```



- ```
# снимки публикуются вместе с приложением: HTML
# в автономный файл прямо из /export/viz-shots
mkdir -p public/export/viz-shots
cp tmp/viz-shots/*.png tmp/viz-shots/manifest
```
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full\_code\_all\_pages` | "
 echo "| \`public/export/full\_code\_all\_pages` | "
 echo "| \`public/export/guide\_lite.pdf\` | "
 echo "| Снимков визуализаций | \${ls tmp/viz-shots/*} | "
 echo "| PNG-страниц | \${ls tmp/export-pages/*} | "
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 if-no-files-found: error
 retention-days: 30
 - name: Upload artifacts (PNG-страницы)
uses: actions/upload-artifact@v7
with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png

```
  paths-ignore:
    - "public/export/**"
    - "**/*.md"
pull_request:
workflow_dispatch:

concurrency:
  group: verify-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  verify:
    name: unit + эталонный код + DOM
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      # puppeteer здесь не нужен (он для растеризации Р
      # тянет Chromium – пропускаем, чтобы установка не
      - name: Зависимости
        run: npm ci
        env:
          PUPPETEER_SKIP_DOWNLOAD: 1
          PUPPETEER_SKIP_CHROMIUM_DOWNLOAD: 1

      - name: Типы и сборка
        run: |
          npx tsc --noEmit -p tsconfig.json
          npm run build
```

1. Объект главы

```
```ts
import { Chapter } from "../../types";

export const myChapters: Chapter[] = [
 {
 id: "dijkstra", // = ключ VIZ_REGISTRY и
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html", // всегда "html": страница
 category: "Графы. Пути", // группа в навигации
 content: `...HTML ниже...`,
 },
];
```
```

Правила:

- `id` — kebab-case, уникальный, совпадает с ключом `VIZ\_REGISTRY` и подставляется автоматически, вручную `div#slot-\*` вставляются только такие заглушки остались в старых главах и считаются мусором
- `title` начинается с номера билета: сортировка оглавления по номеру цифре заголовка. Одна страница может закрывать несколько билетов (например `mst` → 18, 19, 20) — тогда номер в заголовке один, а билет объявляется в `chapterTopics` (`src/data/content.ts`)
- `content` — шаблонная строка. Внутри **никаких** `` ` ` `` экранирования: это сломает сборку.

2. Каркас страницы

```
```html
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <!-- шапка: бейдж билета + заголовок -->
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```
````html
<p class="text-slate-300 text-sm mb-4">
  Текст с <b>акцентом</b> и инлайн-кодом
  <span class="font-mono text-emerald-300">dist[v]</span>
</p>
````
```

Инлайн-«пилюля» для имени переменной/константы:

```
````html
<span class="bg-slate-950 px-2 py-0.5 rounded text-xs f
````
```

### ### 3.3 Сетка ключевых чисел (3 ячейки)

```
````html
<div class="grid grid-cols-1 sm:grid-cols-3 gap-3 my-6">
  <div class="bg-slate-900 border border-slate-700 rounded">
    <p class="text-[10px] uppercase tracking-wider text">
    <p class="font-mono text-emerald-300 text-sm">0((V+H
  </div>
  <!-- 2-я и 3-я ячейки — тот же markup -->
</div>
````
```

### ### 3.4 Сетка аналогий — **\*\*обязательна\*\*** на каждой странице

Слева — неформальная аналогия, справа — техническая аль

```
````html
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border border">
    <div class="absolute -top-3 left-4 bg-slate-700 text
      ald-500 shadow-md">
      Аналогия 1: Позитивное планирование
    </div>
    <p class="text-slate-300 text-sm mb-4">...</p>
  </div>
</div>
````
```

```
```html
<details class="bg-slate-900/40 rounded-lg border border-
  <summary class="p-4 font-bold text-slate-400 outline-l
    e-700/50">
    ▶ Код: Дейкстра на куче (Python)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <pre class="bg-slate-950 p-4 rounded overflow-x-aut
...</code></pre>
    <p class="text-slate-300 text-sm">Комментарий после
  </div>
</details>
```
```

- Вложенные списки внутри спойлера: `<ul class="list-di
- Подзаголовок внутри блока: `<h4 class="text-lg font-b
- или `<p class="text-lg font-bold text-white mt-8 mb-3

### ### 3.7 Ловушки / частые ошибки

```
```html
<div class="bg-rose-950/30 p-5 rounded-lg border border
  <p class="text-sm font-bold text-rose-300 mb-3">△ Гд
  <ul class="list-disc pl-5 space-y-2 text-sm text-slat
    <li><b>Отрицательные веса.</b> Дейкстра их не перев
    <li>Переполнение: <span class="font-mono text-emera
  </ul>
</div>
```
```

---

## ## 4. Порядок разделов на странице

1. Шапка: бейдж билета + `h2`.
2. «Одной фразой» / «Суть алгоритма» + центрированное о
3. 1–3 абзаца объяснения (3.2), затем сетка ключевых чи

- заголовки — только ``h2/h3/h4``; списки — ``ul/ol/li``; т  
код — ``pre``; определения — обычный ``p`` (см. 3.1);
- **\*\*смысл не должен держаться на эмодзи\*\***: в PDF использ  
нет эмодзи-глифов — пиктограммы вырезаются, остаётся  
« Операции на отрезках» доедет как «Операции на отр  
а вот « значит да / значит нет» потеряет смысл, пи
- спойлеры в PDF раскрываются полностью — не прячьте ту  
минимум, но и не дублируйте его в основном тексте;
- картинки страниц берутся из ``CHAPTER_IMAGES`` (``script``  
снимки демонстраций — из ``tmp/viz-shots/*.png`` (их ре  
``npm run export:shots`` в CI); локально без браузера в  
описание вкладок и переменных из ``PAGE_SYNC``.

Цель по объёму: 40–60 страниц, жёсткий предел — 100.